

COMPLETE COURSE REFERENCE

Data Engineering

From Foundations to Production

Modules 0 – 10 · Hands-on Practitioner's Guide

2026 Edition

Table of Contents

- Module 0: Welcome & Setup
- Module 1: Data Modeling Foundations
- Module 2: SQL for Data Engineers
- Module 3: Python for Data Engineers
- Module 4: Apache Airflow — Orchestration
- Module 5: Apache Spark & Big Data Processing
- Module 6: Apache Kafka & Streaming
- Module 7: Data Quality & Testing
- Module 8: Cloud Infrastructure for Data Engineers
- Module 9: Data Engineering for AI
- Module 10: Capstone Project — Real-Time E-Commerce Analytics Platform

Module 0: Welcome & Setup

0.1 Who This Course Is For (and Who It's NOT For)

TL;DR

- You need working knowledge of SQL (JOINS, subqueries), basic Python, and the command line before starting.
- This course targets analysts leveling up, backend devs pivoting, and junior DEs filling gaps.
- Complete beginners and senior DEs wanting distributed systems theory should look elsewhere.
- By the end you'll have a portfolio-ready, end-to-end data platform project.

Let's save you 40 hours right now. If this course isn't the right fit, better to know that upfront than halfway through Module 5.

Who This IS For

You should already be comfortable with:

- **SQL** — not just `SELECT *`. You can write JOINS, maybe window functions, subqueries. You can get data out of a database.
- **Python** — you can write a function, install a package with pip, and read documentation. You don't need to be a software engineer.
- **The command line** — `cd`, `ls`, running scripts. The basics.
- **Databases** — you understand tables, schemas, and queries at a conceptual level.

If that sounds like you — an analyst wanting to level up, a backend developer looking to pivot, a junior DE who learned on the job and wants to fill the gaps — you're in the right place.

Who This is NOT For

- **Complete beginners.** If you've never written a line of code, go learn Python and SQL first. There are plenty of free resources to get you there.
- **Senior DEs wanting distributed systems theory.** This is a practitioner's crash course, not a PhD program.
- **People who want a 6-month bootcamp.** This is ~40 hours. It's fast and dense by design.

What You'll Be Able to Do by the End

You'll build a complete, portfolio-ready project that you can show in interviews. Not a toy — a real pipeline with real tools. Specifically, you'll be able to:

1. Design dimensional data models from scratch

2. Write production-quality SQL transformations
 3. Build Python data ingestion pipelines
 4. Orchestrate workflows with Apache Airflow
 5. Process data at scale with Spark
 6. Build real-time streaming pipelines with Kafka
 7. Implement data quality checks
 8. Deploy to the cloud
 9. Build AI/embedding pipelines
 10. Ship a polished portfolio project
-

0.2 The Data Engineering Landscape in 2026

TL;DR

- The modern data stack flows: Sources → Ingestion → Storage → Processing → Orchestration → Quality → Serving.
- DuckDB, AI pipelines, mature table formats, and cost-consciousness are the biggest recent shifts.
- You don't need to learn everything — learn the fundamentals deeply and the tools well enough to be productive.

When I started in data, the stack was Hadoop, Hive, and a prayer. In 2026, the landscape is completely different — and honestly, way better. But it can feel overwhelming when you look at it for the first time.

The Modern Data Stack — Layer by Layer

[DIAGRAM: A horizontal flow diagram showing seven layers from left to right. Each layer is a box with its name and example tools. Arrows connect them sequentially: Data Sources → Ingestion → Storage → Processing/Transformation → Orchestration → Data Quality → Serving/Consumption. The diagram should feel like a conveyor belt moving data from raw to refined.]

1. Data Sources — APIs, databases, event streams, files, SaaS tools.

Your job starts here. Data is messy, comes from everywhere, and nobody documented it.

2. Ingestion — Getting data from sources into your systems.

Tools: Python scripts, Kafka, Airbyte, Fivetran. Some companies buy tools for this (Fivetran), some build custom. You need to know both approaches.

3. Storage — Where data lives.

Data lakes (S3 + table formats like Delta Lake, Iceberg), data warehouses (Snowflake, BigQuery, Redshift, Postgres). The data lake vs. data warehouse debate is mostly over. Most companies use both — it's called a **lakehouse**.

4. Processing/Transformation — Making raw data useful.

Batch: Spark, dbt, Python, SQL. Streaming: Kafka consumers, Flink. This is where you spend most of your time. Taking garbage data and making it gold.

5. Orchestration — Making it all run on schedule, with error handling.

Airflow (dominant), Dagster, Prefect. Think of this as the conductor of your data orchestra.

6. Data Quality — Making sure data is correct.

Great Expectations, Soda, dbt tests. The difference between a junior and senior DE? The senior one checks their work.

7. Serving / Consumption — Getting data to people who need it.

Dashboards (Looker, Tableau, Metabase), ML models, AI applications, APIs. You build the highway. Analysts and data scientists drive on it.

What's Changed Recently

- **DuckDB exploded.** You don't always need Spark anymore. Sometimes a single laptop is enough.
- **AI changed everything.** Data engineers now build embedding pipelines, vector stores, RAG systems. We cover this in Module 9.
- **Table formats matured.** Delta Lake and Iceberg mean your data lake isn't a swamp anymore.
- **Cloud costs matter.** Companies are actually watching their spend now. You need to think about cost.

□ Key Concept

You don't need to learn every tool. You need to learn the **fundamentals** deeply and the **tools** well enough to be productive. That's exactly what this course does.

□ Checkpoint

1. What's the difference between the ingestion layer and the processing layer?
2. What is a "lakehouse" and why has it become popular?
3. Name one tool from each layer of the modern data stack.

0.3 Your Career Path — DE Roles at Startups vs. Big Tech

TL;DR

- Startup DEs do everything; Big Tech DEs specialize deeply. Both are valid paths.
- Salary ranges from ~\$100k (startup) to \$350k+ (FAANG) in the US (2026).
- Portfolio projects beat certifications every time. Nobody cares about your Coursera certificate — they care about your GitHub.

There's no single "data engineer" job. The role at a 20-person startup looks nothing like the role at Google.

Startup DE (< 100 employees)

You ARE the data team. Maybe you and one other person. You do everything: ingestion, modeling, dashboards, infra, some analytics. The stack is simpler: Postgres, Python, Airflow, maybe dbt, one cloud provider.

Honestly? This is where you learn the fastest. You own everything.

Pay range: \$100k–\$160k (US, 2026)

Mid-size Company DE (100–2,000 employees)

Dedicated data team of 3–10 people. You specialize a bit. There's probably a data platform — Snowflake or BigQuery, Airflow, dbt. More process: code reviews, CI/CD, on-call rotations.

This is the sweet spot for a lot of people. Enough structure to learn, enough autonomy to build.

Pay range: \$130k–\$200k

Big Tech / Enterprise DE (2,000+ employees)

Highly specialized. You might just work on one pipeline or one system. Internal tools everywhere. Custom orchestrators, custom frameworks. Scale problems: petabytes, millions of events per second.

Amazing for your resume, but you might not learn as broadly.

Pay range: \$160k–\$350k+ (FAANG)

Adjacent Roles to Know About

- **Analytics Engineer** — dbt-focused, SQL-heavy, closer to analysts
- **Data Platform Engineer** — infra-focused, Kubernetes, Terraform
- **ML Engineer** — model training pipelines, feature stores

These aren't better or worse — they're different specializations. This course prepares you for all of them.

How to Get Your First Role

- **Portfolio > certifications.** Nobody cares about your certificate. They care about your GitHub. The capstone project in Module 10 is designed to be your portfolio piece.
- **Network.** Join communities, post on LinkedIn, contribute to open source.

❏ Pro Tip

When applying, tailor your portfolio project description to match the job posting's tech stack. Same project, different emphasis.

0.4 Dev Environment Setup

TL;DR

- Docker gives us a reproducible environment: Postgres, Airflow, Kafka, and MinIO running locally.
- Clone the course repo, `docker compose up`, set up a Python venv — done in 15 minutes.
- Everything you need for the entire course runs on your laptop.

Docker gives us a reproducible environment. Fifteen minutes and you're coding.

Prerequisites

Make sure you have these installed before continuing:

Tool	How to verify	Minimum version
Docker Desktop	<code>docker --version</code>	24+
Git	<code>git --version</code>	2.30+
VS Code (or preferred editor)	—	—
Python	<code>python --version</code>	3.11+

Step 1: Clone the Course Repo

```
git clone https://github.com/gyatesofficial/de-fast-track.git
cd de-fast-track
```

Everything for this course lives in this repo. Every module has its own directory with starter code, solutions, and instructions.

Step 2: Review the Project Structure

```
de-fast-track/
├── docker-compose.yml      # All infrastructure
├── .env.example            # Environment variables
├── modules/
│   ├── module-0/
│   ├── module-1/
│   │   ├── starter/      # Your starting point
│   │   └── solution/     # Reference solution
│   └── module-2/
│       └── ...
├── data/                  # Sample datasets
├── airflow/
│   └── dags/              # Your DAGs go here
└── README.md
```

Step 3: Set Up Environment Variables

```
cp .env.example .env
# Open .env and review – most defaults are fine for local dev
```

Step 4: Start the Docker Stack

```
docker compose up -d
```

This spins up four services:


```
# docker-compose.yml (simplified view)
services:
  postgres:
    image: postgres:16
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: dataeng
      POSTGRES_PASSWORD: dataeng
      POSTGRES_DB: warehouse
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d

  airflow-webserver:
    image: apache/airflow:2.9.0
    ports:
      - "8080:8080"
    depends_on:
      - postgres

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    ports:
      - "9092:9092"

  minio:
    image: minio/minio:latest
    ports:
      - "9000:9000"
      - "9001:9001"
    # S3-compatible local storage
```

Postgres for your warehouse, Airflow for orchestration, Kafka for streaming, and MinIO as a local S3 replacement. This is a real production-like stack running on your laptop.

Step 5: Verify Everything Is Running

```
# Check all services are up
docker compose ps
# All services should show "running"
```

```
# Test Postgres
psql -h localhost -U dataeng -d warehouse -c "SELECT 1;"
```

Expected output:

```
?column?
-----
      1
(1 row)
```

```
# Test Airflow UI – open http://localhost:8080 (admin/admin)
# Test MinIO UI – open http://localhost:9001 (minioadmin/minioadmin)
```

Step 6: Set Up Python Virtual Environment

```
python -m venv .venv
source .venv/bin/activate # or .venv\Scripts\activate on Windows

pip install -r requirements.txt
```

Use a standard venv here. If you prefer uv, conda, or poetry — go for it. The `requirements.txt` has everything you need:

```
# Core
psycopg2-binary==2.9.9
sqlalchemy==2.0.30
pandas==2.2.2
polars==0.20.25
duckdb==0.10.3
pydantic==2.7.1

# Testing
pytest==8.2.1
pytest-cov==5.0.0

# API & HTTP
requests==2.32.3
httpx==0.27.0

# File formats
pyarrow==16.1.0
fastparquet==2024.5.0

# Utilities
python-dotenv==1.0.1
rich==13.7.1
```

⚠ Common Mistake

- **Port conflicts:** If Docker says a port is already in use, something else is running on 5432 or 8080. Change the left-side port in `docker-compose.yml` (e.g., `"5433:5432"`).
- **Airflow slow to start:** Normal on first run. It needs to initialize its database. Give it 1–2 minutes.
- **Windows users:** Make sure WSL2 is enabled for Docker. Seriously. It won't work well without it.

□ Checkpoint

1. What command checks if all Docker services are running?
2. What credentials do you use to log into the Airflow UI?
3. Why do we use MinIO instead of real S3 for local development?

0.5 Course Project Overview — What You'll Build

TL;DR

- Every module builds toward one capstone: a Real-Time E-Commerce Analytics Platform.
- The dataset is ~1M orders, ~50k customers, ~10k products — realistic fake data with realistic messiness.
- By Module 10, you'll have a portfolio-ready GitHub repo that demonstrates end-to-end DE skills.

Most courses have you build tiny, disconnected projects that don't mean anything on a resume. We're doing something different. Every module builds toward one big project — a complete, production-grade data platform.

The Capstone: Real-Time E-Commerce Analytics Platform

[DIAGRAM: A vertical architecture diagram showing the full capstone project. Seven horizontal layers stacked top-to-bottom, connected by downward arrows:

1. **DATA SOURCES** (top) — three boxes side by side: "REST API (products)", "Kafka (events)", "CSV (inventory)"
2. **INGESTION LAYER** (Modules 3, 4, 6) — "Python scripts | Kafka consumers | File watchers"
3. **RAW STORAGE** (Module 8) — "S3 / MinIO (Delta Lake format)"
4. **PROCESSING** (Modules 2, 3, 5) — "Spark (batch) | Python (transforms) | SQL (modeling)"
5. **WAREHOUSE** (Modules 1, 2) — "PostgreSQL — Star Schema: Facts (orders, events) | Dims (products, customers, time)"
6. **QUALITY & ORCHESTRATION** (Modules 4, 7) — "Airflow DAGs | Great Expectations | Alerts"
7. **AI / SERVING** (Module 9, bottom) — "Embeddings pipeline | Vector store | Retrieval API"

Each layer should be labeled with the module numbers that build it.]

Module-by-Module Build-Up

Module	What You Build	Capstone Component
1	Data model design	Star schema that everything feeds into
2	SQL transformations	Queries that transform raw data into the model
3	Python ingestion	Scripts that pull data from APIs
4	Airflow orchestration	Scheduling, retries, monitoring
5	Spark processing	Heavy lifting when data gets big
6	Kafka streaming	Real-time event processing
7	Data quality	Checks so it doesn't silently break
8	Cloud deployment	Running it all in the cloud
9	AI components	Embeddings and a vector store
10	Portfolio polish	A GitHub repo that gets you hired

By the end, you'll have a GitHub repo that makes hiring managers say "okay, this person knows what they're doing." That's the whole point.

The Sample Data

- **~1M orders** over 2 years
- **~50k customers**
- **~10k products** across 20 categories
- **Event stream:** page views, add-to-cart, purchases

It's fake data, but it's *realistic* fake data. Same distributions, same messiness you'd see in a real company.

□ Pro Tip

As you work through each module, commit your work to a personal GitHub repo. By Module 10, you'll already have a rich commit history showing your progression — interviewers love seeing that.

Module 0 Project: Environment Verification

Objective

Verify your entire development environment is working correctly before moving on.

Prerequisites

- All steps from Section 0.4 completed
- Docker stack running
- Python venv activated

Step 1: Docker Stack Is Running

```
docker compose ps
```

Expected output: All services show "Up" or "running".

Step 2: PostgreSQL Is Accessible

```
psql -h localhost -U dataeng -d warehouse -c "  
    CREATE TABLE test_setup (  
        id SERIAL PRIMARY KEY,  
        message TEXT,  
        created_at TIMESTAMP DEFAULT NOW()  
    );  
    INSERT INTO test_setup (message) VALUES ('Environment is working!');  
    SELECT * FROM test_setup;  
    DROP TABLE test_setup;  
"
```

Expected output:

```
CREATE TABLE  
INSERT 0 1  
 id |          message          |          created_at  
----+-----  
  1 | Environment is working! | 2026-02-19 10:00:00.000000  
(1 row)  
DROP TABLE
```

Step 3: Python Environment Works

Create and run `test_setup.py`:

```
# test_setup.py
import psycopg2
import pandas as pd
import polars as pl
import duckdb
import pyarrow
from pydantic import BaseModel

# Print library versions
print(f"pandas: {pd.__version__}")
print(f"polars: {pl.__version__}")
print(f"duckdb: {duckdb.__version__}")
print(f"pyarrow: {pyarrow.__version__}")

# Test database connection
conn = psycopg2.connect(
    host="localhost",
    database="warehouse",
    user="dataeng",
    password="dataeng"
)
cur = conn.cursor()
cur.execute("SELECT version();")
print(f"PostgreSQL: {cur.fetchone()[0]}")
conn.close()

print("\n👉 All good! You're ready for the course.")
```

```
python test_setup.py
```

Expected output:

```
pandas: 2.2.2
polars: 0.20.25
duckdb: 0.10.3
pyarrow: 16.1.0
PostgreSQL: PostgreSQL 16.x ...

👉 All good! You're ready for the course.
```

Step 4: Airflow UI Is Accessible

1. Navigate to <http://localhost:8080>
2. Log in with **admin / admin**
3. You should see the Airflow dashboard with no errors

Step 5: MinIO Is Accessible

1. Navigate to **http://localhost:9001**
2. Log in with **minioadmin / minioadmin**
3. Create a bucket called `raw-data` (you'll use this later)

Verification Complete

All checks pass, all UIs load. If anything fails, check the troubleshooting section in the course repo README or post in the Discord.

❏ Pro Tip

Take a screenshot of your working environment. If something breaks later, you'll have a reference point to compare against.

Module 1: Data Modeling Foundations

1.1 Why Data Modeling Matters

TL;DR

- Data modeling is the foundation everything else sits on — bad models mean bad data, no matter how good your pipelines are.
- It's the #1 interview topic for data engineering roles.
- Good modeling = clean queries, single source of truth, fast performance. Bad modeling = `orders_final_FINAL_v2`.

Here's a story that'll sound familiar if you've worked with data. A company I consulted for had 400 tables in their warehouse. Nobody knew which ones were "right." Analysts had six different revenue numbers depending on which table they used. The CEO would ask "what's our revenue?" and get a different answer from every team.

That's what happens when you skip data modeling.

Why It Matters

- Data modeling is the foundation **everything** else sits on. You can have the best pipeline in the world, but if your model sucks, your data is useless.
- It's the **#1 interview topic**. Modeling comes up in almost every DE interview.
- It's the difference between a query that takes 2 seconds and one that takes 20 minutes.

□Key Concept

Data modeling is like an architect's blueprint. You wouldn't build a house by throwing up walls and hoping for the best. You'd design it first — where do the rooms go, how do the pipes connect. Data modeling is the same thing: you design **WHERE** your data lives and **HOW** it connects before you write a single pipeline.

What Bad Modeling Looks Like

```
-- This is what happens without modeling.
-- Someone dumped the production database into the warehouse.
SELECT
    o.id,
    o.customer_id,
    c.name,
    c.email,
    p.product_name,
    oi.quantity,
    oi.price,
    -- Wait, is this the unit price or total price?
    -- Is it pre-tax or post-tax?
    -- Does this include discounts?
    oi.price * oi.quantity as total -- maybe??
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
-- Which orders table? There are three...
-- orders, orders_v2, orders_final_FINAL
```

If you've seen tables named `orders_final_FINAL_v2`, you know the pain.

What Good Modeling Looks Like

```
-- Clean dimensional model. Any analyst can use this.
SELECT
    d.calendar_date,
    p.product_category,
    SUM(f.revenue) as total_revenue,
    COUNT(DISTINCT f.customer_key) as unique_customers
FROM fact_orders f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_product p ON f.product_key = p.product_key
WHERE d.calendar_year = 2026
GROUP BY 1, 2
ORDER BY total_revenue DESC;
```

Anyone can read it. There's one source of truth for revenue. That's the power of modeling.

⚠ Common Mistake

Don't confuse **data modeling** with **database design**. Database design is about the physical implementation — indexes, partitions, storage. Data modeling is the logical design — entities, relationships, how things connect. We'll cover both, but modeling comes first.

□Checkpoint

1. Why can't you just copy production tables into your warehouse and call it done?
 2. What's the difference between data modeling and database design?
 3. What problem does a "single source of truth" solve?
-

1.2 OLTP vs. OLAP

TL;DR

- OLTP databases run your app (fast, small queries). OLAP databases power your analytics (complex, large scans).
- Your job as a DE is to move data FROM OLTP TO OLAP, transforming it along the way.
- Postgres can serve as both for small-to-medium companies — don't assume you need Snowflake.

"Why can't we just run our analytics queries on the production database?" Every new data engineer asks this. And the answer is: you can! Until your app grinds to a halt because an analyst's `GROUP BY` locked the orders table for 30 seconds.

OLTP — Online Transaction Processing

This is your application database. The thing behind your web app.

- **Optimized for:** lots of small, fast reads and writes
- **Query pattern:** INSERT one row, UPDATE one row, SELECT one row by ID
- **Data shape:** Normalized (3NF) — minimal data duplication
- **Examples:** PostgreSQL, MySQL — the database behind every web app

```
-- Typical OLTP query — fast, targeted, one row
SELECT name, email, address
FROM customers
WHERE id = 42;

-- Or an insert
INSERT INTO orders (customer_id, product_id, quantity, price)
VALUES (42, 101, 2, 29.99);
```

These queries touch maybe one row. They complete in milliseconds. That's what OLTP is optimized for.

OLAP — Online Analytical Processing

This is your analytics database / data warehouse.

- **Optimized for:** reading and aggregating large volumes of data

- **Query pattern:** scan millions of rows, GROUP BY, SUM, AVG, COUNT
- **Data shape:** Denormalized — we duplicate data on purpose to avoid JOINS
- **Examples:** Snowflake, BigQuery, Redshift, ClickHouse, DuckDB

```
-- Typical OLAP query – scans millions of rows
SELECT
    DATE_TRUNC('month', order_date) as month,
    product_category,
    SUM(revenue) as total_revenue,
    COUNT(*) as order_count,
    AVG(revenue) as avg_order_value
FROM fact_orders
JOIN dim_product USING (product_key)
WHERE order_date >= '2025-01-01'
GROUP BY 1, 2
ORDER BY 1, 2;
```

This query might scan 10 million rows. In an OLTP database, it'd take forever and block other queries. In an OLAP database, it's 2 seconds.

Key Differences

	OLTP	OLAP
Purpose	Run the app	Analyze the data
Queries	Simple, targeted	Complex, aggregated
Data shape	Normalized (3NF)	Denormalized (star/snowflake)
Rows per query	1–100	Millions
Users	App servers	Analysts, dashboards
Optimize for	Write speed, consistency	Read speed, scan throughput

□Key Concept

Your job as a data engineer is to move data **FROM** OLTP systems **TO** OLAP systems, transforming it along the way. That's literally what a data pipeline does. The OLTP database is the source. The OLAP database is the destination. And the data model defines what the destination looks like.

The Difference in Practice

The same business question answered from normalized vs. denormalized schemas:

```
-- Normalized (OLTP style) — 4 JOINS to answer a simple question
SELECT
    c.name,
    cat.category_name,
    SUM(oi.quantity * oi.unit_price) as total_spent
FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
JOIN categories cat ON p.category_id = cat.id
GROUP BY c.name, cat.category_name;

-- Denormalized (OLAP style) — same answer, way simpler
SELECT
    customer_name,
    product_category,
    SUM(order_revenue) as total_spent
FROM fact_orders
GROUP BY customer_name, product_category;
```

Same answer. Way less complexity. Way faster at scale. That's why we model data differently for analytics.

❏ Pro Tip

Postgres can be BOTH OLTP and OLAP for small-to-medium companies. Don't assume you need Snowflake. I've seen companies waste \$50k/month on Snowflake when Postgres would've been fine. We'll use Postgres in this course — the concepts transfer to any warehouse.

❏ Checkpoint

1. Why would running a heavy analytical query on a production OLTP database be a problem?
2. What does "denormalized" mean, and why is it beneficial for analytics?
3. What kind of company might be fine using Postgres for both OLTP and OLAP?

1.3 Dimensional Modeling — Facts, Dimensions, Star Schemas

TL;DR

- **Fact tables** record business events (orders, clicks, payments) — they contain foreign keys and numeric measures.
- **Dimension tables** provide context (who, what, where, when) — they contain descriptive attributes.
- A **star schema** places the fact table at the center with dimensions radiating outward — intuitive, fast, and flexible.
- Always use surrogate keys, pre-calculate measures, and build a date dimension.

Dimensional modeling was invented by Ralph Kimball in the 1990s. People have been predicting its death every year since. And yet, it's still the most common approach in data warehouses everywhere. Why? Because it works. It's intuitive, it's fast, and analysts can actually use it without a PhD.

The Two Building Blocks

FACT TABLES — "WHAT HAPPENED"

Facts record business events or measurements. Each row equals one event: an order, a click, a payment, a login.

They contain:

- **Foreign keys** to dimension tables
- **Numeric measures** (revenue, quantity, cost)

Think of a fact table like a receipt. It says WHAT happened, WHEN, and HOW MUCH. But it doesn't tell you everything about the customer or the product — that's in the dimensions.

Types of fact tables:

Type	Description	Example
Transaction	One row per event	Order placed, payment made
Snapshot	State at a point in time	Daily account balance, daily inventory count
Accumulating	Tracks a process through milestones	Order → shipped → delivered (one row, multiple dates)

DIMENSION TABLES — "THE CONTEXT"

Dimensions describe the WHO, WHAT, WHERE, and WHEN. Each row equals one entity: a customer, a product, a store, a date.

They contain descriptive attributes, categories, and hierarchies. They're usually small-to-medium (thousands to low millions of rows).

If the fact table is the receipt, dimensions are the product catalog, the customer profile, and the calendar.

The Star Schema

[DIAGRAM: A classic star schema diagram with `fact_orders` in the center. Three dimension tables radiate outward from it: `dim_customer` (top), `dim_product` (left), and `dim_date` (right). Lines connect each dimension's primary key to the corresponding foreign key in the fact table. The fact table shows columns: `order_key`, `customer_key`, `product_key`, `date_key`, `quantity`, `unit_price`, `discount`, `revenue`, `cost`, `profit`. Each dimension shows its key column plus 4-5 descriptive attributes.]

The fact table is in the center, dimensions radiate out like points of a star. That's why it's called a star schema.

Why it works:

1. **Intuitive** — Analysts can look at it and immediately understand it.
2. **Fast** — Queries only JOIN what they need. No chain of 7 JOINS.
3. **Flexible** — Want to slice revenue by category AND month? Just join both dimensions.

Building It in SQL

```

-- =====
-- DIMENSION: Date
-- Build this once. Never do date math in your
-- fact queries – the date dimension handles it.
-- =====
CREATE TABLE dim_date (
    date_key          INT PRIMARY KEY,          -- YYYYMMDD format (e.g., 20260115)
    calendar_date     DATE NOT NULL UNIQUE,
    day_of_week       VARCHAR(10) NOT NULL,     -- Monday, Tuesday, etc.
    day_of_month       INT NOT NULL,
    month             INT NOT NULL,
    month_name        VARCHAR(10) NOT NULL,
    quarter           INT NOT NULL,
    year              INT NOT NULL,
    is_weekend        BOOLEAN NOT NULL,
    is_holiday        BOOLEAN NOT NULL DEFAULT FALSE
);

-- =====
-- DIMENSION: Customer
-- Uses SCD Type 2 for tracking changes over time
-- (covered in detail in section 1.4)
-- =====
CREATE TABLE dim_customer (
    customer_key      SERIAL PRIMARY KEY,       -- surrogate key (auto-generated)
    customer_id       INT NOT NULL,            -- natural key from source system
    name              VARCHAR(200) NOT NULL,
    email             VARCHAR(200),
    city              VARCHAR(100),
    state             VARCHAR(50),
    country           VARCHAR(50) DEFAULT 'US',
    segment           VARCHAR(50),             -- Enterprise, SMB, Consumer
    created_date      DATE,
    -- SCD Type 2 metadata
    effective_date    DATE NOT NULL,
    expiry_date       DATE NOT NULL DEFAULT '9999-12-31',
    is_current        BOOLEAN NOT NULL DEFAULT TRUE
);

-- =====
-- DIMENSION: Product
-- =====
CREATE TABLE dim_product (
    product_key       SERIAL PRIMARY KEY,
    product_id        INT NOT NULL,            -- natural key from source system
    product_name      VARCHAR(300) NOT NULL,
    category          VARCHAR(100),
    subcategory       VARCHAR(100),
    brand             VARCHAR(100),
    unit_cost         NUMERIC(10,2),
    unit_price        NUMERIC(10,2),
    effective_date    DATE NOT NULL,
    expiry_date       DATE NOT NULL DEFAULT '9999-12-31',

```



```

        is_current      BOOLEAN NOT NULL DEFAULT TRUE
    );

    -- =====
    -- FACT: Orders
    -- Grain: one row per order line item
    -- =====
    CREATE TABLE fact_orders (
        order_key        SERIAL PRIMARY KEY,
        order_id         INT NOT NULL,           -- natural key from source
        customer_key     INT NOT NULL REFERENCES dim_customer(customer_key),
        product_key      INT NOT NULL REFERENCES dim_product(product_key),
        date_key         INT NOT NULL REFERENCES dim_date(date_key),
        quantity         INT NOT NULL,
        unit_price       NUMERIC(10,2) NOT NULL,
        discount_amount  NUMERIC(10,2) DEFAULT 0,
        revenue         NUMERIC(12,2) NOT NULL,  -- quantity * unit_price - discount
        cost             NUMERIC(12,2),         -- quantity * unit_cost
        profit           NUMERIC(12,2),         -- revenue - cost
        order_status     VARCHAR(20)           -- completed, returned, cancelled
    );

    -- Indexes for common query patterns
    CREATE INDEX idx_fact_orders_date ON fact_orders(date_key);
    CREATE INDEX idx_fact_orders_customer ON fact_orders(customer_key);
    CREATE INDEX idx_fact_orders_product ON fact_orders(product_key);

```

Design Decisions Worth Highlighting

□ Key Concept: Surrogate Keys vs. Natural Keys

`customer_key` is our **surrogate key** (auto-generated, meaningless integer). `customer_id` is the **natural key** from the source system. Always use surrogate keys in your warehouse — natural keys change, break, and can't handle SCD Type 2 (more on this in the next section).

□ Key Concept: Pre-calculated Measures

`revenue`, `cost`, and `profit` are calculated at load time, not query time. This saves analysts from doing math in every query and ensures consistency — everyone uses the same revenue formula.

□ Key Concept: Date Dimension

Always build a date dimension. Never do date math in your fact queries. The date dimension gives you `is_holiday`, `is_weekend`, fiscal quarters — stuff you can't derive from a raw date without business logic.

⚠ Common Mistake

Don't put high-cardinality text fields in fact tables. If you have 10 million orders, don't put `customer_name` in the fact table — that's what the dimension is for. Facts should be lean: keys and numbers.

Star Schema vs. Snowflake Schema

A **snowflake schema** is a star schema where the dimensions are further normalized. Instead of `dim_product` having a `category` column, you'd have a separate `dim_category` table.

It saves storage but adds JOINS. In 2026, storage is cheap and JOINS are expensive — **stick with star schemas** unless you have a specific reason to snowflake.

□ Checkpoint

1. What's the difference between a fact table and a dimension table?
2. Why should you use surrogate keys instead of natural keys in a fact table?
3. What are the three types of fact tables, and when would you use each?

□ Try It Yourself

Take the star schema above and add a `dim_geography` dimension. What columns would it have? How would it connect to the fact table? Would you keep `city` and `state` in `dim_customer` or move them to the new dimension?

1.4 Slowly Changing Dimensions (SCD Types 1, 2, 3)

TL;DR

- **SCD Type 1:** Overwrite the old value. Simple but you lose history.
- **SCD Type 2:** Add a new row with effective/expiry dates. Preserves full history. This is the default choice.
- **SCD Type 3:** Add a column for the previous value. Limited history (one level only).
- Most warehouses use a mix: Type 2 for important attributes, Type 1 for unimportant ones.

A customer signs up in New York. Six months later, they move to LA. Simple question: when you look at their orders from before the move, should the city show New York or LA?

The answer depends on what the business needs — and it's a question you'll face in literally every data warehouse project.

The Problem

Dimensions change over time. Customers move, products get recategorized, employees change departments. How do you handle this?

SCD Type 0: Retain Original

Don't change anything. Whatever was there first, stays forever. Useful for things that shouldn't change — like a customer's original signup date. Rare in practice.

SCD Type 1: Overwrite

The simplest approach. Just update the value. No history preserved.

```
-- Customer moved from New York to LA
-- Type 1: Just update the row
UPDATE dim_customer
SET city = 'Los Angeles', state = 'CA'
WHERE customer_id = 42 AND is_current = TRUE;
```

Before:

customer_key	customer_id	name	city	state
1	42	Jane Smith	New York	NY

After:

customer_key	customer_id	name	city	state
1	42	Jane Smith	Los Angeles	CA

Pros: Simple.

Cons: You lose history. All of Jane's old orders now look like they came from LA. If your analyst asks "what was our NYC revenue last quarter?" — it'll be wrong.

When to use: Corrections (fixing a typo), attributes that don't matter historically (email address updates), or when the business truly doesn't care about history.

SCD Type 2: Add New Row (Full History)

The gold standard. Create a new row for each change. Track effective dates.

```

-- Customer moved from New York to LA
-- Type 2: Expire old row, insert new row

-- Step 1: Expire the current row
UPDATE dim_customer
SET expiry_date = '2026-02-18',
    is_current = FALSE
WHERE customer_id = 42 AND is_current = TRUE;

-- Step 2: Insert new row with updated values
INSERT INTO dim_customer (
    customer_id, name, email, city, state, segment,
    created_date, effective_date, expiry_date, is_current
)
VALUES (
    42, 'Jane Smith', 'jane@email.com', 'Los Angeles', 'CA', 'Consumer',
    '2025-06-15', '2026-02-19', '9999-12-31', TRUE
);

```

Before:

customer_key	customer_id	name	city	state	effective_date	expiry_date	is_current
1	42	Jane Smith	New York	NY	2025-06-15	9999-12-31	TRUE

After:

customer_key	customer_id	name	city	state	effective_date	expiry_date	is_current
1	42	Jane Smith	New York	NY	2025-06-15	2026-02-18	FALSE
2	42	Jane Smith	Los Angeles	CA	2026-02-19	9999-12-31	TRUE

Now Jane's old orders (linked to `customer_key=1`) still show New York. New orders (linked to `customer_key=2`) show LA. Full history preserved.

Pro Tip

The `9999-12-31` expiry date is a common convention meaning "still active." Some people use NULL, but a far-future date makes range queries easier: `WHERE order_date BETWEEN effective_date AND expiry_date`. No need for COALESCE gymnastics.

When to use: Almost always. Any time history matters. This is the default choice.

SCD Type 3: Add New Column

Keep the current and previous value in separate columns.

```
ALTER TABLE dim_customer ADD COLUMN previous_city VARCHAR(100);
ALTER TABLE dim_customer ADD COLUMN previous_state VARCHAR(50);

UPDATE dim_customer
SET previous_city = city,
    previous_state = state,
    city = 'Los Angeles',
    state = 'CA'
WHERE customer_id = 42;
```

customer_key	name	city	state	previous_city	previous_state
1	Jane Smith	Los Angeles	CA	New York	NY

Pros: Simple, no row explosion.

Cons: You only get one level of history. If Jane moves to Chicago next, you lose the New York data.

When to use: When you only care about "before and after" for a specific known change — like a company reorg where you want `current_department` and `previous_department`. Rare in practice.

In the Real World

Most warehouses use a mix. Type 2 for important attributes (customer location, product category) and Type 1 for unimportant ones (email, phone). You define this **per attribute**, not per table.

⚠ Common Mistake

Type 2 can explode your dimension table if you're not careful. If a customer updates their email 50 times, you get 50 rows. Ask yourself: does email really need history? Probably not — use Type 1 for that attribute, even if the same table uses Type 2 for location.

☐ Checkpoint

1. When would you choose SCD Type 1 over Type 2?
2. What does the `9999-12-31` expiry date convention signify?
3. Why does SCD Type 2 require surrogate keys?

☐ Try It Yourself

A product changes from category "Electronics" to "Smart Home". Write the SQL for both a Type 1 and a Type 2 update on `dim_product`. Which approach preserves historical revenue-by-category accuracy?

1.5 Modern Approaches — OBT, Wide Event Tables

TL;DR

- **One Big Table (OBT):** Denormalize everything into one massive table. No JOINS needed. Great for small-to-medium data and columnar warehouses.
- **Wide Event Tables:** Log everything as events with a flexible JSONB column. Great for product analytics and evolving schemas.
- In practice, most companies use event tables for raw ingestion and star schemas (or OBTs) for reporting.

Here's a hot take: not everything needs a star schema. Star schemas are great. But modern tools have changed the calculus.

One Big Table (OBT)

The idea is simple: denormalize everything into one massive table. No JOINS needed.

```
-- Pre-join everything into one table
CREATE TABLE obt_orders AS
SELECT
    o.order_id,
    o.order_date,
    o.quantity,
    o.revenue,
    o.cost,
    o.profit,
    o.order_status,
    -- Customer attributes (denormalized in)
    c.name as customer_name,
    c.email as customer_email,
    c.city as customer_city,
    c.state as customer_state,
    c.segment as customer_segment,
    -- Product attributes
    p.product_name,
    p.category as product_category,
    p.subcategory as product_subcategory,
    p.brand as product_brand,
    -- Date attributes
    d.day_of_week,
    d.month_name,
    d.quarter,
    d.year,
    d.is_weekend,
    d.is_holiday
FROM fact_orders o
JOIN dim_customer c ON o.customer_key = c.customer_key
JOIN dim_product p ON o.product_key = p.product_key
JOIN dim_date d ON o.date_key = d.date_key;
```

Now analysts can do everything with a single `FROM` clause:

```
SELECT
  product_category,
  customer_segment,
  year,
  quarter,
  SUM(revenue) as total_revenue
FROM obt_orders
WHERE year = 2026
GROUP BY 1, 2, 3, 4;
```

When OBT makes sense:

- Small-to-medium data (< 100M rows)
- Your warehouse is columnar (BigQuery, Snowflake, ClickHouse) — repeated values compress well
- Analysts are less SQL-savvy — JOINS confuse them
- Dead-simple dashboards — one table per dashboard
- DuckDB or Parquet file analytics

When OBT doesn't make sense:

- Huge dimensions that change frequently (Type 2 SCDs are messy in an OBT)
- Multiple fact tables sharing the same dimensions (you'd duplicate dimension data everywhere)
- Strict consistency requirements — if `customer_name` is in 10 OBTs, updating it means updating 10 tables

Wide Event Tables

Popular at tech companies and in product analytics. Instead of modeling into facts and dimensions upfront, you log everything as events.

```

CREATE TABLE events (
  event_id      UUID DEFAULT gen_random_uuid(),
  event_timestamp TIMESTAMPTZ NOT NULL,
  event_type    VARCHAR(100) NOT NULL,      -- page_view, add_to_cart, purchase
  user_id       INT,
  session_id    UUID,
  -- Semi-structured properties – the secret weapon
  properties    JSONB,                      -- flexible key-value pairs
  -- Common dimensions pulled out for query performance
  platform      VARCHAR(20),                -- web, ios, android
  country       VARCHAR(2),
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Example events
INSERT INTO events (event_timestamp, event_type, user_id, session_id,
  properties, platform, country)
VALUES
  (NOW(), 'page_view', 42, 'abc-123',
   '{"page": "/products/shoes", "referrer": "google.com"}', 'web', 'US'),
  (NOW(), 'add_to_cart', 42, 'abc-123',
   '{"product_id": 101, "price": 79.99}', 'web', 'US'),
  (NOW(), 'purchase', 42, 'abc-123',
   '{"order_id": 5001, "total": 79.99, "items": 1}', 'web', 'US');

```

The `JSONB` column means you don't need to `ALTER TABLE` every time the product team adds a new event property.

```

-- Query: conversion funnel
SELECT
  event_type,
  COUNT(*) as event_count,
  COUNT(DISTINCT user_id) as unique_users
FROM events
WHERE event_timestamp >= NOW() - INTERVAL '7 days'
  AND event_type IN ('page_view', 'add_to_cart', 'purchase')
GROUP BY event_type;

```

The Practical Approach: Use Both

In most companies, you use event tables for raw data ingestion (flexible, fast to land) and then model into star schemas or OBTs for reporting:

Raw events → Staging → Dimensional model

That's the pattern.

❑ Pro Tip

Start with a star schema. If analysts complain about JOINS, create OBT views on top of it. You get the best of both worlds: clean modeling underneath, simple queries on top.

❑ Checkpoint

1. What's the main tradeoff of an OBT compared to a star schema?
2. Why is the JSONB column useful in event tables?
3. When would you choose a star schema over an OBT?

1.6 Data Vault Basics

TL;DR

- Data Vault is an enterprise modeling methodology built on three concepts: Hubs (business keys), Links (relationships), and Satellites (descriptive attributes + history).
- It's designed for auditability, flexibility, and handling many source systems.
- You probably won't build one early in your career, but you should know it exists because it comes up in interviews.

I debated whether to include Data Vault in this course. It's not something you'll build at a startup. But it comes up in interviews, and if you work at a large enterprise, you'll encounter it.

What Is Data Vault?

A modeling methodology for enterprise data warehouses. Invented by Dan Linstedt, formalized as Data Vault 2.0. It's designed for auditability, flexibility, and handling many source systems. Used at banks, insurance companies, government agencies, and large enterprises.

Three Building Blocks

1. **Hubs** — Business keys. The core identity of an entity.

```
CREATE TABLE hub_customer (  
    hub_customer_hash CHAR(32) PRIMARY KEY, -- MD5 hash of business key  
    customer_bk       VARCHAR(50) NOT NULL,  -- business key (natural key)  
    load_date         TIMESTAMP NOT NULL,     -- when we first saw this key  
    record_source      VARCHAR(100) NOT NULL  -- which system it came from  
);
```

A hub just says "this customer exists." That's it. No attributes.

2. **Links** — Relationships between hubs.

```
CREATE TABLE link_order (
  link_order_hash    CHAR(32) PRIMARY KEY,
  hub_customer_hash  CHAR(32) NOT NULL,      -- FK to hub_customer
  hub_product_hash   CHAR(32) NOT NULL,      -- FK to hub_product
  load_date          TIMESTAMP NOT NULL,
  record_source       VARCHAR(100) NOT NULL
);
```

3. Satellites — Descriptive attributes + history (like SCD Type 2).

```
CREATE TABLE sat_customer_details (
  hub_customer_hash  CHAR(32) NOT NULL,
  load_date          TIMESTAMP NOT NULL,
  name               VARCHAR(200),
  email              VARCHAR(200),
  city               VARCHAR(100),
  state              VARCHAR(50),
  hash_diff          CHAR(32),              -- hash of all attributes for change
  detection
  record_source       VARCHAR(100),
  PRIMARY KEY (hub_customer_hash, load_date)
);
```

[DIAGRAM: A Data Vault diagram for a customer-order-product scenario. Show three Hubs (hub_customer, hub_product, hub_order) as circles. A Link (link_order) connects all three as a diamond shape. Satellites hang off each hub as rectangles: sat_customer_details off hub_customer, sat_product_details off hub_product. Use distinct shapes/colors for Hubs, Links, and Satellites.]

When Data Vault Makes Sense

- Many source systems feeding into one warehouse
- Regulatory requirements for auditability (you need to prove where every data point came from)
- Large teams where modeling needs to scale across many developers

When It Doesn't

- Startups, small companies — way too much overhead
- 1–3 data sources — just use a star schema
- Teams smaller than 5 people

□ Key Concept

Data Vault is a **raw/staging layer** methodology. You still build star schemas on top for analyst consumption. It's an intermediate layer, not a replacement for dimensional modeling.

□ Checkpoint

1. What are the three building blocks of Data Vault?
 2. Why does Data Vault use hash keys instead of auto-increment IDs?
 3. When would you choose Data Vault over a star schema?
-

1.7 Hands-On — Design a Schema for an E-Commerce Company

TL;DR

- Start with the business process (the grain), then identify dimensions, measures, and SCD decisions.
- Getting the grain right is THE most important decision — everything downstream depends on it.
- Don't try to model everything on day one. Start with one fact table and 3–4 dimensions. Ship first, iterate later.

Time to apply everything. We're going to design a data model from scratch for a fictional e-commerce company, walking through the same thinking process you'd use for real clients.

The Scenario

ShopFast is an e-commerce company selling products online. They have a web app backed by PostgreSQL. Here's their source system:

```

-- =====
-- SOURCE SYSTEM (OLTP) – this is what we're working with
-- =====

CREATE TABLE src_customers (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200),
    email VARCHAR(200) UNIQUE,
    address_line1 VARCHAR(200),
    city VARCHAR(100),
    state VARCHAR(50),
    zip VARCHAR(10),
    country VARCHAR(50) DEFAULT 'US',
    segment VARCHAR(50), -- consumer, business, enterprise
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE src_products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(300),
    description TEXT,
    category VARCHAR(100),
    subcategory VARCHAR(100),
    brand VARCHAR(100),
    cost NUMERIC(10,2),
    price NUMERIC(10,2),
    weight_kg NUMERIC(6,2),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE src_orders (
    id SERIAL PRIMARY KEY,
    customer_id INT REFERENCES src_customers(id),
    order_date TIMESTAMP DEFAULT NOW(),
    status VARCHAR(20), -- pending, shipped, delivered, returned, cancelled
    shipping_address VARCHAR(500),
    total_amount NUMERIC(12,2),
    discount_amount NUMERIC(10,2) DEFAULT 0,
    shipping_cost NUMERIC(8,2) DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE src_order_items (
    id SERIAL PRIMARY KEY,
    order_id INT REFERENCES src_orders(id),
    product_id INT REFERENCES src_products(id),
    quantity INT,
    unit_price NUMERIC(10,2),
    discount NUMERIC(10,2) DEFAULT 0,

```

```
created_at TIMESTAMP DEFAULT NOW()  
);
```

Step 1: Identify the Business Process (the Grain)

What's the core event? An order. More specifically, an **order line item** — because one order can have multiple products.

Our fact table grain: one row per order item.

⚠ Common Mistake

Getting the grain wrong is the most expensive mistake in data modeling. If you set the grain at the order level (one row per order), you can't answer "which products are selling best?" without a separate fact table. Order-item level gives you maximum flexibility.

Step 2: Identify the Dimensions

For each order item, what context do we need?

- **WHO** ordered? → `dim_customer`
- **WHAT** was ordered? → `dim_product`
- **WHEN?** → `dim_date`
- **WHERE** was it shipped? → could be `dim_geography` or part of `dim_customer`

Step 3: Identify the Measures

What numbers do we want to aggregate?

- `quantity`
- `revenue` (quantity × unit_price – discount)
- `cost` (quantity × product cost)
- `profit` (revenue – cost)
- `discount_amount`

Step 4: SCD Decisions

Which dimension attributes need history?

Attribute	SCD Type	Reasoning
customer.name	Type 1	Name corrections, not meaningful history
customer.email	Type 1	Don't need email history
customer.city/state	Type 2	Geographic analysis needs historical accuracy
customer.segment	Type 2	Segment changes affect business analysis
product.price	Type 2	Price history matters for revenue analysis
product.category	Type 2	Category changes affect reporting
product.name	Type 1	Usually just corrections

Step 5: Populate the Date Dimension

Build it once and forget about it:

```
-- Generate date dimension for 2020-2030
INSERT INTO dim_date (
    date_key, calendar_date, day_of_week, day_of_month,
    month, month_name, quarter, year, is_weekend, is_holiday
)
SELECT
    TO_CHAR(d, 'YYYYMMDD')::INT as date_key,
    d as calendar_date,
    TO_CHAR(d, 'Day') as day_of_week,
    EXTRACT(DAY FROM d)::INT as day_of_month,
    EXTRACT(MONTH FROM d)::INT as month,
    TO_CHAR(d, 'Month') as month_name,
    EXTRACT(QUARTER FROM d)::INT as quarter,
    EXTRACT(YEAR FROM d)::INT as year,
    EXTRACT(ISODOW FROM d) IN (6, 7) as is_weekend,
    FALSE as is_holiday -- update holidays separately
FROM generate_series('2020-01-01'::date, '2030-12-31'::date, '1 day'::interval) d;
```

Expected output: 4,018 rows inserted (11 years × 365.25 days).

Step 6: Verify with a Sample Query

```
-- Top categories by revenue this year
SELECT
    p.category,
    COUNT(DISTINCT f.order_id) as total_orders,
    SUM(f.revenue) as total_revenue,
    AVG(f.revenue) as avg_item_revenue
FROM fact_orders f
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_date d ON f.date_key = d.date_key
WHERE d.year = 2026
    AND p.is_current = TRUE
GROUP BY p.category
ORDER BY total_revenue DESC
LIMIT 10;
```

❏ Pro Tip

Don't try to model everything on day one. Start with one fact table and 3–4 dimensions. You can always add more. I've seen teams spend 3 months on a "perfect" model and never ship anything. Ship first, iterate.

❏ Checkpoint

1. Why is "one row per order item" a better grain than "one row per order"?
2. Which customer attributes should use SCD Type 2, and why?
3. Why do we pre-calculate `revenue` and `profit` in the fact table instead of computing them at query time?

Module 1 Project: Design and Implement a Dimensional Model

Objective

Design and implement a complete dimensional model for ShopFast's e-commerce data. Write the DDL, load sample data, document your design decisions, and write analytical queries.

Prerequisites

- Module 0 environment setup complete (Docker stack running, Postgres accessible)
- Familiarity with all concepts from Lessons 1.1–1.7

Expected Time

2–3 hours

Starter Code

`modules/module-1/starter/` contains:

- `source_data.sql` — Source system tables with ~10k sample rows
- `template_ddl.sql` — Empty template for your warehouse tables
- `requirements.md` — Business requirements from the "analytics team"

Step 1: Review the Business Requirements

Read `requirements.md`. The analytics team needs to answer:

1. Monthly revenue by product category
2. Customer lifetime value by segment
3. Return rate by product brand
4. Revenue by geography over time
5. Average order value trends

Step 2: Design Your Schema

- Draw an ER diagram (use draw.io, dbdiagram.io, or pen and paper)
- Identify: fact table(s), dimensions, grain, measures
- Document SCD type for each dimension attribute
- Save your diagram as `schema_design.png`

Step 3: Write the DDL

```
-- Save as warehouse_ddl.sql
-- Include:
-- 1. dim_date (with generated dates)
-- 2. dim_customer (with SCD Type 2 columns)
-- 3. dim_product (with SCD Type 2 columns)
-- 4. fact_orders (at order-item grain)
-- 5. Appropriate indexes
-- 6. Comments explaining design decisions
```

Step 4: Write the ETL Load Scripts

```
-- Save as initial_load.sql
-- Transform source data into your dimensional model
-- Handle:
-- 1. Surrogate key generation
-- 2. Date dimension population
-- 3. Initial dimension loads (all records are "current")
-- 4. Fact table load with lookups to dimension surrogate keys
```


Step 5: Write Analytical Queries

```
-- Save as analytical_queries.sql
-- Write queries that answer the 5 business requirements
-- Each query should have a comment explaining what it answers
```

Step 6: Document Your Decisions

Create `DESIGN_DECISIONS.md` explaining:

- Why you chose the grain you did
- Which SCD types you chose and why
- Any tradeoffs you made
- What you'd do differently at larger scale

Expected Deliverables

File	Description
<code>schema_design.png</code>	ER diagram
<code>warehouse_ddl.sql</code>	Table definitions
<code>initial_load.sql</code>	ETL scripts
<code>analytical_queries.sql</code>	5+ analytical queries
<code>DESIGN_DECISIONS.md</code>	Written justification

Evaluation Criteria

- ☐ Correct grain (order-item level)
- ☐ Proper surrogate keys (not using natural keys in fact table)
- ☐ SCD Type 2 implemented for at least customer location and product price
- ☐ Date dimension populated correctly
- ☐ Pre-calculated measures in fact table
- ☐ Queries actually run and return correct results
- ☐ Design decisions are justified, not just stated

Solution

Check `modules/module-1/solution/` after completing your own attempt. There's no single right answer — but there are wrong ones. Compare your approach.

Module 2: SQL for Data Engineers

2.1 CTEs and Recursive CTEs for Pipeline Logic

TL;DR

- CTEs (Common Table Expressions) replace nested subqueries with named, readable steps.
- Chain CTEs to create pipeline-style SQL: staging → dedup → enrich → validate → output.
- Recursive CTEs traverse hierarchies (org charts, category trees). Always add a depth limit to prevent infinite loops.

If you're still writing nested subqueries five levels deep, it's time to stop. CTEs will change how you write SQL — they make it readable, debuggable, and maintainable. In data engineering, you'll use them in literally every transformation.

CTEs — Common Table Expressions

Think of a CTE as a named temporary result set. It's like defining a variable in Python, but for SQL.

```
-- WITHOUT CTEs: nested subquery hell
SELECT category, avg_revenue
FROM (
    SELECT
        p.category,
        AVG(sub.customer_revenue) as avg_revenue
    FROM (
        SELECT
            f.customer_key,
            f.product_key,
            SUM(f.revenue) as customer_revenue
        FROM fact_orders f
        WHERE f.date_key >= 20260101
        GROUP BY f.customer_key, f.product_key
    ) sub
    JOIN dim_product p ON sub.product_key = p.product_key
    GROUP BY p.category
) final
WHERE avg_revenue > 100;
```

Now the same thing with CTEs:

```

-- WITH CTEs: same logic, actually readable
WITH customer_product_revenue AS (
  -- Step 1: Revenue per customer per product
  SELECT
    customer_key,
    product_key,
    SUM(revenue) as total_revenue
  FROM fact_orders
  WHERE date_key >= 20260101
  GROUP BY customer_key, product_key
),

category_avg AS (
  -- Step 2: Average revenue by category
  SELECT
    p.category,
    AVG(cpr.total_revenue) as avg_revenue
  FROM customer_product_revenue cpr
  JOIN dim_product p ON cpr.product_key = p.product_key
  GROUP BY p.category
)

-- Step 3: Filter to high-value categories
SELECT category, avg_revenue
FROM category_avg
WHERE avg_revenue > 100
ORDER BY avg_revenue DESC;

```

Same result. But now you can read it top to bottom like a story. And you can test each CTE independently by replacing the final `SELECT` with `SELECT * FROM customer_product_revenue LIMIT 10`.

The Pipeline Pattern

This is how most data engineering SQL looks. Each CTE is a transformation step:

```

WITH
-- Step 1: Get raw data
raw_orders AS (
    SELECT * FROM staging.orders
    WHERE loaded_at >= CURRENT_DATE - INTERVAL '1 day'
),

-- Step 2: Deduplicate (keep most recent version of each order)
deduped AS (
    SELECT DISTINCT ON (order_id) *
    FROM raw_orders
    ORDER BY order_id, loaded_at DESC
),

-- Step 3: Enrich with business logic
enriched AS (
    SELECT
        *,
        quantity * unit_price as gross_revenue,
        quantity * unit_price * (1 - discount_pct) as net_revenue,
        CASE
            WHEN total_amount > 500 THEN 'high_value'
            WHEN total_amount > 100 THEN 'medium_value'
            ELSE 'low_value'
        END as order_tier
    FROM deduped
),

-- Step 4: Validate (filter out bad records)
validated AS (
    SELECT *
    FROM enriched
    WHERE net_revenue > 0
        AND customer_id IS NOT NULL
        AND order_date IS NOT NULL
)

-- Step 5: Final output
SELECT * FROM validated;

```

□ Key Concept

The CTE pipeline pattern: Staging → Dedup → Enrich → Validate → Output. This pattern appears in virtually every data engineering SQL transformation. Learn it once, use it everywhere.

Recursive CTEs

Recursive CTEs can traverse hierarchies — org charts, category trees, bill of materials.

```

-- Product category hierarchy
CREATE TABLE categories (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    parent_id INT REFERENCES categories(id)
);

INSERT INTO categories VALUES
    (1, 'All Products', NULL),
    (2, 'Electronics', 1),
    (3, 'Clothing', 1),
    (4, 'Computers', 2),
    (5, 'Phones', 2),
    (6, 'Laptops', 4),
    (7, 'Desktops', 4),
    (8, 'Men's', 3),
    (9, 'Women's', 3);

-- Recursive CTE: Build full path for each category
WITH RECURSIVE category_tree AS (
    -- Base case: top-level categories (no parent)
    SELECT
        id,
        name,
        parent_id,
        name::TEXT as full_path,
        1 as depth
    FROM categories
    WHERE parent_id IS NULL

    UNION ALL

    -- Recursive case: join children to their parents
    SELECT
        c.id,
        c.name,
        c.parent_id,
        ct.full_path || ' > ' || c.name as full_path,
        ct.depth + 1
    FROM categories c
    JOIN category_tree ct ON c.parent_id = ct.id
    WHERE ct.depth < 10 -- safety limit!
)
SELECT * FROM category_tree ORDER BY full_path;

```

Expected output:

id	name	full_path	depth
1	All Products	All Products	1
3	Clothing	All Products > Clothing	2
8	Men's	All Products > Clothing > Men's	3
9	Women's	All Products > Clothing > Women's	3
2	Electronics	All Products > Electronics	2
4	Computers	All Products > Electronics > Computers	3
7	Desktops	All Products > Electronics > Computers > Desktops	4
6	Laptops	All Products > Electronics > Computers > Laptops	4
5	Phones	All Products > Electronics > Phones	3

⚠ Common Mistake

Recursive CTEs can loop forever if your data has cycles ($A \rightarrow B \rightarrow A$). Always add a depth limit: `WHERE ct.depth < 10` in the recursive part. Your future 3 AM self will thank you.

□ Checkpoint

1. How do you test an individual CTE in a multi-CTE query?
2. What are the two parts of a recursive CTE?
3. In the pipeline pattern, why does deduplication come before enrichment?

□ Try It Yourself

Take the pipeline pattern above and add a Step 2.5 that flags potential fraud: any order where

`total_amount > 5000` AND `customer_id` has been active for less than 7 days. Add a `is_suspicious` boolean column.

2.2 Window Functions Deep Dive

TL;DR

- Window functions perform calculations across related rows **without** collapsing them like GROUP BY.
- **ROW_NUMBER** is the deduplication workhorse. **LAG/LEAD** compare rows to their neighbors. **Running aggregations** compute cumulative sums and moving averages.
- The **WINDOW** clause lets you define a window once and reuse it across multiple function calls.

Window functions are the single biggest SQL skill gap in interviews. People either don't know them at all, or they know ROW_NUMBER and nothing else. By the end of this section, you'll know 90% of what you need.

What's a Window Function?

A window function performs a calculation across a set of rows that are somehow related to the current row — **without collapsing the rows** like GROUP BY does.

```
-- GROUP BY collapses rows: one row per customer
SELECT customer_key, SUM(revenue)
FROM fact_orders
GROUP BY customer_key;

-- Window function KEEPS all rows, adds the calculation as a new column
SELECT
    order_id,
    customer_key,
    revenue,
    SUM(revenue) OVER (PARTITION BY customer_key) as customer_total_revenue
FROM fact_orders;
-- Still one row per order, but now each row knows its customer's total
```

The Syntax

```
function_name() OVER (
    PARTITION BY column    -- which groups to calculate within
    ORDER BY column        -- how to order within each group
    ROWS/RANGE frame       -- which rows to include (optional)
)
```

ROW_NUMBER — The Deduplication Workhorse

The most common use in data engineering: **deduplication**.

```
-- Scenario: Your staging table has duplicate records
-- (same order loaded multiple times due to overlapping extracts)
WITH ranked AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY order_id    -- group by the natural key
            ORDER BY loaded_at DESC  -- most recent first
        ) as rn
    FROM staging.orders
)
SELECT * FROM ranked WHERE rn = 1;
-- Keeps only the most recent version of each order
```

This is THE deduplication technique for data engineering. You'll use it almost daily.

RANK vs. DENSE_RANK vs. ROW_NUMBER

```
-- Given scores: 100, 95, 95, 90
SELECT
    student,
    score,
    ROW_NUMBER() OVER (ORDER BY score DESC) as row_num,    -- 1, 2, 3, 4 (no ties)
    RANK()        OVER (ORDER BY score DESC) as rank,      -- 1, 2, 2, 4 (skips 3)
    DENSE_RANK() OVER (ORDER BY score DESC) as dense_rank  -- 1, 2, 2, 3 (no skip)
FROM scores;
```

Expected output:

student	score	row_num	rank	dense_rank
Alice	100	1	1	1
Bob	95	2	2	2
Carol	95	3	2	2
Dave	90	4	4	3

Use **ROW_NUMBER** for dedup (you want exactly one row). Use **RANK/DENSE_RANK** for actual rankings.

LAG and LEAD — Time-Series Magic

```
-- Revenue growth: compare each month to the previous month
WITH monthly_revenue AS (
    SELECT
        DATE_TRUNC('month', d.calendar_date) as month,
        SUM(f.revenue) as revenue
    FROM fact_orders f
    JOIN dim_date d ON f.date_key = d.date_key
    GROUP BY 1
)
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) as prev_month_revenue,
    revenue - LAG(revenue) OVER (ORDER BY month) as revenue_change,
    ROUND(
        (revenue - LAG(revenue) OVER (ORDER BY month))
        / LAG(revenue) OVER (ORDER BY month) * 100, 2
    ) as growth_pct
FROM monthly_revenue
ORDER BY month;
```

Expected output (sample):

month	revenue	prev_month_revenue	revenue_change	growth_pct
2026-01-01	125000.00			
2026-02-01	132000.00	125000.00	7000.00	5.60
2026-03-01	128000.00	132000.00	-4000.00	-3.03

LAG looks backward, **LEAD** looks forward. The second argument is the offset (default 1), third is the default value:

```
LAG(revenue, 1, 0) OVER (ORDER BY month)
-- 1 row back, default to 0 if no previous row exists
```

Running Aggregations

```
-- Cumulative sum (running total)
SELECT
    d.calendar_date,
    f.revenue,
    SUM(f.revenue) OVER (
        ORDER BY d.calendar_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) as running_total
FROM fact_orders f
JOIN dim_date d ON f.date_key = d.date_key
ORDER BY d.calendar_date;

-- 7-day moving average
SELECT
    order_date,
    daily_revenue,
    AVG(daily_revenue) OVER (
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) as moving_avg_7d
FROM daily_revenue_summary;
```

FIRST_VALUE / LAST_VALUE

```
-- For each customer: first and most recent purchase
SELECT DISTINCT
  customer_key,
  FIRST_VALUE(order_date) OVER w as first_order_date,
  LAST_VALUE(order_date) OVER w as last_order_date,
  FIRST_VALUE(product_key) OVER w as first_product_purchased
FROM fact_orders
WINDOW w AS (
  PARTITION BY customer_key
  ORDER BY order_date
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
);
```

The `WINDOW` clause lets you define a window once and reuse it. Much cleaner than repeating `OVER(...)` three times.

⚠ Common Mistake

`LAST_VALUE` doesn't do what you think by default. The default window frame is

`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — which means `LAST_VALUE` just returns the current row's value. You **must** explicitly set `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to get the actual last value in the partition.

Real-World Pattern: Sessionization

```
-- Group user events into sessions (30-minute gap between events = new session)
WITH events_with_gap AS (
    SELECT
        user_id,
        event_timestamp,
        -- Time since this user's previous event
        event_timestamp - LAG(event_timestamp) OVER (
            PARTITION BY user_id ORDER BY event_timestamp
        ) as time_since_last_event
    FROM events
),
session_starts AS (
    SELECT
        *,
        CASE
            WHEN time_since_last_event IS NULL -- first event ever
            OR time_since_last_event > INTERVAL '30 minutes' -- gap too large
            THEN 1
            ELSE 0
        END as is_new_session
    FROM events_with_gap
)
SELECT
    user_id,
    event_timestamp,
    -- Running sum of session starts = session ID
    SUM(is_new_session) OVER (
        PARTITION BY user_id ORDER BY event_timestamp
    ) as session_id
FROM session_starts;
```

This sessionization pattern is bread and butter for product analytics. If you're working on clickstream data, you'll use this weekly.

□ Checkpoint

1. What's the difference between `ROW_NUMBER`, `RANK`, and `DENSE_RANK`?
2. Why do you need `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` for `LAST_VALUE`?
3. How does the sessionization pattern use `LAG` and running sums together?

□ Try It Yourself

Write a query that calculates each customer's **rank** by total revenue within their segment (e.g., rank 1 = highest spender in the "Enterprise" segment). Use `DENSE_RANK` partitioned by `segment`.

2.3 MERGE/UPSERT Patterns for Incremental Loading

TL;DR

- **INSERT ON CONFLICT** (Postgres UPSERT) handles simple "insert or update" scenarios — perfect for SCD Type 1.
- **SCD Type 2 incremental loads** require a multi-step process: find changes → expire old rows → insert new versions → handle new records.
- **MERGE** (SQL standard, Postgres 15+) is cleaner for simple cases but can't handle the full Type 2 expire-and-insert in one statement.

Reloading your entire warehouse every night is fine when you have 10k rows. When you have 10 billion? Not so much. Incremental loading is how you handle growing data — you only process what's new or changed.

The Problem

Every day, you get new orders AND updates to existing orders (status changes, refunds). You need to:

1. **INSERT** new records
2. **UPDATE** existing records that changed
3. **Leave unchanged records alone**

Pattern 1: INSERT ON CONFLICT (Postgres UPSERT)

```
-- Upsert: insert new products, update existing ones (SCD Type 1)
INSERT INTO dim_product (
    product_id, product_name, category, subcategory,
    brand, unit_cost, unit_price, effective_date
)
SELECT
    id, name, category, subcategory,
    brand, cost, price, CURRENT_DATE
FROM src_products
WHERE updated_at >= CURRENT_DATE - INTERVAL '1 day'

ON CONFLICT (product_id) WHERE is_current = TRUE
DO UPDATE SET
    product_name = EXCLUDED.product_name, -- EXCLUDED = the incoming row
    category     = EXCLUDED.category,
    unit_cost    = EXCLUDED.unit_cost,
    unit_price   = EXCLUDED.unit_price;
```

This is SCD Type 1 in one statement. New products get inserted. Existing products get updated. Simple.

□ **Key Concept**

EXCLUDED refers to the row that **would have been inserted** if there were no conflict. It's how you say "use the new values."

Pattern 2: SCD Type 2 Incremental Load

Type 2 is trickier because you need to expire the old row AND insert a new one.

```

-- Step 1: Find what changed (only SCD Type 2 attributes)
WITH changes AS (
    SELECT
        s.id as product_id,
        s.name, s.category, s.subcategory,
        s.brand, s.cost, s.price
    FROM src_products s
    JOIN dim_product d ON s.id = d.product_id AND d.is_current = TRUE
    WHERE s.updated_at >= CURRENT_DATE - INTERVAL '1 day'
    AND (
        s.category != d.category      -- Track category changes
        OR s.price != d.unit_price    -- Track price changes
    )
),

-- Step 2: Expire old rows
expired AS (
    UPDATE dim_product d
    SET expiry_date = CURRENT_DATE - 1,
        is_current = FALSE
    FROM changes c
    WHERE d.product_id = c.product_id
        AND d.is_current = TRUE
    RETURNING d.product_id
)

-- Step 3: Insert new current rows
INSERT INTO dim_product (
    product_id, product_name, category, subcategory,
    brand, unit_cost, unit_price,
    effective_date, expiry_date, is_current
)
SELECT
    product_id, name, category, subcategory,
    brand, cost, price,
    CURRENT_DATE, '9999-12-31', TRUE
FROM changes;

-- Step 4: Handle Type 1 updates (name changes – just overwrite)
UPDATE dim_product d
SET product_name = s.name
FROM src_products s
WHERE d.product_id = s.id
    AND d.is_current = TRUE
    AND d.product_name != s.name;

-- Step 5: Insert brand new products (never seen before)
INSERT INTO dim_product (
    product_id, product_name, category, subcategory,
    brand, unit_cost, unit_price,
    effective_date, expiry_date, is_current
)
SELECT

```

```

    s.id, s.name, s.category, s.subcategory,
    s.brand, s.cost, s.price,
    CURRENT_DATE, '9999-12-31', TRUE
FROM src_products s
LEFT JOIN dim_product d ON s.id = d.product_id
WHERE d.product_id IS NULL
    AND s.created_at >= CURRENT_DATE - INTERVAL '1 day';

```

This is the full pattern. It handles Type 2 changes, Type 1 updates, and new records. It's a lot of SQL — and this is why tools like dbt exist. But you need to understand the underlying SQL.

Pattern 3: MERGE (SQL Standard, Postgres 15+)

```

MERGE INTO dim_product AS target
USING (
    SELECT id, name, category, subcategory, brand, cost, price
    FROM src_products
    WHERE updated_at >= CURRENT_DATE - INTERVAL '1 day'
) AS source
ON target.product_id = source.id AND target.is_current = TRUE

WHEN MATCHED AND (target.category != source.category
    OR target.unit_price != source.price) THEN
    UPDATE SET
        expiry_date = CURRENT_DATE - 1,
        is_current = FALSE

WHEN NOT MATCHED THEN
    INSERT (product_id, product_name, category, subcategory,
        brand, unit_cost, unit_price,
        effective_date, expiry_date, is_current)
    VALUES (source.id, source.name, source.category, source.subcategory,
        source.brand, source.cost, source.price,
        CURRENT_DATE, '9999-12-31', TRUE);

```

MERGE is cleaner but has limitations — you can't do the full Type 2 (expire + insert new version) in a single MERGE. You'd still need a separate INSERT for the new version.

Incremental Fact Loading

```
-- Fact tables are simpler – mostly insert-only
INSERT INTO fact_orders (
    order_id, customer_key, product_key, date_key,
    quantity, unit_price, discount_amount,
    revenue, cost, profit, order_status
)
SELECT
    s.id as order_id,
    dc.customer_key,
    dp.product_key,
    TO_CHAR(s.order_date, 'YYYYMMDD')::INT as date_key,
    si.quantity,
    si.unit_price,
    si.discount,
    si.quantity * si.unit_price - si.discount as revenue,
    si.quantity * dp.unit_cost as cost,
    (si.quantity * si.unit_price - si.discount) - (si.quantity * dp.unit_cost) as
profit,
    s.status
FROM src_orders s
JOIN src_order_items si ON s.id = si.order_id
-- Look up current dimension keys
JOIN dim_customer dc ON s.customer_id = dc.customer_id AND dc.is_current = TRUE
JOIN dim_product dp ON si.product_id = dp.product_id AND dp.is_current = TRUE
-- Only new records (not already loaded)
LEFT JOIN fact_orders f ON s.id = f.order_id AND si.product_id = dp.product_id
WHERE f.order_key IS NULL -- doesn't exist yet
    AND s.created_at >= CURRENT_DATE - INTERVAL '1 day';
```

⚠ Common Mistake

Late-arriving facts. An order from last week finally shows up in the source system. Your incremental load based on `created_at >= yesterday` will miss it. Solutions: (1) Use a wider lookback window (7 days). (2) Track a high-water mark (max `order_id` you've loaded). (3) Use CDC (change data capture) from the source database.

□ Checkpoint

1. What does `EXCLUDED` refer to in an `INSERT ON CONFLICT` statement?
2. Why can't you do a full SCD Type 2 update in a single `MERGE` statement?
3. How do you prevent loading duplicate facts during incremental loads?

2.4 Surrogate Keys and SCD SQL Mechanics

TL;DR

- Use `SERIAL` / `IDENTITY` for surrogate keys in single-database warehouses. Use hash-based keys for distributed systems.
- Use `IS DISTINCT FROM` instead of `!=` when comparing nullable columns — it handles NULLs correctly.
- A complete SCD Type 2 process: expire changed rows → insert new versions → insert brand-new records.

Surrogate Key Strategies

```
-- Option 1: SERIAL / IDENTITY (auto-increment)
-- Simple, works great for single-database warehouses
CREATE TABLE dim_customer (
    customer_key INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ...
);

-- Option 2: Hash-based keys (deterministic – same input = same key)
-- Useful for distributed systems like Spark
SELECT MD5(customer_id::TEXT || '|' || effective_date::TEXT) as customer_key;

-- Option 3: UUID (guaranteed unique, but larger and slower to join)
SELECT gen_random_uuid() as customer_key;
```

For this course, we use SERIAL/IDENTITY. It's the simplest and works perfectly for Postgres. If you move to Spark or a distributed warehouse, hash-based keys become more practical.

Complete SCD Type 2 Load — Working Example

```

-- Complete SCD Type 2 procedure for dim_customer
-- Run this daily after extracting source data
DO $$
DECLARE
    v_load_date DATE := CURRENT_DATE;
    v_rows_expired INT;
    v_rows_inserted INT;
    v_rows_new INT;
BEGIN
    -- 1. Expire changed rows
    WITH changed AS (
        SELECT s.id as customer_id
        FROM src_customers s
        JOIN dim_customer d ON s.id = d.customer_id AND d.is_current = TRUE
        WHERE s.updated_at >= v_load_date - INTERVAL '1 day'
        AND (
            -- IS DISTINCT FROM handles NULLs correctly!
            d.city IS DISTINCT FROM s.city
            OR d.state IS DISTINCT FROM s.state
            OR d.segment IS DISTINCT FROM s.segment
        )
    )
    UPDATE dim_customer d
    SET expiry_date = v_load_date - 1,
        is_current = FALSE
    FROM changed c
    WHERE d.customer_id = c.customer_id
        AND d.is_current = TRUE;

    GET DIAGNOSTICS v_rows_expired = ROW_COUNT;
    RAISE NOTICE 'Expired % rows', v_rows_expired;

    -- 2. Insert new versions of changed records
    INSERT INTO dim_customer (
        customer_id, name, email, city, state,
        country, segment, created_date,
        effective_date, expiry_date, is_current
    )
    SELECT
        s.id, s.name, s.email, s.city, s.state,
        s.country, s.segment, s.created_at::DATE,
        v_load_date, '9999-12-31', TRUE
    FROM src_customers s
    JOIN dim_customer d ON s.id = d.customer_id
    WHERE d.expiry_date = v_load_date - 1
        AND d.is_current = FALSE
        AND NOT EXISTS (
            SELECT 1 FROM dim_customer d2
            WHERE d2.customer_id = s.id AND d2.is_current = TRUE
        );

    GET DIAGNOSTICS v_rows_inserted = ROW_COUNT;
    RAISE NOTICE 'Inserted % new versions', v_rows_inserted;

```

```

-- 3. Insert brand new customers
INSERT INTO dim_customer (
    customer_id, name, email, city, state,
    country, segment, created_date,
    effective_date, expiry_date, is_current
)
SELECT
    s.id, s.name, s.email, s.city, s.state,
    s.country, s.segment, s.created_at::DATE,
    v_load_date, '9999-12-31', TRUE
FROM src_customers s
LEFT JOIN dim_customer d ON s.id = d.customer_id
WHERE d.customer_key IS NULL;

GET DIAGNOSTICS v_rows_new = ROW_COUNT;
RAISE NOTICE 'Inserted % new customers', v_rows_new;
END $$;

```

❏ Pro Tip

Use `IS DISTINCT FROM` instead of `!=` when comparing columns that might be NULL. `NULL != 'New York'` returns NULL (not TRUE), but `NULL IS DISTINCT FROM 'New York'` returns TRUE. This trips people up constantly and causes silent bugs where changes aren't detected.

❏ Checkpoint

1. When would you choose hash-based surrogate keys over auto-increment?
2. What's the difference between `!=` and `IS DISTINCT FROM` when NULLs are involved?
3. Why does the SCD Type 2 procedure have three separate steps instead of one?

2.5 Performance — EXPLAIN Plans, Indexing, Partitioning

TL;DR

- Always run `EXPLAIN ANALYZE` before optimizing — don't guess.
- Index your foreign keys in fact tables and columns you frequently filter on. Use partial indexes for `is_current = TRUE`.
- Partition large fact tables by date — queries that filter by date will only scan the relevant partition.

Your SQL works. It returns the right answer. But it takes 45 minutes. Your Airflow DAG times out. Your stakeholder complains. This is a Tuesday in data engineering.

EXPLAIN ANALYZE — Your Best Friend

```
EXPLAIN ANALYZE
SELECT
    p.category,
    SUM(f.revenue) as total_revenue
FROM fact_orders f
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_date d ON f.date_key = d.date_key
WHERE d.year = 2026
GROUP BY p.category;
```

Key things to look for in the output:

What to check	What it means
Seq Scan vs Index Scan	Seq Scan reads the entire table. Fine for small tables, terrible for large ones.
actual time	Real execution time in milliseconds.
rows vs actual rows	If wildly different, statistics are stale — run <code>ANALYZE</code> .
Nested Loop vs Hash Join	Hash joins are usually best for analytical queries.

Indexing Strategies

```
-- Index the columns you filter and join on
CREATE INDEX idx_fact_orders_date ON fact_orders(date_key);
CREATE INDEX idx_fact_orders_customer ON fact_orders(customer_key);
CREATE INDEX idx_fact_orders_product ON fact_orders(product_key);

-- Composite index for common query patterns
CREATE INDEX idx_fact_orders_date_customer
    ON fact_orders(date_key, customer_key);

-- Partial index — only index the rows you actually query
CREATE INDEX idx_dim_customer_current
    ON dim_customer(customer_id) WHERE is_current = TRUE;
-- Smaller and faster than indexing all rows
```

❏ Pro Tip

The partial index `WHERE is_current = TRUE` is a game-changer for dimension lookups. Since you almost always query current dimension records, this index is smaller, faster, and more targeted than a full index.

Table Partitioning

```
-- Partition fact table by date (the most common pattern)
CREATE TABLE fact_orders_partitioned (
    order_key      SERIAL,
    order_id       INT NOT NULL,
    customer_key   INT NOT NULL,
    product_key    INT NOT NULL,
    date_key       INT NOT NULL,
    quantity       INT NOT NULL,
    revenue        NUMERIC(12,2) NOT NULL,
    order_date     DATE NOT NULL
) PARTITION BY RANGE (order_date);

-- Create monthly partitions
CREATE TABLE fact_orders_2026_01 PARTITION OF fact_orders_partitioned
    FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
CREATE TABLE fact_orders_2026_02 PARTITION OF fact_orders_partitioned
    FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');
-- ... one per month
```

When you query `WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31'`, Postgres only scans the January partition. Instead of reading 100M rows across all time, it reads 5M rows for that month.

Quick Wins Checklist

1. ☐ Run `EXPLAIN ANALYZE` before optimizing (don't guess)
2. ☐ Add indexes on foreign keys and filter columns
3. ☐ Partition large fact tables by date
4. ☐ Run `ANALYZE` after large data loads (updates query planner statistics)
5. ☐ Use `WHERE` clauses — don't scan what you don't need
6. ☐ Avoid `SELECT *` — only select columns you need

⚠ Common Mistake

Premature optimization. If your table has 100k rows and your query takes 200ms, you don't need partitioning or fancy indexes. Focus on correctness first, optimize when you hit a real performance problem. Most data engineering performance issues come from missing indexes on fact table foreign keys — start there.

☐ Checkpoint

1. What's the first thing you should do before trying to optimize a slow query?
2. Why is a partial index on `is_current = TRUE` better than a full index?
3. What's the main benefit of partitioning a fact table by date?

2.6 SQL for Data Quality

TL;DR

- Your pipeline can succeed and still load garbage. Data quality checks catch what status codes miss.
- The five essential checks: completeness, freshness, uniqueness, volume anomalies, and referential integrity.
- Build a reusable quality check template that returns a pass/fail summary after every load.

Your pipeline ran successfully. Airflow shows all green. Everything loaded. And your data is completely wrong. This happens ALL the time. The pipeline succeeded — it just loaded garbage. Data quality checks are how you catch this.

Completeness Checks

```
-- Are required fields populated?
SELECT
    'fact_orders' as table_name,
    COUNT(*) as total_rows,
    COUNT(*) FILTER (WHERE customer_key IS NULL) as null_customer,
    COUNT(*) FILTER (WHERE product_key IS NULL) as null_product,
    COUNT(*) FILTER (WHERE revenue IS NULL) as null_revenue,
    COUNT(*) FILTER (WHERE revenue < 0) as negative_revenue,
    ROUND(
        COUNT(*) FILTER (WHERE customer_key IS NULL)::NUMERIC
        / COUNT(*) * 100, 2
    ) as null_customer_pct
FROM fact_orders
WHERE date_key >= TO_CHAR(CURRENT_DATE - INTERVAL '1 day', 'YYYYMMDD')::INT;
```

Freshness Checks

```
-- When was data last loaded?
SELECT
    MAX(d.calendar_date) as most_recent_data,
    CURRENT_DATE - MAX(d.calendar_date) as days_behind,
    CASE
        WHEN CURRENT_DATE - MAX(d.calendar_date) > 2
        THEN '❑ STALE — INVESTIGATE'
        ELSE '❑ OK'
    END as freshness_status
FROM fact_orders f
JOIN dim_date d ON f.date_key = d.date_key;
```

Uniqueness / Duplicate Checks

```
-- Are there duplicate orders?
SELECT order_id, COUNT(*) as occurrence_count
FROM fact_orders
GROUP BY order_id
HAVING COUNT(*) > 1;
-- Should return 0 rows
```

Volume Anomaly Detection

```
-- Compare today's load to the 30-day average using a z-score
WITH daily_counts AS (
    SELECT
        d.calendar_date,
        COUNT(*) as row_count
    FROM fact_orders f
    JOIN dim_date d ON f.date_key = d.date_key
    WHERE d.calendar_date >= CURRENT_DATE - INTERVAL '30 days'
    GROUP BY d.calendar_date
),
stats AS (
    SELECT
        AVG(row_count) as avg_daily,
        STDDEV(row_count) as stddev_daily
    FROM daily_counts
    WHERE calendar_date < CURRENT_DATE
),
today AS (
    SELECT row_count as today_count
    FROM daily_counts
    WHERE calendar_date = CURRENT_DATE
)
SELECT
    t.today_count,
    ROUND(s.avg_daily) as avg_daily,
    CASE
        WHEN t.today_count < s.avg_daily - 2 * s.stddev_daily
        THEN '☐ ANOMALY: Unusually LOW volume'
        WHEN t.today_count > s.avg_daily + 2 * s.stddev_daily
        THEN '☐ ANOMALY: Unusually HIGH volume'
        ELSE '☐ NORMAL'
    END as volume_status
FROM today t, stats s;
```

If today's count is more than 2 standard deviations from the 30-day average, something might be wrong.

Referential Integrity Checks

```
-- Do all fact table keys have matching dimensions?
SELECT 'orphan_customers' as check_name, COUNT(*) as failures
FROM fact_orders f
LEFT JOIN dim_customer c ON f.customer_key = c.customer_key
WHERE c.customer_key IS NULL

UNION ALL

SELECT 'orphan_products', COUNT(*)
FROM fact_orders f
LEFT JOIN dim_product p ON f.product_key = p.product_key
WHERE p.product_key IS NULL

UNION ALL

SELECT 'orphan_dates', COUNT(*)
FROM fact_orders f
LEFT JOIN dim_date d ON f.date_key = d.date_key
WHERE d.date_key IS NULL;
-- All should return 0
```

Business Rule Checks

```
-- Revenue should equal quantity * unit_price - discount
SELECT COUNT(*) as mismatched_rows
FROM fact_orders
WHERE ABS(revenue - (quantity * unit_price - discount_amount)) > 0.01;
-- Allow small floating-point tolerance
```

Reusable Quality Check Template

```
-- Run this as a post-load validation step in your pipeline
WITH checks AS (
    SELECT 'null_customer_key' as check_name,
           COUNT(*) FILTER (WHERE customer_key IS NULL) as failures
    FROM fact_orders
    WHERE date_key = TO_CHAR(CURRENT_DATE, 'YYYYMMDD')::INT

    UNION ALL

    SELECT 'duplicate_orders',
           COUNT(*) - COUNT(DISTINCT order_id)
    FROM fact_orders
    WHERE date_key = TO_CHAR(CURRENT_DATE, 'YYYYMMDD')::INT

    UNION ALL

    SELECT 'negative_revenue',
           COUNT(*) FILTER (WHERE revenue < 0)
    FROM fact_orders
    WHERE date_key = TO_CHAR(CURRENT_DATE, 'YYYYMMDD')::INT
)
SELECT
    check_name,
    failures,
    CASE WHEN failures > 0 THEN '❌ FAIL' ELSE '✅ PASS' END as status
FROM checks;
```

Expected output:

check_name	failures	status
null_customer_key	0	✅ PASS
duplicate_orders	0	✅ PASS
negative_revenue	0	✅ PASS

❏ Pro Tip

In Module 7, we'll use Great Expectations and Soda to do this at scale. But understanding the raw SQL is essential — you'll always write custom checks that no framework covers. Build this template once and reuse it across every pipeline.

❏ Checkpoint

1. Why can a pipeline succeed (status: green) and still load bad data?
2. What does a volume anomaly check detect that a completeness check doesn't?
3. Why do we allow a 0.01 tolerance in the business rule check for revenue?

Module 2 Project: Incremental ETL Pipeline in Pure SQL

Objective

Write a complete incremental ETL pipeline in SQL that handles late-arriving data, deduplication, and SCD Type 2 updates. Include data quality checks.

Prerequisites

- Module 1 project complete (dimensional model created)
- Comfort with CTEs, window functions, and UPSERT patterns

Expected Time

2–3 hours

Starter Code

`modules/module-2/starter/` contains:

- `source_simulation.sql` — Script that simulates daily source data changes
- `warehouse_schema.sql` — The dimensional model from Module 1
- `sample_data_day1.sql` — Initial load data
- `sample_data_day2.sql` — Day 2 changes (new orders, customer moves, price changes)
- `sample_data_day3.sql` — Day 3 changes (late-arriving orders, corrections)

Step-by-Step

STEP 1: INITIAL LOAD (DAY 1)

Write SQL to do the full initial load of all dimensions and facts.

```
-- Save as 01_initial_load.sql
-- Load all dimensions (every record is is_current = TRUE)
-- Load all facts with proper surrogate key lookups
-- Populate date dimension
```

STEP 2: INCREMENTAL DIMENSION LOAD (DAY 2)

```
-- Save as 02_incremental_dimensions.sql
-- Handle:
--   New customers (INSERT)
--   Customer address changes (SCD Type 2: expire + insert)
--   Customer email updates (SCD Type 1: UPDATE)
--   New products (INSERT)
--   Product price changes (SCD Type 2)
--   Product name corrections (SCD Type 1)
```

STEP 3: INCREMENTAL FACT LOAD (DAY 2)

```
-- Save as 03_incremental_facts.sql
-- Handle:
--   New orders (INSERT with dimension lookups)
--   Deduplication (don't load orders that already exist)
--   Order status updates (UPDATE existing fact rows)
```

STEP 4: LATE-ARRIVING DATA (DAY 3)

```
-- Save as 04_late_arriving.sql
-- Handle:
--   Orders from Day 1 that arrived late
--   Look up the CORRECT dimension key
--   (the one that was current on the order date, not today)
```

STEP 5: DATA QUALITY CHECKS

```
-- Save as 05_quality_checks.sql
-- After each load, run:
-- 1. Completeness: no NULL foreign keys in facts
-- 2. Uniqueness: no duplicate order_id in facts
-- 3. Referential integrity: all fact keys exist in dimensions
-- 4. Volume: row count within expected range
-- 5. Business rules: revenue = quantity * price - discount
-- Return a summary table of check results
```

STEP 6: RUN THE FULL PIPELINE

```
# Execute in order
psql -h localhost -U dataeng -d warehouse -f 01_initial_load.sql
psql -h localhost -U dataeng -d warehouse -f source_simulation.sql
psql -h localhost -U dataeng -d warehouse -f 02_incremental_dimensions.sql
psql -h localhost -U dataeng -d warehouse -f 03_incremental_facts.sql
psql -h localhost -U dataeng -d warehouse -f 05_quality_checks.sql
```

Expected Output

- All quality checks pass (✅)
- SCD Type 2 history preserved (query `dim_customer` and see multiple rows for moved customers)
- Late-arriving facts link to the correct historical dimension key
- No duplicate orders in fact table
- Revenue calculations are consistent

Evaluation Criteria

- ✅ CTEs used for readable, step-by-step logic
- ✅ SCD Type 2 correctly implemented (expire + insert)
- ✅ SCD Type 1 correctly implemented (overwrite)
- ✅ Late-arriving data handled (historical dimension lookup)
- ✅ Deduplication prevents double-loading
- ✅ Data quality checks comprehensive and passing
- ✅ All queries use EXPLAIN-friendly patterns (proper JOINS, indexed lookups)

Module 3: Python for Data Engineers

3.1 Project Structure for DE Projects

TL;DR

- Use the `src/` layout: your package lives in `src/yourproject/`, clearly separated from tests, scripts, and config.
- Never hardcode credentials. Use `pydantic-settings` to load environment variables with type validation.
- Commit `.env.example` (template), never commit `.env` (actual secrets).

I've reviewed so many data engineering repos where everything is in one giant `main.py` with 2,000 lines. No structure, no tests, no config management. It works on their laptop and nowhere else.

Here's how professionals organize Python DE projects.

The Standard Layout

```
shopfast-pipeline/
├── pyproject.toml           # Project metadata + dependencies
├── README.md
├── .env.example            # Template for env vars (committed to git)
├── .env                   # Actual env vars (NOT committed – in .gitignore)
├── .gitignore
├── Dockerfile
├── docker-compose.yml
├──
├── src/
│   ├── shopfast/          # Your package
│   │   ├── __init__.py
│   │   ├── config.py      # Settings management
│   │   ├── models.py      # Pydantic models / dataclasses
│   │   ├── extractors/    # Data extraction (API calls, file reads)
│   │   │   ├── __init__.py
│   │   │   ├── api_client.py
│   │   │   └── file_reader.py
│   │   ├── transformers/  # Data transformation logic
│   │   │   ├── __init__.py
│   │   │   └── orders.py
│   │   ├── loaders/      # Data loading (DB writes, file writes)
│   │   │   ├── __init__.py
│   │   │   ├── postgres.py
│   │   │   └── parquet.py
│   │   └── utils/        # Shared utilities
│   │       ├── __init__.py
│   │       ├── logging.py
│   │       └── retry.py
│   ├── tests/
│   │   ├── __init__.py
│   │   ├── conftest.py    # Shared test fixtures
│   │   ├── test_extractors/
│   │   ├── test_transformers/
│   │   └── test_loaders/
│   ├──
│   ├── scripts/          # One-off scripts, not core logic
│   │   └── seed_data.py
│   ├──
│   └── data/             # Sample/test data (small files only)
│       ├── sample_orders.csv
│       └── sample_products.json
```

This is the `src` layout. Your actual code lives in `src/shopfast/`. It prevents import issues and clearly separates your package from tests, scripts, and config.

Config Management with Environment Variables

```
# src/shopfast/config.py
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    """Application settings loaded from environment variables."""

    # Database
    db_host: str = "localhost"
    db_port: int = 5432
    db_user: str = "dataeng"
    db_password: str = "dataeng"
    db_name: str = "warehouse"

    # API
    api_base_url: str = "https://api.shopfast.example.com"
    api_key: str = ""
    api_timeout: int = 30

    # Storage
    s3_bucket: str = "raw-data"
    s3_endpoint: str = "http://localhost:9000"

    # Pipeline
    batch_size: int = 1000
    max_retries: int = 3
    log_level: str = "INFO"

    class Config:
        env_file = ".env"
        env_prefix = "" # or "SHOPFAST_" for namespacing

@lru_cache # singleton – only loads once
def get_settings() -> Settings:
    return Settings()
```

```
# .env.example (committed to git)
DB_HOST=localhost
DB_PORT=5432
DB_USER=dataeng
DB_PASSWORD=changeme
DB_NAME=warehouse
API_BASE_URL=https://api.shopfast.example.com
API_KEY=your-key-here
S3_BUCKET=raw-data
LOG_LEVEL=INFO
```


Never hardcode credentials. Never commit `.env` files. `pydantic-settings` loads them with validation and type coercion — if `DB_PORT` is set to `"abc"`, it'll fail immediately with a clear error instead of three steps into your pipeline.

The .gitignore Essentials

```
.env
__pycache__/
*.pyc
.venv/
dist/
*.egg-info/
.pytest_cache/
data/output/
*.parquet
```

⚠ Common Mistake

Don't name your virtual environment `venv/` (without a dot). Use `.venv/` — it's hidden by default, and every `.gitignore` template already ignores it.

□ Checkpoint

1. Why does the `src/` layout prevent import issues?
2. What happens if you set `DB_PORT=abc` with `pydantic-settings`?
3. Why should `.env.example` be committed but `.env` should not?

3.2 Working with APIs — Requests, Pagination, Rate Limiting

TL;DR

- Use `httpx` over `requests` — same API, but with async support, HTTP/2, and better timeouts.
- Use `tenacity` for retry logic with exponential backoff. Don't write retry loops manually.
- Use generators (`yield`) for pagination to avoid loading all pages into memory at once.

About half of data engineering is just getting data out of APIs. And the APIs are never as clean as the documentation suggests.

The Naive Approach (Don't Do This in Production)

```
import requests

response = requests.get("https://api.example.com/orders")
data = response.json()
```

This works for demos. In production, it'll fail in 47 different ways. Let's build it properly.

Production API Client

```

# src/shopfast/extractors/api_client.py
import time
import logging
from typing import Generator

import httpx
from tenacity import (
    retry,
    stop_after_attempt,
    wait_exponential,
    retry_if_exception_type,
)

from shopfast.config import get_settings

logger = logging.getLogger(__name__)

class APIClient:
    """Production-quality API client with pagination, retries, and rate limiting."""

    def __init__(self, base_url: str | None = None, api_key: str | None = None):
        settings = get_settings()
        self.base_url = base_url or settings.api_base_url
        self.api_key = api_key or settings.api_key
        self.timeout = settings.api_timeout

        # httpx.Client is a session – reuses connections
        self.client = httpx.Client(
            base_url=self.base_url,
            headers={
                "Authorization": f"Bearer {self.api_key}",
                "Accept": "application/json",
            },
            timeout=self.timeout,
        )

    def __enter__(self):
        return self

    def __exit__(self, *args):
        self.client.close()

    @retry(
        stop=stop_after_attempt(3), # max 3 tries
        wait=wait_exponential(multiplier=1, min=2, max=30), # 2s, 4s, 8s...
        retry=retry_if_exception_type( # only retry on these
            (httpx.HTTPStatusError, httpx.ConnectError)
        ),
        before_sleep=lambda retry_state: logger.warning(
            f"Retry {retry_state.attempt_number}/3 after error: "
            f"{retry_state.outcome.exception()}"
        ),
    ),

```

```

)
def _request(self, method: str, endpoint: str, **kwargs) -> dict:
    """Make a single API request with retry logic."""
    response = self.client.request(method, endpoint, **kwargs)

    # Handle rate limiting (HTTP 429)
    if response.status_code == 429:
        retry_after = int(response.headers.get("Retry-After", 60))
        logger.warning(f"Rate limited. Waiting {retry_after}s...")
        time.sleep(retry_after)
        response = self.client.request(method, endpoint, **kwargs)

    response.raise_for_status() # raises on 4xx/5xx
    return response.json()

def get_paginated(
    self, endpoint: str, params: dict | None = None, page_size: int = 100
) -> Generator[list[dict], None, None]:
    """
    Fetch all pages from a paginated endpoint.

    Handles two common pagination styles:
    - Offset-based: ?page=1&per_page=100
    - Cursor-based: ?cursor=abc123&limit=100

    Yields one page (list of records) at a time.
    """
    params = params or {}
    page = 1

    while True:
        params.update({"page": page, "per_page": page_size})
        logger.info(f"Fetching {endpoint} page {page}")

        data = self._request("GET", endpoint, params=params)

        # Handle different response shapes
        if isinstance(data, list):
            records = data
        elif "data" in data:
            records = data["data"]
        elif "results" in data:
            records = data["results"]
        else:
            records = data

        if not records:
            logger.info(f"No more records. Total pages: {page - 1}")
            break

        yield records # return this page, don't load everything into memory

    # Check for cursor-based pagination
    if isinstance(data, dict) and data.get("next_cursor"):

```

```

        params["cursor"] = data["next_cursor"]
    elif len(records) < page_size:
        break # fewer records than page size = last page
    else:
        page += 1

    time.sleep(0.5) # be a good API citizen

def get_all_records(
    self, endpoint: str, params: dict | None = None
) -> list[dict]:
    """Fetch ALL records from a paginated endpoint into memory."""
    all_records = []
    for page in self.get_paginated(endpoint, params):
        all_records.extend(page)
    logger.info(f"Total records fetched: {len(all_records)}")
    return all_records

```

Using It

```

# scripts/extract_orders.py
from shopfast.extractors.api_client import APIClient
from datetime import date, timedelta

yesterday = (date.today() - timedelta(days=1)).isoformat()

with APIClient() as client:
    # Option 1: Get all pages into memory
    orders = client.get_all_records(
        "/v1/orders",
        params={"created_after": yesterday, "status": "completed"}
    )
    print(f"Extracted {len(orders)} orders")

    # Option 2: Process page by page (memory efficient)
    for page in client.get_paginated("/v1/orders", params={"created_after": yesterday}):
        process_batch(page)

```

Key Design Decisions

□ Key Concept: httpx over requests

`httpx` is the modern replacement for `requests`. Same API, but with async support, HTTP/2, and better timeout handling.

□ Key Concept: tenacity for retries

Don't write retry loops manually. `tenacity` handles exponential backoff, max attempts, and conditional retry (only retry on network errors, not on 400 Bad Request).

□ Key Concept: Generators for pagination

Using `yield` means you don't load all pages into memory at once. For a million records, you process batch by batch. Memory stays flat.

△ Common Mistake

Don't ignore API response bodies. A `200 OK` with `{"data": [], "error": "invalid_token"}` is not a success. Always validate the actual response content, not just the status code.

□ Checkpoint

1. Why use `httpx` instead of `requests`?
2. What's the advantage of `yield` ing pages instead of collecting them all into a list?
3. How does the client handle HTTP 429 (rate limit) responses?

□ Try It Yourself

Modify the `get_paginated` method to also support Link-header-based pagination (where the API returns a `Link: <url>; rel="next"` header). Hint: check `response.headers.get("Link")`.

3.3 File Formats — Parquet, JSON, CSV

TL;DR

- **CSV:** No types, no compression, slow. Use it only when humans need to open the file in Excel.
- **Parquet:** Columnar, compressed, typed, supports column pruning and predicate pushdown. 5–10x smaller, 5–20x faster than CSV.
- **Rule of thumb:** Receive data as CSV/JSON → immediately convert to Parquet for all downstream processing.

CSV is how people send you data. Parquet is how you should store it. JSON is for APIs.

The Contenders

CSV — THE LEGACY FORMAT

```
import pandas as pd

df = pd.DataFrame({
    "order_id": range(1, 100_001),
    "customer_id": [i % 1000 for i in range(1, 100_001)],
    "revenue": [round(i * 0.99, 2) for i in range(1, 100_001)],
    "order_date": pd.date_range("2026-01-01", periods=100_000, freq="min"),
})

df.to_csv("orders.csv", index=False)
# File size: ~4.2 MB
```

Problems with CSV:

- No types — everything is a string. Is `"2026-01-15"` a date or a string? Is `"42"` an int?
- No built-in compression
- Slow to read (must parse every character)
- No column pruning — need 2 columns out of 50? You still read all 50.

JSON / JSONL — THE API FORMAT

```
import json

# JSONL: one JSON object per line (better than JSON arrays for streaming)
with open("orders.jsonl", "w") as f:
    for record in orders:
        f.write(json.dumps(record) + "\n")
```

Good for API responses, semi-structured data, logging. Bad for analytics and large datasets.

PARQUET — THE WINNER

```
# Write Parquet
df.to_parquet("orders.parquet", engine="pyarrow", compression="snappy")
# File size: ~0.8 MB (5x smaller than CSV!)

# Read only specific columns (column pruning)
df_small = pd.read_parquet("orders.parquet", columns=["order_id", "revenue"])
# Only reads the bytes for those 2 columns – way faster

# Read with filter (predicate pushdown)
df_filtered = pd.read_parquet(
    "orders.parquet",
    filters=[("revenue", ">", 500)]
)
```

Why Parquet Wins — Benchmarks

```
import os
import time

# File size comparison
csv_size = os.path.getsize("orders.csv")
parquet_size = os.path.getsize("orders.parquet")
print(f"CSV:      {csv_size / 1024 / 1024:.1f} MB")
print(f"Parquet: {parquet_size / 1024 / 1024:.1f} MB")
print(f"Parquet is {csv_size / parquet_size:.1f}x smaller")

# Read speed comparison
start = time.time()
df_csv = pd.read_csv("orders.csv")
csv_time = time.time() - start

start = time.time()
df_pq = pd.read_parquet("orders.parquet")
pq_time = time.time() - start

print(f"\nCSV read:      {csv_time:.3f}s")
print(f"Parquet read: {pq_time:.3f}s")
print(f"Parquet is {csv_time / pq_time:.1f}x faster")
```

Typical output:

```
CSV:      4.2 MB
Parquet:  0.8 MB
Parquet is 5.3x smaller

CSV read:  0.180s
Parquet read: 0.025s
Parquet is 7.2x faster
```

Inspecting Parquet Files Without Loading Them

```
import pyarrow.parquet as pq

parquet_file = pq.ParquetFile("orders.parquet")
print(parquet_file.schema)                # column names and types
print(f"Rows: {parquet_file.metadata.num_rows}")
print(f"Row groups: {parquet_file.metadata.num_row_groups}")
print(f"Columns: {parquet_file.metadata.num_columns}")
```

The Rule of Thumb

Scenario	Format
Receive data from external source	CSV or JSON (their choice)
Store/process in your pipeline	Parquet
Data lake storage	Parquet (or Delta Lake / Iceberg, which are Parquet underneath)
Human needs to open it in Excel	CSV
API communication	JSON

⚠ Common Mistake

Deeply nested JSON structures don't always map cleanly to Parquet. Flatten your data first. A column of type `List[Dict[str, Any]]` will cause headaches — extract it into a separate table or flatten to top-level columns.

❑ Checkpoint

1. What is "column pruning" and why does it matter for wide tables?
2. Why is Parquet smaller than CSV even though it stores type information?
3. When is CSV still the right choice?

3.4 Pandas vs. Polars vs. DuckDB

TL;DR

- **Pandas:** The standard. Best ecosystem compatibility. Slower on large data.
- **Polars:** Rust-based, multi-threaded, 5–10x faster. Different API — not a drop-in replacement.
- **DuckDB:** SQL on files. Zero loading step. Great for exploration and SQL-heavy work.
- Learn all three. Use the right tool for the job.

Three years ago, it was just Pandas. Now you've got Polars, DuckDB, and people arguing about which is best. Here's the actual answer: they're all good, and they're good at different things.

The Same Task in All Three

Load 1M orders, join with products, filter, aggregate, and sort.

PANDAS

```
import pandas as pd

# Read
orders = pd.read_parquet("data/orders.parquet")
products = pd.read_parquet("data/products.parquet")

# Join
df = orders.merge(products, on="product_id", how="left")

# Filter + Transform
df = df[df["order_date"] >= "2026-01-01"]
df["profit"] = df["revenue"] - df["cost"]

# Aggregate
result = (
    df.groupby("category")
    .agg(
        total_revenue=("revenue", "sum"),
        total_profit=("profit", "sum"),
        order_count=("order_id", "nunique"),
        avg_order_value=("revenue", "mean"),
    )
    .sort_values("total_revenue", ascending=False)
    .reset_index()
)

print(result.head(10))
```

POLARS

```
import polars as pl

# Read
orders = pl.read_parquet("data/orders.parquet")
products = pl.read_parquet("data/products.parquet")

# Everything in one chain (lazy evaluation – builds a query plan first)
result = (
    orders.lazy()
    .join(products.lazy(), on="product_id", how="left")
    .filter(pl.col("order_date") >= "2026-01-01")
    .with_columns(
        (pl.col("revenue") - pl.col("cost")).alias("profit")
    )
    .group_by("category")
    .agg(
        pl.col("revenue").sum().alias("total_revenue"),
        pl.col("profit").sum().alias("total_profit"),
        pl.col("order_id").n_unique().alias("order_count"),
        pl.col("revenue").mean().alias("avg_order_value"),
    )
    .sort("total_revenue", descending=True)
    .collect() # executes the lazy query plan
)

print(result.head(10))
```

DUCKDB

```
import duckdb

# DuckDB reads Parquet files directly in SQL – no loading step!
result = duckdb.sql("""
    SELECT
        p.category,
        SUM(o.revenue) as total_revenue,
        SUM(o.revenue - o.cost) as total_profit,
        COUNT(DISTINCT o.order_id) as order_count,
        AVG(o.revenue) as avg_order_value
    FROM 'data/orders.parquet' o
    LEFT JOIN 'data/products.parquet' p ON o.product_id = p.product_id
    WHERE o.order_date >= '2026-01-01'
    GROUP BY p.category
    ORDER BY total_revenue DESC
    LIMIT 10
""").df() # .df() converts to a pandas DataFrame

print(result)
```

Notice: DuckDB reads Parquet files directly in the SQL query. No loading step. It's like having a database that just works on files.

When to Use What

Scenario	Best tool	Why
Quick data exploration	DuckDB	SQL is fastest for ad-hoc queries
Complex transformations with business logic	Polars	Expressive API, very fast
Integration with ML libraries (sklearn, etc.)	Pandas	Best ecosystem compatibility
Files larger than your RAM	DuckDB or Polars (lazy)	Both handle out-of-core
Team knows SQL well	DuckDB	Zero learning curve
Max performance on single machine	Polars	Rust-based, multi-threaded by default
Legacy codebase	Pandas	It's everywhere, everyone knows it

Performance Comparison (1M rows)

Typical benchmark results:

```
Pandas: 0.450s
Polars: 0.085s
DuckDB: 0.072s
```

Polars and DuckDB are 5–6x faster than Pandas on this workload. For bigger data, the gap widens.

❏ Pro Tip

Learn all three. Use DuckDB for exploration and SQL-heavy work. Use Polars for complex Python pipelines. Use Pandas when you need library compatibility. In this course, you'll see all three.

⚠ Common Mistake

Polars is NOT a drop-in replacement for Pandas. The API is different. `df.groupby()` in Pandas is `df.group_by()` in Polars. Column selection, filtering, and aggregation syntax all differ. Budget time for the learning curve.

❏ Checkpoint

1. What's the key advantage of DuckDB's approach to reading Parquet files?
2. When would you choose Pandas over Polars despite the performance difference?
3. What does "lazy evaluation" mean in Polars, and why is it beneficial?

□ Try It Yourself

Write the same query in all three tools: "Find the top 5 customers by total revenue in January 2026, including their name and order count." Compare the code length and readability.

3.5 Type Hints, Dataclasses, and Pydantic

TL;DR

- **Type hints** make your code self-documenting and enable IDE autocomplete.
- **Dataclasses** are lightweight structured data containers for internal use.
- **Pydantic** validates incoming data at the boundary — catching bad API responses, malformed CSVs, and type mismatches before they corrupt your warehouse.

An API returns `"price": "N/A"` instead of a number. Your pipeline inserts it into the database. PostgreSQL rejects it. Your Airflow task fails at 3 AM. You get paged.

Pydantic would have caught it at ingestion time, not load time.

Type Hints — The Foundation

```
# Without type hints – what does this function expect?
def process_order(order, customer, settings):
    ...

# With type hints – crystal clear
def process_order(
    order: dict[str, any],
    customer: Customer,
    settings: Settings,
) -> ProcessedOrder:
    ...
```

Type hints don't enforce anything at runtime by default. But they make your code self-documenting, enable IDE autocomplete, and tools like `mypy` can catch bugs statically.

Dataclasses — Lightweight Structured Data

```
from dataclasses import dataclass
from datetime import date
from decimal import Decimal

@dataclass
class Order:
    order_id: int
    customer_id: int
    order_date: date
    revenue: Decimal
    status: str = "pending" # default value

# Usage
order = Order(
    order_id=1,
    customer_id=42,
    order_date=date(2026, 1, 15),
    revenue=Decimal("99.99"),
)
print(order.order_id) # 1
print(order.status)   # "pending"
```

Dataclasses give you a class that holds data without boilerplate. Great for internal data structures.


```

# src/shopfast/models.py
from pydantic import BaseModel, Field, field_validator, model_validator
from datetime import datetime, date
from decimal import Decimal
from enum import Enum

class OrderStatus(str, Enum):
    """Valid order statuses."""
    PENDING = "pending"
    COMPLETED = "completed"
    SHIPPED = "shipped"
    RETURNED = "returned"
    CANCELLED = "cancelled"

class OrderItem(BaseModel):
    product_id: int = Field(gt=0, description="Must be positive")
    quantity: int = Field(gt=0, le=10000)
    unit_price: Decimal = Field(gt=0, max_digits=10, decimal_places=2)
    discount: Decimal = Field(ge=0, default=Decimal("0.00"))

    @field_validator("unit_price", mode="before")
    @classmethod
    def parse_price(cls, v):
        """Handle messy price strings like '$29.99' or 'N/A'."""
        if isinstance(v, str):
            v = v.replace("$", "").replace(",", "").strip()
            if v.upper() in ("N/A", "NULL", "NONE", ""):
                raise ValueError(f"Invalid price value: {v}")
        return v

class Order(BaseModel):
    order_id: int = Field(gt=0)
    customer_id: int = Field(gt=0)
    order_date: datetime  # Pydantic auto-parses ISO strings!
    status: OrderStatus
    items: list[OrderItem] = Field(min_length=1)  # at least 1 item
    total_amount: Decimal = Field(gt=0)
    currency: str = Field(default="USD", pattern=r"^[A-Z]{3}$")

    @model_validator(mode="after")
    def validate_total(self):
        """Total should roughly match sum of items."""
        expected = sum(
            item.quantity * item.unit_price - item.discount
            for item in self.items
        )
        if abs(self.total_amount - expected) > Decimal("0.01"):
            raise ValueError(
                f"Total {self.total_amount} doesn't match calculated {expected}"
            )

```

```
)  
return self
```

Using Pydantic with API Data

```
from shopfast.models import Order  
from pydantic import ValidationError  
  
# Raw API response – messy types, string numbers, $ signs  
raw_api_data = {  
    "order_id": 5001,  
    "customer_id": 42,  
    "order_date": "2026-02-15T14:30:00Z",          # string → datetime ✓  
    "status": "completed",  
    "items": [  
        {"product_id": 101, "quantity": 2, "unit_price": "29.99"},  
        {"product_id": 205, "quantity": 1, "unit_price": "$15.00"}, # $ handled ✓  
    ],  
    "total_amount": "74.98",                      # string → Decimal ✓  
    "currency": "USD",  
}  
  
try:  
    order = Order(**raw_api_data)  
    print(f"Valid order: {order.order_id}")  
    print(f"Order date (parsed): {order.order_date}") # datetime object  
    print(f"Total (Decimal): {order.total_amount}")  # Decimal  
except ValidationError as e:  
    print(f"Validation failed:\n{e}")
```

Expected output:

```
Valid order: 5001  
Order date (parsed): 2026-02-15 14:30:00+00:00  
Total (Decimal): 74.98
```

Now with invalid data:

```

bad_data = {
    "order_id": -1,          # ▫ Must be > 0
    "customer_id": 42,
    "order_date": "not-a-date", # ▫ Can't parse
    "status": "unknown",      # ▫ Not in enum
    "items": [],              # ▫ Must have at least 1
    "total_amount": 0,        # ▫ Must be > 0
}

try:
    order = Order(**bad_data)
except ValidationError as e:
    print(e)
    # Shows ALL validation errors at once – not just the first one

```

Batch Validation Pattern

```

def validate_orders(raw_orders: list[dict]) -> tuple[list[Order], list[dict]]:
    """Validate a batch of orders. Return valid orders and errors."""
    valid = []
    errors = []

    for raw in raw_orders:
        try:
            order = Order(**raw)
            valid.append(order)
        except ValidationError as e:
            errors.append({
                "raw_data": raw,
                "errors": e.errors(),
                "order_id": raw.get("order_id", "UNKNOWN"),
            })

    logger.info(
        f"Validated {len(raw_orders)} orders: "
        f"{len(valid)} valid, {len(errors)} invalid"
    )

    if errors:
        # Write bad records to dead letter queue for investigation
        save_to_dead_letter(errors)

    return valid, errors

```

❑ Key Concept: Dead Letter Queue

Never silently drop bad data. Good records proceed through the pipeline. Bad records go to a **dead letter queue** (a file, a table, or a message queue) where you can investigate and reprocess them later. Always track what failed and why.

⚠ Common Mistake

Don't validate inside a tight loop for millions of records if performance matters. Pydantic V2 is much faster than V1, but validating 10 million records one by one is still slow. For bulk data, validate a sample and use schema-level checks (column types, null counts).

❑ Checkpoint

1. What's the difference between a dataclass and a Pydantic model?
2. How does the `@field_validator` on `unit_price` handle the string `"$29.99"`?
3. What is a dead letter queue and why is it important?

3.6 Testing Your Data Code

TL;DR

- Test **transformations** (given this input, get this output), **validation** (bad data gets rejected), and **edge cases** (empty DataFrames, NULLs, duplicates).
- Mock all external dependencies (APIs, databases) in unit tests. Save real integration tests for CI/CD.
- Use `pytest` fixtures to share test data across tests. Use `@pytest.mark.parametrize` for multiple input variations.

"We don't have time for tests." I hear this constantly. And then the same team spends 3 days debugging a pipeline that broke because someone changed a column name.

Project Setup

```
pip install pytest pytest-cov
```

```
# pyproject.toml
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
addopts = "-v --tb=short"
```

Shared Fixtures

```
# tests/conftest.py
import pytest
import pandas as pd

@pytest.fixture
def sample_orders_raw():
    """Raw API response data."""
    return [
        {
            "order_id": 1,
            "customer_id": 42,
            "order_date": "2026-01-15T10:00:00Z",
            "status": "completed",
            "items": [{"product_id": 101, "quantity": 2, "unit_price": "29.99"}],
            "total_amount": "59.98",
        },
        {
            "order_id": 2,
            "customer_id": 43,
            "order_date": "2026-01-16T14:30:00Z",
            "status": "completed",
            "items": [{"product_id": 205, "quantity": 1, "unit_price": "15.00"}],
            "total_amount": "15.00",
        },
    ]

@pytest.fixture
def sample_orders_df():
    """Orders as a DataFrame."""
    return pd.DataFrame({
        "order_id": [1, 2, 3],
        "customer_id": [42, 43, 42],
        "revenue": [59.98, 15.00, 120.00],
        "order_date": pd.to_datetime(["2026-01-15", "2026-01-16", "2026-01-15"]),
        "status": ["completed", "completed", "returned"],
    })
```

Testing Transformations

```

# tests/test_transformers/test_orders.py
import pandas as pd
import pytest
from shopfast.transformers.orders import (
    calculate_metrics,
    filter_valid_orders,
    deduplicate_orders,
)

def test_calculate_metrics(sample_orders_df):
    """Verify that calculated columns are added correctly."""
    result = calculate_metrics(sample_orders_df)

    assert "profit" in result.columns
    assert "order_tier" in result.columns
    assert result.loc[result["order_id"] == 3, "order_tier"].iloc[0] == "high_value"

def test_filter_valid_orders(sample_orders_df):
    """Returned orders should be excluded."""
    result = filter_valid_orders(sample_orders_df)

    assert len(result) == 2
    assert "returned" not in result["status"].values

def test_deduplicate_orders():
    """Keep only the most recently loaded version of each order."""
    df = pd.DataFrame({
        "order_id": [1, 1, 2],
        "revenue": [59.98, 59.98, 15.00],
        "loaded_at": pd.to_datetime([
            "2026-01-15 10:00",
            "2026-01-15 11:00", # more recent
            "2026-01-16 10:00",
        ]),
    })

    result = deduplicate_orders(df)

    assert len(result) == 2
    # Should keep the latest loaded version
    assert result.loc[
        result["order_id"] == 1, "loaded_at"
    ].iloc[0] == pd.Timestamp("2026-01-15 11:00")

def test_empty_dataframe():
    """Edge case: empty input shouldn't crash."""
    empty_df = pd.DataFrame(columns=["order_id", "revenue", "status"])
    result = filter_valid_orders(empty_df)
    assert len(result) == 0

```

Mocking API Calls

```
# tests/test_extractors/test_api_client.py
from unittest.mock import patch, MagicMock
import pytest
from shopfast.extractors.api_client import APIClient

@pytest.fixture
def mock_api_response():
    return {
        "data": [
            {"order_id": 1, "revenue": 59.98},
            {"order_id": 2, "revenue": 15.00},
        ],
        "next_cursor": None,
    }

def test_get_all_records(mock_api_response):
    """API client should fetch and flatten all pages."""
    with patch("shopfast.extractors.api_client.httpx.Client") as MockClient:
        mock_response = MagicMock()
        mock_response.status_code = 200
        mock_response.json.return_value = mock_api_response

        mock_client_instance = MockClient.return_value
        mock_client_instance.request.return_value = mock_response

        client = APIClient(base_url="https://fake.api.com", api_key="test")
        records = client.get_all_records("/orders")

        assert len(records) == 2
        assert records[0]["order_id"] == 1
```


Testing Pydantic Models

```
# tests/test_models.py
import pytest
from pydantic import ValidationError
from shopfast.models import Order, OrderItem

def test_valid_order(sample_orders_raw):
    """Valid data should parse without errors."""
    order = Order(**sample_orders_raw[0])
    assert order.order_id == 1
    assert order.status.value == "completed"

def test_rejects_negative_order_id():
    with pytest.raises(ValidationError, match="greater than 0"):
        OrderItem(product_id=-1, quantity=1, unit_price="10.00")

def test_rejects_na_price():
    with pytest.raises(ValidationError, match="Invalid price"):
        OrderItem(product_id=1, quantity=1, unit_price="N/A")

@pytest.mark.parametrize("bad_status", ["unknown", "COMPLETED", "", None])
def test_rejects_invalid_status(bad_status, sample_orders_raw):
    """Multiple invalid statuses should all be rejected."""
    data = sample_orders_raw[0].copy()
    data["status"] = bad_status
    with pytest.raises(ValidationError):
        Order(**data)
```

Running Tests

```
# Run all tests
pytest

# With coverage report
pytest --cov=shopfast --cov-report=term-missing

# Run a specific file
pytest tests/test_models.py -v

# Run tests matching a pattern
pytest -k "test_valid" -v
```

Expected output:

```
tests/test_models.py: test_valid_order PASSED
tests/test_models.py: test_rejects_negative_order_id PASSED
tests/test_models.py: test_rejects_na_price PASSED
tests/test_models.py: test_rejects_invalid_status[unknown] PASSED
tests/test_models.py: test_rejects_invalid_status[COMPLETED] PASSED
tests/test_models.py: test_rejects_invalid_status[-None] PASSED

----- coverage: 87% -----
```

What to Test (and What Not To)

Test:

- ☐ Transformations — given this input, do I get this output?
- ☐ Validation — does bad data get rejected with the right error?
- ☐ Edge cases — empty DataFrames, NULL values, duplicates
- ☐ API client mocking — does my client handle errors gracefully?

Don't test:

- ☐ Pandas/Polars internals — they work
- ☐ Database connectivity in unit tests — that's integration testing
- ☐ 100% coverage as a goal — test the risky parts

⚠ Common Mistake

Don't test against real APIs or databases in unit tests. That's slow, flaky, and requires credentials. Mock everything external. Save real integration tests for your CI/CD pipeline.

☐ Checkpoint

1. Why use fixtures instead of creating test data inside each test function?
2. What does `@pytest.mark.parametrize` do, and when is it useful?
3. Why should you mock API calls in unit tests instead of hitting the real API?

3.7 Packaging and Dependency Management

TL;DR

- `pyproject.toml` replaces `setup.py` and `requirements.txt` for modern Python projects.
- Use version ranges in `pyproject.toml` for flexibility, lock files for production reproducibility.
- `uv` is the fast (Rust-based) alternative to `pip` — 10x faster installs.

`requirements.txt` is fine for small projects. But when you need dev dependencies, optional extras, entry-point scripts, and version pinning — it falls apart. `pyproject.toml` is the modern answer.


```

[project]
name = "shopfast-pipeline"
version = "0.1.0"
description = "Data pipeline for ShopFast e-commerce"
readme = "README.md"
requires-python = ">=3.11"
authors = [
    {name = "George Yates", email = "george@example.com"},
]

# Production dependencies (version ranges for flexibility)
dependencies = [
    "httpx>=0.27,<1.0",
    "pandas>=2.2,<3.0",
    "polars>=0.20,<1.0",
    "duckdb>=0.10,<1.0",
    "pyarrow>=16.0,<17.0",
    "pydantic>=2.7,<3.0",
    "pydantic-settings>=2.0,<3.0",
    "psycpg2-binary>=2.9,<3.0",
    "sqlalchemy>=2.0,<3.0",
    "tenacity>=8.0,<9.0",
    "python-dotenv>=1.0,<2.0",
    "rich>=13.0,<14.0",
]

# Dev-only dependencies (not needed in production)
[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-cov>=5.0",
    "ruff>=0.4",
    "mypy>=1.10",
]

# CLI entry points
[project.scripts]
extract-orders = "shopfast.extractors.api_client:main"

[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.backends._legacy:_Backend"

[tool.setuptools.packages.find]
where = ["src"]

# Linter config
[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "UP"]

```

```
# Type checker config
[tool.mypy]
python_version = "3.11"
warn_return_any = true
warn_unused_configs = true
```

Installing Your Project

```
# Install in development mode (editable – code changes take effect immediately)
pip install -e ".[dev]"

# Now you can import your code from anywhere
python -c "from shopfast.config import get_settings; print(get_settings())"

# And run your entry-point scripts
extract-orders # calls shopfast.extractors.api_client:main
```

uv — The Fast Alternative to pip

```
# Install uv
pip install uv

# Create venv + install deps (10x faster than pip)
uv venv
uv pip install -e ".[dev]"

# Lock dependencies for reproducibility
uv pip compile pyproject.toml -o requirements.lock
```

uv is written in Rust and installs packages insanely fast. It's becoming the standard. But pip still works fine.

Lock Files for Reproducibility

```
# Generate a lock file with exact versions
pip freeze > requirements.lock

# In CI/CD or production, install from the lock file
pip install -r requirements.lock
```

Your **pyproject.toml** specifies **version ranges** (`>=2.2, <3.0`) for development flexibility. Your lock file pins **exact versions** (`pandas==2.2.2`) for production reproducibility.

⚠ Common Mistake

Don't `pip install pandas` without specifying a version range in your project file. Six months later, Pandas 3.0 comes out with breaking changes and your pipeline explodes. Always pin major versions:

```
"pandas>=2.2,<3.0"
```

□ Checkpoint

1. What's the difference between `dependencies` and `[project.optional-dependencies]` in `pyproject.toml`?
2. Why use version ranges in `pyproject.toml` but exact versions in a lock file?
3. What does `pip install -e ".[dev]"` do, and why is the `-e` flag useful during development?

Module 3 Project: Python Data Ingestion Pipeline

Objective

Build a complete Python data ingestion pipeline that pulls from a REST API, validates with Pydantic, converts to Parquet, and includes unit tests.

Prerequisites

- Module 0 environment setup complete
- Familiarity with all concepts from Lessons 3.1–3.7

Expected Time

2–3 hours

Starter Code

`modules/module-3/starter/` contains:

- `mock_api/` — A Flask/FastAPI mock API server that simulates the ShopFast API
- `pyproject.toml` — Project skeleton
- `src/shopfast/__init__.py` — Empty package
- `tests/conftest.py` — Some fixture starters
- `data/expected_output.parquet` — Expected output for validation

The Mock API

The mock API (runs on `http://localhost:8000`) provides:

Endpoint	Description
<code>GET /v1/products</code>	Paginated product catalog (200 products, 50 per page)
<code>GET /v1/orders?date=YYYY-MM-DD</code>	Orders for a given date (~500/day)
<code>GET /v1/customers/{id}</code>	Customer details

Constraints:

- Rate limit: 10 requests per second (returns 429 if exceeded)
- Some records intentionally have bad data (null prices, invalid dates, extra fields)

Step-by-Step

STEP 1: PROJECT SETUP

```
cd modules/module-3/starter
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
docker compose up mock-api -d
```

STEP 2: CREATE PYDANTIC MODELS

`src/shopfast/models.py` — Define:

- `Product` model with validation (positive prices, valid categories)
- `Order` model with nested `OrderItem` list
- `Customer` model
- Custom validators to handle the API's messy data: string prices, null fields, unexpected formats

STEP 3: BUILD THE API CLIENT

`src/shopfast/extractors/api_client.py` — Implement:

- Paginated fetching for products and orders
- Rate limit handling (respect 429 + Retry-After)
- Retry on transient failures (5xx, connection errors)
- Logging at each step

STEP 4: BUILD THE PIPELINE

```
# src/shopfast/pipeline.py
def run_daily_extract(date: str) -> dict:
    """
    Full daily extraction pipeline.

    Returns:
        dict with keys: products_count, orders_count, errors_count,
                        output_path, duration_seconds
    """
    # 1. Extract products (full refresh daily – small table)
    # 2. Extract orders for the given date (incremental)
    # 3. Validate all records with Pydantic
    # 4. Log validation errors to data/errors/{date}_errors.jsonl
    # 5. Convert valid records to DataFrames
    # 6. Save as Parquet to data/output/{date}/
    #     - products.parquet
    #     - orders.parquet
    # 7. Print summary statistics
    pass
```

STEP 5: WRITE TESTS

Minimum 10 test cases across:

- `test_models.py` — Pydantic validation (valid data, invalid data, edge cases)
- `test_api_client.py` — API client with mocked responses (pagination, rate limiting, errors)
- `test_pipeline.py` — Full pipeline with mocked API

STEP 6: RUN AND VERIFY

```
# Run the pipeline
python -m shopfast.pipeline --date 2026-02-18

# Run tests
pytest -v --cov=shopfast --cov-report=term-missing

# Verify output
python -c "
import pandas as pd
df = pd.read_parquet('data/output/2026-02-18/orders.parquet')
print(f'Orders: {len(df)}')
print(f'Columns: {list(df.columns)}')
print(df.dtypes)
print(df.head())
"
```


Expected Output

```
data/output/2026-02-18/  
├─ products.parquet      (~200 valid products)  
├─ orders.parquet        (~480 valid orders out of ~500)  
└─ errors/  
    └─ 2026-02-18_errors.jsonl (~20 rejected records with reasons)
```

Evaluation Criteria

- ☐ Pydantic models catch all bad data (null prices, invalid dates, wrong types)
- ☐ API client handles pagination correctly (all pages fetched)
- ☐ Rate limiting handled gracefully (no crashes on 429)
- ☐ Parquet files have correct schemas (proper types, not all strings)
- ☐ Bad records saved to error file with clear error messages
- ☐ Tests pass with >80% code coverage
- ☐ Code follows the project structure from Lesson 3.1
- ☐ No hardcoded credentials or URLs

Module 4: Apache Airflow — Orchestration

What you'll learn: Build production-grade DAGs, not toy examples. Airflow is the most in-demand orchestration tool in data engineering, and by the end of this module you'll know how to use it like a practitioner.

4.1 Why Orchestration Matters (Cron Is Not Enough)

TL;DR

- Cron jobs can't handle dependencies, retries, visibility, or backfills
- Orchestrators manage the *flow* of your pipeline, not just the timing
- Airflow is the industry standard (~70% of DE job postings mention it)
- Don't wait for a production incident to migrate off cron — start with Airflow

You've got a Python script. It pulls data from an API, transforms it, dumps it into Postgres. Works great. So you throw it on a cron job — `0 6 * * *` — runs every morning at 6 AM. Life is good.

Then one morning, the API is down at 6 AM. Your script fails silently. Nobody knows until 2 PM when the VP of Sales asks why the dashboard is empty. You SSH into the server, check the logs, re-run it manually. Crisis averted.

But then next week, your script runs fine, but the *downstream* dbt model that depends on it didn't wait — it ran at 6:15 AM like it always does, but your script took 45 minutes instead of 10 because the API was slow. Now your dashboard has yesterday's data and nobody knows.

That's why cron is not enough.

The Four Problems Orchestration Solves

Orchestration addresses four problems that cron simply can't:

1. **Dependencies** — "Don't run step 3 until steps 1 and 2 are done." Cron doesn't know about dependencies. It just fires at a time.
2. **Retries and error handling** — If something fails, retry it 3 times with a 5-minute delay. If it still fails, alert someone. Cron? It either ran or it didn't.
3. **Visibility** — A UI where you can see: what ran, when, did it succeed, how long did it take, what's running right now. Cron gives you... log files. If you remembered to set up logging.
4. **Backfills** — "The API was returning wrong data for the last 3 days. Re-run everything for those dates." With cron, you're writing bash scripts. With orchestration, it's a button click.

□ Key Concept: Orchestration vs. Scheduling

Cron is a *scheduler* — it fires jobs at specific times. An orchestrator like Airflow is a *workflow manager* — it manages the flow of tasks, their dependencies, retries, and state. Think of the difference between a kitchen timer and a head chef who coordinates every dish to arrive at the table together.

The Orchestration Landscape (2025-2026)

The most common orchestrators you'll encounter:

- **Apache Airflow** — the industry standard, and what we're learning
- **Prefect** — newer, more Pythonic, gaining traction
- **Dagster** — asset-centric, great developer experience
- **Mage** — newer, notebook-style, gaining popularity

We're teaching Airflow because ~70% of job postings mention it and it's what you'll most likely encounter in the real world.

⚠ Common Mistake

People build a pipeline with cron, it works for 3 months, then they spend 2 weeks migrating to Airflow in a panic after a major incident. Just start with Airflow. The learning curve is worth it.

□ Checkpoint

1. Name two problems that cron can't solve but Airflow can.
2. What does "backfill" mean in the context of orchestration?
3. Why would you choose Airflow over Prefect or Dagster for a new job?

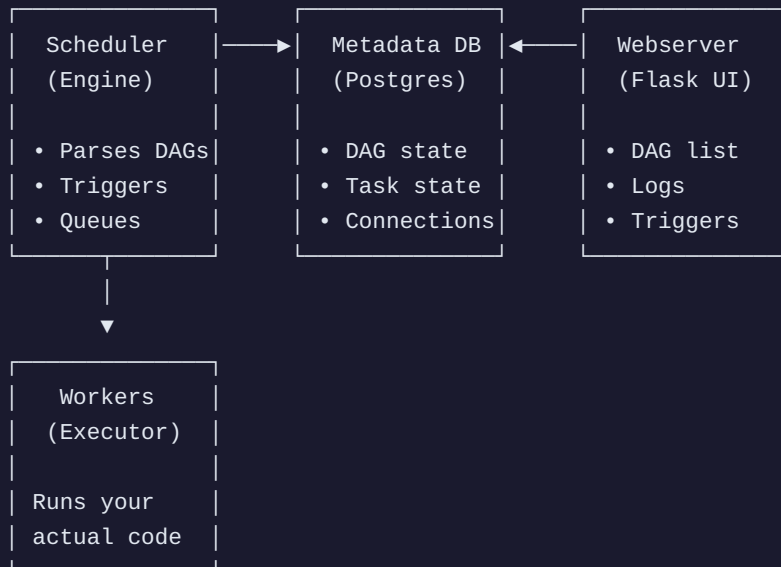
4.2 Airflow Architecture — Scheduler, Webserver, Workers, Metadata DB

TL;DR

- Airflow has four components: Metadata DB, Scheduler, Webserver, and Workers
- The Scheduler is the engine — it decides what runs and when
- The Webserver is just a UI; it doesn't execute tasks
- Use LocalExecutor for learning and small-to-medium production workloads

Before you write a single line of DAG code, you need a mental model of what's actually happening when Airflow runs your pipeline.

[DIAGRAM: Airflow Architecture]



The Four Components

1. The Metadata Database (Postgres)

This is Airflow's brain. It stores everything: what DAGs exist, when they last ran, what state each task is in, connections, variables, user info. It's just a Postgres database (or MySQL, but use Postgres).

Every other component reads from and writes to this database. If the metadata DB goes down, Airflow is dead.

2. The Scheduler

This is the engine. The scheduler does three things in a loop:

- Scans your DAG files to find new/updated DAGs
- Checks if any DAG runs need to be triggered (based on schedule)
- Checks if any tasks are ready to run (dependencies met) and queues them

The scheduler is the *only* component that decides what runs and when. It's the most critical piece.

3. The Webserver

This is the UI — the thing you open in your browser. It reads from the metadata DB and shows you DAG status, task logs, Gantt charts, and trigger buttons.

Important: the webserver does NOT run your tasks. It's just a Flask app showing you what's happening. You could turn it off and Airflow would still run pipelines. You just couldn't see them.

4. The Workers (Executor)

This is where your actual code runs. When the scheduler says "run this task," a worker picks it up and executes it. How workers behave depends on your *executor*:

Executor	Description	When to Use
SequentialExecutor	One task at a time	Development only. Never production.
LocalExecutor	Multiple tasks as separate processes	Small-to-medium scale. What we'll use.
CeleryExecutor	Distributes across machines via Redis/ RabbitMQ	Production at scale.
KubernetesExecutor	Spins up a K8s pod per task	Modern cloud-native deployments.

Setting Up Airflow with Docker Compose

Here's a complete Docker Compose file for a local Airflow environment:

```

# docker-compose.yml
# Airflow with LocalExecutor – suitable for learning and small-to-medium workloads
version: '3.8'

x-airflow-common: &airflow-common
  image: apache/airflow:2.9.1-python3.11
  environment: &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: LocalExecutor # Multiple tasks in parallel
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/
  airflow
    AIRFLOW__CORE__FERNET_KEY: '' # Encryption key (set in
  prod)
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true' # New DAGs start paused
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
# No example DAGs cluttering the UI
    AIRFLOW__API__AUTH_BACKENDS:
'airflow.api.auth.backend.basic_auth,airflow.api.auth.backend.session'
    _PIP_ADDITIONAL_REQUIREMENTS: ${_PIP_ADDITIONAL_REQUIREMENTS:-}
  volumes:
    - ./dags:/opt/airflow/dags # Your DAG files
    - ./logs:/opt/airflow/logs # Task logs
    - ./plugins:/opt/airflow/plugins # Custom plugins
    - ./data:/opt/airflow/data # Data staging area
  user: "${AIRFLOW_UID:-50000}:0"
  depends_on:
    postgres:
      condition: service_healthy

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
      start_period: 5s
    ports:
      - "5432:5432"

  airflow-init:
    <<: *airflow-common
    entrypoint: /bin/bash
    command:
      - -c
      - |
        airflow db migrate
        airflow users create \

```

```

        --username admin \
        --firstname Admin \
        --lastname User \
        --role Admin \
        --email admin@example.com \
        --password admin

depends_on:
  postgres:
    condition: service_healthy

airflow-webserver:
  <<: *airflow-common
  command: webserver
  ports:
    - "8080:8080"
  healthcheck:
    test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  depends_on:
    airflow-init:
      condition: service_completed_successfully

airflow-scheduler:
  <<: *airflow-common
  command: scheduler
  healthcheck:
    test: ["CMD-SHELL", "airflow jobs check --job-type SchedulerJob --hostname $(hostname)"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  depends_on:
    airflow-init:
      condition: service_completed_successfully

volumes:
  postgres-data:

```

Start it up:

```
# Set the Airflow UID (avoids permission issues on mounted volumes)
echo -e "AIRFLOW_UID=$(id -u)" > .env

# Create the required directories
mkdir -p dags logs plugins data

# Start everything
docker compose up -d

# Check that everything is running
docker compose ps
```

Expected output:

NAME	SERVICE	STATUS
airflow-init	airflow-init	exited (0)
airflow-scheduler	airflow-scheduler	running (healthy)
airflow-webserver	airflow-webserver	running (healthy)
postgres	postgres	running (healthy)

Open `http://localhost:8080` — username `admin`, password `admin`. You'll see an empty DAGs page.

⚠ Common Mistake

Not setting `AIRFLOW_UID` — you'll get permission errors on the mounted volumes. Always create the `.env` file first.

Using `SequentialExecutor` in production — it runs one task at a time. Fine for dev, disaster for prod.

Forgetting `LOAD_EXAMPLES: 'false'` — you'll get 30+ example DAGs cluttering your UI.

□ Checkpoint

1. Which component decides what tasks to run and when?
2. What happens if the metadata database goes down?
3. Why should you never use `SequentialExecutor` in production?

4.3 Your First DAG — Tasks, Dependencies, Operators

TL;DR

- A DAG is a Directed Acyclic Graph — your pipeline definition
- Tasks are units of work; Operators define the *type* of task
- The `>>` operator defines dependencies between tasks
- Always set `catchup=False` during development

Time to write some code. By the end of this section, you'll have a working DAG in Airflow — not a "hello world," but an actual pipeline that does something useful.

Vocabulary

□Key Concept: DAG Terminology

- **DAG** = Directed Acyclic Graph. Your pipeline definition — what tasks to run and in what order. "Directed" means tasks flow in one direction (A → B). "Acyclic" means no loops.
- **Task** = A single unit of work. "Extract data from API," "Transform the data," "Load into Postgres."
- **Operator** = The *type* of task. `PythonOperator` runs a Python function. `BashOperator` runs a bash command.
- **Task Instance** = A specific run of a task at a specific time. Your "extract" task on February 19th at 6 AM is one task instance.

Building an ETL DAG

This DAG will:

1. Create a JSON file with sample data (simulating an API extraction)
2. Validate the data quality
3. Transform it to CSV with computed fields
4. Create a Postgres table
5. Send a notification

Save this as `dags/my_first_dag.py`:

```

"""
My First Airflow DAG
Demonstrates: tasks, dependencies, operators, and basic patterns.
"""

from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from airflow.providers.postgres.operators.postgres import PostgresOperator
import csv
import os

# Default arguments applied to all tasks in this DAG
default_args = {
    "owner": "george",          # Who owns this DAG
    "depends_on_past": False,    # Don't wait for previous runs to succeed
    "email_on_failure": False,  # We'll use callbacks instead
    "email_on_retry": False,
    "retries": 2,               # Retry failed tasks twice
    "retry_delay": timedelta(minutes=2), # Wait 2 min between retries
}

DATA_PATH = "/opt/airflow/data"

def extract_data(**kwargs):
    """Simulate extracting data from an API.
    In production, this would be requests.get("https://api.example.com/orders")
    """
    import json

    orders = [
        {"order_id": 1, "customer": "Alice", "amount": 99.99, "date": "2026-02-19"},
        {"order_id": 2, "customer": "Bob", "amount": 149.50, "date": "2026-02-19"},
        {"order_id": 3, "customer": "Charlie", "amount": 29.99, "date": "2026-02-19"},
        {"order_id": 4, "customer": "Diana", "amount": 199.00, "date": "2026-02-19"},
        {"order_id": 5, "customer": "Eve", "amount": 75.25, "date": "2026-02-19"},
    ]

    filepath = os.path.join(DATA_PATH, "raw_orders.json")
    with open(filepath, "w") as f:
        json.dump(orders, f)

    print(f"Extracted {len(orders)} orders to {filepath}")
    return filepath

def validate_data(**kwargs):
    """Check data quality before loading."""
    import json

    filepath = os.path.join(DATA_PATH, "raw_orders.json")

```

```

with open(filepath, "r") as f:
    orders = json.load(f)

# Quality checks
assert len(orders) > 0, "No orders found!"
for order in orders:
    assert "order_id" in order, f"Missing order_id in {order}"
    assert "amount" in order, f"Missing amount in {order}"
    assert order["amount"] > 0, f"Invalid amount: {order['amount']}"

print(f"Validation passed: {len(orders)} orders, all valid")
return len(orders)

def transform_data(**kwargs):
    """Transform raw data into a clean CSV for loading."""
    import json

    filepath = os.path.join(DATA_PATH, "raw_orders.json")
    with open(filepath, "r") as f:
        orders = json.load(f)

    # Add computed fields
    for order in orders:
        order["amount_with_tax"] = round(order["amount"] * 1.08, 2)
        order["processed_at"] = datetime.now().isoformat()

    # Write as CSV for Postgres COPY
    csv_path = os.path.join(DATA_PATH, "clean_orders.csv")
    with open(csv_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=orders[0].keys())
        writer.writeheader()
        writer.writerows(orders)

    print(f"Transformed {len(orders)} orders → {csv_path}")
    return csv_path

# Define the DAG
with DAG(
    dag_id="my_first_pipeline",
    default_args=default_args,
    description="Extract, validate, transform, and load order data",
    schedule="@daily",  # Runs once per day
    start_date=datetime(2026, 2, 1),
    catchup=False,  # Don't backfill past dates
    tags=["tutorial", "etl"],
) as dag:

    extract = PythonOperator(
        task_id="extract_data",
        python_callable=extract_data,
    )

```

```

validate = PythonOperator(
    task_id="validate_data",
    python_callable=validate_data,
)

transform = PythonOperator(
    task_id="transform_data",
    python_callable=transform_data,
)

create_table = PostgresOperator(
    task_id="create_table",
    postgres_conn_id="postgres_default",
    sql="""
        CREATE TABLE IF NOT EXISTS orders (
            order_id INTEGER PRIMARY KEY,
            customer VARCHAR(100),
            amount DECIMAL(10,2),
            date DATE,
            amount_with_tax DECIMAL(10,2),
            processed_at TIMESTAMP
        );
    """
)

notify = BashOperator(
    task_id="notify_complete",
    bash_command='echo "Pipeline complete at $(date). Orders loaded successfully!"',
)

# Define dependencies – this is the "graph" part of DAG
extract >> validate >> transform >> create_table >> notify

```

Expected output (in Airflow task logs):

```

[extract_data] Extracted 5 orders to /opt/airflow/data/raw_orders.json
[validate_data] Validation passed: 5 orders, all valid
[transform_data] Transformed 5 orders to /opt/airflow/data/clean_orders.csv
[notify_complete] Pipeline complete at Thu Feb 19 06:00:00 UTC 2026. Orders loaded
successfully!

```

Key Concepts Breakdown

default_args — Applied to every task. **retries: 2** means if a task fails, Airflow tries again twice.

retry_delay is the wait between retries.

schedule="@daily" — Runs once per day. You can also use cron syntax: **"0 6 * * *"** for 6 AM daily.

catchup=False — If your **start_date** is in the past and catchup is True, Airflow tries to run the DAG for every day since start_date. That's usually not what you want during development.

`>>` — The bitshift operator defines dependencies. `extract >> validate` means "run extract first, then validate."

Verify Your DAG

```
# Check that Airflow can parse your DAG (catches syntax errors)
docker compose exec airflow-scheduler airflow dags list
```

Expected output:

```
dag_id          | filepath          | owner   | paused
=====|=====|=====|=====
my_first_pipeline | my_first_dag.py | george  | True
```

Then go to the Airflow UI, find `my_first_pipeline`, unpause it, and trigger it manually. Watch the tasks go green one by one in the Graph view.

⚠ Common Mistake

Import errors at the top level — If your DAG file has an import error, the ENTIRE file is silently skipped. Always verify with `airflow dags list`.

Heavy logic at module level — Don't do API calls or DB queries outside your task functions. The scheduler reads your DAG file every 30 seconds. A `requests.get()` at the top level hits the API every 30 seconds.

Forgetting `catchup=False` — You'll trigger hundreds of backfill runs and wonder why your Airflow is on fire.

□ Try It Yourself

Modify the DAG to add a sixth task that counts the rows in the CSV file and prints the total revenue. Chain it after `transform` but before `create_table`.

□ Checkpoint

1. What does "acyclic" mean in DAG, and why does it matter?
2. What happens if you forget to set `catchup=False` with a `start_date` in the past?
3. Where should heavy logic (API calls, DB queries) live — at the module level or inside task functions?

4.4 TaskFlow API — The Modern Way to Write DAGs

TL;DR

- The TaskFlow API uses `@task` decorators instead of explicit Operator objects
- Return values automatically become XComs (cross-task communication)
- Dependencies are inferred from function calls — no `>>` needed
- XCom is for small data only (configs, paths, counts) — never DataFrames

The DAG we just wrote works, but it's kind of verbose. Defining `PythonOperator` separately, passing `python_callable`, managing data between tasks with file paths... There's a cleaner way. Airflow 2.0 introduced the TaskFlow API, and once you use it, you'll never go back.

The TaskFlow Rewrite

```

"""
TaskFlow API version of our pipeline.
Cleaner, more Pythonic, and handles data passing automatically.
"""

from datetime import datetime, timedelta
from airflow.decorators import dag, task
import json
import os

DATA_PATH = "/opt/airflow/data"

@dag(
    dag_id="taskflow_pipeline",
    schedule="@daily",
    start_date=datetime(2026, 2, 1),
    catchup=False,
    default_args={
        "owner": "george",
        "retries": 2,
        "retry_delay": timedelta(minutes=2),
    },
    tags=["tutorial", "taskflow"],
)
def my_taskflow_pipeline():
    """An ETL pipeline using the TaskFlow API."""

    @task()
    def extract() -> list[dict]:
        """Extract orders from source (simulated)."""
        orders = [
            {"order_id": 1, "customer": "Alice", "amount": 99.99, "date": "2026-02-19"},
            {"order_id": 2, "customer": "Bob", "amount": 149.50, "date": "2026-02-19"},
            {"order_id": 3, "customer": "Charlie", "amount": 29.99, "date":
"2026-02-19"},
            {"order_id": 4, "customer": "Diana", "amount": 199.00, "date":
"2026-02-19"},
            {"order_id": 5, "customer": "Eve", "amount": 75.25, "date": "2026-02-19"},
        ]
        print(f"Extracted {len(orders)} orders")
        return orders # Automatically stored as XCom

    @task()
    def validate(orders: list[dict]) -> list[dict]:
        """Validate data quality."""
        assert len(orders) > 0, "No orders!"
        for order in orders:
            assert order["amount"] > 0, f"Bad amount: {order['amount']}"
        print(f"Validated {len(orders)} orders – all clean")
        return orders # Pass along to next task

    @task()

```



```
def transform(orders: list[dict]) -> list[dict]:
    """Add computed fields."""
    for order in orders:
        order["amount_with_tax"] = round(order["amount"] * 1.08, 2)
        order["processed_at"] = datetime.now().isoformat()
    print(f"Transformed {len(orders)} orders")
    return orders

@task()
def load(orders: list[dict]):
    """Load into destination (simulated)."""
    filepath = os.path.join(DATA_PATH, "final_orders.json")
    with open(filepath, "w") as f:
        json.dump(orders, f, indent=2)
    print(f"Loaded {len(orders)} orders to {filepath}")

# Just call functions – Airflow infers dependencies from data flow
raw_data = extract()
validated_data = validate(raw_data)
transformed_data = transform(validated_data)
load(transformed_data)

# Don't forget this line! Instantiates the DAG.
my_taskflow_pipeline()
```

Expected output (task logs):

```
[extract] Extracted 5 orders
[validate] Validated 5 orders - all clean
[transform] Transformed 5 orders
[load] Loaded 5 orders to /opt/airflow/data/final_orders.json
```

Old Style vs. TaskFlow — Side by Side

Old Style	TaskFlow
Define function separately	Decorate function with <code>@task</code>
Create <code>PythonOperator(python_callable=func)</code>	Just call the function
Manual XCom push/pull	Return values = automatic XCom
<code>extract >> validate >> transform</code>	<code>validate(extract())</code> — implicit

❑ Key Concept: XCom (Cross-Communication)

XCom is how tasks pass data to each other. With TaskFlow, return values are automatically serialized to JSON and stored in the metadata database. The next task receives them as function arguments. Simple.

Size limits matter: XCom stores data in Postgres. Small data (configs, file paths, row counts, small lists) is perfect. Large data (DataFrames, millions of rows) will either crash your metadata DB or make it enormous. For large datasets, write to a file and pass the *path* via XCom.

⚠ Common Mistake

Passing DataFrames through XCom — A 1GB DataFrame serialized to JSON in Postgres is a recipe for disaster. Pass file paths instead.

Forgetting to instantiate the DAG — See that last line `my_taskflow_pipeline()` ? Without it, the DAG won't appear. Easy to miss.

❑ Checkpoint

1. What does `@task()` replace in the old-style DAG definition?
2. Why shouldn't you pass a Pandas DataFrame through XCom?
3. What happens if you forget the `my_taskflow_pipeline()` call at the bottom?

4.5 Connections, Variables, and Secrets Management

TL;DR

- Use **Connections** for external credentials (databases, APIs, cloud)
- Use **Variables** for non-sensitive config (thresholds, paths, flags)
- Never call `Variable.get()` at the module level — put it inside tasks
- In production, use a secrets backend (AWS Secrets Manager, Vault, etc.)

Pop quiz: what's the fastest way to get fired as a data engineer? Hardcode your database password in a DAG file and push it to GitHub. Let's talk about how to manage credentials properly.

Connections — External System Credentials

```
# Set via CLI
docker compose exec airflow-scheduler airflow connections add 'postgres_warehouse' \
  --conn-type 'postgres' \
  --conn-host 'postgres' \
  --conn-schema 'warehouse' \
  --conn-login 'warehouse_user' \
  --conn-password 'warehouse_pass' \
  --conn-port 5432

# Or via environment variable (useful for Docker/K8s)
export AIRFLOW_CONN_POSTGRES_WAREHOUSE='postgresql://warehouse_user:warehouse_pass@postgres:5432/warehouse'
```

Then use it in your DAG:

```
from airflow.providers.postgres.hooks.postgres import PostgresHook

@task()
def query_warehouse():
    # The Hook looks up credentials from the connection by ID
    hook = PostgresHook(postgres_conn_id="postgres_warehouse")
    df = hook.get_pandas_df("SELECT COUNT(*) as cnt FROM orders")
    print(f"Row count: {df['cnt'].iloc[0]}")
    return df['cnt'].iloc[0]
```

Variables — Non-Sensitive Configuration

```
from airflow.models import Variable

@task()
def check_threshold():
    # Variable.get() queries the metadata DB
    threshold = int(Variable.get("my_threshold", default_var=50))
    print(f"Using threshold: {threshold}")
```

Set variables via CLI: `airflow variables set my_threshold 100`

⚠ Common Mistake: `Variable.get()` at Module Level

The scheduler parses DAG files every 30 seconds. If `Variable.get()` is at the top level, that's a database query every 30 seconds per DAG.

```
```python
```

## ❑ BAD — runs every time scheduler parses the file (every 30 seconds)

```
threshold = Variable.get("my_threshold")

@task()
def my_task():
 # ❑ GOOD — runs only when the task executes
 threshold = Variable.get("my_threshold")
 ...
```

## Secrets Backend — For Production

In production, use an external secrets manager instead of storing credentials in the Airflow metadata DB:

```
In airflow.cfg or environment variables:
#
AIRFLOW__SECRETS__BACKEND=airflow.providers.amazon.aws.secrets.secrets_manager.SecretsManager
AIRFLOW__SECRETS__BACKEND_KWARGS={"connections_prefix": "airflow/connections",
"variables_prefix": "airflow/variables"}
```

This pulls secrets from AWS Secrets Manager, HashiCorp Vault, or GCP Secret Manager transparently. Your DAG code doesn't change — Airflow just looks up the credentials from a different place.

### ❑ Pro Tip

Use the "Test" button in the Airflow UI when setting up connections. It validates the connection immediately and saves you debugging time later.

## ❑ Checkpoint

1. What's the difference between a Connection and a Variable in Airflow?
2. Why is `Variable.get()` at the module level dangerous?
3. What's a secrets backend, and when should you use one?

## 4.6 Error Handling — Retries, SLAs, Alerting

### TL;DR

- Always configure retries with exponential backoff for external calls
- Set `execution_timeout` on every task to prevent stuck jobs
- Use callbacks for Slack/PagerDuty alerts on final failure
- SLA monitoring alerts you if a DAG isn't done by a deadline

Data pipelines break. APIs go down, databases timeout, files are malformed, services hit rate limits. The question isn't IF your pipeline will fail, it's HOW it handles failure.

### Retries with Exponential Backoff

```
@task(
 retries=3, # Try 3 more times after first failure
 retry_delay=timedelta(minutes=5), # Start with 5 min delay
 retry_exponential_backoff=True, # 5 min → 10 min → 20 min
 max_retry_delay=timedelta(minutes=30),
)
def flaky_api_call():
 """Call an API that sometimes fails."""
 import requests
 response = requests.get("https://api.example.com/data", timeout=30)
 response.raise_for_status() # Raises on 4xx/5xx
 return response.json()
```

### ❏ Pro Tip

`retry_exponential_backoff=True` is essential for API calls. If an API is overloaded, hammering it every 5 minutes makes things worse. Exponential backoff gives it time to recover.

### Timeouts

```
@task(
 execution_timeout=timedelta(minutes=30), # Kill task if it runs > 30 min
)
def long_running_transform():
 """Transform that should complete in 30 minutes."""
 # ... processing code ...
 pass
```

Without a timeout, a stuck task can block your entire pipeline for hours.

## Alerting with Callbacks

This is how you actually get notified when things go wrong:

```

from airflow.decorators import dag, task
from airflow.models import Variable
from datetime import datetime, timedelta
import requests

def send_slack_alert(context):
 """Send a Slack notification on task failure."""
 task_instance = context.get("task_instance")
 dag_id = context.get("dag").dag_id
 task_id = task_instance.task_id
 log_url = task_instance.log_url

 message = (
 f"❏ *Task Failed*\n"
 f"DAG: `{dag_id}`\n"
 f"Task: `{task_id}`\n"
 f"Execution: {context.get('execution_date')}\n"
 f"<{log_url}|View Logs>"
)

 webhook_url = Variable.get("slack_webhook_url")
 requests.post(webhook_url, json={"text": message})

def send_success_notification(context):
 """Notify on DAG success."""
 dag_id = context.get("dag").dag_id
 webhook_url = Variable.get("slack_webhook_url")
 requests.post(webhook_url, json={
 "text": f"❏ DAG `{dag_id}` completed successfully!"
 })

@dag(
 dag_id="alerting_pipeline",
 schedule="@daily",
 start_date=datetime(2026, 2, 1),
 catchup=False,
 default_args={
 "owner": "george",
 "retries": 3,
 "retry_delay": timedelta(minutes=5),
 "on_failure_callback": send_slack_alert, # Per-task failure alert
 },
 on_success_callback=send_success_notification, # DAG-level success
)
def alerting_pipeline():

 @task()
 def risky_extraction():
 """This might fail – and that's okay, we have retries and alerts."""
 import random

```

```

 if random.random() < 0.3:
 raise Exception("API rate limited! (simulated)")
 return {"status": "success", "rows": 1000}

@task()
def process(data: dict):
 print(f"Processing {data['rows']} rows")

data = risky_extraction()
process(data)

alerting_pipeline()

```

## SLA Monitoring

SLAs alert you when a DAG isn't done by a certain time — critical for the CFO's morning report that better be ready by 7 AM:

```

@task(sla=timedelta(hours=1)) # Must complete within 1 hour of DAG start
def critical_task():
 pass

```

### ⚠ Common Mistake

**No retries on API calls** — Always add retries. External APIs are unreliable.

**No execution\_timeout** — A stuck task can silently block everything for hours.

**Alerting on every retry** — Don't spam Slack on retries. The `on_failure_callback` fires only on *final* failure (after all retries), which is the right behavior.

## □ Checkpoint

1. What does `retry_exponential_backoff=True` do, and why is it useful for API calls?
2. What's the difference between `on_failure_callback` on `default_args` vs. on the `@dag` decorator?
3. Why should you always set `execution_timeout`?



## 4.7 Dynamic DAGs and DAG Factories

### TL;DR

- DAG factories generate multiple DAGs from a config — no copy-pasting
- Dynamic task mapping ( `.expand()` ) creates parallel tasks at runtime
- Use factory functions (not bare loops) to avoid Python closure issues
- Don't generate more than ~200 DAGs from a factory or you'll slow the scheduler

If you work at a company with 50 clients and each one gets the same ETL pipeline with different configs, you're not going to copy-paste a DAG file 50 times. You're going to use a DAG factory.

## Pattern 1: Config-Driven DAGs

```

"""
dags/dag_factory.py
Generate one DAG per client from a configuration list.
"""

from datetime import datetime, timedelta
from airflow.decorators import dag, task

Config could come from a file, database, or API
CLIENTS = [
 {
 "client_id": "acme_corp",
 "api_url": "https://api.acme.com/v1/orders",
 "schedule": "0 6 * * *", # 6 AM daily
 "owner": "george",
 },
 {
 "client_id": "globex",
 "api_url": "https://api.globex.com/v2/data",
 "schedule": "0 8 * * *", # 8 AM daily
 "owner": "george",
 },
 {
 "client_id": "initech",
 "api_url": "https://initech.io/api/reports",
 "schedule": "0 7 * * 1-5", # 7 AM weekdays only
 "owner": "george",
 },
]

def create_client_dag(client_config: dict):
 """Factory function that creates a DAG for a single client."""

 client_id = client_config["client_id"]

 @dag(
 dag_id=f"client_pipeline_{client_id}",
 schedule=client_config["schedule"],
 start_date=datetime(2026, 2, 1),
 catchup=False,
 default_args={
 "owner": client_config["owner"],
 "retries": 3,
 "retry_delay": timedelta(minutes=5),
 },
 tags=["client", client_id],
)
 def client_pipeline():

 @task()
 def extract(api_url: str) -> dict:
 print(f"Extracting from {api_url}")

```

```

 return {"client": client_id, "rows": 100} # Simulated

 @task()
 def transform(data: dict) -> dict:
 data["transformed"] = True
 data["processed_at"] = datetime.now().isoformat()
 return data

 @task()
 def load(data: dict):
 print(f>Loading {data['rows']} rows for {data['client']}")

 raw = extract(api_url=client_config["api_url"])
 clean = transform(raw)
 load(clean)

 return client_pipeline()

Generate a DAG for each client
for client in CLIENTS:
 create_client_dag(client)

```

When Airflow parses this file, it creates three DAGs: `client_pipeline_acme_corp`, `client_pipeline_globex`, and `client_pipeline_initech`. Each with its own schedule and config.

## Pattern 2: Dynamic Task Mapping (Airflow 2.3+)

This is even more powerful — expand tasks dynamically at runtime:

```

@dag(
 dag_id="dynamic_mapping_example",
 schedule="@daily",
 start_date=datetime(2026, 2, 1),
 catchup=False,
)
def dynamic_pipeline():

 @task()
 def get_files() -> list[str]:
 """Discover files to process – could be from S3, API, etc."""
 return ["file_a.csv", "file_b.csv", "file_c.csv"]

 @task()
 def process_file(filename: str) -> dict:
 """Process a single file."""
 print(f"Processing {filename}")
 return {"file": filename, "rows": 1000}

 @task()
 def summarize(results: list[dict]):
 """Aggregate results."""
 total = sum(r["rows"] for r in results)
 print(f"Total rows processed: {total}")

 files = get_files()
 # .expand() creates one task instance per item in the list – in parallel!
 results = process_file.expand(filename=files)
 summarize(results)

dynamic_pipeline()

```

The `.expand()` call is the magic. If `get_files()` returns 3 files, Airflow creates 3 parallel `process_file` tasks. If tomorrow it returns 10 files, you get 10 tasks. No code changes needed.

#### ⚠ Common Mistake

**Creating DAGs in a loop with mutable variables** — Classic Python closure issue. Always use a factory function (like `create_client_dag` above) to capture the config properly.

**Too many dynamic DAGs** — 500 DAGs from a factory will slow down the scheduler. If you have that many, consider grouping them.

## □ Checkpoint

1. Why do you need a factory *function* instead of creating DAGs directly in a loop?
2. What does `.expand()` do in dynamic task mapping?
3. At what point should you worry about having too many factory-generated DAGs?

---

## 4.8 Sensors and Triggers — Waiting for Data and Events

### TL;DR

- Sensors wait for a condition to be true before allowing downstream tasks to run
- Always use `mode="reschedule"` or `deferrable=True` to avoid hogging worker slots
- Always set a `timeout` — sensors without timeouts wait forever
- `soft_fail=True` marks as skipped instead of failed on timeout

What if your pipeline shouldn't run at 6 AM — it should run when the data is *ready*? Maybe a vendor uploads a file to S3 sometime between 6 AM and 9 AM. A schedule won't work. You need a sensor.

### □Key Concept: Sensors

A sensor is a special type of operator that periodically checks ("pokes") for a condition. When the condition is met, it succeeds and allows downstream tasks to proceed. Think of it as a bouncer checking IDs — nobody gets in until the condition is verified.

```

from airflow.sensors.filesystem import FileSensor
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from airflow.sensors.external_task import ExternalTaskSensor
from datetime import datetime, timedelta
from airflow.decorators import dag, task

@dag(
 dag_id="sensor_pipeline",
 schedule="0 6 * * *",
 start_date=datetime(2026, 2, 1),
 catchup=False,
)
def sensor_pipeline():

 # Wait for a local file to appear
 wait_for_file = FileSensor(
 task_id="wait_for_file",
 filepath="/opt/airflow/data/incoming/daily_feed.csv",
 poke_interval=60, # Check every 60 seconds
 timeout=3 * 60 * 60, # Give up after 3 hours
 mode="reschedule", # Free up the worker slot while waiting
 soft_fail=True, # Mark as skipped (not failed) on timeout
)

 # Wait for a file in S3
 wait_for_s3 = S3KeySensor(
 task_id="wait_for_s3_file",
 bucket_name="my-data-lake",
 bucket_key="raw/{{ ds }}/orders.parquet", # Jinja template for date
 aws_conn_id="aws_default",
 poke_interval=120,
 timeout=4 * 60 * 60,
 mode="reschedule",
)

 # Wait for another DAG to finish
 wait_for_upstream = ExternalTaskSensor(
 task_id="wait_for_ingestion_dag",
 external_dag_id="data_ingestion_pipeline",
 external_task_id=None, # None = wait for the entire DAG
 timeout=2 * 60 * 60,
 mode="reschedule",
)

 @task()
 def process():
 print("All conditions met – processing!")

 [wait_for_file, wait_for_s3, wait_for_upstream] >> process()

sensor_pipeline()

```

## Key Parameters

- `poke_interval` — How often to check (seconds). Don't set too low or you'll hammer the external system.
- `timeout` — Maximum wait time. After this, the sensor fails (or skips if `soft_fail=True`).
- `mode="reschedule"` — Frees the worker slot between checks. The default `poke` mode keeps the slot occupied the entire time. Always use `reschedule` unless you have a specific reason not to.

## Deferrable Operators (Airflow 2.6+)

Even better than `reschedule` — deferrable operators use almost zero resources while waiting:

```
wait_for_s3 = S3KeySensor(
 task_id="wait_for_s3",
 bucket_name="my-bucket",
 bucket_key="data/{{ ds }}/file.parquet",
 deferrable=True, # Uses the Triggerer – minimal resource usage
)
```

### ⚠ Common Mistake

`mode="poke"` on long-running sensors — Occupies a worker slot the entire time. A sensor waiting 3 hours in poke mode is a wasted worker for 3 hours.

**No timeout** — A sensor without a timeout waits forever. Always set one.

**Poke interval too short** — Checking S3 every 5 seconds is unnecessary and costly (S3 LIST operations aren't free at scale).

## □ Checkpoint

1. What's the difference between `mode="poke"` and `mode="reschedule"` ?
2. Why should you always set a `timeout` on sensors?
3. What does `deferrable=True` do that `reschedule` mode doesn't?

## 4.9 Testing DAGs Locally

### TL;DR

- DAG integrity tests (does it parse?) should run in CI/CD — catches 80% of deployment issues
- Separate business logic from DAG files so you can unit test normally
- Use `airflow dags test` for free integration testing
- Don't test Airflow internals — test YOUR logic



Would you deploy application code without tests? Then why do so many people deploy DAGs without tests? It's easier than you think.

## Level 1: DAG Integrity Tests

Does the DAG even parse? Run this in CI/CD:

```
tests/test_dag_integrity.py
import pytest
from airflow.models import DagBag

@pytest.fixture()
def dagbag():
 return DagBag(dag_folder="dags/", include_examples=False)

def test_no_import_errors(dagbag):
 """Verify no DAG import errors."""
 assert len(dagbag.import_errors) == 0, f"Import errors: {dagbag.import_errors}"

def test_dag_count(dagbag):
 """Verify expected number of DAGs."""
 assert len(dagbag.dags) >= 1, "No DAGs found!"

@pytest.mark.parametrize("dag_id", [
 "my_first_pipeline",
 "taskflow_pipeline",
])
def test_dag_exists(dagbag, dag_id):
 """Check that expected DAGs are loaded."""
 assert dag_id in dagbag.dags, f"DAG '{dag_id}' not found"

def test_no_cycles(dagbag):
 """Verify no DAGs have circular dependencies."""
 for dag_id, dag in dagbag.dags.items():
 assert dag.topological_sort() # Raises if there's a cycle
```

## Level 2: Task Logic Tests

### ❏ Pro Tip: Separate Logic from DAGs

The biggest testing tip: **don't put business logic inside your DAG file**. Put it in separate Python modules, test those normally, and have your DAG tasks just call the functions.

```

project/
├── dags/
│ └── my_pipeline.py # DAG definition — thin, just wiring
├── src/
│ ├── extractors.py # API calls, file reads
│ ├── transforms.py # Business logic
│ ├── validators.py # Data quality checks
│ └── loaders.py # Database writes
└── tests/
 ├── test_dag_integrity.py # DAGs parse correctly
 ├── test_transforms.py # Unit tests for logic
 └── test_validators.py # Unit tests for validation

```

```

src/transforms.py
def calculate_tax(amount: float, rate: float = 0.08) -> float:
 """Calculate amount with tax."""
 if amount < 0:
 raise ValueError(f"Amount cannot be negative: {amount}")
 return round(amount * (1 + rate), 2)

def validate_order(order: dict) -> bool:
 """Validate a single order record."""
 required_fields = ["order_id", "customer", "amount", "date"]
 return all(field in order for field in required_fields) and order["amount"] > 0

```

```
tests/test_transforms.py
import pytest
from src.transforms import calculate_tax, validate_order

def test_calculate_tax():
 assert calculate_tax(100.00) == 108.00
 assert calculate_tax(0) == 0.00

def test_calculate_tax_negative():
 with pytest.raises(ValueError):
 calculate_tax(-50.00)

def test_validate_order_valid():
 order = {"order_id": 1, "customer": "Alice", "amount": 99.99, "date": "2026-02-19"}
 assert validate_order(order) is True

def test_validate_order_missing_field():
 order = {"order_id": 1, "customer": "Alice"}
 assert validate_order(order) is False

def test_validate_order_negative_amount():
 order = {"order_id": 1, "customer": "Alice", "amount": -10, "date": "2026-02-19"}
 assert validate_order(order) is False
```

### Level 3: Integration Tests

```
Test a specific task
docker compose exec airflow-scheduler \
 airflow tasks test my_first_pipeline extract_data 2026-02-19

Test the whole DAG (one-off run without the scheduler)
docker compose exec airflow-scheduler \
 airflow dags test my_first_pipeline 2026-02-19
```

#### ⚠ Common Mistake

**No tests at all** — At minimum, run DAG integrity tests in CI. Takes 30 seconds, catches 80% of deployment issues.

**Testing Airflow internals** — Don't test that `PythonOperator` works. Airflow already tests that. Test YOUR business logic.

## □ Checkpoint

1. What do DAG integrity tests verify?
2. Why should business logic live in separate modules, not inside DAG files?
3. What's the difference between `airflow tasks test` and `airflow dags test`?

## 4.10 Airflow in Production — Best Practices at Scale

### TL;DR

- Deploy DAGs via Git + CI/CD — never edit files directly on the server
- Every task must be idempotent (safe to run twice)
- Use pools to prevent resource exhaustion on shared systems
- Monitor the scheduler heartbeat — if it dies, nothing runs

Running Airflow on your laptop is easy. Running it in production where hundreds of DAGs execute reliably every day — that's where the real lessons are.

### 1. Git-Based DAG Deployment

Never edit DAG files directly on the Airflow server:

```
.github/workflows/deploy-dags.yml
name: Deploy DAGs
on:
 push:
 branches: [main]
 paths: ['dags/**']

jobs:
 test-and-deploy:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4

 - name: Install dependencies
 run: pip install apache-airflow pytest

 - name: Run DAG integrity tests
 run: pytest tests/test_dag_integrity.py -v

 - name: Sync DAGs to S3
 run: aws s3 sync dags/ s3://my-airflow-dags/ --delete
 env:
 AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
 AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
```

## 2. Resource Management with Pools

```
Set pool slots to prevent resource exhaustion
@task(pool="api_calls", pool_slots=1)
def call_external_api():
 """Only N concurrent API calls across all DAGs."""
 pass

Set priority for important DAGs
@task(priority_weight=10) # Higher = runs first when resources are scarce
def critical_transform():
 pass
```

Pools are essential in production. If you have 20 DAGs that all call the same API, you'll rate-limit yourself instantly without pools.

## 3. Idempotency — The Golden Rule

### □ Key Concept: Idempotency

Every task should be **idempotent** — running it twice with the same inputs produces the same output. Airflow WILL retry tasks. If your task inserts rows without checking for duplicates, you'll get double data.

```
@task()
def idempotent_load(data: list[dict], execution_date: str):
 """Use MERGE/upsert, not INSERT, for idempotency."""
 hook = PostgresHook(postgres_conn_id="warehouse")

 # □ Delete-then-insert (atomic replace for a partition)
 hook.run(f"DELETE FROM orders WHERE date = '{execution_date}'")
 # ... insert new data ...

 # □ Or use upsert
 # INSERT INTO orders ... ON CONFLICT (order_id) DO UPDATE SET ...
```

## 4. Templating with Jinja

Airflow passes execution context as Jinja templates. Use them for date partitioning:

```

from airflow.operators.bash import BashOperator

{{ ds }} = execution date (YYYY-MM-DD)
{{ ds_nodash }} = YYYYMMDD
{{ data_interval_start }} / {{ data_interval_end }}
extract = BashOperator(
 task_id="extract",
 bash_command="python extract.py --date {{ ds }} --output /data/raw/
{{ ds_nodash }}/",
)

```

## 5. Monitoring

Set up StatsD + Grafana for Airflow metrics:

- DAG/task success/failure rates
- Task duration trends (is something getting slower?)
- Scheduler heartbeat (is it alive?)
- Queue depth (are tasks piling up?)

## Production Checklist

- ❑ LocalExecutor or CeleryExecutor (never Sequential)
- ❑ External Postgres for metadata (not SQLite)
- ❑ Secrets backend (AWS SM, Vault, etc.)
- ❑ Git-based DAG deployment with CI/CD
- ❑ DAG integrity tests in CI
- ❑ All tasks are idempotent
- ❑ Retries + timeouts on every task
- ❑ Pools for shared resources
- ❑ Alerting on failure (Slack/PagerDuty)
- ❑ Monitoring (StatsD + Grafana)
- ❑ Log rotation configured
- ❑ Airflow upgraded regularly (security patches)

### ⚠ Common Mistake

**Not making tasks idempotent** — The #1 cause of duplicate data in production.

**Not monitoring the scheduler** — If the scheduler dies, nothing runs. Monitor its heartbeat.

**Massive DAGs** — If a DAG has 100+ tasks, break it into smaller DAGs connected by sensors.

## ❑ Checkpoint

1. Why must every Airflow task be idempotent?
2. What's the purpose of pools in Airflow?
3. What happens if the Airflow scheduler goes down?

---

## Module 4 Project: Multi-Step Production ETL Pipeline

Build a production-grade e-commerce ETL pipeline with extraction, validation, transformation, loading, and alerting.

### Project Overview

Your pipeline will:

1. Check for new data via a sensor
2. Extract data from a simulated API
3. Validate data quality
4. Transform using computed fields
5. Load to a warehouse table (with upserts for idempotency)
6. Run post-load quality checks
7. Send notification on success/failure

### Step 1: Set Up the Project Structure

```
module-4-project/
├── docker-compose.yaml # Airflow + Postgres (from Section 4.2)
├── dags/
│ └── ecommerce_pipeline.py # Your main DAG
├── src/
│ ├── __init__.py
│ ├── extractors.py # API extraction logic
│ ├── validators.py # Data quality checks
│ └── transforms.py # Transform functions
├── sql/
│ └── create_tables.sql
├── tests/
│ ├── test_dag_integrity.py
│ ├── test_validators.py
│ └── test_transforms.py
├── data/ # Mounted volume for file staging
└── .env
```

## Step 2: Create the Warehouse Tables

```
-- sql/create_tables.sql
CREATE TABLE IF NOT EXISTS raw_orders (
 order_id INTEGER,
 customer_id INTEGER,
 product_id INTEGER,
 quantity INTEGER,
 unit_price DECIMAL(10,2),
 order_date DATE,
 status VARCHAR(20),
 ingested_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS dim_customers (
 customer_id INTEGER PRIMARY KEY,
 name VARCHAR(100),
 email VARCHAR(100),
 segment VARCHAR(50),
 created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS fact_orders (
 order_id INTEGER PRIMARY KEY,
 customer_id INTEGER REFERENCES dim_customers(customer_id),
 product_id INTEGER,
 quantity INTEGER,
 unit_price DECIMAL(10,2),
 total_amount DECIMAL(10,2),
 tax_amount DECIMAL(10,2),
 order_date DATE,
 status VARCHAR(20),
 processed_at TIMESTAMP DEFAULT NOW()
);
```



### Step 3: Build the Extraction Logic

```
src/extractors.py
"""Simulated API extraction – in production, this would call an actual API."""

import random
from datetime import datetime

def extract_orders(execution_date: str) -> list[dict]:
 """Simulate extracting orders for a given date."""
 customers = list(range(1, 21))
 products = list(range(100, 120))
 statuses = ["completed", "pending", "shipped", "cancelled"]

 num_orders = random.randint(10, 50)
 orders = []

 for i in range(num_orders):
 orders.append({
 "order_id": int(datetime.now().timestamp() * 1000) + i,
 "customer_id": random.choice(customers),
 "product_id": random.choice(products),
 "quantity": random.randint(1, 5),
 "unit_price": round(random.uniform(9.99, 199.99), 2),
 "order_date": execution_date,
 "status": random.choice(statuses),
 })

 return orders

def extract_customers() -> list[dict]:
 """Simulate customer dimension data."""
 segments = ["premium", "standard", "basic"]
 return [
 {
 "customer_id": i,
 "name": f"Customer_{i}",
 "email": f"customer_{i}@example.com",
 "segment": segments[i % 3],
 }
 for i in range(1, 21)
]
```

**Step 4: Build the Validation Logic**

```

src/validators.py
"""Data quality validation functions."""

class DataQualityError(Exception):
 """Raised when data quality checks fail."""
 pass

def validate_orders(orders: list[dict]) -> list[dict]:
 """Validate order data. Returns valid orders, raises on critical issues."""
 if not orders:
 raise DataQualityError("No orders received – extraction may have failed")

 required_fields = ["order_id", "customer_id", "product_id",
 "quantity", "unit_price", "order_date"]
 valid_statuses = {"completed", "pending", "shipped", "cancelled"}

 valid_orders = []
 issues = []

 for order in orders:
 # Check required fields
 missing = [f for f in required_fields if f not in order]
 if missing:
 issues.append(f"Order {order.get('order_id', '?')}: missing {missing}")
 continue

 # Check value ranges
 if order["quantity"] <= 0:
 issues.append(f"Order {order['order_id']}: invalid quantity {order['quantity']}")
 continue

 if order["unit_price"] <= 0:
 issues.append(f"Order {order['order_id']}: invalid price {order['unit_price']}")
 continue

 if order.get("status") not in valid_statuses:
 issues.append(f"Order {order['order_id']}: unknown status {order.get('status')}")
 continue

 valid_orders.append(order)

 # Log issues
 if issues:
 print(f"⚠ Found {len(issues)} data quality issues:")
 for issue in issues[:10]:
 print(f" - {issue}")

 # Fail if more than 20% of records are bad

```

```
error_rate = len(issues) / len(orders) if orders else 0
if error_rate > 0.2:
 raise DataQualityError(
 f"Error rate too high: {error_rate:.1%} ({len(issues)}/{len(orders)}
records)"
)

print(f"▯ Validation passed: {len(valid_orders)}/{len(orders)} orders valid")
return valid_orders
```

**Step 5: Build the DAG**

```

dags/ecommerce_pipeline.py
"""
E-Commerce ETL Pipeline
=====
Production-grade pipeline: extract, validate, transform, load
with error handling and post-load quality checks.
"""

from datetime import datetime, timedelta
from airflow.decorators import dag, task
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.providers.postgres.operators.postgres import PostgresOperator
from airflow.sensors.filesystem import FileSensor
import os
import sys

Add src to path so we can import our modules
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))

def on_failure(context):
 """Called when any task fails (after retries exhausted)."""
 ti = context["task_instance"]
 print(f"⚠ ALERT: Task {ti.task_id} in DAG {ti.dag_id} failed!")
 print(f" Execution date: {context['ds']}")
 print(f" Log URL: {ti.log_url}")
 # In production: send to Slack, PagerDuty, etc.

def on_success(context):
 """Called when the entire DAG succeeds."""
 print(f"✅ DAG {context['dag'].dag_id} completed for {context['ds']}")

@dag(
 dag_id="ecommerce_etl_pipeline",
 description="Extract, validate, transform, and load e-commerce order data",
 schedule="0 6 * * *",
 start_date=datetime(2026, 2, 1),
 catchup=False,
 default_args={
 "owner": "george",
 "retries": 3,
 "retry_delay": timedelta(minutes=2),
 "retry_exponential_backoff": True,
 "max_retry_delay": timedelta(minutes=15),
 "execution_timeout": timedelta(minutes=30),
 "on_failure_callback": on_failure,
 },
 on_success_callback=on_success,
 tags=["ecommerce", "etl", "production"],
)
def ecommerce_pipeline():

```

```

Create tables if they don't exist
create_tables = PostgresOperator(
 task_id="create_tables",
 postgres_conn_id="postgres_default",
 sql="sql/create_tables.sql",
)

Wait for a trigger file (simulates waiting for upstream data)
wait_for_trigger = FileSensor(
 task_id="wait_for_trigger",
 filepath="/opt/airflow/data/trigger_{{ ds_nodash }}.flag",
 poke_interval=30,
 timeout=60, # Short for demo – use hours in production
 mode="reschedule",
 soft_fail=True, # Skip if no trigger
)

@task()
def extract_orders(ds=None) -> list[dict]:
 from extractors import extract_orders as _extract
 orders = _extract(execution_date=ds)
 print(f"▣ Extracted {len(orders)} orders for {ds}")
 return orders

@task()
def extract_customers() -> list[dict]:
 from extractors import extract_customers as _extract
 customers = _extract()
 print(f"▣ Extracted {len(customers)} customers")
 return customers

@task()
def validate(orders: list[dict]) -> list[dict]:
 from validators import validate_orders
 return validate_orders(orders)

@task()
def transform(orders: list[dict]) -> list[dict]:
 """Add computed fields."""
 TAX_RATE = 0.08
 for order in orders:
 order["total_amount"] = round(order["quantity"] * order["unit_price"], 2)
 order["tax_amount"] = round(order["total_amount"] * TAX_RATE, 2)
 order["processed_at"] = datetime.now().isoformat()
 print(f"▣ Transformed {len(orders)} orders (added tax + totals)")
 return orders

@task()
def load_customers(customers: list[dict]):
 """Upsert customer dimension – idempotent."""
 hook = PostgresHook(postgres_conn_id="postgres_default")
 for c in customers:
 hook.run("""

```

```

 INSERT INTO dim_customers (customer_id, name, email, segment)
 VALUES (%s, %s, %s, %s)
 ON CONFLICT (customer_id) DO UPDATE SET
 name = EXCLUDED.name,
 email = EXCLUDED.email,
 segment = EXCLUDED.segment
 """
 parameters=(c["customer_id"], c["name"], c["email"], c["segment"])
 print(f"▢ Loaded {len(customers)} customers")

@task()
def load_orders(orders: list[dict]):
 """Upsert fact orders – idempotent."""
 hook = PostgresHook(postgres_conn_id="postgres_default")
 for o in orders:
 hook.run("""
 INSERT INTO fact_orders
 (order_id, customer_id, product_id, quantity, unit_price,
 total_amount, tax_amount, order_date, status)
 VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
 ON CONFLICT (order_id) DO UPDATE SET
 status = EXCLUDED.status,
 processed_at = NOW()
 """, parameters=(
 o["order_id"], o["customer_id"], o["product_id"],
 o["quantity"], o["unit_price"], o["total_amount"],
 o["tax_amount"], o["order_date"], o["status"]
))
 print(f"▢ Loaded {len(orders)} orders to fact_orders")

@task()
def post_load_check(ds=None):
 """Verify data was loaded correctly."""
 hook = PostgresHook(postgres_conn_id="postgres_default")

 row_count = hook.get_first(
 "SELECT COUNT(*) FROM fact_orders WHERE order_date = %s",
 parameters=(ds,)
)
 print(f"▢ Post-load check: {row_count[0]} orders for {ds}")

 if row_count[0] == 0:
 raise ValueError(f"No orders loaded for {ds} – something went wrong!")

 orphans = hook.get_first("""
 SELECT COUNT(*) FROM fact_orders f
 LEFT JOIN dim_customers d ON f.customer_id = d.customer_id
 WHERE d.customer_id IS NULL AND f.order_date = %s
 """, parameters=(ds,))

 if orphans[0] > 0:
 print(f"⚠ Warning: {orphans[0]} orders with no matching customer")

 print("▢ Post-load quality checks passed")

```



```

Wire it all together
raw_orders = extract_orders()
customers = extract_customers()
validated = validate(raw_orders)
transformed = transform(validated)

create_tables >> [wait_for_trigger, customers]
wait_for_trigger >> raw_orders
load_customers_task = load_customers(customers)
load_orders_task = load_orders(transformed)

[load_customers_task, load_orders_task] >> post_load_check()

ecommerce_pipeline()

```

## Step 6: Write Tests

```

tests/test_dag_integrity.py
import pytest
from airflow.models import DagBag

@pytest.fixture()
def dagbag():
 return DagBag(dag_folder="dags/", include_examples=False)

def test_no_import_errors(dagbag):
 assert len(dagbag.import_errors) == 0, f"Errors: {dagbag.import_errors}"

def test_ecommerce_pipeline_exists(dagbag):
 assert "ecommerce_etl_pipeline" in dagbag.dags

```

```
tests/test_validators.py
import pytest
import sys
sys.path.insert(0, "src")
from validators import validate_orders, DataQualityError

def test_valid_orders():
 orders = [
 {"order_id": 1, "customer_id": 1, "product_id": 100,
 "quantity": 2, "unit_price": 29.99, "order_date": "2026-02-19",
 "status": "completed"},
]
 result = validate_orders(orders)
 assert len(result) == 1

def test_empty_orders_raises():
 with pytest.raises(DataQualityError):
 validate_orders([])

def test_high_error_rate_raises():
 bad_orders = [{"order_id": i, "quantity": -1} for i in range(10)]
 with pytest.raises(DataQualityError, match="Error rate too high"):
 validate_orders(bad_orders)
```

## Step 7: Run It

```
Start Airflow
docker compose up -d

Create a trigger file so the sensor passes
touch data/trigger_20260219.flag

Create the Postgres connection
docker compose exec airflow-scheduler airflow connections add 'postgres_default' \
 --conn-type 'postgres' \
 --conn-host 'postgres' \
 --conn-schema 'airflow' \
 --conn-login 'airflow' \
 --conn-password 'airflow' \
 --conn-port 5432

Trigger the DAG
docker compose exec airflow-scheduler airflow dags trigger ecommerce_etl_pipeline

Watch in the UI at http://localhost:8080
```

## Expected Output

- All tasks turn green in the Airflow UI
- `fact_orders` table has 10-50 rows
- `dim_customers` table has 20 rows
- Post-load checks pass
- Logs show emoji-annotated progress messages

## Stretch Goals

1. Add a second DAG that runs a daily report query on the loaded data
  2. Use `ExternalTaskSensor` to chain the two DAGs
  3. Add dynamic task mapping to process multiple product categories in parallel
  4. Set up a pool to limit concurrent database connections
- 

*End of Module 4*

# Module 5: Apache Spark & Big Data Processing

**What you'll learn:** When your data gets too big for a single machine, Spark is the industry standard. By the end of this module you'll know PySpark the way it's actually used in production — and just as importantly, when NOT to use it.

## 5.1 When You Need Spark (And When You Don't)

### TL;DR

- Most data engineers don't need Spark for their day-to-day work
- DuckDB/Polars handle up to ~100GB on a single machine, often faster than Spark
- Spark shines at 100GB+ or when you need distributed compute across a cluster
- Learn it because job postings require it and you WILL eventually hit big data

Here's something that might be controversial: most data engineers don't need Spark. If you're processing 500MB of data daily, Spark is like renting a school bus to drive yourself to work. It works, but you're paying for 40 seats you're not using and it takes 10 minutes just to start the engine.

So why learn it? Because at some point in your career, you WILL hit data that's too big for a single machine. And when that happens, Spark is the industry standard.

### The Decision Framework

Data Size	Recommended Tool	Why
< 1 GB	Pandas, Polars	Fits in memory, simple
1–10 GB	DuckDB, Polars	Single machine, crazy fast
10–100 GB	DuckDB (maybe Spark)	DuckDB handles this surprisingly well
100 GB+	Spark	You actually need distributed compute
1 TB+	Spark (cluster)	No question

The twist: DuckDB and Polars have gotten ridiculously good. A modern laptop with 32GB RAM can process 50–100GB files with DuckDB. That wasn't true 3 years ago.

## Quick Comparison: DuckDB vs. Spark on 1GB

```
DuckDB — processes 1GB CSV in seconds
import duckdb

con = duckdb.connect()
result = con.sql("""
 SELECT
 vendor_id,
 COUNT(*) as trips,
 AVG(total_amount) as avg_amount
 FROM read_csv('nyc_taxi_2024.csv')
 GROUP BY vendor_id
""").fetchdf()
Time: ~3 seconds on a laptop
```

```
Spark on the same data — significant overhead
from pyspark.sql import SparkSession
from pyspark.sql.functions import count, avg

spark = SparkSession.builder.appName("test").getOrCreate()
df = spark.read.csv("nyc_taxi_2024.csv", header=True, inferSchema=True)
result = df.groupBy("vendor_id").agg(
 count("*").alias("trips"),
 avg("total_amount").alias("avg_amount")
)
result.show()
Time: ~15-30 seconds (most of it is JVM startup)
```

DuckDB: 3 seconds. Spark: 20 seconds. On the same data.

## When You Actually Need Spark

1. **Data literally doesn't fit on one machine** — 500GB+ with no single server large enough
2. **You need cluster-level processing** — EMR, Databricks, Dataproc
3. **Your company already uses Spark** — ecosystem is there, team knows it, infra exists
4. **Streaming at scale** — Spark Structured Streaming for real-time processing
5. **Job postings require it** — this is a valid reason

### ❏ Pro Tip

In production, companies sometimes spin up a 20-node Spark cluster to process 5GB daily. That's \$3,000/month in cloud costs for something DuckDB could do on a \$20/month server. Know your data size. Pick the right tool.

## □Checkpoint

1. At what data size does Spark start to make sense over DuckDB?
2. Why is Spark slower than DuckDB on small datasets?
3. Name two scenarios where Spark is the right choice regardless of data size.

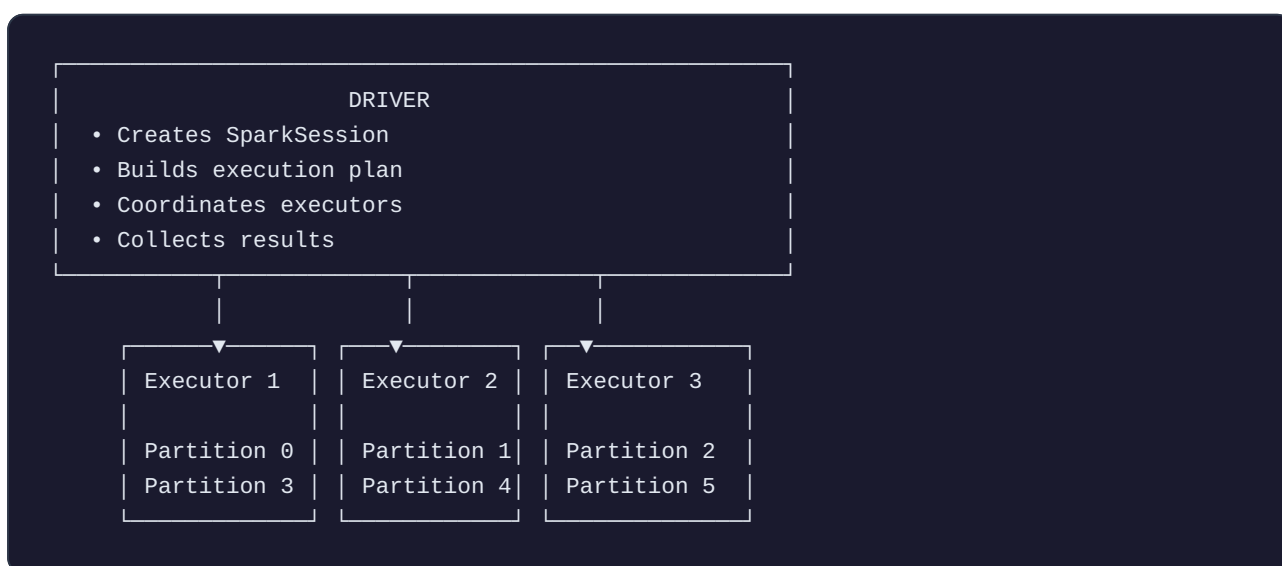
## 5.2 Spark Architecture — Drivers, Executors, Partitions

### TL;DR

- Spark splits data into partitions and processes them in parallel across executors
- The Driver coordinates everything; Executors do the actual work
- Transformations are lazy (build a plan); Actions trigger execution
- Shuffles (data exchange between executors) are the most expensive operation

If you don't understand Spark's architecture, you'll write code that's 10x slower than it should be. PySpark jobs that take 4 hours can often be optimized to 15 minutes just by understanding how data moves through the system.

[DIAGRAM: Spark Architecture]



### The Three Components

- 1. The Driver** — The boss. Your main program runs here. It creates the `SparkSession`, reads your code, builds an execution plan, coordinates executors, and collects results. The driver runs on ONE machine. If you accidentally pull all your data to the driver (with `.collect()`), you'll crash it.
- 2. Executors** — The workers. They run on separate machines (or separate cores in local mode), store data partitions in memory, execute tasks, and report results back to the driver.

**3. Partitions** — The data chunks. This is THE concept in Spark:

- Your data is split into partitions (chunks)
- Each partition is processed independently and in parallel
- More partitions = more parallelism (up to a point)
- Default: 200 partitions for shuffles, but configurable

Think of it like counting words in a library of 10,000 books. Without Spark, you read every book yourself. With Spark, you hire 10 workers, split the books into 10 piles, and each worker counts their pile simultaneously. 10x faster.

But if you need to SORT all the words alphabetically, the workers need to exchange data — Worker A has some "Z" words that Worker B needs. This data exchange is called a **shuffle**, and it's the most expensive operation in Spark.

#### □Key Concept: Transformations vs. Actions

Spark is **lazy**. It doesn't do anything until it absolutely has to.

**Transformations** = instructions that build a plan (lazy): `filter()`, `select()`, `groupBy()`, `join()`, `withColumn()`

**Actions** = triggers that execute the plan: `show()`, `count()`, `collect()`, `write()`

This matters because Spark optimizes the entire chain before running it — reordering operations, skipping unnecessary columns, combining steps. This is called the **Catalyst optimizer**.

```
This does NOTHING yet (just builds a plan)
df = spark.read.parquet("data/")
filtered = df.filter(df.amount > 100)
grouped = filtered.groupBy("category").count()

THIS triggers execution of the entire chain
grouped.show()
```

## Getting Started with PySpark

```
Start PySpark with Docker
docker run -it --rm \
 -p 4040:4040 \
 -v $(pwd)/data:/data \
 apache/spark:3.5.1-python3 \
 /opt/spark/bin/pyspark
```

```
In the PySpark shell – SparkSession is pre-created as `spark`
data = [(i, f"product_{i % 10}", i * 9.99) for i in range(1000000)]
df = spark.createDataFrame(data, ["id", "product", "amount"])

Check partitions
print(f"Number of partitions: {df.rdd.getNumPartitions()}")

This is lazy – nothing happens yet
filtered = df.filter(df.amount > 5000)
print("Filtered defined – nothing executed yet")

This triggers execution
count = filtered.count()
print(f"Count: {count}")
Now check the Spark UI at http://localhost:4040 – you'll see the completed job
```

### Expected output:

```
Number of partitions: 8
Filtered defined – nothing executed yet
Count: 499500
```

#### ⚠ Common Mistake

**Calling `.collect()` on large data** — Pulls everything to the driver. 50GB of data = dead driver.

Use `.show()` for previews, `.write()` for output.

**Too few partitions** — 100 cores but 10 partitions means 90 cores sit idle.

**Too many partitions** — 100,000 partitions for 1GB creates scheduling overhead. Rule of thumb: target ~128MB per partition.

### □ Checkpoint

1. What's the difference between a transformation and an action in Spark?
2. Why are shuffles expensive?
3. What happens if you call `.collect()` on a 50GB DataFrame?



## 5.3 DataFrames API — Reading, Transforming, Writing Data

### TL;DR

- The DataFrame API is where you spend 90% of your time in Spark
- Always define schemas explicitly in production (don't use `inferSchema` )
- Use Parquet for output — it's 10x smaller and faster than CSV
- `F.col()` is your best friend for column references

The DataFrame API is similar to Pandas, but distributed. If you know Pandas, you'll pick this up fast — just some syntax differences.

## Reading Data

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
 .appName("DataFrames Tutorial") \
 .config("spark.sql.adaptive.enabled", "true") \
 .getOrCreate()

CSV (quick and dirty)
df_csv = spark.read \
 .option("header", "true") \
 .option("inferSchema", "true") \
 .csv("data/orders.csv")

Parquet (preferred — columnar, compressed, schema included)
df_parquet = spark.read.parquet("data/orders.parquet")

JSON
df_json = spark.read.json("data/events.json")

With explicit schema (faster and more reliable than inferSchema)
from pyspark.sql.types import (
 StructType, StructField, StringType,
 IntegerType, DoubleType, DateType
)

order_schema = StructType([
 StructField("order_id", IntegerType(), False), # Not nullable
 StructField("customer_id", IntegerType(), False),
 StructField("product", StringType(), True), # Nullable
 StructField("quantity", IntegerType(), True),
 StructField("price", DoubleType(), True),
 StructField("order_date", DateType(), True),
])

df = spark.read.schema(order_schema).csv("data/orders.csv", header=True)

Always check your data after loading
df.printSchema()
df.show(5)
print(f"Rows: {df.count()}, Partitions: {df.rdd.getNumPartitions()}")
```

### ❏ Pro Tip

Always define your schema explicitly in production. `inferSchema=True` reads the entire file twice — once to figure out types, once to actually load. On large files, that doubles your read time.

## Transformations

```
from pyspark.sql import functions as F

Select columns
df.select("order_id", "customer_id", "price").show()

Filter rows
expensive = df.filter(F.col("price") > 100)
Equivalently:
expensive = df.where(df.price > 100)

Add computed columns
df_enriched = df.withColumn(
 "total", F.col("quantity") * F.col("price")
).withColumn(
 "tax", F.col("quantity") * F.col("price") * 0.08
).withColumn(
 "order_year", F.year("order_date")
)

Rename columns
df_renamed = df.withColumnRenamed("price", "unit_price")

Drop columns
df_slim = df.drop("some_unnecessary_column")

Handle nulls
df_clean = df.fillna({"quantity": 0, "product": "unknown"})
df_no_nulls = df.dropna(subset=["order_id", "customer_id"])

Type casting
df_typed = df.withColumn("price", F.col("price").cast("decimal(10,2)"))

String operations
df_strings = df.withColumn(
 "product_upper", F.upper(F.col("product"))
).withColumn(
 "product_trimmed", F.trim(F.col("product"))
)

Date operations
df_dates = df.withColumn(
 "order_month", F.date_format("order_date", "yyyy-MM")
).withColumn(
 "days_ago", F.datediff(F.current_date(), F.col("order_date"))
)
```

## Aggregations

```
Basic groupby
summary = df.groupBy("product").agg(
 F.count("*").alias("order_count"),
 F.sum("price").alias("total_revenue"),
 F.avg("price").alias("avg_price"),
 F.min("order_date").alias("first_order"),
 F.max("order_date").alias("last_order"),
)
summary.show()

Multiple groupby columns with ordering
monthly = df.withColumn("month", F.date_format("order_date", "yyyy-MM")) \
 .groupBy("month", "product") \
 .agg(F.sum(F.col("quantity") * F.col("price")).alias("revenue")) \
 .orderBy("month", F.desc("revenue"))

monthly.show(20)
```

## Writing Data

```
Parquet – the go-to for data lakes
df_enriched.write \
 .mode("overwrite") \
 .parquet("output/orders_enriched")

Partitioned by date – standard for data lakes
df_enriched.write \
 .mode("overwrite") \
 .partitionBy("order_year") \
 .parquet("output/orders_by_year")

CSV (for humans or legacy systems)
summary.coalesce(1).write \
 .mode("overwrite") \
 .option("header", "true") \
 .csv("output/summary")

To a database via JDBC
df_enriched.write \
 .format("jdbc") \
 .option("url", "jdbc:postgresql://localhost:5432/warehouse") \
 .option("dbtable", "public.orders_enriched") \
 .option("user", "postgres") \
 .option("password", "postgres") \
 .mode("overwrite") \
 .save()
```

### Write modes:

- `overwrite` — replace everything
- `append` — add to existing data
- `ignore` — do nothing if data exists
- `error` / `errorifexists` — fail if data exists (default)

#### ⚠ Common Mistake

`inferSchema=True` on large files — Define your schema. It's faster and more reliable.

**Writing CSV when you should write Parquet** — Parquet is 10x smaller, 10x faster to read, and includes the schema. Always use Parquet unless someone specifically needs CSV.

`.coalesce(1)` on large data — Merges all partitions into 1 file. Fine for small summary tables. Do NOT coalesce 10GB of data into one file.

### □ Checkpoint

1. Why should you define schemas explicitly instead of using `inferSchema` ?
2. What's the difference between `overwrite` and `append` write modes?
3. When would you use `.coalesce(1)` , and when would it be a bad idea?

## 5.4 Spark SQL — Running SQL on Distributed Data

### TL;DR

- Register DataFrames as views, then query with standard SQL
- Spark SQL supports window functions, CTEs, and everything you'd expect
- You can query Parquet files directly without registering views
- Mix DataFrame API and SQL freely — use whatever's clearest

You can use Spark and barely write any Python. Spark SQL lets you run standard SQL on distributed datasets.

## Basic Usage

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Spark SQL").getOrCreate()

Read data
orders = spark.read.parquet("data/orders.parquet")
customers = spark.read.parquet("data/customers.parquet")

Register as SQL views
orders.createOrReplaceTempView("orders")
customers.createOrReplaceTempView("customers")

Now write standard SQL!
result = spark.sql("""
 SELECT
 c.name,
 c.segment,
 COUNT(o.order_id) AS total_orders,
 SUM(o.quantity * o.price) AS total_revenue,
 AVG(o.quantity * o.price) AS avg_order_value
 FROM orders o
 JOIN customers c ON o.customer_id = c.customer_id
 WHERE o.order_date >= '2026-01-01'
 GROUP BY c.name, c.segment
 HAVING total_revenue > 1000
 ORDER BY total_revenue DESC
""")

result.show(20)
```

It's the exact same SQL you'd write in Postgres. But it runs distributed across your Spark cluster.

## Window Functions

```
spark.sql("""
 SELECT
 order_id,
 customer_id,
 price,
 order_date,
 ROW_NUMBER() OVER (
 PARTITION BY customer_id ORDER BY order_date DESC
) as rn,
 SUM(price) OVER (
 PARTITION BY customer_id ORDER BY order_date
 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) as running_total,
 LAG(price) OVER (
 PARTITION BY customer_id ORDER BY order_date
) as prev_order_amount
 FROM orders
""").show()
```

## Querying Files Directly

You don't even need to register views for ad-hoc analysis:

```
Query Parquet files directly
spark.sql("""
 SELECT * FROM parquet.`data/orders.parquet`
 WHERE price > 100
 LIMIT 10
""").show()

Query CSV files directly
spark.sql("SELECT * FROM csv.`data/orders.csv`").show()
```

## When to Use DataFrame API vs. SQL

Use Case	Preference
Complex multi-step transformations	DataFrame API (chaining)
Ad-hoc analysis / exploration	SQL
Analysts maintaining the code	SQL
Dynamic column operations	DataFrame API
Joins with complex logic	SQL (often easier to read)

In practice, use both. Many production jobs mix DataFrame API and SQL in the same script.

### □ Checkpoint

1. How do you make a DataFrame queryable via SQL?
2. Can you query a Parquet file without loading it into a DataFrame first?
3. Name a scenario where SQL is clearer than the DataFrame API.

## 5.5 Joins, Aggregations, and Window Functions

### TL;DR

- Broadcast small tables in joins to avoid expensive shuffles
- Anti joins are great for data quality checks (finding orphaned records)
- Window functions without `partitionBy` send all data to one executor — very slow
- Check for data skew on join keys before running large joins

Joins are where most Spark performance problems live. Understanding how Spark joins data will make or break your job execution times.



## Join Types

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("Joins").getOrCreate()

Sample data
orders = spark.createDataFrame([
 (1, 101, 29.99), (2, 102, 49.99), (3, 101, 19.99),
 (4, 103, 99.99), (5, 999, 14.99), # 999 = no matching customer
], ["order_id", "customer_id", "amount"])

customers = spark.createDataFrame([
 (101, "Alice", "premium"), (102, "Bob", "standard"),
 (103, "Charlie", "basic"), (104, "Diana", "premium"), # 104 = no orders
], ["customer_id", "name", "segment"])

INNER JOIN – only matching rows
inner = orders.join(customers, "customer_id", "inner")
inner.show()
Result: 4 rows (order 5 excluded, customer 104 excluded)

LEFT JOIN – all orders, even without customer match
left = orders.join(customers, "customer_id", "left")
left.show()
Result: 5 rows (order 5 has null name/segment)

ANTI JOIN – orders WITHOUT matching customers (data quality check!)
orphans = orders.join(customers, "customer_id", "left_anti")
orphans.show()
Result: just order 5 – great for finding data quality issues

SEMI JOIN – orders that HAVE matching customers (no customer columns added)
matched = orders.join(customers, "customer_id", "left_semi")
matched.show()
Result: 4 order rows, no customer columns
```

## Broadcast Joins — The Key Optimization

When you join two DataFrames, Spark shuffles data across the network so matching keys land on the same executor. This is slow. If one side is small, you can avoid the shuffle entirely:

```
from pyspark.sql.functions import broadcast

Send the entire small table to every executor – no shuffle needed
result = orders.join(broadcast(customers), "customer_id", "inner")
```

### ❏Key Concept: Broadcast Joins

Spark auto-broadcasts tables under 10MB. For tables up to ~100MB, manually broadcast them with `broadcast()`. If one table has 100MB and the other has 100GB, broadcasting saves massive shuffle time.

## Window Functions with DataFrame API

```
from pyspark.sql.window import Window

orders_data = spark.createDataFrame([
 (1, 101, 100.0, "2026-01-01"),
 (2, 101, 200.0, "2026-01-15"),
 (3, 101, 150.0, "2026-02-01"),
 (4, 102, 300.0, "2026-01-10"),
 (5, 102, 250.0, "2026-01-20"),
 (6, 102, 175.0, "2026-02-05"),
], ["order_id", "customer_id", "amount", "order_date"])

Define window specs
customer_window = Window.partitionBy("customer_id").orderBy("order_date")
customer_all = Window.partitionBy("customer_id")

result = orders_data.select(
 "*",
 F.row_number().over(customer_window).alias("order_number"),

 F.sum("amount").over(
 customer_window.rowsBetween(Window.unboundedPreceding, Window.currentRow)
).alias("running_total"),

 F.lag("amount", 1).over(customer_window).alias("prev_amount"),
 F.lead("amount", 1).over(customer_window).alias("next_amount"),

 F.sum("amount").over(customer_all).alias("customer_total"),

 (F.col("amount") / F.sum("amount").over(customer_all) * 100)
 .cast("decimal(5,2)").alias("pct_of_total"),
)

result.show()
```

Expected output:

```

+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|order_id|customer_id|amount|order_date|order_number|running_total|prev_amount|
next_amount|customer_total|pct_of_total|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
| 1| 101| 100.0|2026-01-01| 1| 100.0| null|
200.0| 450.0| 22.22|
| 2| 101| 200.0|2026-01-15| 2| 300.0| 100.0|
150.0| 450.0| 44.44|
| 3| 101| 150.0|2026-02-01| 3| 450.0| 200.0|
null| 450.0| 33.33|
| 4| 102| 300.0|2026-01-10| 1| 300.0| null|
250.0| 725.0| 41.38|
| 5| 102| 250.0|2026-01-20| 2| 550.0| 300.0|
175.0| 725.0| 34.48|
| 6| 102| 175.0|2026-02-05| 3| 725.0| 250.0|
null| 725.0| 24.14|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+

```

#### ⚠ Common Mistake

**Joining two huge tables without checking for data skew** — If 90% of your data has `customer_id = NULL`, one partition processes 90% of the work. Filter nulls before joining.

**Window functions without `partitionBy`** — `Window.orderBy(...)` without `partitionBy(...)` creates a single partition for ALL data. Everything goes to one executor. Very slow.

#### □ Checkpoint

1. When should you use a broadcast join?
2. What's an anti join useful for?
3. Why is a window function without `partitionBy` dangerous?

## 5.6 UDFs — When to Use Them (Rarely) and Alternatives

### TL;DR

- UDFs are 10–100x slower than built-in functions due to serialization overhead
- Always try built-in functions first (`F.when`, `F.regexp_extract`, etc.)
- If you must use UDFs, use Pandas UDFs (vectorized) for 10–100x speedup over regular UDFs
- Regular UDFs are a last resort

UDFs (User Defined Functions) let you run custom Python code on Spark data. The problem: they're slow. Like, 10–100x slower than built-in functions.

## Why UDFs Are Slow

Built-in Spark functions run in the JVM — compiled, optimized, fast. UDFs run in Python. For every row, Spark serializes data from JVM → Python (via a socket), runs your function, then serializes back. That per-row overhead kills performance.

## Don't Use a UDF When Built-In Functions Work

```
from pyspark.sql import functions as F
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

▢ BAD — UDF for something built-in functions handle perfectly
@udf(returnType=StringType())
def categorize_amount_udf(amount):
 if amount > 100: return "high"
 elif amount > 50: return "medium"
 else: return "low"

df.withColumn("tier", categorize_amount_udf(df.amount)) # Slow!

▢ GOOD — Built-in when/otherwise (runs in JVM, no serialization)
df.withColumn("tier",
 F.when(F.col("amount") > 100, "high")
 .when(F.col("amount") > 50, "medium")
 .otherwise("low")
)
```

## When You Genuinely Need a UDF

```
import re
from pyspark.sql.functions import udf
from pyspark.sql.types import ArrayType, StringType

Complex regex that can't be expressed with built-in functions
@udf(returnType=ArrayType(StringType()))
def extract_emails(text):
 """Extract all email addresses from text."""
 if text is None:
 return []
 pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
 return re.findall(pattern, text)

df.withColumn("emails", extract_emails(df.description)).show()
```

## Better Alternative: Pandas UDFs (Vectorized)

If you must use UDFs, Pandas UDFs process batches of rows instead of one at a time — 10–100x faster:

```
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")
def calculate_discount(amounts: pd.Series, quantities: pd.Series) -> pd.Series:
 """Vectorized UDF — processes batches, not individual rows."""
 base_discount = 0.05
 volume_discount = (quantities > 10).astype(float) * 0.1
 return amounts * (1 - base_discount - volume_discount)

df.withColumn("discounted", calculate_discount(df.amount, df.quantity)).show()
```

## The UDF Decision Ladder

1. First, try **built-in functions** ( `F.when`, `F.regexp_extract`, `F.split`, etc.)
2. If that fails, try **Spark SQL expressions**
3. If that fails, use a **Pandas UDF**
4. **Last resort**: regular UDF

### □ Checkpoint

1. Why are regular UDFs so much slower than built-in Spark functions?
2. What's the performance difference between a regular UDF and a Pandas UDF?
3. Give an example of something that requires a UDF vs. something that doesn't.

## 5.7 Performance Tuning — Partitioning, Caching, Broadcast Joins, AQE

### TL;DR

- The four performance killers: data skew, wrong partition count, no caching, unnecessary shuffles
- Enable AQE (Adaptive Query Execution) — it's a free performance boost
- Cache DataFrames you use multiple times; unpersist when done
- Read execution plans to understand what Spark is actually doing

This is the section that'll make you look like a Spark wizard. Most PySpark jobs are slow because of a handful of common issues.

### Performance Killer #1: Data Skew

Data skew means some partitions have way more data than others. If partition 1 has 10 rows and partition 2 has 10 million rows, your cluster sits idle while one executor does all the work.

```
Detect skew – check key distribution
df.groupBy("join_key").count().orderBy(F.desc("count")).show(10)
If one key has 10M rows and others have 1K, you have skew

Fix: salt the skewed key
num_salts = 10
skewed_df = skewed_df.withColumn(
 "salt", (F.rand() * num_salts).cast("int")
)
Explode salts on the other table to match
other_df = other_df.withColumn(
 "salt", F.explode(F.array([F.lit(i) for i in range(num_salts)]))
)
Join on key + salt distributes the work evenly
result = skewed_df.join(other_df, ["join_key", "salt"]).drop("salt")
```

## Performance Killer #2: Wrong Partition Count

```
Check current partition count
print(f"Partitions: {df.rdd.getNumPartitions()}")

Repartition – more partitions (triggers shuffle)
df_repartitioned = df.repartition(200)

Coalesce – fewer partitions (no shuffle, faster)
df_small = df.coalesce(10)

Rule of thumb: target 128MB per partition
10GB of data: 10,000MB / 128MB ≈ 80 partitions
```

## Performance Killer #3: Not Caching

If you read from the same DataFrame multiple times, cache it:

```
Without cache – reads from disk TWICE
count = df.filter(df.status == "active").count()
total = df.filter(df.status == "active").agg(F.sum("amount")).collect()

With cache – reads once, stores in memory
active = df.filter(df.status == "active").cache()
count = active.count() # First call: reads + caches
total = active.agg(F.sum("amount")).collect() # Second call: from memory

IMPORTANT: free memory when done
active.unpersist()
```

**Cache when:** you use the same DataFrame in multiple actions, or it's expensive to compute.

**Don't cache when:** you only use it once, or it's too big to fit in memory.

## Performance Killer #4: Unnecessary Shuffles

```
▢ Repartition then immediately coalesce – useless shuffle
df.repartition(200).coalesce(50)

▢ Distinct before a join – two shuffles when one might do
df1.distinct().join(df2, "key")

▢ If possible, deduplicate AFTER the join
```

## Adaptive Query Execution (AQE) — Free Performance

AQE (Spark 3.0+) optimizes your job DURING execution based on actual data statistics:

```
Enable AQE (default in Spark 3.2+)
spark.conf.set("spark.sql.adaptive.enabled", "true")

Auto-coalesce: combines small shuffle partitions
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")

Auto-broadcast: detects small join sides at runtime
spark.conf.set("spark.sql.adaptive.autoBroadcastJoinThreshold", "10MB")

Skew optimization: splits skewed partitions automatically
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

## Reading Execution Plans

```
df.filter(df.amount > 100) \
 .groupBy("category") \
 .agg(F.sum("amount")) \
 .explain(mode="formatted")

Look for:
- "BroadcastHashJoin" (good) vs "SortMergeJoin" (potentially slow)
- "Exchange" = shuffle (expensive)
- "FileScan" – are you reading all columns or just what you need?
```

## Quick Wins Checklist

- Enable AQE: `spark.sql.adaptive.enabled = true`
- Select only columns you need (don't `SELECT *`)
- Filter early (push filters before joins)
- Use Parquet (column pruning + predicate pushdown)
- Broadcast small tables in joins
- Cache DataFrames used multiple times
- Partition writes by date for downstream efficiency
- Check for data skew on join keys

### □ Checkpoint

1. How do you detect data skew?
2. What's the difference between `repartition()` and `coalesce()`?
3. What three things does AQE optimize automatically?

## 5.8 Delta Lake / Apache Iceberg — Modern Table Formats

### TL;DR

- Plain Parquet files lack transactions, updates, deletes, and time travel
- Delta Lake adds ACID transactions, MERGE/upsert, time travel, and schema evolution
- Delta Lake (Databricks) vs. Iceberg (Netflix) — both solve the same problems
- Use Delta if on Databricks; use Iceberg for multi-engine flexibility

Here's a real scenario: you have a data lake with millions of Parquet files in S3. A pipeline writes new data while a Spark job reads. The read job picks up a half-written file. Corrupt data. Dashboard breaks.

Or: you need to delete all data for a GDPR request. Good luck finding and rewriting every Parquet file containing that user.

### The Problems with Plain Parquet

1. **No ACID transactions** — reads can see partially written data
2. **No updates/deletes** — Parquet files are immutable
3. **No schema evolution** — adding a column breaks old files
4. **No time travel** — can't roll back mistakes
5. **Small file problem** — streaming creates thousands of tiny files



## Delta Lake in Action

```
from pyspark.sql import SparkSession
from delta import configure_spark_with_delta_pip

Setup Spark with Delta Lake
builder = SparkSession.builder \
 .appName("DeltaLake") \
 .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
 .config("spark.sql.catalog.spark_catalog",
 "org.apache.spark.sql.delta.catalog.DeltaCatalog")

spark = configure_spark_with_delta_pip(builder).getOrCreate()

Write a Delta table
orders = spark.createDataFrame([
 (1, "Alice", 99.99, "2026-02-19"),
 (2, "Bob", 149.50, "2026-02-19"),
 (3, "Charlie", 29.99, "2026-02-19"),
], ["order_id", "customer", "amount", "date"])

orders.write.format("delta").mode("overwrite").save("data/delta/orders")

Read it back
df = spark.read.format("delta").load("data/delta/orders")
df.show()
```

### Expected output:

```
+-----+-----+-----+-----+
|order_id|customer|amount| date|
+-----+-----+-----+-----+
| 1| Alice| 99.99|2026-02-19|
| 2| Bob|149.50|2026-02-19|
| 3| Charlie| 29.99|2026-02-19|
+-----+-----+-----+-----+
```

## Updates, Deletes, and MERGE

```
from delta.tables import DeltaTable

delta_table = DeltaTable.forPath(spark, "data/delta/orders")

UPDATE — change Alice's amount
delta_table.update(
 condition="customer = 'Alice'",
 set={"amount": "109.99"}
)

DELETE — remove Charlie
delta_table.delete("customer = 'Charlie'")

MERGE (upsert) — the most powerful operation
new_data = spark.createDataFrame([
 (1, "Alice", 119.99, "2026-02-20"), # Update existing
 (4, "Diana", 199.00, "2026-02-20"), # Insert new
], ["order_id", "customer", "amount", "date"])

delta_table.alias("target").merge(
 new_data.alias("source"),
 "target.order_id = source.order_id"
).whenMatchedUpdateAll() \
.whenNotMatchedInsertAll() \
.execute()
```

## Time Travel

```
Read a previous version
df_v0 = spark.read.format("delta") \
 .option("versionAsOf", 0) \
 .load("data/delta/orders")

Read as of a specific timestamp
df_yesterday = spark.read.format("delta") \
 .option("timestampAsOf", "2026-02-18") \
 .load("data/delta/orders")

View history
delta_table.history().show()

Restore to a previous version (undo a mistake!)
delta_table.restoreToVersion(0)
```

## Schema Evolution

```
Add a new column – existing data gets nulls
new_with_status = spark.createDataFrame([
 (5, "Eve", 75.25, "2026-02-20", "completed"),
], ["order_id", "customer", "amount", "date", "status"])

new_with_status.write.format("delta") \
 .mode("append") \
 .option("mergeSchema", "true") \
 .save("data/delta/orders")
```

## OPTIMIZE — Fix the Small Files Problem

```
Compact small files into larger ones
delta_table.optimize().executeCompaction()

Z-ORDER — co-locate related data for faster queries
delta_table.optimize().executeZOrderBy("date", "customer")
```

## Delta Lake vs. Iceberg

Feature	Delta Lake	Apache Iceberg
Creator	Databricks	Netflix
Ecosystem	Tight Spark integration	Multi-engine (Spark, Trino, Flink)
Industry adoption	Dominant in Databricks shops	Growing fast, especially multi-cloud
Community	Open source + Databricks extras	Fully open source

If you're on Databricks, use Delta Lake. If you want engine flexibility, Iceberg is the better bet. Both solve the same problems.

### □ Checkpoint

1. Name three problems with plain Parquet that Delta Lake solves.
2. What does MERGE do, and why is it important for production pipelines?
3. How does time travel work in Delta Lake?

## 5.9 Running Spark on AWS EMR and Databricks

### TL;DR

- In production, Spark runs on managed clusters: EMR, Databricks, Dataproc
- EMR Serverless is the simplest AWS option — no cluster management
- Databricks is the most popular Spark platform overall
- Your PySpark code is identical everywhere — only deployment differs

Running Spark on your laptop is great for learning. In production, you need managed clusters.

### Option 1: AWS EMR

```
Submit a PySpark job to EMR
aws emr add-steps \
 --cluster-id j-XXXXXXXXXXXX \
 --steps Type=Spark,Name="Daily ETL",\
 ActionOnFailure=CONTINUE,\
 Args[--deploy-mode,cluster,s3://my-bucket/scripts/etl.py]
```

EMR Serverless — no cluster management at all:

```
aws emr-serverless start-job-run \
 --application-id 00xxxxxxxxxxxxxxxx \
 --execution-role-arn arn:aws:iam::123456789:role/emr-role \
 --job-driver '{
 "sparkSubmit": {
 "entryPoint": "s3://my-bucket/scripts/etl.py",
 "sparkSubmitParameters": "--conf spark.sql.adaptive.enabled=true"
 }
 }'
```

### Option 2: Databricks

The most popular Spark platform. Founded by the creators of Spark. Features: managed notebooks, built-in Delta Lake, Unity Catalog for governance, auto-scaling clusters, and built-in job scheduling.

### Option 3: Google Dataproc / Azure Synapse

Same concept, different cloud: Dataproc for GCP, Synapse for Azure.

## Option 4: Kubernetes

```
spark-submit \
 --master k8s://https://k8s-apserver:443 \
 --deploy-mode cluster \
 --conf spark.kubernetes.container.image=my-spark:latest \
 --conf spark.executor.instances=5 \
 s3://my-bucket/etl.py
```

### □ Pro Tip

For job hunting: learn EMR and Databricks terminology. Most job postings mention one or both. The actual PySpark code is identical everywhere — it's just the deployment wrapper that differs.

### □ Checkpoint

1. What's the advantage of EMR Serverless over regular EMR?
2. Why is Databricks so popular for Spark workloads?
3. Does your PySpark code change when moving from local to EMR?

## Module 5 Project: Processing NYC Taxi Data with PySpark

Process the NYC Taxi dataset using PySpark — exploration, data quality checks, transformations, analytics with window functions, and Delta Lake output.

## Step 1: Set Up the Environment

```
docker-compose.yml
version: '3.8'

services:
 spark-master:
 image: bitnami/spark:3.5.1
 environment:
 - SPARK_MODE=master
 - SPARK_MASTER_HOST=spark-master
 ports:
 - "8081:8080" # Spark Master UI
 - "7077:7077"
 - "4040:4040" # Spark App UI
 volumes:
 - ./data:/data
 - ./scripts:/scripts
 - ./output:/output

 spark-worker:
 image: bitnami/spark:3.5.1
 environment:
 - SPARK_MODE=worker
 - SPARK_MASTER_URL=spark://spark-master:7077
 - SPARK_WORKER_MEMORY=4g
 - SPARK_WORKER_CORES=2
 depends_on:
 - spark-master
 volumes:
 - ./data:/data
 - ./output:/output
```

```
mkdir -p data scripts output

Download NYC taxi data (3-6 months, ~100MB each)
for month in 01 02 03 04 05 06; do
 wget -P data/ \
 "https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-${month}.parquet"
done

docker compose up -d
```

## Step 2: Explore the Data

```
scripts/01_explore.py
"""Explore and understand the dataset."""

from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
 .appName("NYC Taxi - Explore") \
 .config("spark.sql.adaptive.enabled", "true") \
 .getOrCreate()

Read all months at once with glob pattern
df = spark.read.parquet("data/yellow_tripdata_2023-*.parquet")

print(f"Total rows: {df.count():,}")
print(f"Partitions: {df.rdd.getNumPartitions()}")

print("\nSchema:")
df.printSchema()

print("\nSample data:")
df.show(5, truncate=False)

Check for nulls across all columns
print("\nNull counts per column:")
null_counts = df.select([
 F.sum(F.when(F.col(c).isNull(), 1).otherwise(0)).alias(c)
 for c in df.columns
])
null_counts.show(truncate=False)

Check value distributions
print("\nPassenger count distribution:")
df.groupBy("passenger_count").count().orderBy("passenger_count").show()

spark.stop()
```

Expected output (approximate):

```
Total rows: 18,456,231
Partitions: 24
...
```

**Step 3: Data Quality Checks**



```

scripts/02_quality_checks.py
"""Identify and handle data quality issues."""

from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder \
 .appName("NYC Taxi - Quality") \
 .config("spark.sql.adaptive.enabled", "true") \
 .getOrCreate()

df = spark.read.parquet("data/yellow_tripdata_2023-*.parquet")
total = df.count()

print("=== DATA QUALITY REPORT ===\n")

neg_fares = df.filter(F.col("fare_amount") < 0).count()
print(f"Negative fares: {neg_fares:,} ({neg_fares/total*100:.2f}%)")

long_trips = df.filter(F.col("trip_distance") > 200).count()
print(f"Trips > 200 miles: {long_trips:,}")

zero_pax = df.filter(
 (F.col("passenger_count") == 0) | F.col("passenger_count").isNull()
).count()
print(f"Zero/null passengers: {zero_pax:,}")

time_travel = df.filter(
 F.col("tpep_dropoff_datetime") < F.col("tpep_pickup_datetime")
).count()
print(f"Dropoff before pickup: {time_travel:,}")

Apply quality filters
print(f"\n=== CLEANING ===")
print(f"Before: {total:,} rows")

df_clean = df.filter(
 (F.col("fare_amount") >= 0) &
 (F.col("fare_amount") < 1000) &
 (F.col("trip_distance") > 0) &
 (F.col("trip_distance") < 200) &
 (F.col("passenger_count") > 0) &
 (F.col("tpep_dropoff_datetime") > F.col("tpep_pickup_datetime")) &
 (F.col("tpep_pickup_datetime").between("2023-01-01", "2023-12-31"))
)

clean_count = df_clean.count()
print(f"After: {clean_count:,} rows")
print(f"Removed: {total - clean_count:,} rows ({(total-clean_count)/total*100:.2f}%)")

df_clean.write.mode("overwrite").parquet("output/cleaned_taxi_data")

spark.stop()

```

**Step 4: Transformations and Analytics**

```

scripts/03_transform.py
"""Transform data and compute analytics with window functions."""

from pyspark.sql import SparkSession, functions as F
from pyspark.sql.window import Window

spark = SparkSession.builder \
 .appName("NYC Taxi - Transform") \
 .config("spark.sql.adaptive.enabled", "true") \
 .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
 .config("spark.sql.catalog.spark_catalog",
 "org.apache.spark.sql.delta.catalog.DeltaCatalog") \
 .getOrCreate()

df = spark.read.parquet("output/cleaned_taxi_data")

=== ENRICHMENT ===
df_enriched = df.withColumn(
 "trip_duration_minutes",
 (F.unix_timestamp("tpep_dropoff_datetime") -
 F.unix_timestamp("tpep_pickup_datetime")) / 60
).withColumn(
 "avg_speed_mph",
 F.when(
 F.col("trip_duration_minutes") > 0,
 F.col("trip_distance") / (F.col("trip_duration_minutes") / 60)
).otherwise(0)
).withColumn(
 "pickup_hour", F.hour("tpep_pickup_datetime")
).withColumn(
 "pickup_date", F.to_date("tpep_pickup_datetime")
).withColumn(
 "pickup_month", F.month("tpep_pickup_datetime")
).withColumn(
 "is_weekend", F.dayofweek("tpep_pickup_datetime").isin([1, 7]).cast("int")
).withColumn(
 "tip_percentage",
 F.when(F.col("fare_amount") > 0,
 (F.col("tip_amount") / F.col("fare_amount") * 100).cast("decimal(5,2)"))
 .otherwise(0)
).withColumn(
 "fare_category",
 F.when(F.col("total_amount") > 100, "premium")
 .when(F.col("total_amount") > 50, "high")
 .when(F.col("total_amount") > 20, "medium")
 .otherwise("budget")
)

Filter unreasonable computed values
df_enriched = df_enriched.filter(
 (F.col("trip_duration_minutes") > 0) &
 (F.col("trip_duration_minutes") < 300) &
 (F.col("avg_speed_mph") < 100)
)

```

```

)

=== ANALYTICS ===

Hourly demand pattern
hourly_demand = df_enriched.groupBy("pickup_hour").agg(
 F.count("*").alias("trip_count"),
 F.avg("fare_amount").alias("avg_fare"),
 F.avg("trip_duration_minutes").alias("avg_duration"),
 F.avg("tip_percentage").alias("avg_tip_pct"),
).orderBy("pickup_hour")

print("=== Hourly Demand ===")
hourly_demand.show(24)

Daily revenue with window functions (7-day and 30-day moving averages)
daily_revenue = df_enriched.groupBy("pickup_date").agg(
 F.count("*").alias("trips"),
 F.sum("total_amount").alias("revenue"),
 F.avg("total_amount").alias("avg_fare"),
)

window_7d = Window.orderBy("pickup_date").rowsBetween(-6, 0)
window_30d = Window.orderBy("pickup_date").rowsBetween(-29, 0)

daily_with_trends = daily_revenue.select(
 "*",
 F.avg("revenue").over(window_7d).alias("revenue_7d_avg"),
 F.avg("revenue").over(window_30d).alias("revenue_30d_avg"),
 F.avg("trips").over(window_7d).alias("trips_7d_avg"),
).orderBy("pickup_date")

print("=== Daily Revenue with Trends ===")
daily_with_trends.show(10)

=== WRITE TO DELTA LAKE ===
df_enriched.write \
 .format("delta") \
 .mode("overwrite") \
 .partitionBy("pickup_month") \
 .save("output/delta/taxi_enriched")

daily_with_trends.write \
 .format("delta") \
 .mode("overwrite") \
 .save("output/delta/daily_revenue")

hourly_demand.write \
 .format("delta") \
 .mode("overwrite") \
 .save("output/delta/hourly_demand")

print("\n All outputs written to Delta Lake")
spark.stop()

```

## Step 5: Run Everything

```
Submit jobs to Spark cluster
docker compose exec spark-master spark-submit \
 --master spark://spark-master:7077 \
 /scripts/01_explore.py

docker compose exec spark-master spark-submit \
 --master spark://spark-master:7077 \
 /scripts/02_quality_checks.py

docker compose exec spark-master spark-submit \
 --master spark://spark-master:7077 \
 --packages io.delta:delta-spark_2.12:3.1.0 \
 /scripts/03_transform.py
```

## Expected Output

- Data quality report showing issues found and percentage cleaned
- Enriched dataset with trip duration, speed, tip %, fare category
- Hourly demand analysis showing peak hours (typically 6–8 PM)
- Daily revenue with 7-day and 30-day moving averages
- All outputs in Delta Lake format with month partitioning

## Stretch Goals

1. **Zone lookup** — Join with the taxi zone shapefile to get pickup/dropoff neighborhood names
2. **Surge pricing indicator** — Flag hours above the 90th percentile of demand
3. **Delta time travel** — Compare this month's data vs. last month's
4. **Anomaly detection** — Flag days where revenue is > 2 standard deviations from the 30-day average

---

*End of Module 5*

# Module 6: Apache Kafka & Streaming

**What you'll learn:** Real-time data is no longer optional for many use cases. By the end of this module you'll understand Kafka's architecture, be able to build producers and consumers in Python, use Kafka Connect for zero-code integrations, and build a complete streaming pipeline.

## 6.1 Batch vs. Streaming — When You Actually Need Real-Time

### TL;DR

- 80% of the time, batch processing is the right choice — simpler and cheaper
- Streaming is for fraud detection, real-time recommendations, IoT alerts, live dashboards
- Streaming is harder, more expensive, and has more failure modes than batch
- Most production systems use both (hot path for speed, cold path for accuracy)

"We need real-time data!" — every startup founder, every VP of Product, every stakeholder who watched a conference talk about streaming. And 80% of the time? They don't.

### When Batch Is Fine (And Often Better)

- Daily/hourly dashboards and reports
- Data warehouse loading
- ML model training
- Any use case where "a few hours old" data is acceptable
- Most analytics

### When You Actually Need Streaming

- **Fraud detection** — you can't wait an hour to flag a suspicious transaction
- **Real-time recommendations** — "users viewing this RIGHT NOW also viewed..."
- **IoT monitoring** — factory sensor goes out of range, alert immediately
- **Live dashboards** — stock tickers, live sports, operations monitoring
- **Event-driven microservices** — order placed → trigger fulfillment → update inventory

## The Cost of Streaming

Streaming is harder and more expensive than batch:

- More complex infrastructure (Kafka cluster, schema registry, monitoring)
- Harder to debug (try debugging a race condition at 3 AM)
- More failure modes (consumer lag, partition rebalancing, exactly-once delivery)
- Higher cloud costs (always-on infrastructure vs. spin-up-then-down)

### □Key Concept: The Hybrid Approach

Most production systems use BOTH batch and streaming:

- **Hot path:** Kafka → real-time consumer → quick aggregations → live dashboard
- **Cold path:** Kafka → S3 → Spark batch job → data warehouse → detailed analytics

The hot path gives you speed. The cold path gives you accuracy and depth.

### □Pro Tip

At most companies, the journey looks like: (1) start with batch, (2) build Airflow pipelines, (3) one use case genuinely needs real-time, (4) add Kafka for THAT use case, (5) gradually expand. Don't try to stream everything on day one.

### □Checkpoint

1. Name three use cases where streaming is genuinely necessary.
2. Why is streaming more expensive than batch processing?
3. What's the difference between a "hot path" and "cold path" in a hybrid architecture?

## 6.2 Kafka Architecture — Brokers, Topics, Partitions, Consumer Groups

### TL;DR

- Kafka is a distributed log, not a message queue — messages persist and multiple consumers read independently
- Topics are split into partitions for parallelism; ordering is guaranteed only within a partition
- Consumer groups enable independent processing — analytics and fraud detection read the same data without interference
- Use KRaft mode (no Zookeeper) for new setups

Kafka isn't a message queue. That's the first misconception to clear up. It's a distributed *log*. Think of an append-only journal that multiple writers can write to and multiple readers can read from, independently, at their own pace.

[DIAGRAM: Kafka Architecture]

```
Partition 0: [msg1][msg4][msg7]... || Broker 1
Partition 1: [msg2][msg5][msg8]... || Broker 2
Partition 2: [msg3][msg6][msg9]... || Broker 3
```



## Setting Up Kafka with Docker

```
docker-compose.yml
version: '3.8'

services:
 kafka:
 image: bitnami/kafka:3.7
 ports:
 - "9092:9092"
 environment:
 - KAFKA_CFG_NODE_ID=1
 - KAFKA_CFG_PROCESS_ROLES=controller,broker # KRaft mode (no Zookeeper)
 - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
 - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092
 -
 KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
 - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
 - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
 - KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE=false
 volumes:
 - kafka-data:/bitnami/kafka

 kafka-ui:
 image: provectuslabs/kafka-ui:latest
 ports:
 - "8080:8080"
 environment:
 - KAFKA_CLUSTERS_0_NAME=local
 - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:9092
 depends_on:
 - kafka

volumes:
 kafka-data:
```

```

docker compose up -d

Create a topic with 3 partitions
docker compose exec kafka kafka-topics.sh \
 --bootstrap-server localhost:9092 \
 --create \
 --topic orders \
 --partitions 3 \
 --replication-factor 1

List topics
docker compose exec kafka kafka-topics.sh \
 --bootstrap-server localhost:9092 \
 --list

Describe a topic (shows partition details)
docker compose exec kafka kafka-topics.sh \
 --bootstrap-server localhost:9092 \
 --describe --topic orders

```

Expected output (describe):

```

Topic: orders TopicId: abc123 PartitionCount: 3 ReplicationFactor: 1
 Topic: orders Partition: 0 Leader: 1 Replicas: 1 Isr: 1
 Topic: orders Partition: 1 Leader: 1 Replicas: 1 Isr: 1
 Topic: orders Partition: 2 Leader: 1 Replicas: 1 Isr: 1

```

#### ⚠ Common Mistake

**Too few partitions** — You can't have more consumers (in a group) than partitions. 3 partitions = max 3 consumers. Plan ahead.

**Assuming global ordering** — Ordering is only guaranteed within a partition. If you need ordered processing per customer, use `customer_id` as the partition key.

#### ☐ Checkpoint

1. What's the difference between Kafka and a traditional message queue?
2. Why do partitions matter for parallelism?
3. Can two different consumer groups read the same topic independently?

## 6.3 Producing Messages — Serialization, Partitioning Strategies

### TL;DR

- Use `acks="all"` and `enable.idempotence=True` for reliable production
- Partition keys ensure all messages for the same entity go to the same partition (ordering)
- Always call `producer.flush()` before your program exits
- Batch messages with `linger.ms` for higher throughput with minimal latency increase

```
pip install confluent-kafka
```

```

producer.py
"""Kafka Producer – sends e-commerce order events."""

from confluent_kafka import Producer
import json
import time
import random
from datetime import datetime

def delivery_callback(err, msg):
 """Called once per message to indicate delivery result."""
 if err:
 print(f"❏ Delivery failed: {err}")
 else:
 print(f"❏ Delivered to {msg.topic()} [{msg.partition()}] @ offset {msg.offset()}")

def create_producer() -> Producer:
 """Create a Kafka producer with production-ready config."""
 config = {
 "bootstrap.servers": "localhost:9092",

 # Reliability
 "acks": "all", # Wait for all replicas to acknowledge
 "retries": 5, # Retry on transient failures
 "retry.backoff.ms": 1000, # 1s between retries

 # Performance
 "linger.ms": 10, # Batch messages for up to 10ms
 "batch.size": 65536, # 64KB batch size
 "compression.type": "snappy", # Compress messages

 # Idempotence – prevents duplicates on retry
 "enable.idempotence": True,
 }
 return Producer(config)

def generate_order_event() -> dict:
 """Generate a realistic e-commerce order event."""
 products = [
 {"id": "PROD-001", "name": "Wireless Headphones", "price": 79.99},
 {"id": "PROD-002", "name": "USB-C Hub", "price": 49.99},
 {"id": "PROD-003", "name": "Mechanical Keyboard", "price": 149.99},
 {"id": "PROD-004", "name": "Monitor Stand", "price": 39.99},
 {"id": "PROD-005", "name": "Webcam HD", "price": 69.99},
]

 product = random.choice(products)
 customer_id = f"CUST-{random.randint(1, 100):04d}"
 quantity = random.randint(1, 3)

```

```

 return {
 "event_type": "order_placed",
 "order_id": f"ORD-{int(time.time() * 1000)}-{random.randint(100, 999)}",
 "customer_id": customer_id,
 "product_id": product["id"],
 "product_name": product["name"],
 "quantity": quantity,
 "unit_price": product["price"],
 "total_amount": round(product["price"] * quantity, 2),
 "currency": "USD",
 "timestamp": datetime.utcnow().isoformat() + "Z",
 }

def main():
 producer = create_producer()
 topic = "orders"

 print(f"▣ Starting producer – sending events to '{topic}'")
 print("Press Ctrl+C to stop\n")

 try:
 count = 0
 while True:
 event = generate_order_event()

 # key = partition key. Same customer_id → same partition → ordered
 producer.produce(
 topic=topic,
 key=event["customer_id"],
 value=json.dumps(event),
 callback=delivery_callback,
)

 producer.poll(0) # Trigger delivery callbacks

 count += 1
 if count % 10 == 0:
 print(f"▣ Sent {count} events")

 time.sleep(random.uniform(0.1, 0.5))

 except KeyboardInterrupt:
 print(f"\n▣ Stopping. Flushing remaining messages...")
 producer.flush(10)
 print(f"Total sent: {count}")

if __name__ == "__main__":
 main()

```

## Key Producer Concepts

**Partition Key:** Setting `key=event["customer_id"]` means Kafka hashes the key and always routes it to the same partition. All events for `CUST-0001` go to the same partition, guaranteeing order for that customer. Without a key, Kafka round-robins (good for throughput, bad for ordering).

`acks="all"`: Waits for ALL replicas to confirm. Safest setting. Alternatives: `acks=0` (fire and forget), `acks=1` (leader only).

`enable.idempotence=True`: If the producer retries (network blip), Kafka deduplicates. Without this, retries cause duplicate messages. Always enable in production.

`linger.ms` and `batch.size`: Instead of sending one message at a time, the producer batches them. A 10ms linger dramatically improves throughput with a tiny latency increase.

## Verify Messages Are Flowing

```
docker compose exec kafka kafka-console-consumer.sh \
 --bootstrap-server localhost:9092 \
 --topic orders \
 --from-beginning \
 --property print.key=true \
 --property key.separator=": "
```

### ⚠ Common Mistake

**Not calling `producer.flush()`** — The producer buffers messages. If your program exits without flushing, buffered messages are lost.

**Not setting `enable.idempotence`** — Network retries cause duplicate messages without it.

**Producing massive messages** — Kafka's default max is 1MB. Keep messages small; use S3 URLs for large payloads.

## ☐ Checkpoint

1. What does the partition key determine?
2. Why is `enable.idempotence=True` important?
3. What happens if you don't call `flush()` before your program exits?

## 6.4 Consuming Messages — Offsets, Consumer Groups, Exactly-Once

### TL;DR

- Disable auto-commit and commit manually AFTER successful processing
- At-least-once + idempotent processing is the production sweet spot
- Always close the consumer cleanly for proper group rebalancing
- If processing is too slow, Kafka kicks you from the group ( `max.poll.interval.ms` )

Producing is the easy part. Consuming is where the real engineering happens — handling crashes, duplicate processing, and offset management.

```

consumer.py
"""Kafka Consumer – processes e-commerce order events."""

from confluent_kafka import Consumer, KafkaError, KafkaException
import json
from datetime import datetime
import signal
import sys

class OrderProcessor:
 """Processes order events and maintains running aggregations."""

 def __init__(self):
 self.total_revenue = 0.0
 self.order_count = 0
 self.customer_totals = {}

 def process(self, event: dict) -> bool:
 """Process a single order event. Returns True if successful."""
 try:
 order_id = event["order_id"]
 customer_id = event["customer_id"]
 amount = event["total_amount"]

 self.total_revenue += amount
 self.order_count += 1
 self.customer_totals[customer_id] = \
 self.customer_totals.get(customer_id, 0) + amount

 print(
 f"▢ Order {order_id}: ${amount:.2f} from {customer_id} "
 f"| Running total: ${self.total_revenue:.2f} ({self.order_count}
orders)"
)
 return True

 except (KeyError, TypeError) as e:
 print(f"△ Bad message format: {e} – skipping")
 return True # Don't retry malformed messages
 except Exception as e:
 print(f"▢ Processing error: {e} – will retry")
 return False

 def print_summary(self):
 print(f"\n{'='*50}")
 print(f"▢ SUMMARY")
 print(f"Total orders: {self.order_count}")
 print(f"Total revenue: ${self.total_revenue:.2f}")
 print(f"Avg order value: ${self.total_revenue/max(self.order_count,1):.2f}")
 print(f"Unique customers: {len(self.customer_totals)}")
 if self.customer_totals:
 top = max(self.customer_totals, key=self.customer_totals.get)

```



```

 print(f"Top customer: {top} (${self.customer_totals[top]:.2f})")
 print(f"{' '*50}\n")

def create_consumer(group_id: str) -> Consumer:
 """Create a Kafka consumer with production-ready config."""
 config = {
 "bootstrap.servers": "localhost:9092",
 "group.id": group_id,

 # Offset management
 "auto.offset.reset": "earliest", # Start from beginning if no offset
 "enable.auto.commit": False, # Manual commit for safety

 # Performance
 "max.poll.interval.ms": 300000, # 5 min max between polls
 "session.timeout.ms": 45000, # 45s heartbeat timeout
 "fetch.min.bytes": 1024, # Min data per fetch
 }
 return Consumer(config)

def main():
 consumer = create_consumer("order-processing-group")
 processor = OrderProcessor()
 running = True

 def shutdown(signum, frame):
 nonlocal running
 print("\n▣ Shutdown signal received...")
 running = False

 signal.signal(signal.SIGINT, shutdown)
 signal.signal(signal.SIGTERM, shutdown)

 consumer.subscribe(["orders"])
 print("▣ Consumer started – waiting for messages...\n")

 try:
 while running:
 msg = consumer.poll(timeout=1.0)

 if msg is None:
 continue

 if msg.error():
 if msg.error().code() == KafkaError._PARTITION_EOF:
 continue # End of partition – not an error
 else:
 raise KafkaException(msg.error())

 # Deserialize
 try:
 event = json.loads(msg.value().decode("utf-8"))

```

```

except json.JSONDecodeError as e:
 print(f"⚠ Invalid JSON: {e} – skipping")
 consumer.commit(msg) # Commit to skip this bad message
 continue

Process
success = processor.process(event)

if success:
 consumer.commit(msg) # Commit AFTER successful processing
else:
 print("🔄 Will retry on next poll")

finally:
 processor.print_summary()
 consumer.close() # Triggers clean group rebalance
 print("Consumer closed cleanly.")

if __name__ == "__main__":
 main()

```

## Delivery Semantics

### ❑ Key Concept: Delivery Guarantees

- **At-most-once:** Commit before processing. If processing fails, the message is lost. Fast but unreliable.
- **At-least-once:** Commit after processing. If the commit fails, the message is reprocessed. Safe but may have duplicates. **This is what we're using.**
- **Exactly-once:** Requires Kafka transactions or idempotent consumers. Complex but possible.

The production sweet spot: **at-least-once delivery + idempotent processing.**

```

Idempotent processing example – upsert instead of insert
def process_order_idempotent(event: dict, db_conn):
 """Safe to run multiple times – uses upsert."""
 db_conn.execute("""
 INSERT INTO processed_orders (order_id, customer_id, amount, processed_at)
 VALUES (%s, %s, %s, NOW())
 ON CONFLICT (order_id) DO NOTHING -- Idempotent!
 """, (event["order_id"], event["customer_id"], event["total_amount"]))

```

## Scaling with Consumer Groups

Run multiple consumers in the same group — Kafka distributes partitions automatically:

```
Terminal 1 – gets partitions 0, 1
python consumer.py

Terminal 2 (same group) – gets partition 2
python consumer.py

Kafka automatically rebalances!
```

#### ⚠ Common Mistake

**Auto-commit enabled** — Leads to silent data loss. Kafka commits offsets on a timer regardless of whether you processed the message. Always commit manually.

**Processing too slowly** — If `max.poll.interval.ms` expires, Kafka kicks you from the group. Process fast or increase the timeout.

**Not closing the consumer** — `consumer.close()` triggers a clean rebalance. Without it, Kafka waits for the session timeout before reassigning partitions.

#### ☐ Checkpoint

1. Why should you disable auto-commit?
2. What's the difference between at-most-once and at-least-once delivery?
3. What happens when you start a second consumer with the same group ID?

## 6.5 Kafka Connect — Plugging Into Databases Without Code

### TL;DR

- Source connectors stream data FROM external systems INTO Kafka
- Sink connectors stream data FROM Kafka TO external systems
- Debezium provides CDC (Change Data Capture) — every database change becomes a Kafka message
- Manage connectors via REST API; monitor them — they fail silently

Writing a custom producer for every data source gets old fast. Kafka Connect comes with pre-built connectors for Postgres, MySQL, S3, Elasticsearch, and hundreds more.

## Adding Kafka Connect to Docker Compose

```
kafka-connect:
 image: confluentinc/cp-kafka-connect:7.6.0
 ports:
 - "8083:8083"
 environment:
 CONNECT_BOOTSTRAP_SERVERS: kafka:9092
 CONNECT_REST_PORT: 8083
 CONNECT_GROUP_ID: connect-cluster
 CONNECT_CONFIG_STORAGE_TOPIC: _connect-configs
 CONNECT_OFFSET_STORAGE_TOPIC: _connect-offsets
 CONNECT_STATUS_STORAGE_TOPIC: _connect-status
 CONNECT_CONFIG_STORAGE_REPLICATION_FACTOR: 1
 CONNECT_OFFSET_STORAGE_REPLICATION_FACTOR: 1
 CONNECT_STATUS_STORAGE_REPLICATION_FACTOR: 1
 CONNECT_KEY_CONVERTER: org.apache.kafka.connect.json.JsonConverter
 CONNECT_VALUE_CONVERTER: org.apache.kafka.connect.json.JsonConverter
 CONNECT_PLUGIN_PATH: /usr/share/java,/usr/share/confluent-hub-components
 command:
 - bash
 - -c
 - |
 confluent-hub install --no-prompt debezium/debezium-connector-postgresql:2.5.0
 confluent-hub install --no-prompt confluentinc/kafka-connect-s3:10.5.7
 /etc/confluent/docker/run
 depends_on:
 - kafka
```

### Example: Postgres → Kafka (CDC with Debezium)

Change Data Capture streams every INSERT, UPDATE, and DELETE from Postgres into Kafka in real-time:

```
Deploy the connector via REST API
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
 "name": "postgres-source",
 "config": {
 "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
 "database.hostname": "postgres",
 "database.port": "5432",
 "database.user": "postgres",
 "database.password": "postgres",
 "database.dbname": "warehouse",
 "database.server.name": "warehouse",
 "table.include.list": "public.orders,public.customers",
 "topic.prefix": "cdc",
 "plugin.name": "pgoutput",
 "slot.name": "debezium_slot"
 }
}'
```

Now every change to `orders` or `customers` automatically appears in Kafka topics `cdc.public.orders` and `cdc.public.customers`. No code required.

### Example: Kafka → S3 (Sink)

```
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
 "name": "s3-sink",
 "config": {
 "connector.class": "io.confluent.connect.s3.S3SinkConnector",
 "tasks.max": "1",
 "topics": "orders",
 "s3.bucket.name": "my-data-lake",
 "s3.region": "us-east-1",
 "format.class": "io.confluent.connect.s3.format.parquet.ParquetFormat",
 "flush.size": "1000",
 "rotate.interval.ms": "60000",
 "partitioner.class":
"io.confluent.connect.storage.partitionner.TimeBasedPartitionner",
 "path.format": "year=YYYY/month=MM/day=dd/hour=HH",
 "timestamp.extractor": "Record"
 }
}'
```

This writes Kafka messages to S3 in Parquet format, partitioned by date/hour. This is how most companies build their data lake ingestion layer.

## Managing Connectors

```
List all connectors
curl http://localhost:8083/connectors

Check status
curl http://localhost:8083/connectors/postgres-source/status

Pause / Resume / Delete
curl -X PUT http://localhost:8083/connectors/postgres-source/pause
curl -X PUT http://localhost:8083/connectors/postgres-source/resume
curl -X DELETE http://localhost:8083/connectors/postgres-source
```

### ⚠ Common Mistake

**Not monitoring connector status** — Connectors fail silently. Check status regularly or set up alerts.

**No dead letter queue** — If a message can't be processed, it blocks the connector. Configure

`errors.deadletterqueue.topic.name` to route bad messages aside.

### ☐ Checkpoint

1. What's the difference between a source connector and a sink connector?
2. What is CDC and why is Debezium useful?
3. How do you check if a connector is healthy?

## 6.6 Schema Registry and Avro — Managing Data Contracts

### TL;DR

- Without schemas, producer changes break consumers silently at 3 AM
- Avro is a compact binary format with embedded schema — smaller and faster than JSON
- Schema Registry validates schemas before messages enter Kafka
- Schema evolution rules (BACKWARD, FORWARD, FULL) prevent breaking changes

Here's a production horror story. Team A produces messages to the `orders` topic with a field called `price`. Team B's consumer reads `price` and does math with it. One day, Team A renames `price` to `unit_price` and deploys. Team B's consumer crashes on every message. No warning. No review. Just a 3 AM page.

Schema Registry prevents this.

## How It Works

**Avro** — A binary serialization format with a schema. Messages are smaller, faster, and always match a defined structure.

**Schema Registry** — Stores and validates Avro schemas. When a producer sends a message, the schema is validated. If it doesn't match, the message is rejected BEFORE it hits Kafka.

**Schema Evolution** — Rules for how schemas can change:

- **BACKWARD** — new schema can read old data (add optional fields, remove fields)
- **FORWARD** — old schema can read new data
- **FULL** — both directions
- **NONE** — no checks (don't do this)

## Setup

```
schema-registry:
 image: confluentinc/cp-schema-registry:7.6.0
 ports:
 - "8081:8081"
 environment:
 SCHEMA_REGISTRY_HOST_NAME: schema-registry
 SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: kafka:9092
 depends_on:
 - kafka
```

## Producer with Avro

```
pip install confluent-kafka[avro]
```

```
Define an Avro schema
ORDER_EVENT_SCHEMA = """
{
 "type": "record",
 "name": "OrderEvent",
 "namespace": "com.ecommerce.events",
 "fields": [
 {"name": "order_id", "type": "string"},
 {"name": "customer_id", "type": "string"},
 {"name": "product_id", "type": "string"},
 {"name": "quantity", "type": "int"},
 {"name": "unit_price", "type": "double"},
 {"name": "total_amount", "type": "double"},
 {"name": "currency", "type": {"type": "string", "default": "USD"}},
 {"name": "timestamp", "type": "string"},
 {"name": "event_type", "type": {
 "type": "enum",
 "name": "EventType",
 "symbols": ["ORDER_PLACED", "ORDER_SHIPPED", "ORDER_CANCELLED"]
 }}
]
}
"""
```



```

avro_producer.py
from confluent_kafka import SerializingProducer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
import time
from datetime import datetime

Connect to Schema Registry
schema_registry_client = SchemaRegistryClient({"url": "http://localhost:8081"})

Create Avro serializer – validates against the schema
avro_serializer = AvroSerializer(
 schema_registry_client,
 schema_str=ORDER_EVENT_SCHEMA,
 to_dict=lambda obj, ctx: obj,
)

producer = SerializingProducer({
 "bootstrap.servers": "localhost:9092",
 "key.serializer": lambda k, ctx: k.encode("utf-8") if k else None,
 "value.serializer": avro_serializer,
})

This event matches the schema – it will be accepted
event = {
 "order_id": f"ORD-{int(time.time())}",
 "customer_id": "CUST-0001",
 "product_id": "PROD-001",
 "quantity": 2,
 "unit_price": 79.99,
 "total_amount": 159.98,
 "currency": "USD",
 "timestamp": datetime.utcnow().isoformat() + "Z",
 "event_type": "ORDER_PLACED",
}

producer.produce(topic="orders-avro", key=event["customer_id"], value=event)
producer.flush()
print(f"▯ Sent: {event['order_id']}")

This would FAIL – "INVALID_STATUS" isn't in the enum:
bad_event = {**event, "event_type": "INVALID_STATUS"}
producer.produce(topic="orders-avro", value=bad_event) # ▯ Schema violation!

```

## Consumer with Avro

```
avro_consumer.py
from confluent_kafka import DeserializingConsumer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroDeserializer

schema_registry_client = SchemaRegistryClient({"url": "http://localhost:8081"})
avro_deserializer = AvroDeserializer(schema_registry_client)

consumer = DeserializingConsumer({
 "bootstrap.servers": "localhost:9092",
 "group.id": "avro-consumer-group",
 "auto.offset.reset": "earliest",
 "key.deserializer": lambda k, ctx: k.decode("utf-8") if k else None,
 "value.deserializer": avro_deserializer,
})
consumer.subscribe(["orders-avro"])

while True:
 msg = consumer.poll(1.0)
 if msg is None:
 continue
 if msg.error():
 print(f"Error: {msg.error()}")
 continue

 # msg.value() is already a Python dict – Avro handles deserialization
 event = msg.value()
 print(f"📄 {event['order_id']}: {event['quantity']}x "
 f"📦 {event['product_id']} = ${event['total_amount']}")
```

## Schema Evolution

Want to add a `discount_code` field? Add it with a default value:

```
{"name": "discount_code", "type": ["null", "string"], "default": null}
```

This is **backward compatible** — new consumers can read old messages (`discount_code` defaults to null). The Schema Registry validates this automatically. If you try to remove a required field or change a type, the registry rejects the new schema.

### ⚠ Common Mistake

**Not using schemas at all** — JSON in Kafka works but leads to the Team A/B scenario. Use schemas for anything beyond prototyping.

**Setting compatibility to NONE** — Defeats the entire purpose.

## □ Checkpoint

1. What problem does Schema Registry solve?
  2. What does "backward compatible" mean for schema changes?
  3. Why is Avro preferred over JSON for Kafka messages?
- 

## 6.7 Building a Streaming Pipeline End-to-End

### TL;DR

- A complete pipeline: event generator → Kafka → stream processor → metrics topic → DB writer → dashboard
- The stream processor computes sliding-window aggregations in real-time
- The DB writer stores metrics in Postgres for querying
- This is a mini version of what Uber or DoorDash actually runs

We've learned the pieces. Now let's assemble them into a real streaming pipeline.

[DIAGRAM: Pipeline Architecture]

```
Event Generator → Kafka "raw-events" → Stream Processor → Kafka "metrics" → DB Writer →
Postgres
```

```
|
```

```
Dashboard Query
```

## Docker Compose (Full Stack)

```
version: '3.8'

services:
 kafka:
 image: bitnami/kafka:3.7
 ports:
 - "9092:9092"
 environment:
 - KAFKA_CFG_NODE_ID=1
 - KAFKA_CFG_PROCESS_ROLES=controller,broker
 - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
 - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092
 - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
 - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
 - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
 - KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE=true
 volumes:
 - kafka-data:/bitnami/kafka

 postgres:
 image: postgres:16
 ports:
 - "5432:5432"
 environment:
 POSTGRES_DB: streaming
 POSTGRES_USER: streaming
 POSTGRES_PASSWORD: streaming
 volumes:
 - pg-data:/var/lib/postgresql/data

 kafka-ui:
 image: provectuslabs/kafka-ui:latest
 ports:
 - "8080:8080"
 environment:
 - KAFKA_CLUSTERS_0_NAME=local
 - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:9092
 depends_on:
 - kafka

volumes:
 kafka-data:
 pg-data:
```

**Component 1: Event Generator**

```

event_generator.py
"""Simulates a busy e-commerce website generating click and purchase events."""

from confluent_kafka import Producer
import json
import random
import time
from datetime import datetime

producer = Producer({"bootstrap.servers": "localhost:9092"})

PAGES = ["home", "search", "product/headphones", "product/keyboard",
 "product/monitor", "cart", "checkout"]
Weighted toward page views (most common event)
EVENT_TYPES = ["page_view", "page_view", "page_view", "add_to_cart", "purchase"]

def generate_event():
 user_id = f"user_{random.randint(1, 500)}"
 event_type = random.choice(EVENT_TYPES)

 event = {
 "user_id": user_id,
 "event_type": event_type,
 "page": random.choice(PAGES),
 "timestamp": datetime.utcnow().isoformat() + "Z",
 "session_id": f"sess_{random.randint(1, 1000)}",
 }

 if event_type == "purchase":
 event["amount"] = round(random.uniform(19.99, 299.99), 2)
 event["product_id"] = f"prod_{random.randint(1, 50)}"

 if event_type == "add_to_cart":
 event["product_id"] = f"prod_{random.randint(1, 50)}"

 return user_id, event

def main():
 print("▣ Event generator started – simulating website traffic")
 count = 0

 try:
 while True:
 key, event = generate_event()
 producer.produce("raw-events", key=key, value=json.dumps(event))
 producer.poll(0)

 count += 1
 if count % 100 == 0:
 print(f" Generated {count} events")
 except KeyboardInterrupt:
 print("▣ Event generator stopped")

```

```
 time.sleep(random.uniform(0.01, 0.1)) # Simulate bursty traffic

 except KeyboardInterrupt:
 producer.flush()
 print(f"\n📊 Generated {count} total events")

if __name__ == "__main__":
 main()
```

**Component 2: Stream Processor**



```

stream_processor.py
"""Consumes raw events, computes real-time aggregations, produces metrics."""

from confluent_kafka import Consumer, Producer
import json
from datetime import datetime, timedelta
from collections import defaultdict
import threading
import time

class RealTimeAggregator:
 """Maintains sliding window aggregations."""

 def __init__(self, window_seconds: int = 60):
 self.window_seconds = window_seconds
 self.events = []
 self.lock = threading.Lock()

 def add_event(self, event: dict):
 with self.lock:
 ts = datetime.fromisoformat(event["timestamp"].rstrip("Z"))
 self.events.append((ts, event))
 self._prune()

 def _prune(self):
 """Remove events outside the window."""
 cutoff = datetime.utcnow() - timedelta(seconds=self.window_seconds)
 self.events = [(ts, e) for ts, e in self.events if ts > cutoff]

 def get_metrics(self) -> dict:
 with self.lock:
 self._prune()

 if not self.events:
 return {"window_seconds": self.window_seconds, "events": 0}

 events = [e for _, e in self.events]
 purchases = [e for e in events if e["event_type"] == "purchase"]
 unique_users = len(set(e["user_id"] for e in events))
 revenue = sum(e.get("amount", 0) for e in purchases)

 return {
 "timestamp": datetime.utcnow().isoformat() + "Z",
 "window_seconds": self.window_seconds,
 "total_events": len(events),
 "page_views": sum(1 for e in events if e["event_type"] == "page_view"),
 "cart_adds": sum(1 for e in events if e["event_type"] == "add_to_cart"),
 "purchases": len(purchases),
 "revenue": round(revenue, 2),
 "unique_users": unique_users,
 "conversion_rate": round(
 len(purchases) / max(unique_users, 1) * 100, 2
)
 }

```

```

),
 "events_per_second": round(
 len(events) / self.window_seconds, 1
),
 }

def main():
 consumer = Consumer({
 "bootstrap.servers": "localhost:9092",
 "group.id": "stream-processor",
 "auto.offset.reset": "latest",
 "enable.auto.commit": True,
 })

 metrics_producer = Producer({"bootstrap.servers": "localhost:9092"})
 aggregator = RealTimeAggregator(window_seconds=60)

 consumer.subscribe(["raw-events"])
 print("✂ Stream processor started")

 last_emit = time.time()
 events_processed = 0

 try:
 while True:
 msg = consumer.poll(0.1)

 if msg and not msg.error():
 event = json.loads(msg.value().decode("utf-8"))
 aggregator.add_event(event)
 events_processed += 1

 # Emit metrics every 5 seconds
 if time.time() - last_emit >= 5:
 metrics = aggregator.get_metrics()
 metrics_producer.produce(
 "metrics", key="realtime", value=json.dumps(metrics)
)
 metrics_producer.poll(0)

 print(
 f"▣ [{metrics['timestamp'][:19]}] "
 f"Events: {metrics['total_events']} | "
 f"Users: {metrics['unique_users']} | "
 f"Purchases: {metrics['purchases']} | "
 f"Revenue: ${metrics['revenue']:.2f} | "
 f"Conv: {metrics['conversion_rate']}%"
)
 last_emit = time.time()

 except KeyboardInterrupt:
 print(f"\n▣ Processed {events_processed} total events")
 finally:

```

```
consumer.close()
```

```
if __name__ == "__main__":
 main()
```

**Component 3: Database Writer**

```

db_writer.py
"""Consumes metrics from Kafka and stores in Postgres for dashboarding."""

from confluent_kafka import Consumer
import json
import psycopg2

def setup_database():
 conn = psycopg2.connect(
 host="localhost", port=5432,
 dbname="streaming", user="streaming", password="streaming"
)
 conn.autocommit = True
 cur = conn.cursor()
 cur.execute("""
 CREATE TABLE IF NOT EXISTS realtime_metrics (
 id SERIAL PRIMARY KEY,
 timestamp TIMESTAMPTZ NOT NULL,
 window_seconds INT,
 total_events INT,
 page_views INT,
 cart_adds INT,
 purchases INT,
 revenue DECIMAL(10,2),
 unique_users INT,
 conversion_rate DECIMAL(5,2),
 events_per_second DECIMAL(10,1),
 created_at TIMESTAMPTZ DEFAULT NOW()
);

 CREATE INDEX IF NOT EXISTS idx_metrics_timestamp
 ON realtime_metrics(timestamp DESC);
 """)
 return conn

def main():
 conn = setup_database()
 cur = conn.cursor()

 consumer = Consumer({
 "bootstrap.servers": "localhost:9092",
 "group.id": "db-writer",
 "auto.offset.reset": "latest",
 "enable.auto.commit": False,
 })
 consumer.subscribe(["metrics"])

 print("▯ DB writer started – storing metrics to Postgres")

 try:
 while True:

```

```

msg = consumer.poll(1.0)
if msg is None or msg.error():
 continue

metrics = json.loads(msg.value().decode("utf-8"))

cur.execute("""
 INSERT INTO realtime_metrics
 (timestamp, window_seconds, total_events, page_views,
 cart_adds, purchases, revenue, unique_users,
 conversion_rate, events_per_second)
 VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
""", (
 metrics["timestamp"], metrics["window_seconds"],
 metrics["total_events"], metrics["page_views"],
 metrics["cart_adds"], metrics["purchases"],
 metrics["revenue"], metrics["unique_users"],
 metrics["conversion_rate"], metrics["events_per_second"],
))
conn.commit()
consumer.commit(msg)

print(f" Stored metrics for {metrics['timestamp'][:19]}")

except KeyboardInterrupt:
 print("\n DB writer stopped")
finally:
 consumer.close()
 conn.close()

if __name__ == "__main__":
 main()

```

## Running the Complete Pipeline

```
Terminal 1: Start infrastructure
docker compose up -d

Terminal 2: Event generator
python event_generator.py

Terminal 3: Stream processor
python stream_processor.py

Terminal 4: DB writer
python db_writer.py

Terminal 5: Query the live results!
docker compose exec postgres psql -U streaming -d streaming -c "
 SELECT
 timestamp::time as time,
 total_events,
 purchases,
 revenue,
 conversion_rate
 FROM realtime_metrics
 ORDER BY timestamp DESC
 LIMIT 10;
"
```

### Expected output (query):

time	total_events	purchases	revenue	conversion_rate
14:32:15	247	12	891.23	4.85
14:32:10	231	10	745.50	4.33
14:32:05	218	11	823.17	5.05
...				

### ❑ Checkpoint

1. What are the three main components of this streaming pipeline?
2. Why does the stream processor emit metrics every 5 seconds instead of per-message?
3. How would you add a new consumer group that does fraud detection without affecting the existing pipeline?

## 6.8 Kafka in Production — Monitoring, Scaling, Common Failure Modes

### TL;DR

- Consumer lag is the #1 metric to watch — if it's growing, your consumer can't keep up
- Use `CooperativeStickyAssignor` to minimize rebalancing disruptions
- Set retention policies per topic to avoid filling disks
- Consider managed Kafka (Confluent Cloud, Redpanda) unless you have dedicated platform engineers

Running Kafka locally with Docker is one thing. Running it in production with millions of messages per day is where the real lessons are.

### The Top 5 Production Issues

#### 1. Consumer Lag

Consumer lag = how far behind your consumer is from the latest message. If lag is growing, your consumer can't keep up.

```
docker compose exec kafka kafka-consumer-groups.sh \
 --bootstrap-server localhost:9092 \
 --group order-processing-group \
 --describe
```

If lag is growing:

- Scale consumers (add more instances, up to partition count)
- Optimize processing logic (batch DB writes, reduce per-message overhead)
- Increase partition count

#### 2. Rebalancing Storms

When consumers join/leave, Kafka rebalances partitions. During rebalancing, NO messages are processed. Fix:

```
consumer_conf = {
 "partition.assignment.strategy": "cooperative-sticky", # Minimal disruption
 "session.timeout.ms": 45000,
 "heartbeat.interval.ms": 15000,
}
```

#### 3. Hot Partitions

If one key is way more popular, that partition's consumer does all the work. Fix: use a composite key or better hash distribution.



## 4. Disk Space

Kafka stores messages on disk. Default retention is 7 days. High throughput = lots of disk.

```
Set retention per topic (3 days)
docker compose exec kafka kafka-configs.sh \
 --bootstrap-server localhost:9092 \
 --alter --entity-type topics --entity-name orders \
 --add-config retention.ms=259200000
```

## 5. Message Ordering Violations

Retries with `max.in.flight.requests.per.connection > 1` can cause out-of-order delivery. Fix: use `enable.idempotence=true` (handles this automatically).

## Production Checklist

- ▣ Minimum 3 brokers (fault tolerance)
- ▣ Replication factor  $\geq 3$  for important topics
- ▣ Consumer lag monitoring with alerting
- ▣ `enable.idempotence = true` on producers
- ▣ Manual offset commits (no auto-commit)
- ▣ Dead letter queue for poison messages
- ▣ Schema Registry for data contracts
- ▣ Retention policy per topic
- ▣ Monitoring: broker health, partition leadership, under-replicated partitions
- ▣ Backup plan: what happens if Kafka goes down entirely?

## Managed Kafka Options

Running Kafka yourself is complex. Most companies use managed services:

- **Confluent Cloud** — by the creators of Kafka. Full managed + Schema Registry + Connectors
- **AWS MSK** — Amazon Managed Streaming for Kafka. Basic but solid
- **Redpanda** — Kafka-compatible, written in C++. Faster, simpler, no JVM
- **Aiven** — multi-cloud managed Kafka

### ❏ Pro Tip

If you're a small team, use Confluent Cloud or Redpanda Cloud. Don't operate Kafka yourself unless you have dedicated platform engineers.

### ❏ Checkpoint

1. What does growing consumer lag tell you?
2. Why is the CooperativeStickyAssignor better than the default?

3. When should you use managed Kafka vs. self-hosted?

---

## **Module 6 Project: Real-Time E-Commerce Analytics Pipeline**

Build a complete real-time pipeline: event generation → Kafka → stream processing → Postgres → live dashboard.

## Step 1: Set Up the Full Stack

```
docker-compose.yml
version: '3.8'

services:
 kafka:
 image: bitnami/kafka:3.7
 ports:
 - "9092:9092"
 - "9094:9094"
 environment:
 - KAFKA_CFG_NODE_ID=1
 - KAFKA_CFG_PROCESS_ROLES=controller,broker
 - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093,EXTERNAL://:9094
 - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092,EXTERNAL://localhost:9094
 - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
 - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
 - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
 - KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE=true
 volumes:
 - kafka-data:/bitnami/kafka

 postgres:
 image: postgres:16
 ports:
 - "5432:5432"
 environment:
 POSTGRES_DB: ecommerce
 POSTGRES_USER: ecommerce
 POSTGRES_PASSWORD: ecommerce
 volumes:
 - ./sql/init.sql:/docker-entrypoint-initdb.d/init.sql
 - pg-data:/var/lib/postgresql/data

 kafka-ui:
 image: provectuslabs/kafka-ui:latest
 ports:
 - "8080:8080"
 environment:
 - KAFKA_CLUSTERS_0_NAME=local
 - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:9092
 depends_on:
 - kafka

volumes:
 kafka-data:
 pg-data:
```

**Step 2: Database Setup**

```

-- sql/init.sql
CREATE TABLE realtime_metrics (
 id SERIAL PRIMARY KEY,
 window_start TIMESTAMPTZ NOT NULL,
 window_end TIMESTAMPTZ NOT NULL,
 total_events INT DEFAULT 0,
 unique_users INT DEFAULT 0,
 page_views INT DEFAULT 0,
 cart_adds INT DEFAULT 0,
 purchases INT DEFAULT 0,
 revenue DECIMAL(12,2) DEFAULT 0,
 avg_order_value DECIMAL(10,2) DEFAULT 0,
 conversion_rate DECIMAL(5,2) DEFAULT 0,
 top_products JSONB DEFAULT '[]',
 created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE raw_events (
 id SERIAL PRIMARY KEY,
 event_id VARCHAR(100) UNIQUE,
 user_id VARCHAR(50) NOT NULL,
 event_type VARCHAR(50) NOT NULL,
 page VARCHAR(200),
 product_id VARCHAR(50),
 amount DECIMAL(10,2),
 session_id VARCHAR(100),
 event_timestamp TIMESTAMPTZ NOT NULL,
 kafka_partition INT,
 kafka_offset BIGINT,
 processed_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_raw_events_type ON raw_events(event_type);
CREATE INDEX idx_raw_events_timestamp ON raw_events(event_timestamp DESC);
CREATE INDEX idx_raw_events_user ON raw_events(user_id);

CREATE TABLE hourly_stats (
 hour TIMESTAMPTZ PRIMARY KEY,
 events INT,
 unique_users INT,
 purchases INT,
 revenue DECIMAL(12,2),
 avg_order_value DECIMAL(10,2)
);

-- Live dashboard view
CREATE VIEW live_dashboard AS
SELECT
 date_trunc('minute', event_timestamp) as minute,
 COUNT(*) as events,
 COUNT(DISTINCT user_id) as users,
 COUNT(*) FILTER (WHERE event_type = 'purchase') as purchases,
 COALESCE(SUM(amount) FILTER (WHERE event_type = 'purchase'), 0) as revenue

```

```
FROM raw_events
WHERE event_timestamp > NOW() - INTERVAL '1 hour'
GROUP BY 1
ORDER BY 1 DESC;
```

**Step 3: Event Producer with Realistic Funnel Behavior**

```

src/producer.py
"""Simulates realistic user behavior with sessions and conversion funnels."""

from confluent_kafka import Producer
import json
import random
import time
import uuid
from datetime import datetime

PRODUCTS = {
 "prod_001": {"name": "Wireless Headphones", "price": 79.99, "category": "audio"},
 "prod_002": {"name": "Mechanical Keyboard", "price": 149.99, "category":
"peripherals"},
 "prod_003": {"name": "4K Monitor", "price": 399.99, "category": "displays"},
 "prod_004": {"name": "USB-C Hub", "price": 49.99, "category": "accessories"},
 "prod_005": {"name": "Webcam Pro", "price": 129.99, "category": "video"},
 "prod_006": {"name": "Mouse Pad XL", "price": 24.99, "category": "accessories"},
 "prod_007": {"name": "Standing Desk", "price": 599.99, "category": "furniture"},
 "prod_008": {"name": "Laptop Stand", "price": 39.99, "category": "accessories"},
}

class UserSession:
 """Simulates a user browsing session with realistic drop-off."""

 FUNNEL = ["page_view", "page_view", "page_view", "add_to_cart", "purchase"]

 def __init__(self):
 self.user_id = f"user_{random.randint(1, 1000)}"
 self.session_id = str(uuid.uuid4())[:8]
 self.step = 0
 self.viewed_product = random.choice(list(PRODUCTS.keys()))

 def next_event(self) -> dict | None:
 if self.step >= len(self.FUNNEL):
 return None

 event_type = self.FUNNEL[self.step]

 # Realistic drop-off: 60% continue to next step
 if self.step > 0 and random.random() > 0.6:
 return None

 event = {
 "event_id": str(uuid.uuid4()),
 "user_id": self.user_id,
 "session_id": self.session_id,
 "event_type": event_type,
 "timestamp": datetime.utcnow().isoformat() + "Z",
 }

 if event_type == "page_view":

```



```

 event["page"] = random.choice([
 "home", "search", f"product/{self.viewed_product}"
])
 elif event_type == "add_to_cart":
 event["product_id"] = self.viewed_product
 elif event_type == "purchase":
 product = PRODUCTS[self.viewed_product]
 quantity = random.randint(1, 3)
 event["product_id"] = self.viewed_product
 event["quantity"] = quantity
 event["amount"] = round(product["price"] * quantity, 2)

 self.step += 1
 return event

def main():
 producer = Producer({
 "bootstrap.servers": "localhost:9094",
 "acks": "all",
 "enable.idempotence": True,
 "compression.type": "snappy",
 "linger.ms": 5,
 })

 sessions = []
 total_sent = 0

 print("▣ E-commerce event generator started")
 print(" Press Ctrl+C to stop\n")

 try:
 while True:
 # Start new sessions (new visitors arriving)
 if random.random() < 0.3:
 sessions.append(UserSession())

 active_sessions = []
 for session in sessions:
 event = session.next_event()
 if event:
 producer.produce("raw-events", key=event["user_id"],
 value=json.dumps(event))

 total_sent += 1
 active_sessions.append(session)

 if event["event_type"] == "purchase":
 print(f" ▣ Purchase! {event['user_id']}: ${event['amount']}")

 sessions = active_sessions
 producer.poll(0)

 if total_sent % 100 == 0 and total_sent > 0:
 print(f" ▣ Sent {total_sent} events | Active sessions:

```

```
{len(sessions)}")

 time.sleep(random.uniform(0.05, 0.2))

except KeyboardInterrupt:
 producer.flush(10)
 print(f"\n Total events sent: {total_sent}")

if __name__ == "__main__":
 main()
```

**Step 4: Stream Processor with Postgres Storage**

```

src/processor.py
"""Consumes raw events, stores them, and computes real-time metrics."""

from confluent_kafka import Consumer, KafkaError
import json
import psycpg2
from datetime import datetime, timedelta
from collections import defaultdict
import time
import signal

class MetricsAggregator:
 """Computes real-time metrics over a sliding window."""

 def __init__(self, window_seconds: int = 60):
 self.window = window_seconds
 self.events = []

 def add(self, event: dict):
 ts = datetime.fromisoformat(event["timestamp"].rstrip("Z"))
 self.events.append((ts, event))
 self._prune()

 def _prune(self):
 cutoff = datetime.utcnow() - timedelta(seconds=self.window)
 self.events = [(ts, e) for ts, e in self.events if ts > cutoff]

 def compute(self) -> dict | None:
 self._prune()
 if not self.events:
 return None

 events = [e for _, e in self.events]
 now = datetime.utcnow()

 purchases = [e for e in events if e["event_type"] == "purchase"]
 unique_users = set(e["user_id"] for e in events)
 revenue = sum(e.get("amount", 0) for e in purchases)

 product_counts = defaultdict(int)
 for p in purchases:
 product_counts[p.get("product_id", "unknown")] += 1
 top_products = sorted(product_counts.items(), key=lambda x: -x[1])[:5]

 return {
 "window_start": (now - timedelta(seconds=self.window)).isoformat() + "Z",
 "window_end": now.isoformat() + "Z",
 "total_events": len(events),
 "unique_users": len(unique_users),
 "page_views": sum(1 for e in events if e["event_type"] == "page_view"),
 "cart_adds": sum(1 for e in events if e["event_type"] == "add_to_cart"),
 "purchases": len(purchases),
 }

```

```

 "revenue": round(revenue, 2),
 "avg_order_value": round(revenue / len(purchases), 2) if purchases else 0,
 "conversion_rate": round(
 len(purchases) / max(len(unique_users), 1) * 100, 2
),
 "top_products": json.dumps(
 [{"product": p, "count": c} for p, c in top_products]
),
 }

def main():
 consumer = Consumer({
 "bootstrap.servers": "localhost:9094",
 "group.id": "ecommerce-processor",
 "auto.offset.reset": "earliest",
 "enable.auto.commit": False,
 })
 consumer.subscribe(["raw-events"])

 conn = psycopg2.connect(
 host="localhost", port=5432,
 dbname="ecommerce", user="ecommerce", password="ecommerce"
)
 conn.autocommit = False
 cur = conn.cursor()

 aggregator = MetricsAggregator(window_seconds=60)
 running = True

 def stop(sig, frame):
 nonlocal running
 running = False
 signal.signal(signal.SIGINT, stop)
 signal.signal(signal.SIGTERM, stop)

 print("> Stream processor started\n")

 events_processed = 0
 batch_size = 0
 last_metrics_time = time.time()
 last_commit_time = time.time()

 try:
 while running:
 msg = consumer.poll(0.1)

 if msg is None:
 pass
 elif msg.error():
 if msg.error().code() != KafkaError._PARTITION_EOF:
 print(f"Consumer error: {msg.error()}")
 else:
 event = json.loads(msg.value().decode("utf-8"))

```

```

Store raw event (idempotent via ON CONFLICT)
cur.execute("""
 INSERT INTO raw_events
 (event_id, user_id, event_type, page, product_id,
 amount, session_id, event_timestamp,
 kafka_partition, kafka_offset)
 VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
 ON CONFLICT (event_id) DO NOTHING
""", (
 event.get("event_id"), event["user_id"],
 event["event_type"], event.get("page"),
 event.get("product_id"), event.get("amount"),
 event.get("session_id"), event["timestamp"],
 msg.partition(), msg.offset()
))

aggregator.add(event)
events_processed += 1
batch_size += 1

Commit every 100 messages or 5 seconds
now = time.time()
if batch_size >= 100 or (batch_size > 0 and now - last_commit_time >= 5):
 conn.commit()
 consumer.commit()
 batch_size = 0
 last_commit_time = now

Emit metrics every 10 seconds
if now - last_metrics_time >= 10:
 metrics = aggregator.compute()
 if metrics:
 cur.execute("""
 INSERT INTO realtime_metrics
 (window_start, window_end, total_events, unique_users,
 page_views, cart_adds, purchases, revenue,
 avg_order_value, conversion_rate, top_products)
 VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
""", (
 metrics["window_start"], metrics["window_end"],
 metrics["total_events"], metrics["unique_users"],
 metrics["page_views"], metrics["cart_adds"],
 metrics["purchases"], metrics["revenue"],
 metrics["avg_order_value"], metrics["conversion_rate"],
 metrics["top_products"],
))
 conn.commit()

 print(
 f"📅 [{datetime.utcnow().strftime('%H:%M:%S')}] "
 f"Events: {metrics['total_events']} | "
 f"Users: {metrics['unique_users']} | "
 f"Revenue: ${metrics['revenue']:.2f} | "
)

```

```

 f"Conv: {metrics['conversion_rate']]% | "
 f"Total: {events_processed}"
)
 last_metrics_time = now

finally:
 if batch_size > 0:
 conn.commit()
 consumer.commit()
 consumer.close()
 cur.close()
 conn.close()
 print(f"\n▯ Processed {events_processed} events total")

if __name__ == "__main__":
 main()

```

**Step 5: Live Dashboard**



```

src/dashboard.py
"""CLI dashboard – queries Postgres for live metrics."""

import psycopg2
import time
import os

def clear_screen():
 os.system("clear" if os.name == "posix" else "cls")

def main():
 conn = psycopg2.connect(
 host="localhost", port=5432,
 dbname="ecommerce", user="ecommerce", password="ecommerce"
)

 print("▣ Live Dashboard – refreshes every 5 seconds\n")

 try:
 while True:
 clear_screen()
 cur = conn.cursor()

 # Latest metrics
 cur.execute("""
 SELECT window_end, total_events, unique_users, purchases,
 revenue, conversion_rate
 FROM realtime_metrics
 ORDER BY created_at DESC LIMIT 1
 """)
 latest = cur.fetchone()

 print("=" * 60)
 print(" ▣ E-COMMERCE REAL-TIME DASHBOARD")
 print("=" * 60)

 if latest:
 print(f"\n ▣ Last update: {latest[0]}")
 print(f" ▣ Events (60s window): {latest[1]}")
 print(f" ▣ Unique users: {latest[2]}")
 print(f" ▣ Purchases: {latest[3]}")
 print(f" ▣ Revenue: ${latest[4]:.2f}")
 print(f" ▣ Conversion rate: {latest[5]}%")

 # Per-minute breakdown (last 10 minutes)
 cur.execute("SELECT * FROM live_dashboard LIMIT 10")
 minutes = cur.fetchall()

 if minutes:
 print(f"\n {'-' * 40}")
 print(f" PER-MINUTE BREAKDOWN:")

```

```

 print(f" {'Time':<12} {'Events':>8} {'Users':>8} "
 f" {'Sales':>8} {'Revenue':>10}")
 for row in minutes:
 print(f" {row[0].strftime('%H:%M'):<12} {row[1]:>8} "
 f" {row[2]:>8} {row[3]:>8} ${row[4]:>9.2f}")

 print(f"\n {'=' * 60}")
 print(f" Refreshing in 5 seconds...")

 cur.close()
 time.sleep(5)

except KeyboardInterrupt:
 print("\n\n Dashboard closed")
finally:
 conn.close()

if __name__ == "__main__":
 main()

```

## Step 6: Run the Full Pipeline

```

Start infrastructure
docker compose up -d
sleep 10

Install dependencies
pip install confluent-kafka psycpg2-binary

Terminal 1: Event generator
python src/producer.py

Terminal 2: Stream processor
python src/processor.py

Terminal 3: Live dashboard
python src/dashboard.py

```

## Expected Output

1. **Event generator** producing 5–20 events/second with realistic funnel drop-off
2. **Stream processor** consuming events, computing 60-second metrics, writing to Postgres
3. **Dashboard** showing live KPIs refreshing every 5 seconds
4. **Kafka UI** at localhost:8080 showing topics, consumer groups, and lag

## Stretch Goals

1. **Dead letter topic** — Route events that fail processing to `raw-events-dlq`

2. **Exactly-once semantics** — Use an idempotent consumer with deduplication (already partially implemented via `ON CONFLICT` )
  3. **Avro serialization** — Add Schema Registry and convert to Avro format
  4. **Fraud detection consumer** — A second consumer group that flags purchases > \$500 or > 3 purchases/minute per user
  5. **S3 archival** — Add Kafka Connect with the S3 sink connector to archive raw events in Parquet
- 

*End of Module 6*

# Module 7: Data Quality & Testing

*The difference between a junior and senior DE is data quality. Learn to build pipelines that don't silently break.*

## 7.1 Why Data Quality Is Your Job (Not the Analyst's)

**TL;DR:** Data quality is the data engineer's responsibility because you control the pipeline. Silent failures — not crashes — are the #1 way data engineers lose credibility. Build quality checks into every stage, not just at the end.

### The Silent Killer

It's 9:15 AM on a Monday. Your VP of Sales opens their dashboard, sees revenue dropped 40% over the weekend, panics, fires off an email to the CEO. By 9:30 the CEO is pinging your manager. By 10 AM your manager is pinging you. And what happened? A third-party API changed their date format from `YYYY-MM-DD` to `MM/DD/YYYY` and your pipeline silently ingested garbage all weekend.

Nobody's pipeline crashed. No errors in the logs. The data just... went bad. And nobody knew until someone looked at a dashboard two days later.

This is the #1 way data engineers lose credibility. Not because of downtime — because of *silent failures*.

### The Six Categories of Data Quality Issues

Here's the thing most people get wrong: data quality is not a separate step you add at the end. It's baked into every single stage of your pipeline — like checking that a foundation is level while building a house, not after the roof is on.

Every data quality issue you'll encounter in the real world falls into one of these categories:

1. **Schema drift** — upstream source adds, removes, or renames columns
2. **Completeness** — missing values, nulls where there shouldn't be any
3. **Freshness** — data is stale; the pipeline didn't run or ran late
4. **Accuracy** — values are present but wrong (duplicates, bad calculations)
5. **Consistency** — the same entity has different values in different tables
6. **Volume anomalies** — suddenly getting 10x or 0.1x the normal row count

The nasty part? Most of these don't show up as errors. They show up as *wrong numbers in dashboards* weeks later.

### ❑ Key Concept: Silent Failures

A silent failure is when your pipeline runs successfully (no errors, exit code 0) but produces incorrect data. These are far more dangerous than crashes because nobody knows anything is wrong until downstream consumers notice bad numbers.

## Real-World Horror Stories

These are from real companies:

- **Fintech startup:** Loaded duplicate transactions for 3 weeks because their deduplication key changed upstream. Cost: \$2M in misreported revenue.
- **E-commerce company:** Product dimension table got a column renamed from `category` to `product_category`. Joins silently returned NULLs. Nobody noticed for 5 days.
- **Healthcare company:** Date parsing silently converted European dates (DD/MM/YYYY) to American (MM/DD/YYYY). Patient records got wrong dates for months.

Every single one of these was preventable with basic data quality checks.

## The Ownership Model

"Isn't data quality the data team's problem? Like, shouldn't the analysts check their own stuff?"

No. Here's why: you, the data engineer, control the pipeline. You decide what gets loaded, how it gets transformed, and where it lands. If bad data gets through, it's because your pipeline let it through. You're the gatekeeper.

That doesn't mean analysts shouldn't validate — they absolutely should. But think of it as defense in depth:

Layer	Owner	What It Checks
Layer 1: Pipeline-level	Data Engineer	Schema validation, null checks, freshness, volume
Layer 2: Transformation-level	Data Engineer	Business logic validation, referential integrity
Layer 3: Dashboard-level	Analyst	Metric validation, trend analysis

You own Layers 1 and 2.

### ⚠ Common Mistake

The biggest mistake: adding data quality checks *after* an incident (reactive, not proactive). Build them in from day one. The second biggest: only checking the happy path. Your checks need to catch *unexpected* data, not just validate expected data.

## □ Checkpoint

1. What's the difference between a pipeline failure and a silent failure? Which is more dangerous and why?
  2. Name the six categories of data quality issues.
  3. Which layers of the data quality defense model are the data engineer's responsibility?
- 

## 7.2 Types of Data Quality Checks

**TL;DR:** There are six core categories of data quality checks: schema validation, completeness, freshness, accuracy, consistency, and volume anomalies. You can't check everything — be strategic about which checks you implement for each table.

You're convinced data quality matters. But what do you actually *check*? You can't check everything — that would make your pipeline take 10x longer. You need to be strategic.

Here are the exact checks to add to every pipeline, with real SQL running against our e-commerce tables.

### 1. Schema Validation

Before you even load data, check that it looks like what you expect.

```
-- Check column count and names match expected schema
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'raw_orders'
ORDER BY ordinal_position;
```

In Python, validate *before* the data touches the database:

```

import pandas as pd

Define the contract for what the data should look like
EXPECTED_COLUMNS = {
 'order_id': 'int64',
 'customer_id': 'int64',
 'order_date': 'datetime64[ns]',
 'total_amount': 'float64',
 'status': 'object',
}

def validate_schema(df: pd.DataFrame) -> bool:
 """Validate DataFrame matches expected schema."""
 missing = set(EXPECTED_COLUMNS.keys()) - set(df.columns)
 extra = set(df.columns) - set(EXPECTED_COLUMNS.keys())

 if missing:
 raise ValueError(f"Missing columns: {missing}")
 if extra:
 # Don't fail on extra columns – but log it
 print(f"WARNING: Extra columns detected: {extra}")

 # Check types
 for col, expected_type in EXPECTED_COLUMNS.items():
 actual_type = str(df[col].dtype)
 if actual_type != expected_type:
 raise TypeError(
 f"Column '{col}' expected {expected_type}, got {actual_type}"
)

 return True

```

Expected output (when validation passes):

```
True
```

Expected output (when a column is missing):

```
ValueError: Missing columns: {'customer_id'}
```

## 2. Completeness Checks

Are there nulls where there shouldn't be?

```
-- Null check for critical columns
SELECT
 COUNT(*) AS total_rows,
 COUNT(*) - COUNT(order_id) AS null_order_ids,
 COUNT(*) - COUNT(customer_id) AS null_customer_ids,
 COUNT(*) - COUNT(order_date) AS null_order_dates,
 COUNT(*) - COUNT(total_amount) AS null_amounts
FROM raw_orders
WHERE loaded_at >= CURRENT_DATE; -- Only check today's load
```

Expected output:

total_rows	null_order_ids	null_customer_ids	null_order_dates	null_amounts
1523	0	0	0	12

#### Key Concept: Not All Nulls Are Bad

A `shipping_date` being null is perfectly fine if the order hasn't shipped yet. But an `order_id` being null? That's a hard failure. Define your null-tolerance rules per column.

### 3. Freshness Checks

Is the data actually current?

```
-- Check that we have data from the expected time window
SELECT
 MAX(order_date) AS most_recent_order,
 NOW() - MAX(order_date) AS data_lag,
 CASE
 WHEN NOW() - MAX(order_date) > INTERVAL '2 hours'
 THEN 'STALE'
 ELSE 'FRESH'
 END AS freshness_status
FROM raw_orders;
```

Expected output:

most_recent_order	data_lag	freshness_status
2026-02-19 14:32:11+00	00:45:22	FRESH

Freshness checks fail when: upstream systems are down, your scheduler skipped a run, or there's a network issue. Freshness is often the *first* signal that something's wrong.



## 4. Accuracy / Validity Checks

Do the values make sense?

```
-- Range checks
SELECT COUNT(*) AS invalid_amounts
FROM raw_orders
WHERE total_amount < 0
 OR total_amount > 100000; -- No single order should be > $100K

-- Enum/category validation
SELECT status, COUNT(*)
FROM raw_orders
WHERE status NOT IN ('pending', 'processing', 'shipped', 'delivered', 'cancelled')
GROUP BY status;
-- Any rows here = unexpected status values

-- Date sanity
SELECT COUNT(*) AS future_orders
FROM raw_orders
WHERE order_date > NOW(); -- Orders from the future = bad data

-- Referential integrity
SELECT o.order_id
FROM raw_orders o
LEFT JOIN dim_customers c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
-- Orphaned orders = customer data missing or mismatched
```

## 5. Consistency Checks

Does the same thing look the same everywhere?

```
-- Cross-table consistency
SELECT
 (SELECT SUM(total_amount) FROM fact_orders WHERE order_date = '2026-02-18') AS
fact_total,
 (SELECT SUM(total_amount) FROM raw_orders WHERE order_date::date = '2026-02-18') AS
raw_total;
-- These should match (or be within acceptable rounding tolerance)

-- Unique constraint validation
SELECT order_id, COUNT(*) AS dupes
FROM fact_orders
GROUP BY order_id
HAVING COUNT(*) > 1;
-- Any results = duplicates in your fact table. Bad.
```

## 6. Volume Anomaly Detection

Did we get a reasonable amount of data?

```
-- Compare today's volume against a rolling 30-day average
WITH daily_volumes AS (
 SELECT
 order_date::date AS load_date,
 COUNT(*) AS row_count
 FROM raw_orders
 WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'
 GROUP BY order_date::date
),
stats AS (
 SELECT
 AVG(row_count) AS avg_count,
 STDDEV(row_count) AS stddev_count
 FROM daily_volumes
 WHERE load_date < CURRENT_DATE
)
SELECT
 dv.load_date,
 dv.row_count,
 s.avg_count,
 CASE
 WHEN dv.row_count < s.avg_count - 2 * s.stddev_count THEN 'ANOMALY_LOW'
 WHEN dv.row_count > s.avg_count + 2 * s.stddev_count THEN 'ANOMALY_HIGH'
 ELSE 'NORMAL'
 END AS volume_status
FROM daily_volumes dv, stats s
WHERE dv.load_date = CURRENT_DATE;
```

Expected output:

load_date	row_count	avg_count	volume_status
2026-02-19	1523	1480.3	NORMAL

This check catches the "pipeline ran but loaded 0 rows" problem. If you're getting 10,000 orders a day and suddenly get 50, something is wrong even if there are no errors.

#### ⚠ Common Mistake

- Checking only for nulls and calling it "data quality." That's maybe 20% of what you need.
- Hard-coding thresholds instead of using statistical baselines. What's "normal" changes over time.
- Running checks *after* the data hits the warehouse. Run them *before* or *during* loading so you can reject bad batches.

#### 📌 Pro Tip

Start with the checks most likely to catch real problems at your company. For most teams, that's: null checks on PKs, freshness, and volume anomalies. You can add more as you discover issues.

#### 📌 Checkpoint

1. Write a SQL query that checks for duplicate `order_id` values in a `raw_orders` table.
2. Why is a volume anomaly check better than a simple "row count > 0" check?
3. What's the difference between an accuracy check and a consistency check?

## 7.3 Great Expectations & Soda — Automated Data Quality Frameworks

**TL;DR:** Great Expectations (GX) and Soda let you define data quality checks as config instead of hand-written SQL. GX is the industry standard with powerful features and auto-generated reports. Soda is simpler and YAML-driven. Use whichever fits your team.

Writing SQL checks by hand for every table gets old fast when you have 50 tables and 200 columns. Data quality frameworks let you define checks as config, run them automatically, and generate reports.

### Great Expectations Setup

```
pip install great_expectations
great_expectations init
```

This creates a project structure:

```

great_expectations/
├── great_expectations.yml # Main config
├── expectations/ # Your expectation suites (the checks)
├── checkpoints/ # How and when to run checks
├── plugins/ # Custom expectations
├── uncommitted/
│ ├── config_variables.yml # Secrets (don't commit this)
│ └── data_docs/ # Generated HTML reports

```

## Connecting to Your Data

```

import great_expectations as gx

context = gx.get_context()

Add a Postgres datasource
datasource = context.sources.add_postgres(
 name="postgres_db",
 connection_string="postgresql+psycopg2://postgres:postgres@localhost:5432/ecommerce",
)

Add a data asset (table)
data_asset = datasource.add_table_asset(
 name="raw_orders",
 table_name="raw_orders",
)

Create a batch request (what data to validate)
batch_request = data_asset.build_batch_request()

```

## Writing Expectations

Expectations are the checks themselves — assertions for your data that read like English:

```

Create an expectation suite
suite = context.add_expectation_suite("raw_orders_suite")

Get a validator – connects the suite to actual data
validator = context.get_validator(
 batch_request=batch_request,
 expectation_suite_name="raw_orders_suite",
)

Schema checks
validator.expect_table_columns_to_match_ordered_list(
 column_list=["order_id", "customer_id", "order_date", "total_amount", "status"]
)

Null checks
validator.expect_column_values_to_not_be_null(column="order_id")
validator.expect_column_values_to_not_be_null(column="customer_id")
validator.expect_column_values_to_not_be_null(column="order_date")

Allow some nulls in total_amount (e.g., pending orders)
validator.expect_column_values_to_not_be_null(
 column="total_amount",
 mostly=0.95, # At least 95% non-null
)

Value ranges
validator.expect_column_values_to_be_between(
 column="total_amount",
 min_value=0,
 max_value=100000,
)

Valid categories
validator.expect_column_values_to_be_in_set(
 column="status",
 value_set=["pending", "processing", "shipped", "delivered", "cancelled"],
)

Uniqueness
validator.expect_column_values_to_be_unique(column="order_id")

Row count within expected range
validator.expect_table_row_count_to_be_between(
 min_value=1000,
 max_value=500000,
)

No future dates
validator.expect_column_values_to_be_between(
 column="order_date",
 max_value="2026-02-20", # Today + 1 day buffer
)

```

```
Save the suite
validator.save_expectation_suite(discard_failed_expectations=False)
```

## Running Validations via Checkpoints

Expectations don't do anything until you run them. Checkpoints handle execution:

```
checkpoint = context.add_or_update_checkpoint(
 name="raw_orders_checkpoint",
 validations=[
 {
 "batch_request": batch_request,
 "expectation_suite_name": "raw_orders_suite",
 },
],
)

Run it
result = checkpoint.run()

Check if everything passed
if not result.success:
 for validation_result in result.run_results.values():
 for expectation_result in validation_result["validation_result"]["results"]:
 if not expectation_result["success"]:
 print(f"FAILED: {expectation_result['expectation_config']
['expectation_type']}")
 print(f" Details: {expectation_result['result']}")
 raise Exception("Data quality checks failed!")
else:
 print("All data quality checks passed!")
```

Expected output (all passing):

```
All data quality checks passed!
```

## Data Docs — Auto-Generated Reports

One of GX's killer features:

```
context.build_data_docs()
context.open_data_docs()
```

This opens a browser with a report showing every expectation, whether it passed or failed, with details. When an analyst asks "how do you know the data is good?" you send them the Data Docs link.

#### ❏Pro Tip

Data Docs are a great way to build trust with stakeholders. Set up automated publishing so the latest quality report is always available at a known URL.

## Soda — The Simpler Alternative

Soda is YAML-based and simpler for basic checks:

```
pip install soda-core-postgres
```

Create `checks/raw_orders.yml`:

```
checks for raw_orders:
 - row_count > 0
 - missing_count(order_id) = 0
 - missing_count(customer_id) = 0
 - missing_percent(total_amount) < 5%
 - duplicate_count(order_id) = 0
 - min(total_amount) >= 0
 - max(total_amount) <= 100000
 - invalid_count(status) = 0:
 valid values: ['pending', 'processing', 'shipped', 'delivered', 'cancelled']
 - freshness(order_date) < 2d
 - anomaly detection for row_count
```

Soda configuration ( `configuration.yml` ):

```
data_source ecommerce:
 type: postgres
 host: localhost
 port: 5432
 username: postgres
 password: postgres
 database: ecommerce
```

Run it:

```
soda scan -d ecommerce -c configuration.yml checks/raw_orders.yml
```

Soda is great for teams that want something quick and YAML-driven. Great Expectations is more powerful and flexible, especially for complex checks and Airflow integration. Use whichever fits your team.

#### ⚠ Common Mistake

- Writing 200 expectations for every table. Start with the critical checks and add more as you discover issues.
- Not versioning your expectation suites. They're config — they belong in Git.
- Running GX checks but not failing the pipeline when they fail. If you don't stop bad data, the checks are pointless.

#### □ Checkpoint

1. What's the difference between an expectation suite and a checkpoint in Great Expectations?
2. When would you choose Soda over Great Expectations?
3. What does the `mostly=0.95` parameter do in a null check expectation?

## 7.4 dbt Tests and Custom Test Macros

**TL;DR:** Great Expectations checks your raw data. dbt tests validate your *transformation logic* — ensuring your SQL produces correct results and your business rules hold. dbt has generic tests (unique, not\_null, relationships) and custom tests (any SQL that returns failing rows).

Great Expectations checks your raw data. But what about the *transformations*? What if your SQL that calculates revenue is wrong? What if your join produces duplicates? That's where dbt tests come in.

### Generic Tests

In your `schema.yml`:



```

version: 2

models:
 - name: fact_orders
 description: "Order fact table – one row per order"
 columns:
 - name: order_id
 description: "Primary key"
 tests:
 - unique
 - not_null

 - name: customer_id
 description: "FK to dim_customers"
 tests:
 - not_null
 - relationships:
 to: ref('dim_customers')
 field: customer_id

 - name: order_status
 tests:
 - accepted_values:
 values: ['pending', 'processing', 'shipped', 'delivered', 'cancelled']

 - name: total_amount
 tests:
 - not_null
 - dbt_utils.expression_is_true:
 expression: ">= 0"

```

```
dbt test
```

### Expected output:

```

Completed successfully

Pass: 7 Warn: 0 Error: 0 Skip: 0 Total: 7

```

## Custom Tests

Real business logic needs custom tests. Create files in `tests/` — a failing test returns rows, and zero rows means pass:

```
tests/assert_order_total_matches_line_items.sql :
```

```
-- Fails if ANY order's total doesn't match its line items sum
SELECT
 o.order_id,
 o.total_amount AS order_total,
 SUM(li.quantity * li.unit_price) AS calculated_total,
 ABS(o.total_amount - SUM(li.quantity * li.unit_price)) AS difference
FROM {{ ref('fact_orders') }} o
JOIN {{ ref('fact_line_items') }} li ON o.order_id = li.order_id
GROUP BY o.order_id, o.total_amount
HAVING ABS(o.total_amount - SUM(li.quantity * li.unit_price)) > 0.01
```

`tests/assert_no_negative_revenue.sql`:

```
-- No revenue should be negative after discounts
SELECT order_id, total_amount
FROM {{ ref('fact_orders') }}
WHERE total_amount < 0
```

## Custom Generic Tests (Macros)

If you write the same pattern for multiple models, make it a reusable macro:

`macros/test_row_count_within_range.sql`:

```
{% test row_count_within_range(model, min_count, max_count) %}

SELECT COUNT(*) AS row_count
FROM {{ model }}
HAVING COUNT(*) < {{ min_count }} OR COUNT(*) > {{ max_count }}

{% endtest %}
```

Use it in `schema.yml`:

```
models:
 - name: fact_orders
 tests:
 - row_count_within_range:
 min_count: 1000
 max_count: 5000000
```

## Freshness Tests

dbt has source freshness built in:

```
sources:
 - name: raw
 database: ecommerce
 schema: raw
 tables:
 - name: orders
 loaded_at_field: loaded_at
 freshness:
 warn_after: {count: 12, period: hour}
 error_after: {count: 24, period: hour}
```

```
dbt source freshness
```

#### ⚠ Common Mistake

- Only using generic tests. `unique` and `not_null` are just the baseline. Custom tests for business logic are where the real value is.
- Not running `dbt test` in CI/CD. Tests should run on every PR and every production deploy.

#### 📌 Pro Tip

Create a `tests/` directory structure that mirrors your models. When a model changes, the associated tests are easy to find and update.

### 📌 Checkpoint

1. What's the difference between a dbt generic test and a custom test?
2. In a custom dbt test SQL file, what does it mean when the query returns rows?
3. How would you test that a fact table's foreign key always has a matching dimension record?

## 7.5 Data Contracts — Defining and Enforcing Them

**TL;DR:** A data contract is a formal agreement between data producers and consumers. It specifies schema, semantics, SLAs, and quality guarantees. Without one, upstream changes silently break your pipeline. Start simple with YAML files in a shared Git repo.

Here's a scenario you'll hit in every data team: the backend team changes a column name. They don't tell you. Your pipeline breaks at 2 AM. You find out at 9 AM when dashboards go red.

Whose fault is it? Neither side's. The fault is that there's no *contract* between the producer and the consumer.

## What a Data Contract Specifies

A data contract covers:

- **Schema** — column names, types, nullable or not
- **Semantics** — what each field means (is `amount` in cents or dollars?)
- **SLAs** — when data will be available, how fresh it'll be
- **Quality guarantees** — max null rate, valid value ranges
- **Change management** — how changes get communicated and versioned

Think of it like a REST API contract. If an API changes its response format without warning, that's a bug. Same principle applies to data.

### □ Key Concept: Data Contract

A formal, versioned agreement between a data producer and consumer that defines the expected schema, semantics, quality, and delivery SLAs. A contract without enforcement is just documentation.

## Implementing a Basic Contract

`contracts/raw_orders_contract.yml` :

```

contract:
 name: raw_orders
 version: "2.1.0"
 owner: backend-team
 consumer: data-engineering

schema:
 fields:
 - name: order_id
 type: integer
 nullable: false
 unique: true
 description: "Unique order identifier from the orders service"

 - name: customer_id
 type: integer
 nullable: false
 description: "FK to customers service"

 - name: order_date
 type: timestamp
 nullable: false
 description: "UTC timestamp when order was placed"

 - name: total_amount
 type: decimal
 nullable: true
 description: "Order total in USD (cents). Null for pending orders."
 constraints:
 min: 0
 max: 10000000 # $100K in cents

 - name: status
 type: string
 nullable: false
 description: "Order lifecycle status"
 constraints:
 allowed_values: ["pending", "processing", "shipped", "delivered", "cancelled"]

sla:
 freshness: "2 hours"
 availability: "99.5%"

quality:
 min_row_count: 100
 max_null_rate:
 total_amount: 0.05

```



```

import yaml
import pandas as pd
from dataclasses import dataclass
from typing import Any
import json

@dataclass
class ContractViolation:
 field: str
 check: str
 details: str

def load_contract(path: str) -> dict:
 with open(path) as f:
 return yaml.safe_load(f)["contract"]

def validate_contract(df: pd.DataFrame, contract_path: str) -> list[ContractViolation]:
 """Validate a DataFrame against a data contract. Returns list of violations."""
 contract = load_contract(contract_path)
 violations: list[ContractViolation] = []

 expected_fields = {f["name"] for f in contract["schema"]["fields"]}
 actual_fields = set(df.columns)

 # Check for missing columns
 for missing in expected_fields - actual_fields:
 violations.append(ContractViolation(
 field=missing, check="schema", details=f"Missing column: {missing}"
))

 # Check each field's constraints
 for field_spec in contract["schema"]["fields"]:
 name = field_spec["name"]
 if name not in df.columns:
 continue

 # Nullable check
 if not field_spec.get("nullable", True):
 null_count = df[name].isnull().sum()
 if null_count > 0:
 violations.append(ContractViolation(
 field=name, check="nullable",
 details=f"{null_count} null values in non-nullable column"
))

 # Unique check
 if field_spec.get("unique", False):
 dupe_count = df[name].duplicated().sum()
 if dupe_count > 0:
 violations.append(ContractViolation(

```

```

 field=name, check="unique",
 details=f"{dupe_count} duplicate values"
))

Min/max constraints
constraints = field_spec.get("constraints", {})

if "min" in constraints:
 below_min = (df[name].dropna() < constraints["min"]).sum()
 if below_min > 0:
 violations.append(ContractViolation(
 field=name, check="min",
 details=f"{below_min} values below minimum {constraints['min']}"
))

if "max" in constraints:
 above_max = (df[name].dropna() > constraints["max"]).sum()
 if above_max > 0:
 violations.append(ContractViolation(
 field=name, check="max",
 details=f"{above_max} values above maximum {constraints['max']}"
))

if "allowed_values" in constraints:
 invalid = ~df[name].dropna().isin(constraints["allowed_values"])
 invalid_count = invalid.sum()
 if invalid_count > 0:
 violations.append(ContractViolation(
 field=name, check="allowed_values",
 details=f"{invalid_count} values not in allowed set"
))

Row count check
quality = contract.get("quality", {})
if "min_row_count" in quality and len(df) < quality["min_row_count"]:
 violations.append(ContractViolation(
 field="_table_", check="row_count",
 details=f"Row count {len(df)} below minimum {quality['min_row_count']}"
))

return violations

Usage
violations = validate_contract(df, "contracts/raw_orders_contract.yml")
if violations:
 for v in violations:
 print(f"▯ [{v.check}] {v.field}: {v.details}")
 raise Exception(f"Contract violated: {len(violations)} issues found")
else:
 print("▯ Contract validated successfully")

```

Expected output (violation detected):



```
❏ [nullable] customer_id: 3 null values in non-nullable column
❏ [allowed_values] status: 7 values not in allowed set
Contract violated: 2 issues found
```

#### □ Pro Tip

At smaller companies, a YAML contract file in a shared Git repo is often enough. The point isn't fancy tooling — it's having an explicit agreement that both sides can reference. Big companies like Google, Airbnb, and Netflix all use data contracts internally.

#### △ Common Mistake

- Making contracts too strict too fast. Start with critical fields and expand.
- Not versioning contracts. When the schema changes, bump the version and communicate.
- Having contracts that nobody checks. A contract without enforcement is just documentation.

#### □ Checkpoint

1. What five things does a data contract specify?
2. Why should you version your data contracts?
3. What's the difference between a data contract and a Great Expectations suite?

## 7.6 Monitoring, Alerting, and Incident Response

**TL;DR:** Monitor at three levels: pipeline health, data quality, and business metrics. Alert only on actionable failures (avoid alert fatigue). When bad data gets through, follow a structured incident response: assess scope → stop the bleeding → communicate → root cause → fix & backfill → post-mortem.

You've built quality checks. They run every day. Everything's green. Until it's not. And when it's not, you need to know *immediately* — not when an analyst Slacks you 3 hours later.

### Three Monitoring Layers

1. **Pipeline health** — Did the DAG run? Did it succeed? How long did it take?
2. **Data quality** — Did the quality checks pass? What's the trend over time?
3. **Business metrics** — Are the downstream metrics in expected ranges?

## Pipeline Health Monitoring in Airflow

```
from airflow import DAG
from airflow.providers.slack.notifications.slack import send_slack_notification
from datetime import datetime, timedelta

default_args = {
 "owner": "data-engineering",
 "retries": 2,
 "retry_delay": timedelta(minutes=5),
 "execution_timeout": timedelta(hours=1),
 "sla": timedelta(hours=2), # Must complete within 2 hours
 "on_failure_callback": send_slack_notification(
 slack_conn_id="slack_webhook",
 text="❏ DAG `{dag.dag_id}` failed on task `{ti.task_id}`\n"
 "Execution date: {{ ds }}\n"
 "Log: {{ ti.log_url }}",
 channel="#data-alerts",
),
 "on_sla_miss_callback": send_slack_notification(
 slack_conn_id="slack_webhook",
 text="⚠ SLA MISS: DAG `{dag.dag_id}` didn't complete in time\n"
 "Expected by: {{ sla }}\n"
 "This affects downstream dashboards.",
 channel="#data-alerts",
),
}

dag = DAG(
 "ecommerce_pipeline",
 default_args=default_args,
 schedule_interval="@daily",
 start_date=datetime(2026, 1, 1),
 catchup=False,
)
```

**Custom Quality Alerting to Slack**

```

import requests

def send_quality_alert(
 webhook_url: str,
 pipeline_name: str,
 check_results: list[dict],
) -> None:
 """Send data quality alert to Slack."""
 failures = [r for r in check_results if not r["success"]]

 if not failures:
 return # No news is good news

 blocks = [
 {
 "type": "header",
 "text": {
 "type": "plain_text",
 "text": f" Data Quality Alert: {pipeline_name}",
 },
 },
 {
 "type": "section",
 "text": {
 "type": "mrkdown",
 "text": f"*{len(failures)} checks failed* out of {len(check_results)}
total",
 },
 },
]

 for failure in failures[:5]: # Cap at 5 to avoid spam
 blocks.append({
 "type": "section",
 "text": {
 "type": "mrkdown",
 "text": (
 f" *{failure['check_name']}*\n"
 f"Column: `{failure.get('column', 'N/A')}`\n"
 f"Details: {failure['details']}"
),
 },
 })

 if len(failures) > 5:
 blocks.append({
 "type": "section",
 "text": {
 "type": "mrkdown",
 "text": f"...and {len(failures) - 5} more failures. Check Data Docs for
full report.",
 },
 })

```

```
})
```

```
requests.post(webhook_url, json={"blocks": blocks})
```

## Incident Response Playbook

When bad data gets through — and it will eventually — follow this playbook:

### Step 1: Assess Scope (5 minutes)

- What data is affected? Which tables, which date ranges?
- Who's impacted? Which dashboards, reports, or downstream systems?
- Is the bad data still flowing, or was it a one-time issue?

### Step 2: Stop the Bleeding (10 minutes)

- Pause the pipeline (Airflow: pause the DAG)
- If bad data is already in the warehouse, mark it — don't delete it yet

```
-- Quarantine bad data (don't delete — you'll need it for investigation)
ALTER TABLE fact_orders ADD COLUMN IF NOT EXISTS _quarantined BOOLEAN DEFAULT FALSE;

UPDATE fact_orders
SET _quarantined = TRUE
WHERE loaded_at BETWEEN '2026-02-18 00:00:00' AND '2026-02-18 23:59:59'
AND <condition_that_identifies_bad_data>;
```

### Step 3: Communicate (immediately)

- Post in #data-alerts with what you know so far
- Be honest: *"We identified a data quality issue. Dashboard X may show incorrect numbers for [date range]. We're investigating."*

### Step 4: Root Cause (30 min – 2 hours)

- Check upstream sources: Did the source data change?
- Check pipeline logs: Any errors or warnings you missed?
- Check transformations: Did a code change introduce a bug?

### Step 5: Fix and Backfill (varies)

- Fix the root cause
- Backfill the affected data
- Re-run quality checks to confirm

### Step 6: Post-Mortem (next day)

- What happened?
- Why didn't our checks catch it?
- What check do we add to prevent this in the future?

#### ⚠ Common Mistake

- **Alert fatigue.** If you alert on everything, people ignore alerts. Only alert on *actionable* failures.
- **No runbook.** When it's 2 AM and you're half asleep, you need a checklist, not a thought process.
- **Deleting bad data** instead of quarantining it. You might need it for the investigation.

#### □ Checkpoint

1. What are the three levels of monitoring for a data pipeline?
2. Why should you quarantine bad data instead of deleting it?
3. What's the purpose of a post-mortem after a data incident?

## 7.7 Module 7 Project — Data Quality Pipeline

**TL;DR:** Add a comprehensive data quality layer to your Airflow pipeline: contract validation before loading, Great Expectations checks after transformation, Slack alerts on failure, and quarantine for bad data.

### Overview

You're going to add comprehensive data quality checks to your Airflow pipeline from Module 4. When complete, your pipeline will:

1. Validate incoming data against a contract before loading
2. Run Great Expectations checks after each transformation step
3. Alert to Slack (or console) when checks fail
4. Quarantine bad data instead of loading it to production tables

## Project Structure

```
module-7-project/
├── dags/
│ └── ecommerce_pipeline_with_quality.py
├── contracts/
│ └── raw_orders_contract.yml
├── great_expectations/
│ ├── great_expectations.yml
│ └── expectations/
│ ├── raw_orders_suite.json
│ └── fact_orders_suite.json
├── quality/
│ ├── __init__.py
│ ├── contract_validator.py
│ └── alerts.py
├── tests/
│ ├── test_contract_validator.py
│ └── test_quality_checks.py
├── docker-compose.yml
└── README.md
```

### Step 1: Set Up the Contract

Use the contract YAML from Section 7.5. Customize it for your specific e-commerce schema.

### Step 2: Build the Contract Validator

Take the `validate_contract()` function from Section 7.5 and enhance it with severity levels and structured results:

```

quality/contract_validator.py
import yaml
import pandas as pd
from dataclasses import dataclass, asdict
from datetime import datetime
import json

@dataclass
class ContractViolation:
 field: str
 check: str
 details: str
 severity: str = "error" # "error" or "warning"

@dataclass
class ValidationResult:
 contract_name: str
 contract_version: str
 validated_at: str
 row_count: int
 violations: list[ContractViolation]

 @property
 def passed(self) -> bool:
 return not any(v.severity == "error" for v in self.violations)

 def to_json(self) -> str:
 return json.dumps(asdict(self), indent=2, default=str)

def validate_contract(df: pd.DataFrame, contract_path: str) -> ValidationResult:
 """Full contract validation. Returns a ValidationResult."""
 # YOUR IMPLEMENTATION HERE
 # Use the code from Section 7.5 as your starting point
 # Return a ValidationResult instead of a list
 pass

```

### Step 3: Set Up Great Expectations

Create expectation suites for:

- `raw_orders_suite` — checks on raw ingested data
- `fact_orders_suite` — checks on transformed fact table

#### Minimum expectations per suite:

- Schema validation (columns exist, correct types)
- Null checks on primary keys and required fields
- Value range checks on numeric columns



- Allowed value checks on categorical columns
- Uniqueness on primary keys
- Row count within expected range

**Step 4: Build the Airflow DAG**

```

dags/ecommerce_pipeline_with_quality.py
from airflow.decorators import dag, task
from datetime import datetime

@dag(
 schedule="@daily",
 start_date=datetime(2026, 1, 1),
 catchup=False,
 tags=["ecommerce", "data-quality"],
)
def ecommerce_pipeline_with_quality():

 @task()
 def extract_orders(**context):
 """Extract orders from source."""
 pass

 @task()
 def validate_contract(**context):
 """Validate raw data against contract BEFORE loading.
 If contract fails, raise exception (pipeline stops)."""
 pass

 @task()
 def load_raw(**context):
 """Load validated data to raw layer."""
 pass

 @task()
 def run_raw_quality_checks(**context):
 """Run Great Expectations on raw data.
 If critical checks fail, raise exception."""
 pass

 @task()
 def transform(**context):
 """Transform raw to fact/dim tables."""
 pass

 @task()
 def run_transformed_quality_checks(**context):
 """Run Great Expectations on transformed data."""
 pass

 @task(trigger_rule="one_failed")
 def quarantine_bad_data(**context):
 """Move bad data to quarantine table instead of deleting."""
 pass

 @task(trigger_rule="all_done")
 def send_quality_report(**context):
 """Send summary of quality results (pass or fail)."""
 pass

```

```

Define the flow
raw_data = extract_orders()
contract_ok = validate_contract()
loaded = load_raw()
raw_quality = run_raw_quality_checks()
transformed = transform()
final_quality = run_transformed_quality_checks()
quarantine = quarantine_bad_data()
report = send_quality_report()

raw_data >> contract_ok >> loaded >> raw_quality >> transformed >> final_quality
[raw_quality, final_quality] >> quarantine
[final_quality, quarantine] >> report

ecommerce_pipeline_with_quality()

```

[DIAGRAM: Airflow DAG showing linear flow from extract → validate\_contract → load\_raw → raw\_quality\_checks → transform → final\_quality\_checks, with a quarantine branch triggered on any quality failure, and a report task at the end triggered after all tasks complete.]

## Step 5: Write Tests

```
tests/test_contract_validator.py
import pytest
import pandas as pd
from quality.contract_validator import validate_contract, ValidationResult

def test_valid_data_passes_contract():
 df = pd.DataFrame({
 "order_id": [1, 2, 3],
 "customer_id": [100, 101, 102],
 "order_date": pd.to_datetime(["2026-01-01", "2026-01-02", "2026-01-03"]),
 "total_amount": [29.99, 49.99, 19.99],
 "status": ["pending", "shipped", "delivered"],
 })
 result = validate_contract(df, "contracts/raw_orders_contract.yml")
 assert result.passed

def test_null_primary_key_fails():
 df = pd.DataFrame({
 "order_id": [1, None, 3],
 "customer_id": [100, 101, 102],
 "order_date": pd.to_datetime(["2026-01-01", "2026-01-02", "2026-01-03"]),
 "total_amount": [29.99, 49.99, 19.99],
 "status": ["pending", "shipped", "delivered"],
 })
 result = validate_contract(df, "contracts/raw_orders_contract.yml")
 assert not result.passed

def test_invalid_status_fails():
 df = pd.DataFrame({
 "order_id": [1, 2, 3],
 "customer_id": [100, 101, 102],
 "order_date": pd.to_datetime(["2026-01-01", "2026-01-02", "2026-01-03"]),
 "total_amount": [29.99, 49.99, 19.99],
 "status": ["pending", "shipped", "YOLO"], # Invalid status
 })
 result = validate_contract(df, "contracts/raw_orders_contract.yml")
 assert not result.passed
 assert any(v.check == "allowed_values" for v in result.violations)
```

## Try It Yourself

- Add a test for negative `total_amount` values.
- Add a test for duplicate `order_id` values.
- Add a test that verifies extra columns generate a warning but not an error.

## Expected Output

When you run the DAG successfully:

- All quality checks pass → green in Airflow, summary report sent
- Bad data detected → pipeline stops at the quality check step, bad data quarantined, alert sent
- Quality report shows: total checks run, pass/fail per check, trend vs previous run

## Completion Checklist

- ☐ Contract YAML file covers all critical fields
- ☐ Contract validator catches: missing columns, nulls, invalid values, range violations
- ☐ Great Expectations suites for both raw and transformed data
- ☐ Airflow DAG has quality checks at multiple stages
- ☐ Pipeline stops on critical failures (doesn't load bad data)
- ☐ Quarantine mechanism for bad data
- ☐ Alerting function sends notifications on failure
- ☐ At least 5 unit tests for the contract validator
- ☐ README documents the quality checks and how to add new ones

---

*End of Module 7*

# Module 8: Cloud Infrastructure for Data Engineers

*You don't need to be a DevOps engineer, but you need to deploy your own stuff. AWS-focused with transferable concepts.*

## 8.1 Cloud Fundamentals for DEs — What You Need to Know

**TL;DR:** Out of 200+ AWS services, you need about 10 for data engineering. The core pattern is: S3 for storage, RDS/Redshift for databases, Athena for ad-hoc queries, ECS for compute, and IAM for security. Learn one cloud well and the others are just different UIs.

Here's a conversation that happens with at least 20 junior data engineers: *"I know Python. I know SQL. I can build pipelines locally. But when it comes to deploying anything to the cloud, I freeze. There are 200+ AWS services and I don't know where to start."*

Good news: you need maybe 10 of them.

### The Three Cloud Superpowers

1. **Elasticity** — scale up when you need it, scale down when you don't
2. **Managed services** — someone else handles the patching, backups, and uptime
3. **Pay-per-use** — you pay for what you consume, not what you provision (mostly)

### The AWS Data Engineering Stack

For data engineers, the cloud maps to your pipeline stages:

Source → Ingest → Store → Process → Serve → Monitor

Pipeline Stage	AWS Service	What It Does
Store (raw)	S3	Object storage — your data lake
Store (structured)	RDS (Postgres)	Managed relational database
Store (warehouse)	Redshift	Columnar analytics database
Process (batch)	EMR / Glue	Managed Spark
Process (serverless)	Lambda	Run code without servers
Query (ad-hoc)	Athena	SQL on S3 files directly
Orchestrate	MWAA / Step Functions	Managed Airflow / workflow
Stream	Kinesis / MSK	Managed Kafka
Compute	EC2 / ECS / Fargate	Run containers
IAM	IAM	Access control
Secrets	Secrets Manager	Store credentials

That's it. That's the 80/20 of AWS for data engineers.

## The Data Lake Architecture on AWS

[DIAGRAM: AWS account containing three main components: (1) ECS running Airflow that ingests from APIs/Kafka/CSV, (2) S3 bucket organized as a data lake with /raw/, /staging/, and /prod/ layers, and (3) RDS Postgres receiving transformed data from S3. Arrows show data flowing from external sources → ECS → S3 → RDS.]

Three layers in S3:

- **/raw/** — data as it arrived, never modified (your safety net)
- **/staging/** — cleaned, validated, intermediate transformations
- **/prod/** — final, analytics-ready tables (Parquet/Delta format)

### □Key Concept: Medallion Architecture

This layered approach is called the **medallion architecture** (bronze/silver/gold) or raw/staging/prod. Same concept, different names. The key principle: raw data is immutable. You never modify it. All transformations produce new data in downstream layers.

## GCP/Azure Equivalents

Quick reference if you end up at a non-AWS shop:



AWS	GCP	Azure
S3	Cloud Storage	Blob Storage
RDS	Cloud SQL	Azure SQL
Redshift	BigQuery	Synapse
EMR	Dataproc	HDInsight
Athena	BigQuery	Synapse Serverless
Lambda	Cloud Functions	Azure Functions
ECS/Fargate	Cloud Run	Container Apps
MWAA	Cloud Composer	Data Factory

The concepts transfer completely. Learn one cloud well, and the others are just different UIs.

#### ⚠ Common Mistake

- Trying to learn all AWS services. Focus on the data stack above.
- Using the AWS Console for everything. That's fine for learning, but production uses Infrastructure as Code (Terraform).
- Running expensive services 24/7 during learning. Set up billing alerts first (covered in Section 8.6).

### ☐ Checkpoint

1. What are the three layers of a data lake on S3?
2. What AWS service would you use to run SQL queries directly on files stored in S3?
3. What's the GCP equivalent of S3?

## 8.2 The AWS Data Stack — S3, RDS, Glue, Athena

**TL;DR:** S3 is your data lake foundation — cheap, durable, infinitely scalable. Store data as Parquet (not CSV) for 10-100x better query performance. Use Athena to query S3 with standard SQL. Always partition your data to reduce costs. For most startups, S3 + Athena + RDS Postgres covers 90% of use cases.

### S3 — Your Data Lake

S3 is the foundation of everything. It's cheap (\$0.023/GB/month for standard storage), durable (99.999999999% — eleven 9s), and infinitely scalable.

Set up a data lake bucket:

```
Create the bucket
aws s3 mb s3://my-ecommerce-data-lake-2026 --region us-east-1

Create the layer structure
aws s3api put-object --bucket my-ecommerce-data-lake-2026 --key raw/
aws s3api put-object --bucket my-ecommerce-data-lake-2026 --key staging/
aws s3api put-object --bucket my-ecommerce-data-lake-2026 --key prod/
```

Upload data with Python and boto3:

```

import boto3
import pandas as pd
from datetime import datetime

s3 = boto3.client("s3")
BUCKET = "my-ecommerce-data-lake-2026"

def upload_to_data_lake(
 df: pd.DataFrame,
 layer: str, # raw, staging, prod
 dataset: str, # orders, customers, etc.
 partition_date: str | None = None,
) -> str:
 """Upload a DataFrame to S3 as Parquet with date partitioning."""
 if partition_date is None:
 partition_date = datetime.now().strftime("%Y-%m-%d")

 # Hive-style partitioning: /layer/dataset/date=YYYY-MM-DD/data.parquet
 key = f"{layer}/{dataset}/date={partition_date}/data.parquet"

 # Write to Parquet in memory, then upload
 buffer = df.to_parquet(index=False)
 s3.put_object(Bucket=BUCKET, Key=key, Body=buffer)

 print(f"Uploaded {len(df)} rows to s3://{BUCKET}/{key}")
 return f"s3://{BUCKET}/{key}"

Example: upload raw orders
orders_df = pd.DataFrame({
 "order_id": range(1, 1001),
 "customer_id": [i % 200 + 1 for i in range(1000)],
 "order_date": pd.date_range("2026-02-01", periods=1000, freq="h"),
 "total_amount": [round(i * 1.5 + 10, 2) for i in range(1000)],
 "status": ["shipped"] * 400 + ["delivered"] * 300 + ["pending"] * 200 +
["cancelled"] * 100,
})

upload_to_data_lake(orders_df, "raw", "orders", "2026-02-18")

```

### Expected output:

```

Uploaded 1000 rows to s3://my-ecommerce-data-lake-2026/raw/orders/date=2026-02-18/
data.parquet

```

#### ❏ Pro Tip: Why Parquet?

Parquet is columnar, compressed, and typed. For analytics workloads, it's 10-100x faster to query than CSV because: (1) query engines only read the columns they need, (2) compression reduces I/O, (3) built-in statistics allow predicate pushdown (skipping rows that don't match your WHERE clause). Never store analytics data as CSV in S3.

## Athena — SQL on S3

Athena lets you query S3 files using standard SQL. No server to manage, no data to load into a database. You pay \$5 per TB scanned.

Create a table definition:

```
-- Partitioned table — critical for cost and performance
CREATE EXTERNAL TABLE IF NOT EXISTS raw_orders_partitioned (
 order_id INT,
 customer_id INT,
 order_date TIMESTAMP,
 total_amount DOUBLE,
 status STRING
)
PARTITIONED BY (date STRING)
STORED AS PARQUET
LOCATION 's3://my-ecommerce-data-lake-2026/raw/orders/'
TBLPROPERTIES ('parquet.compression'='SNAPPY');

-- Tell Athena to discover partitions
MSCK REPAIR TABLE raw_orders_partitioned;
```

Now query it — just SQL:

```
SELECT
 status,
 COUNT(*) AS order_count,
 ROUND(AVG(total_amount), 2) AS avg_amount
FROM raw_orders_partitioned
WHERE date = '2026-02-18' -- Partition filter: only scans this one day
GROUP BY status
ORDER BY order_count DESC;
```

Expected output:

status	order_count	avg_amount
shipped	400	310.25
delivered	300	760.50
pending	200	1210.75
cancelled	100	1511.00

#### ⚠ Common Mistake

The partition filter ( `WHERE date = '2026-02-18'` ) is critical — it means Athena only scans that one partition instead of your entire dataset. Without partitions, every query scans everything. That's expensive and slow.

## AWS Glue — The Catalog

Glue does two things:

1. **Glue Data Catalog** — a metadata store (think: Hive Metastore). Athena uses this to know about your tables.
2. **Glue ETL** — managed Spark jobs for transformations. (Honestly, most teams use Airflow + Spark directly because Glue ETL has weird limitations and debugging is painful.)

Set up a Glue Crawler to auto-discover your S3 data:

```
aws glue create-crawler \
 --name ecommerce-raw-crawler \
 --role AWSGlueServiceRole \
 --database-name ecommerce_db \
 --targets '{"S3Targets": [{"Path": "s3://my-ecommerce-data-lake-2026/raw/"}]}'

aws glue start-crawler --name ecommerce-raw-crawler
```

The crawler scans your S3 data, infers the schema, and creates table definitions in the Glue Catalog. Athena can then query these tables without you manually writing CREATE TABLE statements.

## When to Use What

Scenario	Best Choice
Small data (< 10GB), simple queries	Athena on S3 (cheapest, simplest)
Need ACID transactions, complex joins	RDS Postgres (or Aurora)
Large-scale analytics, many users	Redshift (columnar warehouse)
Spark transformations	EMR (or Glue ETL if you must)

For the course project and most startups, **S3 + Athena + RDS Postgres** covers 90% of use cases.

### □ Checkpoint

1. Why is Parquet dramatically better than CSV for Athena queries?
2. What does `MSCK REPAIR TABLE` do?
3. At what scale would you consider Redshift over Postgres + Athena?

## 8.3 Infrastructure as Code with Terraform

**TL;DR:** Terraform lets you define your cloud infrastructure as version-controlled code files. You describe the desired state, Terraform figures out the steps. Three commands: `init`, `plan`, `apply`. Always read the plan before applying. Store state remotely in S3 for team collaboration.

Imagine you built your whole data platform in AWS. It's running great. Then your company wants a staging environment that mirrors production. You open the AWS Console and think, *"How did I set all this up again?"*

That's the problem Infrastructure as Code solves.

### How Terraform Works

1. **Write** — describe what you want in `.tf` files (HCL language)
2. **Plan** — Terraform compares your config to what exists and shows what it'll change
3. **Apply** — Terraform makes the changes

It's declarative: you describe the *desired state*, not the steps to get there.

## S3 Data Lake in Terraform

```
main.tf
terraform {
 required_providers {
 aws = {
 source = "hashicorp/aws"
 version = "~> 5.0"
 }
 }
 required_version = ">= 1.5.0"
}

provider "aws" {
 region = var.aws_region
}

variable "aws_region" {
 default = "us-east-1"
}

variable "environment" {
 default = "dev"
}

variable "project_name" {
 default = "ecommerce-data-platform"
}
```

```

s3.tf
resource "aws_s3_bucket" "data_lake" {
 bucket = "${var.project_name}-${var.environment}-data-lake"

 tags = {
 Environment = var.environment
 Project = var.project_name
 ManagedBy = "terraform"
 }
}

Enable versioning (safety net for accidental deletes)
resource "aws_s3_bucket_versioning" "data_lake" {
 bucket = aws_s3_bucket.data_lake.id
 versioning_configuration {
 status = "Enabled"
 }
}

Block ALL public access (data lakes should NEVER be public)
resource "aws_s3_bucket_public_access_block" "data_lake" {
 bucket = aws_s3_bucket.data_lake.id

 block_public_acls = true
 block_public_policy = true
 ignore_public_acls = true
 restrict_public_buckets = true
}

Lifecycle rules: move old raw data to cheaper storage tiers
resource "aws_s3_bucket_lifecycle_configuration" "data_lake" {
 bucket = aws_s3_bucket.data_lake.id

 rule {
 id = "archive-raw-data"
 status = "Enabled"

 filter {
 prefix = "raw/"
 }

 transition {
 days = 90
 storage_class = "STANDARD_IA" # Infrequent Access – 45% cheaper
 }

 transition {
 days = 365
 storage_class = "GLACIER" # Long-term archive – 80% cheaper
 }
 }
}

```



## RDS Postgres in Terraform

```
rds.tf
resource "aws_db_instance" "postgres" {
 identifier = "${var.project_name}-${var.environment}-db"
 engine = "postgres"
 engine_version = "16.1"
 instance_class = "db.t3.micro" # Cheapest option for dev

 allocated_storage = 20
 max_allocated_storage = 100 # Auto-scale up to 100GB
 storage_type = "gp3"

 db_name = "ecommerce"
 username = "dataengineer"
 password = var.db_password # Never hardcode passwords!

 publicly_accessible = false # Access only from within VPC
 vpc_security_group_ids = [aws_security_group.db.id]
 db_subnet_group_name = aws_db_subnet_group.db.name

 backup_retention_period = 7
 backup_window = "03:00-04:00"

 # Safety: don't delete production databases on terraform destroy
 deletion_protection = var.environment == "prod" ? true : false
 skip_final_snapshot = var.environment == "dev" ? true : false

 tags = {
 Environment = var.environment
 Project = var.project_name
 ManagedBy = "terraform"
 }
}

variable "db_password" {
 type = string
 sensitive = true # Won't show in terraform plan output
}
```

## IAM Role for Your Pipeline

```
iam.tf – Role that your Airflow tasks assume
resource "aws_iam_role" "pipeline_role" {
 name = "${var.project_name}-${var.environment}-pipeline-role"

 assume_role_policy = jsonencode({
 Version = "2012-10-17"
 Statement = [
 {
 Action = "sts:AssumeRole"
 Effect = "Allow"
 Principal = {
 Service = "ecs-tasks.amazonaws.com"
 }
 },
]
 })
}

Policy: pipeline can read/write to S3 data lake – nothing else
resource "aws_iam_role_policy" "pipeline_s3_access" {
 name = "s3-data-lake-access"
 role = aws_iam_role.pipeline_role.id

 policy = jsonencode({
 Version = "2012-10-17"
 Statement = [
 {
 Effect = "Allow"
 Action = [
 "s3:GetObject",
 "s3:PutObject",
 "s3:ListBucket",
 "s3:DeleteObject",
]
 Resource = [
 aws_s3_bucket.data_lake.arn,
 "${aws_s3_bucket.data_lake.arn}/*",
]
 },
]
 })
}
```

## Running Terraform

```
Initialize – downloads provider plugins
terraform init

Plan – shows what will be created/changed/destroyed
terraform plan -var="db_password=supersecret123"

Apply – actually creates the resources
terraform apply -var="db_password=supersecret123"

When you're done, destroy everything to stop charges
terraform destroy -var="db_password=supersecret123"
```

### Expected plan output:

```
Plan: 6 to add, 0 to change, 0 to destroy.
```

❏ **Pro Tip: Always read the plan before applying.** Especially look for resources being destroyed — that's usually not what you want.

## Remote State Management

In production, store Terraform state in S3 (not locally) so your team can share it:

```
backend.tf
terraform {
 backend "s3" {
 bucket = "my-terraform-state-bucket"
 key = "ecommerce-data-platform/terraform.tfstate"
 region = "us-east-1"
 dynamodb_table = "terraform-locks" # Prevents concurrent applies
 }
}
```

### ⚠ Common Mistake

- Hardcoding secrets in `.tf` files. Use variables with `sensitive = true` or AWS Secrets Manager.
- Not using remote state from day one. Local state files get lost and cause drift.
- Forgetting `terraform destroy` when done experimenting. That's how you get a surprise AWS bill.

### ❏ Checkpoint

1. What are the three Terraform commands and what does each do?

2. Why should you never hardcode passwords in Terraform files?
3. What's the purpose of the DynamoDB table in the remote state backend config?

## 8.4 Docker for Data Engineers — Containerizing Pipelines

**TL;DR:** Docker packages your code AND its environment into a container that runs identically everywhere.

Key practices: specific version tags (not `latest`), non-root user, multi-stage builds for smaller images, and layer ordering optimized for cache (dependencies first, code last).

"It works on my machine." Docker eliminates this by packaging your code and its entire environment into a container that runs the same everywhere.

### Production-Grade Dockerfile

```
Use a specific version – reproducibility matters
FROM python:3.11-slim AS base

Don't run as root in production
RUN groupadd -r pipeline && useradd -r -g pipeline pipeline

Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
 gcc \
 libpq-dev \
 && rm -rf /var/lib/apt/lists/*

WORKDIR /app

Copy dependency files first (Docker caches layers – this rarely changes)
COPY pyproject.toml uv.lock ./

Install Python dependencies
RUN pip install uv && uv sync --frozen --no-dev

Copy the actual code LAST (this layer changes most often)
COPY src/ ./src/
COPY configs/ ./configs/

Switch to non-root user
USER pipeline

ENTRYPOINT ["python", "-m", "src.main"]
```

#### □Key Concept: Layer Ordering

Docker caches each layer. If you put `COPY src/` before `pip install`, every code change invalidates the cache and reinstalls all dependencies. Put dependencies first, code last. This makes rebuilds go from minutes to seconds.

## Docker Compose for Multi-Container Pipelines

```
docker-compose.yml
version: "3.8"

services:
 pipeline:
 build: .
 environment:
 - DATABASE_URL=postgresql://postgres:postgres@db:5432/ecommerce
 - S3_BUCKET=my-ecommerce-data-lake-2026
 - AWS_REGION=us-east-1
 depends_on:
 db:
 condition: service_healthy
 volumes:
 - ./data:/app/data # Mount local data directory for dev

 db:
 image: postgres:16
 environment:
 POSTGRES_USER: postgres
 POSTGRES_PASSWORD: postgres
 POSTGRES_DB: ecommerce
 ports:
 - "5432:5432"
 volumes:
 - pgdata:/var/lib/postgresql/data
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U postgres"]
 interval: 5s
 timeout: 5s
 retries: 5

Local S3-compatible storage (for development)
minio:
 image: minio/minio
 command: server /data --console-address ":9001"
 environment:
 MINIO_ROOT_USER: minioadmin
 MINIO_ROOT_PASSWORD: minioadmin
 ports:
 - "9000:9000"
 - "9001:9001"
 volumes:
 - minio_data:/data

volumes:
 pgdata:
 minio_data:
```

## Multi-Stage Builds for Smaller Images

Production images should be small. Multi-stage builds separate the build environment from runtime:

```
Build stage — has all the build tools
FROM python:3.11-slim AS builder
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen --no-dev

Runtime stage — only what's needed to run
FROM python:3.11-slim AS runtime
RUN groupadd -r pipeline && useradd -r -g pipeline pipeline
RUN apt-get update && apt-get install -y --no-install-recommends \
 libpq5 \
 && rm -rf /var/lib/apt/lists/*
WORKDIR /app

Copy only installed packages from builder
COPY --from=builder /app/.venv /app/.venv
COPY src/ ./src/
COPY configs/ ./configs/

ENV PATH="/app/.venv/bin:$PATH"
USER pipeline
ENTRYPOINT ["python", "-m", "src.main"]
```

This produces images of 200-400MB instead of 1-2GB.

## Deploying to AWS ECS

```
Build the image
docker build -t ecommerce-pipeline .

Tag for ECR (Elastic Container Registry)
aws ecr get-login-password --region us-east-1 | \
 docker login --username AWS --password-stdin <account-id>.dkr.ecr.us-
east-1.amazonaws.com

docker tag ecommerce-pipeline:latest \
 <account-id>.dkr.ecr.us-east-1.amazonaws.com/ecommerce-pipeline:latest

Push to ECR
docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/ecommerce-pipeline:latest
```

#### ⚠ Common Mistake

- Using `latest` tag in production. Always tag with a specific version or Git SHA.
- Running containers as root. Always use a non-root user.
- Putting secrets in Dockerfiles or docker-compose. Use environment variables injected at runtime.

#### □ Checkpoint

1. Why should you copy dependency files before source code in a Dockerfile?
2. What's the benefit of a multi-stage Docker build?
3. Why should you never use the `latest` tag in production?

## 8.5 CI/CD for Data Pipelines — GitHub Actions

**TL;DR:** CI/CD automates testing and deployment triggered by Git pushes. CI runs tests on every push (catches bugs early). CD deploys to production when tests pass on main. Use GitHub Actions with a test Postgres service, and deploy via ECR + ECS. Never deploy from anywhere but your CI pipeline.

You've got Terraform for infrastructure, Docker for packaging. But deploying manually — SSH, pull, restart — is a liability in production.



# The GitHub Actions Workflow

```

.github/workflows/data-pipeline.yml
name: Data Pipeline CI/CD

on:
 push:
 branches: [main, develop]
 pull_request:
 branches: [main]

env:
 AWS_REGION: us-east-1
 ECR_REPOSITORY: ecommerce-pipeline
 ECS_SERVICE: ecommerce-pipeline-service
 ECS_CLUSTER: data-platform

jobs:
 test:
 name: Run Tests
 runs-on: ubuntu-latest

 services:
 postgres:
 image: postgres:16
 env:
 POSTGRES_USER: test
 POSTGRES_PASSWORD: test
 POSTGRES_DB: test_ecommerce
 ports:
 - 5432:5432
 options: >-
 --health-cmd "pg_isready -U test"
 --health-interval 10s
 --health-timeout 5s
 --health-retries 5

 steps:
 - uses: actions/checkout@v4

 - name: Set up Python
 uses: actions/setup-python@v5
 with:
 python-version: "3.11"

 - name: Install dependencies
 run: |
 pip install uv
 uv sync --frozen

 - name: Lint
 run: |
 uv run ruff check src/ tests/
 uv run ruff format --check src/ tests/

```

```

- name: Type check
 run: uv run mypy src/

- name: Run unit tests
 env:
 DATABASE_URL: postgresql://test:test@localhost:5432/test_ecommerce
 run: uv run pytest tests/ -v --tb=short

- name: Run dbt tests
 env:
 DATABASE_URL: postgresql://test:test@localhost:5432/test_ecommerce
 run: |
 cd dbt_project
 uv run dbt deps
 uv run dbt build --target test

build-and-deploy:
 name: Build & Deploy
 needs: test # Waits for test job to pass
 runs-on: ubuntu-latest
 if: github.ref == 'refs/heads/main' # Only deploy from main branch

 permissions:
 id-token: write # Required for AWS OIDC
 contents: read

 steps:
 - uses: actions/checkout@v4

 - name: Configure AWS credentials
 uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: ${ secrets.AWS_DEPLOY_ROLE_ARN }
 aws-region: ${ env.AWS_REGION }

 - name: Login to Amazon ECR
 id: login-ecr
 uses: aws-actions/amazon-ecr-login@v2

 - name: Build, tag, and push image to ECR
 id: build-image
 env:
 ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
 IMAGE_TAG: ${ github.sha }
 run: |
 docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
 docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:latest .
 docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
 docker push $ECR_REGISTRY/$ECR_REPOSITORY:latest
 echo "image=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG" >> $GITHUB_OUTPUT

 - name: Deploy to ECS
 run: |
 aws ecs update-service \

```

```

 --cluster $ECS_CLUSTER \
 --service $ECS_SERVICE \
 --force-new-deployment

- name: Wait for deployment
 run: |
 aws ecs wait services-stable \
 --cluster $ECS_CLUSTER \
 --services $ECS_SERVICE
 echo "👉 Deployment successful!"

```

## How It Works

The workflow has two jobs:

1. **test** — runs on every push and PR. Spins up a test Postgres, runs linting, type checking, unit tests, and dbt tests. If any step fails, the pipeline stops.
2. **build-and-deploy** — only runs on **main** branch, and only if tests pass. Builds the Docker image, pushes to ECR, updates the ECS service.

**needs: test** means deploy waits for tests to succeed. **if: github.ref == 'refs/heads/main'** means PRs get tested but not deployed.

## Terraform Plan in PRs

Add infrastructure validation to pull requests:

```

terraform-plan:
 name: Terraform Plan
 runs-on: ubuntu-latest
 if: github.event_name == 'pull_request'

 steps:
 - uses: actions/checkout@v4
 - uses: hashicorp/setup-terraform@v3

 - name: Terraform Init
 working-directory: terraform/
 run: terraform init

 - name: Terraform Plan
 working-directory: terraform/
 run: terraform plan -no-color

```

### ❑ Pro Tip: Secrets Management

Never put credentials in workflow files. Use GitHub Secrets (repo → Settings → Secrets → Actions). Better yet, use OIDC with `role-to-assume` — no static credentials at all.

### ⚠ Common Mistake

- Not running tests in CI. "I'll test locally" doesn't scale.
- Deploying on every push to main without a staging step.
- Not pinning action versions. Use `actions/checkout@v4` not `@main`.

## ❑ Checkpoint

1. What's the difference between CI and CD?
2. Why does the deploy job have `if: github.ref == 'refs/heads/main'`?
3. What's the advantage of OIDC over static AWS access keys for CI/CD?

## 8.6 Cost Management and Security Basics

**TL;DR:** Set up billing alerts BEFORE doing anything else. A typical dev data platform costs ~\$47/month on AWS. Key cost savers: Spot instances for Spark, S3 lifecycle rules, right-size RDS, schedule dev resources off at night. Security: least privilege IAM, never hardcode secrets, encrypt everything.

### Rule #1: Billing Alerts First

```
In Terraform
resource "aws_cloudwatch_metric_alarm" "billing_alarm" {
 alarm_name = "monthly-billing-alarm"
 comparison_operator = "GreaterThanThreshold"
 evaluation_periods = 1
 metric_name = "EstimatedCharges"
 namespace = "AWS/Billing"
 period = 21600 # 6 hours
 statistic = "Maximum"
 threshold = 50 # Alert at $50
 alarm_actions = [aws_sns_topic.billing_alerts.arn]
 dimensions = {
 Currency = "USD"
 }
}
```

## Cost Estimation for a Typical DE Setup

Service	Config	Monthly Cost
S3	50GB standard	~\$1.15
RDS Postgres	db.t3.micro, 20GB	~\$15
ECS Fargate	2 tasks, 0.5 vCPU, 1GB each	~\$30
Athena	~50 queries/day, 100MB scanned each	~\$0.75
ECR	5 images	~\$0.50
<b>Total</b>		<b>~\$47/month</b>

In production with more data and compute, expect \$200-500/month for a small/medium platform.

## Cost-Saving Tips

1. **Use Spot instances for Spark/EMR** — 60-90% cheaper than on-demand
2. **S3 lifecycle rules** — move old data to Glacier (configured in Section 8.3)
3. **Right-size RDS** — start with `db.t3.micro`, scale up only when needed
4. **Schedule dev resources off at night:**

```
Lambda function to stop dev RDS at night
import boto3

def lambda_handler(event, context):
 rds = boto3.client('rds')
 rds.stop_db_instance(DBInstanceIdentifier='ecommerce-dev-db')
 return {"status": "stopped"}
```

1. **Athena: always use Parquet + partitions** — can reduce query costs by 90%+

## Security: Three Rules

### 1. Principle of Least Privilege

```

BAD – gives access to everything
resource "aws_iam_role_policy_attachment" "bad" {
 role = aws_iam_role.pipeline_role.name
 policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess" # NEVER
}

GOOD – specific permissions for specific resources
resource "aws_iam_role_policy" "good" {
 name = "pipeline-s3-access"
 role = aws_iam_role.pipeline_role.id
 policy = jsonencode({
 Version = "2012-10-17"
 Statement = [{
 Effect = "Allow"
 Action = ["s3:GetObject", "s3:PutObject"]
 Resource = ["${aws_s3_bucket.data_lake.arn}/*"]
 }]
 })
}

```

## 2. Never Hardcode Secrets

Use AWS Secrets Manager:

```

import boto3
import json

def get_secret(secret_name: str, region: str = "us-east-1") -> dict:
 client = boto3.client("secretsmanager", region_name=region)
 response = client.get_secret_value(SecretId=secret_name)
 return json.loads(response["SecretString"])

Usage
db_creds = get_secret("ecommerce/db-credentials")
conn_string = (
 f"postgresql://{db_creds['username']}:{db_creds['password']}@"
 f"{db_creds['host']}:5432/{db_creds['dbname']}"
)

```

## 3. Encrypt Everything

```
resource "aws_s3_bucket_server_side_encryption_configuration" "data_lake" {
 bucket = aws_s3_bucket.data_lake.id
 rule {
 apply_server_side_encryption_by_default {
 sse_algorithm = "aws:kms"
 }
 }
}
```

#### ⚠ Common Mistake

- Using your root AWS account for daily work. Create an IAM user.
- S3 buckets accidentally set to public. Always block public access.
- Committing `.env` files with real credentials to Git. Add `.env` to `.gitignore`.

#### ☐ Checkpoint

1. What should you set up BEFORE creating any AWS resources?
2. How much cheaper is S3 Glacier compared to S3 Standard?
3. What does "principle of least privilege" mean for IAM policies?

## 8.7 Module 8 Project — Deploy Your Pipeline to AWS

**TL;DR:** Deploy your Airflow pipeline to AWS using Terraform for infrastructure, Docker for packaging, and GitHub Actions for CI/CD. Use MinIO locally as an S3 substitute. The Terraform configs themselves are the primary deliverable — you don't have to `apply` to a real AWS account if you don't want to spend money.

### Overview

Deploy your Airflow pipeline (from Modules 4 and 7) to AWS using:

- **Terraform** to provision S3, RDS, ECS, IAM
- **Docker** to containerize Airflow and your pipeline code
- **GitHub Actions** for CI/CD
- **MinIO** as a local S3 substitute for development



## Project Structure

```
module-8-project/
├── terraform/
│ ├── main.tf
│ ├── variables.tf
│ ├── s3.tf
│ ├── rds.tf
│ ├── iam.tf
│ ├── ecs.tf # Bonus
│ ├── outputs.tf
│ └── terraform.tfvars.example
├── docker/
│ ├── Dockerfile
│ └── Dockerfile.airflow
├── .github/
│ └── workflows/
│ └── pipeline.yml
├── src/
│ └── pipeline/
│ ├── __init__.py
│ ├── extract.py
│ ├── transform.py
│ ├── load.py
│ └── quality.py
├── dags/
│ └── ecommerce_pipeline.py
├── tests/
│ ├── test_extract.py
│ ├── test_transform.py
│ └── test_quality.py
├── docker-compose.yml
├── pyproject.toml
└── README.md
```

### Step 1: Terraform Infrastructure

Create the Terraform configs from this module. At minimum:

- S3 bucket with versioning, encryption, lifecycle rules, and public access block
- RDS Postgres instance with security group
- IAM role with least-privilege policy for S3 and RDS access
- CloudWatch billing alarm

```
cd terraform
terraform init
terraform plan -var="db_password=localtest123"
```

#### ❏ Pro Tip

You don't have to actually `terraform apply` to a real AWS account if you don't want to spend money. The configs themselves are the deliverable. But if you have free tier or credits, go for it.

## Step 2: Dockerize the Pipeline

Write a production-grade Dockerfile with:

- Multi-stage build
- Non-root user
- Layer ordering optimized for caching
- Health check

```
docker build -t ecommerce-pipeline .
docker run --rm ecommerce-pipeline --help
```

## Step 3: Docker Compose for Local Dev

Everything should start with one command:

```
docker-compose up -d
```

Include: your pipeline container, Postgres, MinIO, and Airflow (webserver + scheduler).

## Step 4: GitHub Actions CI/CD

Create `.github/workflows/pipeline.yml` that:

1. Runs on push to `main` and on PRs
2. Lints code (ruff)
3. Runs unit tests with a test Postgres service
4. Builds Docker image
5. (Optional) Pushes to ECR and deploys to ECS

## Step 5: Environment-Aware Configuration

```
src/pipeline/config.py
import os

class Config:
 """Environment-aware configuration."""
 ENVIRONMENT = os.getenv("ENVIRONMENT", "dev")

 # S3 / MinIO
 S3_ENDPOINT = os.getenv("S3_ENDPOINT", "http://localhost:9000")
 S3_BUCKET = os.getenv("S3_BUCKET", "ecommerce-data-lake")
 AWS_ACCESS_KEY_ID = os.getenv("AWS_ACCESS_KEY_ID", "minioadmin")
 AWS_SECRET_ACCESS_KEY = os.getenv("AWS_SECRET_ACCESS_KEY", "minioadmin")

 # Database
 DATABASE_URL = os.getenv(
 "DATABASE_URL",
 "postgresql://postgres:postgres@localhost:5432/ecommerce"
)

 @classmethod
 def is_local(cls) -> bool:
 return cls.ENVIRONMENT == "dev"
```

## Expected Output

- `terraform plan` runs without errors and shows resources to be created
- `docker-compose up` starts all services and the pipeline runs
- GitHub Actions workflow passes on push
- Pipeline reads from and writes to MinIO/S3

## Completion Checklist

- ☐ Terraform configs for S3, RDS, IAM, and billing alarm
- ☐ S3 bucket has versioning, encryption, lifecycle rules, public access block
- ☐ RDS has security group restricting access
- ☐ IAM role follows least privilege
- ☐ No secrets hardcoded anywhere
- ☐ Dockerfile uses multi-stage build, non-root user
- ☐ docker-compose.yml starts full stack locally
- ☐ GitHub Actions workflow runs tests and builds Docker image
- ☐ Pipeline code uses environment variables for configuration
- ☐ README explains how to run locally and how to deploy



# Module 9: Data Engineering for AI

*The hottest intersection in tech. Learn to build the data infrastructure that powers AI applications.*

## 9.1 How AI Changes the Data Engineer's Job

**TL;DR:** AI doesn't replace data engineers — it creates MORE work for us. Every AI application needs data pipelines feeding it. The new skills to learn: embeddings pipelines, vector database management, feature stores, RAG data layers, and LLM data preparation. These all build on top of the data engineering skills you already have.

If you've been watching job postings, you've noticed something: "data engineer" roles increasingly mention AI, LLMs, embeddings, and vector databases. This isn't a fad. AI is fundamentally changing what data engineers build.

But here's what nobody tells you: AI doesn't replace data engineers. It creates MORE work for us. Every AI application needs data pipelines feeding it. Every LLM needs clean, chunked, embedded data. Every ML model needs features computed and served.

### Traditional vs AI-Augmented Pipeline

#### Traditional Pipeline:

```
Source → Extract → Transform → Load → Dashboard
```

#### AI-Augmented Pipeline:

```
Source → Extract → Transform → Load → Dashboard
 ↓
 Chunk → Embed → Vector DB → RAG API
 ↓
 Feature Store → ML Model → Predictions
```

Same pipeline, but with new branches. As a data engineer, you're building those branches.

### The New Things You'll Build

1. **Embeddings pipelines** — converting text/images into vectors that ML models can use
2. **Vector database management** — storing and indexing embeddings for similarity search

3. **Feature stores** — pre-computing features for ML models
4. **RAG data layers** — preparing and serving data for Retrieval-Augmented Generation
5. **LLM data prep** — cleaning, chunking, and enriching data for fine-tuning or prompting

[DIAGRAM: The AI Data Stack as a layered architecture. Bottom layer: Ingestion Layer (same pipelines you already know). Storage Layer: Vector DB, Feature Store, Data Lake. Processing Layer: Embeddings, chunking, feature computation. Serving Layer: FastAPI, vector search, feature serving. Top layer: AI Applications (chatbots, search, recommendations).]

#### □ **Key Concept: The AI Data Stack**

The bottom layers of the AI stack are the same data engineering you've been learning this whole course. The top layers are new, but they're built ON TOP of your existing skills. In every AI project, the ML/AI engineer builds the model. The data engineer builds everything that feeds it.

#### △ **Common Mistake**

- Thinking you need to understand deep learning to do AI data engineering. You don't. You need to understand data formats, pipelines, and APIs.
- Ignoring this trend. The DEs who learn these skills now will be the most valuable ones in 2-3 years.

#### □ **Checkpoint**

1. How does an AI-augmented pipeline differ from a traditional data pipeline?
2. Name three new things data engineers build for AI workloads.
3. Why does AI create *more* work for data engineers, not less?

## 9.2 Feature Stores — What They Are and When You Need One

**TL;DR:** A feature store is a centralized system for computing, storing, and serving ML-ready features. Most teams don't need a full-blown feature store (like Feast) until they have 5+ ML models. For 1-2 models, well-organized SQL views and a simple API are enough. Build simple first.

An ML engineer says: *"I need the customer's average order value, their total orders in the last 30 days, and their most recent order date. Updated daily for batch predictions, and in real-time for the API."*

You could write a SQL query. But then another ML engineer needs different features. And another. And they all need the *exact same features* computed the *exact same way* so training and serving are consistent. This is the problem feature stores solve.

### What a Feature Store Does

1. **Computes features** — running transformations that produce ML-ready values
2. **Stores features** — persisting them for both batch (training) and online (serving) use

- 3. **Serves features** — making them available with low latency for real-time predictions
- 4. **Ensures consistency** — same feature definition used in training and serving

[DIAGRAM: Dual-store pattern. Batch Pipeline feeds the Offline Store (data lake/warehouse). Offline Store syncs to Online Store (Redis/DynamoDB). ML Model API reads from Online Store for low-latency serving.]

### Do You Actually Need One?

Honest answer: most teams don't need a full-blown feature store until they have 5+ ML models in production.

Scenario	Recommendation
1-2 models, batch only	SQL views + scheduled materialization. No feature store needed.
1-2 models, real-time	Pre-computed features in Redis. Still no formal feature store.
5+ models, shared features	Feature store (Feast, Tecton, or custom).
Enterprise, 50+ models	Feast, Tecton, or Databricks Feature Store.

### Building a Simple Feature Store

Here's what you'd build at a startup before you need Feast — a minimal feature store in Postgres + Python:

```

-- Feature table: one row per customer, updated daily
CREATE TABLE IF NOT EXISTS feature_store.customer_features (
 customer_id INTEGER PRIMARY KEY,

 -- Order features
 total_orders INTEGER,
 total_revenue DECIMAL(12,2),
 avg_order_value DECIMAL(10,2),
 orders_last_30_days INTEGER,
 days_since_last_order INTEGER,

 -- Behavioral features
 favorite_category VARCHAR(100),
 distinct_products_purchased INTEGER,

 -- Metadata
 computed_at TIMESTAMP DEFAULT NOW(),
 feature_version VARCHAR(20) DEFAULT '1.0'
);

-- Materialization query: compute features from raw data
INSERT INTO feature_store.customer_features
SELECT
 c.customer_id,
 COUNT(DISTINCT o.order_id) AS total_orders,
 COALESCE(SUM(o.total_amount), 0) AS total_revenue,
 COALESCE(AVG(o.total_amount), 0) AS avg_order_value,
 COUNT(DISTINCT CASE
 WHEN o.order_date >= CURRENT_DATE - INTERVAL '30 days'
 THEN o.order_id
 END) AS orders_last_30_days,
 EXTRACT(DAY FROM NOW() - MAX(o.order_date))::INTEGER AS days_since_last_order,
 MODE() WITHIN GROUP (ORDER BY p.category) AS favorite_category,
 COUNT(DISTINCT li.product_id) AS distinct_products_purchased,
 NOW() AS computed_at,
 '1.0' AS feature_version
FROM dim_customers c
LEFT JOIN fact_orders o ON c.customer_id = o.customer_id
LEFT JOIN fact_line_items li ON o.order_id = li.order_id
LEFT JOIN dim_products p ON li.product_id = p.product_id
GROUP BY c.customer_id
ON CONFLICT (customer_id) DO UPDATE SET
 total_orders = EXCLUDED.total_orders,
 total_revenue = EXCLUDED.total_revenue,
 avg_order_value = EXCLUDED.avg_order_value,
 orders_last_30_days = EXCLUDED.orders_last_30_days,
 days_since_last_order = EXCLUDED.days_since_last_order,
 favorite_category = EXCLUDED.favorite_category,
 distinct_products_purchased = EXCLUDED.distinct_products_purchased,
 computed_at = EXCLUDED.computed_at;

```

Python class to serve features:



```

feature_store/store.py
import psycopg2
from dataclasses import dataclass
from typing import Optional

@dataclass
class CustomerFeatures:
 customer_id: int
 total_orders: int
 total_revenue: float
 avg_order_value: float
 orders_last_30_days: int
 days_since_last_order: Optional[int]
 favorite_category: Optional[str]
 distinct_products_purchased: int
 computed_at: str
 feature_version: str

class FeatureStore:
 def __init__(self, connection_string: str):
 self.conn = psycopg2.connect(connection_string)

 def get_customer_features(self, customer_id: int) -> Optional[CustomerFeatures]:
 """Get features for a single customer (online serving)."""
 with self.conn.cursor() as cur:
 cur.execute(
 "SELECT * FROM feature_store.customer_features WHERE customer_id = %s",
 (customer_id,),
)
 row = cur.fetchone()
 if row is None:
 return None
 return CustomerFeatures(*row)

 def get_batch_features(self, customer_ids: list[int]) -> list[CustomerFeatures]:
 """Get features for multiple customers (batch prediction)."""
 with self.conn.cursor() as cur:
 cur.execute(
 "SELECT * FROM feature_store.customer_features WHERE customer_id = ANY(%s)",
 (customer_ids,),
)
 return [CustomerFeatures(*row) for row in cur.fetchall()]

 def get_training_dataset(self, feature_version: str = "1.0") -> list[CustomerFeatures]:
 """Get all features for model training."""
 with self.conn.cursor() as cur:
 cur.execute(
 "SELECT * FROM feature_store.customer_features WHERE feature_version = %s",

```

```

 (feature_version,),
)
 return [CustomerFeatures(*row) for row in cur.fetchall()]

Usage
store = FeatureStore("postgresql://postgres:postgres@localhost:5432/ecommerce")
features = store.get_customer_features(customer_id=42)
print(f"Customer 42: {features.total_orders} orders, ${features.avg_order_value} avg")

```

Expected output:

```
Customer 42: 17 orders, $84.32 avg
```

#### ⚠ Common Mistake

- Over-engineering the feature store before you have any ML models in production. Build simple first.
- **Training-serving skew**: computing features differently in training vs serving. Use the SAME SQL/code for both.
- Not versioning features. When you change how a feature is computed, old models trained on the old version will break.

#### □ Checkpoint

1. What problem does a feature store solve?
2. At what scale should you consider a formal feature store like Feast?
3. What is training-serving skew and why is it dangerous?

## 9.3 Embeddings Pipelines — Turning Text Into Vectors

**TL;DR:** An embedding is a list of numbers (vector) representing the "meaning" of text. Similar meanings produce similar vectors. Build pipelines that generate embeddings incrementally (only new/changed items), in batches (API rate limits), and store the original text alongside the vector. OpenAI's `text-embedding-3-small` is cheap (~\$0.10 per 100K products); open-source `all-MiniLM-L6-v2` is free and runs locally.

How does ChatGPT "understand" that "dog" and "puppy" mean similar things? How does semantic search know that "cheap flights to Paris" should match "budget airline tickets to France"?

Embeddings.

### How Embeddings Work

An embedding is a list of numbers (a vector) that represents the "meaning" of a piece of text:

```
"I love pizza" → [0.023, -0.156, 0.892, ..., 0.044] # 1536 numbers
"Pizza is great" → [0.019, -0.148, 0.887, ..., 0.041] # Similar numbers!
"The stock market crashed" → [-0.445, 0.312, -0.019, ..., 0.678] # Very different
```

Similar meanings → similar vectors → similar numbers. Think of embeddings as coordinates on a map. "Pizza" and "pasta" are close together. "Pizza" and "stock market" are far apart.

#### □Key Concept: Embeddings

An embedding converts unstructured data (text, images) into a fixed-length numerical vector. The key property: semantically similar inputs produce vectors that are close together in vector space. This enables similarity search, clustering, and classification without keyword matching.

## Building an Embeddings Pipeline

A pipeline that generates embeddings for product descriptions — used for "similar products" or semantic search:

```

embeddings/pipeline.py
import openai
import psycopg2
import time
from dataclasses import dataclass

@dataclass
class ProductEmbedding:
 product_id: int
 product_name: str
 description: str
 embedding: list[float]

class EmbeddingsPipeline:
 """Generate and store embeddings for product catalog."""

 def __init__(
 self,
 openai_api_key: str,
 db_connection_string: str,
 model: str = "text-embedding-3-small", # Cheap and good
 batch_size: int = 100,
):
 self.client = openai.OpenAI(api_key=openai_api_key)
 self.conn = psycopg2.connect(db_connection_string)
 self.model = model
 self.batch_size = batch_size

 def generate_embeddings(self, texts: list[str]) -> list[list[float]]:
 """Generate embeddings for a batch of texts using OpenAI API."""
 response = self.client.embeddings.create(
 model=self.model,
 input=texts,
)
 return [item.embedding for item in response.data]

 def get_products_without_embeddings(self) -> list[dict]:
 """Find products that don't have embeddings yet (incremental)."""
 with self.conn.cursor() as cur:
 cur.execute("""
 SELECT p.product_id, p.product_name, p.description
 FROM dim_products p
 LEFT JOIN product_embeddings pe ON p.product_id = pe.product_id
 WHERE pe.product_id IS NULL
 OR pe.updated_at < p.updated_at -- Re-embed if product changed
 ORDER BY p.product_id
 """)
 columns = [desc[0] for desc in cur.description]
 return [dict(zip(columns, row)) for row in cur.fetchall()]

 def store_embeddings(self, product_embeddings: list[ProductEmbedding]) -> None:

```

```

 """Store embeddings in Postgres with pgvector."""
 with self.conn.cursor() as cur:
 for pe in product_embeddings:
 cur.execute("""
 INSERT INTO product_embeddings (product_id, embedding, updated_at)
 VALUES (%s, %s, NOW())
 ON CONFLICT (product_id) DO UPDATE SET
 embedding = EXCLUDED.embedding,
 updated_at = NOW()
 """, (pe.product_id, pe.embedding))
 self.conn.commit()

 def run(self) -> int:
 """Run the full embeddings pipeline. Returns number processed."""
 products = self.get_products_without_embeddings()

 if not products:
 print("No products need embedding updates.")
 return 0

 print(f"Processing {len(products)} products...")
 total_processed = 0

 for i in range(0, len(products), self.batch_size):
 batch = products[i:i + self.batch_size]

 # Combine name + description for richer embeddings
 texts = [
 f"{p['product_name']}: {p['description']}"
 for p in batch
]

 embeddings = self.generate_embeddings(texts)

 product_embeddings = [
 ProductEmbedding(
 product_id=p["product_id"],
 product_name=p["product_name"],
 description=p["description"],
 embedding=emb,
)
 for p, emb in zip(batch, embeddings)
]

 self.store_embeddings(product_embeddings)
 total_processed += len(batch)
 print(f" Processed {total_processed}/{len(products)} products")

 # Rate limiting
 if i + self.batch_size < len(products):
 time.sleep(1)

 return total_processed

```

```
Usage
if __name__ == "__main__":
 import os
 pipeline = EmbeddingsPipeline(
 openai_api_key=os.environ["OPENAI_API_KEY"],
 db_connection_string=os.environ["DATABASE_URL"],
)
 processed = pipeline.run()
 print(f"Done! Processed {processed} products.")
```

Expected output:

```
Processing 500 products...
Processed 100/500 products
Processed 200/500 products
Processed 300/500 products
Processed 400/500 products
Processed 500/500 products
Done! Processed 500 products.
```

## Using Open-Source Models Instead

OpenAI is easy but costs money. For production at scale, use open-source models:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2") # 384 dimensions, fast

texts = [
 "Wireless Bluetooth Headphones: Premium sound quality with noise cancellation",
 "Running Shoes: Lightweight mesh with responsive cushioning",
]

embeddings = model.encode(texts)
print(f"Embedding shape: {embeddings.shape}")
print(f"Embedding sample: {embeddings[0][:5]}")
```

Expected output:

```
Embedding shape: (2, 384)
Embedding sample: [0.0234 -0.0891 0.1123 -0.0456 0.0789]
```

## Model Comparison

Model	Dimensions	Cost	Quality	Speed
OpenAI <code>text-embedding-3-small</code>	1536	\$0.02/1M tokens	Best	API call required
<code>all-MiniLM-L6-v2</code>	384	Free	Good	Fast, runs locally
<code>e5-large-v2</code>	1024	Free	Near-OpenAI	Slower, runs locally

## Cost Estimation

OpenAI `text-embedding-3-small` at ~\$0.02 per 1M tokens:

- Average product description: ~50 tokens
- 100K products: ~5M tokens = **~\$0.10**
- Even at 1M products, you're looking at **\$1**

### ⚠ Common Mistake

- Embedding entire documents as one vector. Long documents should be chunked first (covered in Section 9.5).
- Not storing the text alongside the embedding. You'll need it for debugging and returning results.
- Re-embedding everything on every run. Use incremental logic — only embed new or changed items.
- Using the wrong model for your use case. Multilingual data? Use a multilingual model.

## □ Checkpoint

1. What is an embedding and what key property makes it useful?
2. Why should you combine product name + description before embedding?
3. When would you choose an open-source embedding model over OpenAI?

## 9.4 Vector Databases — Storage and Retrieval

**TL;DR:** A vector database stores and searches high-dimensional vectors efficiently using specialized indexes. Start with **pgvector** (Postgres extension) — no new infrastructure needed. It handles millions of vectors easily. Use HNSW indexes for production (faster queries) and IVFFlat for simpler cases. Cosine similarity is the standard metric.

You've got 100,000 product embeddings — each a list of 1,536 numbers. A customer searches for "comfortable shoes for running." You generate an embedding for that query. Now you need to find the 10 most similar products. Vector databases solve this with special indexes that make similarity search fast.

## Choosing a Vector Database

Database	Type	Best For
pgvector	Postgres extension	Teams already using Postgres — no new infrastructure
Pinecone	Managed cloud	Quick start, no ops (\$70+/month)
Weaviate	Open-source	Self-hosted, full-featured
Qdrant	Open-source	Performance-focused
ChromaDB	Open-source	Prototyping only (not production scale)

**Recommendation: start with pgvector.** If you're already using Postgres, adding pgvector is one command. No new database to manage. It handles millions of vectors easily.

## Setting Up pgvector

```
-- Enable the extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Create the embeddings table
CREATE TABLE product_embeddings (
 product_id INTEGER PRIMARY KEY REFERENCES dim_products(product_id),
 embedding vector(1536), -- OpenAI text-embedding-3-small dimension
 updated_at TIMESTAMP DEFAULT NOW()
);

-- HNSW index (recommended for production — faster queries, more accurate)
CREATE INDEX ON product_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

### ❑ Key Concept: Vector Index Types

- **IVFFlat** — faster to build, slightly less accurate. Good for getting started. Set `lists = sqrt(num_rows)`.
- **HNSW** — slower to build, more accurate, faster queries. Better for production.

Without an index, every query does a full table scan. Fine for 1,000 rows, terrible for 100K+.



## Similarity Search in SQL

```
-- Find 10 most similar products to a given embedding
-- The <=> operator computes cosine distance (1 - cosine_similarity)
SELECT
 p.product_id,
 p.product_name,
 p.description,
 1 - (pe.embedding <=> '[0.023, -0.156, 0.892, ...] '::vector) AS similarity
FROM product_embeddings pe
JOIN dim_products p ON pe.product_id = p.product_id
ORDER BY pe.embedding <=> '[0.023, -0.156, 0.892, ...] '::vector
LIMIT 10;
```



```

vector_search/search.py
import psycopg2
from dataclasses import dataclass

@dataclass
class SearchResult:
 product_id: int
 product_name: str
 description: str
 similarity: float

class VectorSearch:
 def __init__(self, connection_string: str):
 self.conn = psycopg2.connect(connection_string)

 def search_similar_products(
 self,
 query_embedding: list[float],
 limit: int = 10,
 min_similarity: float = 0.5,
) -> list[SearchResult]:
 """Find products similar to the query embedding."""
 with self.conn.cursor() as cur:
 cur.execute("""
 SELECT
 p.product_id,
 p.product_name,
 p.description,
 1 - (pe.embedding <=> %s::vector) AS similarity
 FROM product_embeddings pe
 JOIN dim_products p ON pe.product_id = p.product_id
 WHERE 1 - (pe.embedding <=> %s::vector) > %s
 ORDER BY pe.embedding <=> %s::vector
 LIMIT %s
 """, (str(query_embedding), str(query_embedding),
 min_similarity, str(query_embedding), limit))

 return [
 SearchResult(
 product_id=row[0],
 product_name=row[1],
 description=row[2],
 similarity=round(row[3], 4),
)
 for row in cur.fetchall()
]

 def find_similar_to_product(
 self,
 product_id: int,
 limit: int = 10,
) -> list[SearchResult]:
 """Find products similar to the product with the given id."""
 with self.conn.cursor() as cur:
 cur.execute("""
 SELECT
 p.product_id,
 p.product_name,
 p.description,
 1 - (pe.embedding <=> %s::vector) AS similarity
 FROM product_embeddings pe
 JOIN dim_products p ON pe.product_id = p.product_id
 WHERE p.product_id = %s
 ORDER BY pe.embedding <=> %s::vector
 LIMIT %s
 """, (product_id, product_id, limit))

 return [
 SearchResult(
 product_id=row[0],
 product_name=row[1],
 description=row[2],
 similarity=round(row[3], 4),
)
 for row in cur.fetchall()
]

```

```

) -> list[SearchResult]:
 """Find products similar to a given product."""
 with self.conn.cursor() as cur:
 cur.execute(
 "SELECT embedding FROM product_embeddings WHERE product_id = %s",
 (product_id,),
)
 row = cur.fetchone()
 if row is None:
 return []

 embedding = row[0]

 cur.execute("""
 SELECT
 p.product_id,
 p.product_name,
 p.description,
 1 - (pe.embedding <=> %s::vector) AS similarity
 FROM product_embeddings pe
 JOIN dim_products p ON pe.product_id = p.product_id
 WHERE pe.product_id != %s
 ORDER BY pe.embedding <=> %s::vector
 LIMIT %s
 """, (embedding, product_id, embedding, limit))

 return [
 SearchResult(
 product_id=row[0],
 product_name=row[1],
 description=row[2],
 similarity=round(row[3], 4),
)
 for row in cur.fetchall()
]

Usage
search = VectorSearch("postgresql://postgres:postgres@localhost:5432/ecommerce")
results = search.find_similar_to_product(42, limit=5)
for r in results:
 print(f" {r.product_name} (similarity: {r.similarity})")

```

#### Expected output:

```

ProElite Running Max 72 (similarity: 0.9234)
ActiveGear Running Lite 18 (similarity: 0.9012)
NatureFit Training Pro 45 (similarity: 0.8756)
CoreBasics Running Essential 91 (similarity: 0.8543)
UrbanEdge Shoes Ultra 33 (similarity: 0.8201)

```

## Performance at Scale

Rows	IVFFlat Query Time	HNSW Query Time
100K	~5ms	~2ms
1M	~20ms	~5ms
10M	~100ms	~15ms

For most applications, this is more than fast enough. Sub-millisecond at billions of vectors is when you'd look at Pinecone or a dedicated vector database.

### ⚠ Common Mistake

- Forgetting to create an index. Without one, queries do full table scans.
- Not tuning index parameters. Adjust `lists` (IVFFlat) or `m` / `ef_construction` (HNSW) based on data size.
- Mixing embedding dimensions. If you switch models (1536-dim to 384-dim), you need new columns or tables.

## □ Checkpoint

1. What's the difference between IVFFlat and HNSW indexes?
2. What does the `<=>` operator compute in pgvector?
3. Why should you start with pgvector instead of a dedicated vector database?

## 9.5 RAG Pipeline Architecture — Building the Data Layer

**TL;DR:** RAG (Retrieval-Augmented Generation) is ~20% LLM and ~80% data engineering. The data engineer builds the indexing pipeline (documents → chunk → embed → store) and the retrieval layer (query → embed → vector search → top K chunks). Chunking strategy has the biggest impact on quality. Use 300-500 token chunks with overlap.

RAG — Retrieval-Augmented Generation — is the pattern behind every "chat with your documents" application. And while the ML engineers get credit for the chatbot, guess who builds the entire data layer? Us.

## The Two Phases of RAG

### Phase 1: Indexing (offline — your job)

Documents → Chunk → Embed → Store in Vector DB

### Phase 2: Query (online — real-time)

User Question → Embed → Search Vector DB → Top K Chunks → LLM → Answer

You build Phase 1 and the retrieval part of Phase 2.

**Step 1: Document Ingestion**

```

rag/ingest.py
from pathlib import Path
import fitz # PyMuPDF for PDF parsing
from dataclasses import dataclass

@dataclass
class Document:
 content: str
 metadata: dict # source file, page number, etc.

def load_pdf(path: str) -> list[Document]:
 """Extract text from a PDF file."""
 doc = fitz.open(path)
 documents = []
 for page_num, page in enumerate(doc):
 text = page.get_text().strip()
 if text:
 documents.append(Document(
 content=text,
 metadata={
 "source": Path(path).name,
 "page": page_num + 1,
 "type": "pdf",
 },
))
 return documents

def load_text(path: str) -> list[Document]:
 """Load a plain text or markdown file."""
 content = Path(path).read_text()
 return [Document(
 content=content,
 metadata={"source": Path(path).name, "type": "text"},
)]

def load_documents(directory: str) -> list[Document]:
 """Load all supported documents from a directory."""
 docs = []
 dir_path = Path(directory)

 for pdf in dir_path.glob("**/*.pdf"):
 docs.extend(load_pdf(str(pdf)))
 for txt in dir_path.glob("**/*.txt"):
 docs.extend(load_text(str(txt)))
 for md in dir_path.glob("**/*.md"):
 docs.extend(load_text(str(md)))

 print(f"Loaded {len(docs)} document sections from {directory}")
 return docs

```



## Step 2: Chunking

This is the most underrated step in RAG. How you chunk documents has a massive impact on retrieval quality.

```

rag/chunker.py
from dataclasses import dataclass

@dataclass
class Chunk:
 content: str
 metadata: dict
 chunk_index: int

def chunk_by_tokens(
 text: str,
 chunk_size: int = 500, # tokens per chunk
 chunk_overlap: int = 50, # overlap between chunks
) -> list[str]:
 """Split text into overlapping chunks by approximate token count.

 Rule of thumb: 1 token ≈ 4 characters for English text.
 """
 words = text.split()
 tokens_per_word = 1.3 # average
 words_per_chunk = int(chunk_size / tokens_per_word)
 overlap_words = int(chunk_overlap / tokens_per_word)

 chunks = []
 start = 0
 while start < len(words):
 end = start + words_per_chunk
 chunk = " ".join(words[start:end])
 if chunk.strip():
 chunks.append(chunk)
 start = end - overlap_words

 return chunks

def chunk_documents(
 documents: list[Document],
 chunk_size: int = 500,
 chunk_overlap: int = 50,
) -> list[Chunk]:
 """Chunk all documents with metadata preservation."""
 all_chunks = []

 for doc in documents:
 text_chunks = chunk_by_tokens(doc.content, chunk_size, chunk_overlap)

 for i, text in enumerate(text_chunks):
 chunk = Chunk(
 content=text,
 metadata={
 **doc.metadata,

```

```

 "chunk_index": i,
 "chunk_count": len(text_chunks),
 },
 chunk_index=i,
)
all_chunks.append(chunk)

print(f"Created {len(all_chunks)} chunks from {len(documents)} documents")
return all_chunks

```

### Chunking strategies:

Strategy	When to Use
Fixed-size token chunks	Simple, works for most cases
Sentence-based	Better for Q&A use cases
Paragraph-based (split on <code>\n\n</code> )	Good for well-structured documents
Semantic chunking	Best quality, slowest (uses embedding model to find breakpoints)

#### ❏ Pro Tip

The overlap between chunks is critical — it prevents information from being split across chunk boundaries. 300-500 tokens per chunk with 50-100 token overlap is the sweet spot for most use cases.

**Step 3: Embed and Store**

```

rag/indexer.py
import openai
import psycopg2
import json

class RAGIndexer:
 def __init__(self, openai_api_key: str, db_connection_string: str):
 self.client = openai.OpenAI(api_key=openai_api_key)
 self.conn = psycopg2.connect(db_connection_string)
 self._setup_table()

 def _setup_table(self):
 with self.conn.cursor() as cur:
 cur.execute("CREATE EXTENSION IF NOT EXISTS vector")
 cur.execute("""
 CREATE TABLE IF NOT EXISTS rag_chunks (
 id SERIAL PRIMARY KEY,
 content TEXT NOT NULL,
 metadata JSONB,
 embedding vector(1536),
 created_at TIMESTAMP DEFAULT NOW()
)
 """)
 cur.execute("""
 CREATE INDEX IF NOT EXISTS rag_chunks_embedding_idx
 ON rag_chunks USING hnsw (embedding vector_cosine_ops)
 """)
 self.conn.commit()

 def index_chunks(self, chunks: list[Chunk]) -> int:
 """Embed and store chunks. Returns count indexed."""
 batch_size = 100
 total = 0

 for i in range(0, len(chunks), batch_size):
 batch = chunks[i:i + batch_size]
 texts = [c.content for c in batch]

 response = self.client.embeddings.create(
 model="text-embedding-3-small",
 input=texts,
)
 embeddings = [item.embedding for item in response.data]

 with self.conn.cursor() as cur:
 for chunk, embedding in zip(batch, embeddings):
 cur.execute(
 "INSERT INTO rag_chunks (content, metadata, embedding) VALUES
(%s, %s, %s)",
 (chunk.content, json.dumps(chunk.metadata), str(embedding)),
)
 self.conn.commit()

```

```
total += len(batch)

return total
```

**Step 4: Retrieval**

```

rag/retriever.py
import openai
import psycopg2
import os

class RAGRetriever:
 def __init__(self, openai_api_key: str, db_connection_string: str):
 self.client = openai.OpenAI(api_key=openai_api_key)
 self.conn = psycopg2.connect(db_connection_string)

 def retrieve(self, query: str, top_k: int = 5) -> list[dict]:
 """Retrieve the most relevant chunks for a query."""
 response = self.client.embeddings.create(
 model="text-embedding-3-small",
 input=[query],
)
 query_embedding = response.data[0].embedding

 with self.conn.cursor() as cur:
 cur.execute("""
 SELECT
 content,
 metadata,
 1 - (embedding <=> %s::vector) AS similarity
 FROM rag_chunks
 ORDER BY embedding <=> %s::vector
 LIMIT %s
 """, (str(query_embedding), str(query_embedding), top_k))

 return [
 {
 "content": row[0],
 "metadata": row[1],
 "similarity": round(row[2], 4),
 }
 for row in cur.fetchall()
]

 def query_with_context(self, question: str, top_k: int = 5) -> str:
 """Full RAG: retrieve context, then ask the LLM."""
 chunks = self.retrieve(question, top_k=top_k)

 context = "\n\n---\n\n".join([
 f"[Source: {c['metadata'].get('source', 'unknown')}] ",
 f"Similarity: {c['similarity']}]\n{c['content']}"
 for c in chunks
])

 response = self.client.chat.completions.create(
 model="gpt-4o-mini",
 messages=[
 {

```



```

 "role": "system",
 "content": (
 "You are a helpful assistant. Answer the user's question "
 "based ONLY on the provided context. If the context doesn't "
 "contain enough information, say so. Cite your sources."
),
 },
 {
 "role": "user",
 "content": f"Context:\n{context}\n\nQuestion: {question}",
 },
],
temperature=0,
)

return response.choices[0].message.content

Usage
retriever = RAGRetriever(
 openai_api_key=os.environ["OPENAI_API_KEY"],
 db_connection_string=os.environ["DATABASE_URL"],
)
answer = retriever.query_with_context("What is our return policy for electronics?")
print(answer)

```

#### ⚠ Common Mistake

- Chunks too large (lose specificity) or too small (lose context). 300-500 tokens is the sweet spot.
- No overlap between chunks — you'll miss information at boundaries.
- Not including metadata. Without source/page info, you can't tell users WHERE the answer came from.
- Embedding the query with a different model than the documents. Must use the same model for both.

#### □ Checkpoint

1. What are the two phases of RAG and who typically builds each?
2. Why is overlap between chunks important?
3. What system prompt instruction ensures the LLM only answers from the provided context?

## 9.6 LLM Data Preparation — Cleaning, Chunking, Metadata

**TL;DR:** Real-world documents are messy — PDFs with headers/footers, HTML with navigation menus, broken formatting. Clean text before embedding by removing artifacts, normalizing whitespace, and stripping boilerplate. Enrich metadata for filtering (content type, source, language). Good metadata enables filtered retrieval, which dramatically improves search quality.

The RAG pipeline works great with clean text files. Real-world data is messy. If you feed garbage into your embeddings, you get garbage search results.

**Cleaning Techniques**

```

rag/cleaner.py
import re

def clean_text(text: str) -> str:
 """Clean text for embedding. Order matters."""
 # Remove excessive whitespace
 text = re.sub(r'\s+', ' ', text)

 # Remove common PDF artifacts
 text = re.sub(r'Page \d+ of \d+', '', text)
 text = re.sub(r'(?i)confidential|draft|internal use only', '', text)

 # Remove URLs (usually not useful for semantic meaning)
 text = re.sub(r'https?://\S+', '', text)

 # Remove email addresses
 text = re.sub(r'\S+@\S+\.\S+', '', text)

 # Normalize unicode
 text = text.encode('ascii', 'ignore').decode('ascii')

 # Remove very short lines (likely headers/footers)
 lines = text.split('\n')
 lines = [line for line in lines if len(line.strip()) > 20]
 text = '\n'.join(lines)

 return text.strip()

def clean_html(html: str) -> str:
 """Extract meaningful text from HTML."""
 from bs4 import BeautifulSoup

 soup = BeautifulSoup(html, 'html.parser')

 # Remove non-content elements
 for tag in soup(['script', 'style', 'nav', 'header', 'footer',
 'aside', 'form', 'iframe']):
 tag.decompose()

 text = soup.get_text(separator='\n')
 return clean_text(text)

def enrich_metadata(content: str, source_metadata: dict) -> dict:
 """Add computed metadata useful for retrieval filtering."""
 metadata = {**source_metadata}

 # Approximate token count
 metadata['token_count'] = len(content.split()) * 1.3

 # Language detection (simple heuristic)

```

```

common_english = {'the', 'is', 'at', 'which', 'on', 'a', 'an'}
words = set(content.lower().split()[:100])
metadata['likely_english'] = len(words & common_english) > 3

Content type heuristic
if any(kw in content.lower() for kw in ['def ', 'class ', 'import ', 'function']):
 metadata['content_type'] = 'code'
elif any(kw in content.lower() for kw in ['table', 'figure', 'chart']):
 metadata['content_type'] = 'structured'
else:
 metadata['content_type'] = 'prose'

return metadata

```

## Metadata-Filtered Retrieval

Good metadata enables filtering, which dramatically improves retrieval quality:

```

-- Instead of searching ALL chunks, filter first
SELECT content, metadata, 1 - (embedding <=> %s::vector) AS similarity
FROM rag_chunks
WHERE metadata->>'content_type' = 'prose'
 AND metadata->>'department' = 'customer_service'
ORDER BY embedding <=> %s::vector
LIMIT 5;

```

### ⚠ Common Mistake

- Over-cleaning. Don't remove formatting that carries meaning (bullet points, numbered lists).
- Not deduplicating. Duplicate content produces duplicate chunks and skewed search results.
- Ignoring tables. Tables in PDFs are hard to parse but often contain the most valuable information.

## □ Checkpoint

1. Why should you remove URLs and email addresses before embedding?
2. How does metadata filtering improve RAG retrieval quality?
3. What's the risk of over-cleaning document text?

## 9.7 Building an End-to-End AI Data Pipeline

**TL;DR:** All the AI data components — embeddings, vector search, RAG, feature stores — connect in a production system orchestrated by Airflow. The pipeline ingests new documents, cleans/chunks them, generates embeddings, indexes them in pgvector, computes ML features, and runs quality checks.

[DIAGRAM: Airflow DAG with two parallel branches. Branch 1: Ingest Docs → Clean & Chunk → Embed & Index. Branch 2: Feature Computation → Materialize to Store. Both branches converge at Quality Checks. Below the DAG: Postgres + pgvector serves both a FastAPI API (/search, /features, /ask) and direct analytics queries.]

```

from airflow.decorators import dag, task
from datetime import datetime

@dag(
 schedule="@daily",
 start_date=datetime(2026, 1, 1),
 catchup=False,
 tags=["ai", "embeddings", "rag"],
)
def ai_data_pipeline():

 @task()
 def ingest_new_documents():
 """Check S3 for new documents, download them."""
 pass

 @task()
 def clean_and_chunk(documents):
 """Clean documents and split into chunks."""
 pass

 @task()
 def generate_embeddings(chunks):
 """Generate embeddings for new chunks."""
 pass

 @task()
 def index_in_vector_db(embeddings):
 """Store embeddings in pgvector."""
 pass

 @task()
 def compute_features():
 """Compute ML features from latest data."""
 pass

 @task()
 def run_quality_checks():
 """Validate embedding quality and feature freshness."""
 pass

 docs = ingest_new_documents()
 chunks = clean_and_chunk(docs)
 embeddings = generate_embeddings(chunks)
 indexed = index_in_vector_db(embeddings)
 features = compute_features()

 [indexed, features] >> run_quality_checks()

ai_data_pipeline()

```

## 9.8 Module 9 Project — RAG-Ready Data Pipeline

**TL;DR:** Build a complete RAG pipeline: ingest documents from a local directory, clean and chunk them, generate embeddings, store in pgvector, and serve a retrieval/search API via FastAPI. Everything runs in Docker Compose.

### Overview

Build a RAG-ready data pipeline that:

1. Ingests documents from a local directory (simulating S3)
2. Cleans and chunks them
3. Generates embeddings (OpenAI or sentence-transformers)
4. Stores in Postgres + pgvector
5. Serves a retrieval/search API via FastAPI
6. Orchestrated by Airflow

### Project Structure

```
module-9-project/
├── dags/
│ └── rag_pipeline.py
├── src/
│ ├── __init__.py
│ ├── ingest.py
│ ├── cleaner.py
│ ├── chunker.py
│ ├── embedder.py
│ ├── indexer.py
│ ├── retriever.py
│ └── api.py
├── data/
│ └── documents/
│ ├── product-catalog.md
│ ├── return-policy.pdf
│ └── faq.txt
├── tests/
│ ├── test_chunker.py
│ ├── test_cleaner.py
│ └── test_retriever.py
├── docker-compose.yml
├── Dockerfile
├── pyproject.toml
└── README.md
```



## Step 1: Prepare Sample Documents

Create 3-5 sample documents in `data/documents/`. Use at least 2 different formats (txt, md, pdf).

## Step 2: Build the Pipeline Components

Use the code from Sections 9.3-9.6 as your starting point.

## Step 3: Build the API

```
src/api.py
from fastapi import FastAPI, Query
from pydantic import BaseModel

app = FastAPI(title="RAG Search API")

class SearchResult(BaseModel):
 content: str
 source: str
 similarity: float

class RAGResponse(BaseModel):
 answer: str
 sources: list[SearchResult]

@app.get("/search", response_model=list[SearchResult])
async def search(
 q: str = Query(..., description="Search query"),
 top_k: int = Query(5, ge=1, le=20),
):
 """Semantic search over indexed documents."""
 # Use your retriever here
 pass

@app.get("/ask", response_model=RAGResponse)
async def ask(
 question: str = Query(..., description="Question to answer"),
):
 """RAG: retrieve context and generate an answer."""
 # Use your retriever.query_with_context() here
 pass

@app.get("/health")
async def health():
 return {"status": "healthy", "indexed_chunks": 0} # TODO: actual count
```

## Step 4: Docker Compose

```
version: "3.8"

services:
 api:
 build: .
 command: uvicorn src.api:app --host 0.0.0.0 --port 8000 --reload
 ports:
 - "8000:8000"
 environment:
 - DATABASE_URL=postgresql://postgres:postgres@db:5432/ragdb
 - OPENAI_API_KEY=${OPENAI_API_KEY}
 depends_on:
 db:
 condition: service_healthy
 volumes:
 - ./src:/app/src
 - ./data:/app/data

 db:
 image: pgvector/pgvector:pg16
 environment:
 POSTGRES_USER: postgres
 POSTGRES_PASSWORD: postgres
 POSTGRES_DB: ragdb
 ports:
 - "5432:5432"
 volumes:
 - pgdata:/var/lib/postgresql/data
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U postgres"]
 interval: 5s
 timeout: 5s
 retries: 5

volumes:
 pgdata:
```

## Step 5: Write Tests

```
tests/test_chunker.py
from src.chunker import chunk_by_tokens

def test_chunk_size():
 text = " ".join(["word"] * 1000)
 chunks = chunk_by_tokens(text, chunk_size=100, chunk_overlap=10)
 for chunk in chunks:
 word_count = len(chunk.split())
 assert word_count <= 120 # Allow some variance

def test_chunk_overlap():
 text = " ".join([f"word{i}" for i in range(200)])
 chunks = chunk_by_tokens(text, chunk_size=50, chunk_overlap=10)
 assert len(chunks) > 1
```

## Try It Yourself

```
Start the stack
docker-compose up -d

Index documents
python -m src.indexer

Test the API
curl "http://localhost:8000/search?q=return+policy&top_k=3"
curl "http://localhost:8000/ask?question=What+is+the+return+policy+for+electronics"
```

### Expected output:

```
/search returns top 3 matching chunks with similarity scores
/ask returns an AI-generated answer with source citations
```

## Completion Checklist

- [ ] Document ingestion handles at least 2 file formats
- [ ] Text cleaning removes common artifacts
- [ ] Chunking uses overlap and preserves metadata
- [ ] Embeddings generated and stored in pgvector
- [ ] HNSW or IVFFlat index created on embedding column
- [ ] `/search` endpoint returns ranked results with similarity scores
- [ ] `/ask` endpoint returns RAG-generated answers with source citations
- [ ] Docker Compose starts full stack with one command

- [ ] At least 3 unit tests
- [ ] README with setup instructions and example queries

## Bonus Challenges

- Add a `/index` endpoint that accepts new documents via upload
  - Implement metadata filtering (filter by source, content type)
  - Add a simple web UI with Streamlit or Gradio
  - Use sentence-transformers instead of OpenAI (no API key needed)
  - Add an Airflow DAG that re-indexes documents on a schedule
- 

*End of Module 9*

# Module 10: Capstone Project — Real-Time E-Commerce Analytics Platform

*Bring it all together. Build a complete, production-grade data platform that you can show in any interview.*

## Overview

**TL;DR:** You're building a **Real-Time E-Commerce Analytics Platform** — a full, production-grade data platform that handles batch and streaming data, implements data quality, includes an AI component, and runs entirely in Docker. This is the project you link in your resume, walk through in interviews, and write about on LinkedIn. It takes 5-8 hours across 5 phases.

Everything you've learned across 9 modules comes together here. When you're done, you'll have a portfolio project on GitHub that demonstrates every core data engineering skill.

## Architecture

[DIAGRAM: Full system architecture with five layers:

- **Data Sources** (left): REST API (Products), Kafka Stream (User Events), CSV Upload (Inventory)
- **Ingestion Layer:** Airflow orchestrating three sub-pipelines: API Ingester, Kafka Consumer, File Loader
- **Storage Layer:** S3/MinIO data lake (/raw/, /staging/, /prod/) and PostgreSQL + pgvector (raw schema, analytics star schema, vector embeddings)
- **Processing Layer:** SQL Transformations (CTEs, window functions, SCD Type 2), Great Expectations (schema, completeness, freshness, volume), Embedding Pipeline (OpenAI/sentence-transformers)
- **Serving Layer** (right): Analytics Queries, FastAPI (/similar/{id}, /search), Quality Alerts
- **Infrastructure** (bottom): Docker Compose (local dev), Terraform (AWS deploy), GitHub Actions (CI/CD)]

## What You'll Build

Component	Technology	Module Reference
Data modeling	Star schema in Postgres	Module 1
SQL transforms	CTEs, window functions, MERGE	Module 2
Python ingestion	API calls, Pydantic validation	Module 3
Orchestration	Airflow DAGs	Module 4
Batch processing	DuckDB or PySpark (your choice)	Module 5
Streaming	Kafka producer + consumer	Module 6
Data quality	Great Expectations	Module 7
Infrastructure	Docker Compose, Terraform configs	Module 8
AI component	Embeddings + pgvector	Module 9

## Project Phases

Phase	Focus	Time
1. Foundation	Repo setup, Docker Compose, database schema	~1 hour
2. Ingestion	API, Kafka, and CSV ingestion pipelines	~1.5 hours
3. Transformation	Star schema population, SQL transforms	~1.5 hours
4. Quality & AI	Great Expectations, embeddings, vector search	~1.5 hours
5. Polish	Documentation, testing, README, architecture diagram	~1-2 hours

### Phase 1: Foundation Setup

**TL;DR:** Set up the repo structure, Docker Compose stack (Postgres+pgvector, Airflow, MinIO, Kafka), database schema (raw, staging, analytics, quality schemas), and seed data generation. Everything starts with `docker-compose up -d`.

## Step 1: Repository Structure

```
mkdir ecommerce-data-platform && cd ecommerce-data-platform
git init
```

Create this directory structure:

```

ecommerce-data-platform/
├── .github/
│ ├── workflows/
│ └── ci.yml
├── dags/
│ ├── ingest_products.py
│ ├── ingest_events.py
│ ├── ingest_inventory.py
│ ├── transform_star_schema.py
│ ├── quality_checks.py
│ └── embeddings_pipeline.py
├── src/
│ ├── __init__.py
│ ├── ingestion/
│ │ ├── __init__.py
│ │ ├── api_client.py # REST API product ingestion
│ │ ├── kafka_consumer.py # Kafka event consumer
│ │ ├── kafka_producer.py # Kafka event producer (simulator)
│ │ └── file_loader.py # CSV inventory loader
│ ├── transformation/
│ │ ├── __init__.py
│ │ └── star_schema.py # SQL transformation logic
│ ├── quality/
│ │ ├── __init__.py
│ │ ├── expectations.py # Great Expectations suites
│ │ └── alerts.py # Alerting functions
│ ├── ai/
│ │ ├── __init__.py
│ │ ├── embeddings.py # Embedding generation
│ │ ├── vector_search.py # pgvector search
│ │ └── api.py # FastAPI similarity endpoint
│ ├── common/
│ │ ├── __init__.py
│ │ ├── config.py # Environment config
│ │ └── db.py # Database connection helpers
│ └── sql/
│ ├── 001_create_raw_tables.sql
│ ├── 002_create_dim_tables.sql
│ ├── 003_create_fact_tables.sql
│ ├── 004_create_quality_tables.sql
│ └── 005_create_vector_tables.sql
├── data/
│ ├── seed/ # Sample data for testing
│ │ ├── products.json
│ │ ├── customers.csv
│ │ └── inventory.csv
│ └── sample_events.json # Sample Kafka events
├── terraform/
│ ├── main.tf
│ ├── s3.tf
│ ├── rds.tf
│ ├── iam.tf
│ └── variables.tf

```



```
└─ great_expectations/
 └─ great_expectations.yml
 └─ expectations/
└─ tests/
 └─ test_ingestion.py
 └─ test_transformation.py
 └─ test_quality.py
 └─ test_embeddings.py
└─ docker-compose.yml
└─ Dockerfile
└─ pyproject.toml
└─ .env.example
└─ .gitignore
└─ README.md
```

## Step 2: Docker Compose — The Full Stack

This is the heart of your local development environment. One `docker-compose up -d` starts everything:

```

docker-compose.yml
version: "3.8"

x-airflow-common: &airflow-common
 build:
 context: .
 dockerfile: Dockerfile
 environment: &airflow-env
 AIRFLOW__CORE__EXECUTOR: LocalExecutor
 AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@airflow-
db:5432/airflow
 AIRFLOW__CORE__FERNET_KEY: ""
 AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: "true"
 AIRFLOW__CORE__LOAD_EXAMPLES: "false"
 AIRFLOW__WEBSERVER__EXPOSE_CONFIG: "true"
 DATABASE_URL: postgresql://postgres:postgres@warehouse-db:5432/ecommerce
 S3_ENDPOINT: http://minio:9000
 S3_BUCKET: ecommerce-data-lake
 AWS_ACCESS_KEY_ID: minioadmin
 AWS_SECRET_ACCESS_KEY: minioadmin
 KAFKA_BOOTSTRAP_SERVERS: kafka:29092
 volumes:
 - ./dags:/opt/airflow/dags
 - ./src:/opt/airflow/src
 - ./great_expectations:/opt/airflow/great_expectations
 - ./data:/opt/airflow/data
 depends_on:
 warehouse-db:
 condition: service_healthy
 airflow-db:
 condition: service_healthy

services:
— Warehouse Database (Postgres + pgvector) —————
warehouse-db:
 image: pgvector/pgvector:pg16
 environment:
 POSTGRES_USER: postgres
 POSTGRES_PASSWORD: postgres
 POSTGRES_DB: ecommerce
 ports:
 - "5432:5432"
 volumes:
 - warehouse_data:/var/lib/postgresql/data
 - ./sql:/docker-entrypoint-initdb.d # Auto-run SQL on first start
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U postgres"]
 interval: 5s
 timeout: 5s
 retries: 5

— Airflow Metadata Database —————
airflow-db:

```

```

image: postgres:16
environment:
 POSTGRES_USER: airflow
 POSTGRES_PASSWORD: airflow
 POSTGRES_DB: airflow
volumes:
 - airflow_db_data:/var/lib/postgresql/data
healthcheck:
 test: ["CMD-SHELL", "pg_isready -U airflow"]
 interval: 5s
 timeout: 5s
 retries: 5

— Airflow Webserver —————
airflow-webserver:
 <<: *airflow-common
 command: webserver
 ports:
 - "8080:8080"

— Airflow Scheduler —————
airflow-scheduler:
 <<: *airflow-common
 command: scheduler

— Airflow Init (one-shot) —————
airflow-init:
 <<: *airflow-common
 entrypoint: /bin/bash
 command: >
 -c "
 airflow db migrate &&
 airflow users create
 --username admin
 --password admin
 --firstname Admin
 --lastname User
 --role Admin
 --email admin@example.com || true
 "
 restart: "no"

— MinIO (S3-compatible object storage) —————
minio:
 image: minio/minio
 command: server /data --console-address ":9001"
 environment:
 MINIO_ROOT_USER: minioadmin
 MINIO_ROOT_PASSWORD: minioadmin
 ports:
 - "9000:9000"
 - "9001:9001"
 volumes:
 - minio_data:/data

```

```

— MinIO Bucket Setup (one-shot) —————
minio-setup:
 image: minio/mc
 depends_on:
 - minio
 entrypoint: >
 /bin/sh -c "
 sleep 5 &&
 mc alias set local http://minio:9000 minioadmin minioadmin &&
 mc mb local/ecommerce-data-lake --ignore-existing &&
 mc mb local/ecommerce-data-lake/raw --ignore-existing &&
 mc mb local/ecommerce-data-lake/staging --ignore-existing &&
 mc mb local/ecommerce-data-lake/prod --ignore-existing &&
 echo 'Buckets created!'
 "
 restart: "no"

— Kafka —————
zookeeper:
 image: confluentinc/cp-zookeeper:7.5.0
 environment:
 ZOOKEEPER_CLIENT_PORT: 2181
 ZOOKEEPER_TICK_TIME: 2000

kafka:
 image: confluentinc/cp-kafka:7.5.0
 depends_on:
 - zookeeper
 ports:
 - "9092:9092"
 environment:
 KAFKA_BROKER_ID: 1
 KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
 KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:
9092
 KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
 KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
 KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
 KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

— FastAPI (Similarity Search API) —————
api:
 build: .
 command: uvicorn src.ai.api:app --host 0.0.0.0 --port 8000 --reload
 ports:
 - "8000:8000"
 environment:
 DATABASE_URL: postgresql://postgres:postgres@warehouse-db:5432/ecommerce
 OPENAI_API_KEY: ${OPENAI_API_KEY:-}
 depends_on:
 warehouse-db:
 condition: service_healthy
 volumes:

```

```
- ./src:/app/src

volumes:
 warehouse_data:
 airflow_db_data:
 minio_data:
```

### Step 3: Dockerfile

```
FROM apache/airflow:2.8.1-python3.11

USER root
RUN apt-get update && apt-get install -y --no-install-recommends \
 gcc libpq-dev && rm -rf /var/lib/apt/lists/*

USER airflow

COPY pyproject.toml /opt/airflow/
RUN pip install --no-cache-dir \
 great-expectations \
 confluent-kafka \
 boto3 \
 psycpg2-binary \
 openai \
 sentence-transformers \
 fastapi \
 uvicorn \
 pydantic \
 polars \
 duckdb \
 pyarrow \
 requests \
 ruff \
 pytest

COPY src/ /opt/airflow/src/
COPY dags/ /opt/airflow/dags/
```

### Step 4: Database Schema (DDL)

These SQL files go in the `sql/` directory and run automatically on first `docker-compose up` because they're mounted into `/docker-entrypoint-initdb.d`.

`sql/001_create_raw_tables.sql`:

```

CREATE EXTENSION IF NOT EXISTS vector;

CREATE SCHEMA IF NOT EXISTS raw;
CREATE SCHEMA IF NOT EXISTS staging;
CREATE SCHEMA IF NOT EXISTS analytics;
CREATE SCHEMA IF NOT EXISTS quality;

-- Raw tables: data as it arrives, never modified
CREATE TABLE IF NOT EXISTS raw.products (
 product_id INTEGER,
 product_name VARCHAR(500),
 description TEXT,
 category VARCHAR(200),
 subcategory VARCHAR(200),
 price DECIMAL(10,2),
 brand VARCHAR(200),
 loaded_at TIMESTAMP DEFAULT NOW(),
 source VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS raw.events (
 event_id VARCHAR(50),
 event_type VARCHAR(50), -- page_view, add_to_cart, purchase, search
 customer_id INTEGER,
 product_id INTEGER,
 session_id VARCHAR(100),
 event_timestamp TIMESTAMP,
 event_properties JSONB,
 loaded_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS raw.inventory (
 product_id INTEGER,
 warehouse_id INTEGER,
 quantity_on_hand INTEGER,
 reorder_point INTEGER,
 last_restocked TIMESTAMP,
 loaded_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS raw.customers (
 customer_id INTEGER,
 email VARCHAR(300),
 first_name VARCHAR(100),
 last_name VARCHAR(100),
 signup_date DATE,
 city VARCHAR(200),
 state VARCHAR(100),
 country VARCHAR(100),
 loaded_at TIMESTAMP DEFAULT NOW()
);

```

sql/002\_create\_dim\_tables.sql:

```

-- Dimension tables with SCD Type 2 support
CREATE TABLE IF NOT EXISTS analytics.dim_products (
 product_key SERIAL PRIMARY KEY,
 product_id INTEGER NOT NULL,
 product_name VARCHAR(500),
 description TEXT,
 category VARCHAR(200),
 subcategory VARCHAR(200),
 price DECIMAL(10,2),
 brand VARCHAR(200),
 effective_from TIMESTAMP DEFAULT NOW(),
 effective_to TIMESTAMP DEFAULT '9999-12-31',
 is_current BOOLEAN DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS analytics.dim_customers (
 customer_key SERIAL PRIMARY KEY,
 customer_id INTEGER NOT NULL,
 email VARCHAR(300),
 first_name VARCHAR(100),
 last_name VARCHAR(100),
 signup_date DATE,
 city VARCHAR(200),
 state VARCHAR(100),
 country VARCHAR(100),
 customer_segment VARCHAR(50), -- derived: new, active, lapsed
 effective_from TIMESTAMP DEFAULT NOW(),
 effective_to TIMESTAMP DEFAULT '9999-12-31',
 is_current BOOLEAN DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS analytics.dim_time (
 time_key INTEGER PRIMARY KEY, -- YYYYMMDD format
 full_date DATE NOT NULL,
 day_of_week INTEGER,
 day_name VARCHAR(10),
 week_of_year INTEGER,
 month_number INTEGER,
 month_name VARCHAR(10),
 quarter INTEGER,
 year INTEGER,
 is_weekend BOOLEAN,
 is_holiday BOOLEAN DEFAULT FALSE
);

-- Populate dim_time for 2024-2027
INSERT INTO analytics.dim_time
SELECT
 TO_CHAR(d, 'YYYYMMDD')::INTEGER AS time_key,
 d AS full_date,
 EXTRACT(DOW FROM d) AS day_of_week,
 TO_CHAR(d, 'Day') AS day_name,
 EXTRACT(WEEK FROM d) AS week_of_year,

```



```

 EXTRACT(MONTH FROM d) AS month_number,
 TO_CHAR(d, 'Month') AS month_name,
 EXTRACT(QUARTER FROM d) AS quarter,
 EXTRACT(YEAR FROM d) AS year,
 EXTRACT(DOW FROM d) IN (0, 6) AS is_weekend,
 FALSE AS is_holiday
FROM generate_series('2024-01-01'::date, '2027-12-31'::date, '1 day') AS d
ON CONFLICT DO NOTHING;

```

**sql/003\_create\_fact\_tables.sql :**

```

CREATE TABLE IF NOT EXISTS analytics.fact_orders (
 order_id SERIAL PRIMARY KEY,
 customer_key INTEGER REFERENCES analytics.dim_customers(customer_key),
 time_key INTEGER REFERENCES analytics.dim_time(time_key),
 product_key INTEGER REFERENCES analytics.dim_products(product_key),
 quantity INTEGER,
 unit_price DECIMAL(10,2),
 total_amount DECIMAL(12,2),
 discount_amount DECIMAL(10,2) DEFAULT 0,
 order_status VARCHAR(50),
 order_timestamp TIMESTAMP,
 session_id VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS analytics.fact_events (
 event_id VARCHAR(50) PRIMARY KEY,
 customer_key INTEGER REFERENCES analytics.dim_customers(customer_key),
 product_key INTEGER,
 time_key INTEGER REFERENCES analytics.dim_time(time_key),
 event_type VARCHAR(50),
 session_id VARCHAR(100),
 event_timestamp TIMESTAMP,
 event_properties JSONB
);

-- Indexes for common query patterns
CREATE INDEX IF NOT EXISTS idx_fact_orders_time ON analytics.fact_orders(time_key);
CREATE INDEX IF NOT EXISTS idx_fact_orders_customer ON
analytics.fact_orders(customer_key);
CREATE INDEX IF NOT EXISTS idx_fact_events_type ON analytics.fact_events(event_type);
CREATE INDEX IF NOT EXISTS idx_fact_events_time ON analytics.fact_events(time_key);

```

**sql/004\_create\_quality\_tables.sql :**

```

CREATE TABLE IF NOT EXISTS quality.check_results (
 id SERIAL PRIMARY KEY,
 check_name VARCHAR(200),
 table_name VARCHAR(200),
 check_type VARCHAR(100), -- schema, completeness, freshness, volume, accuracy
 passed BOOLEAN,
 details JSONB,
 executed_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS quality.quarantine (
 id SERIAL PRIMARY KEY,
 source_table VARCHAR(200),
 record_data JSONB,
 violation_reason VARCHAR(500),
 quarantined_at TIMESTAMP DEFAULT NOW()
);

```

`sql/005_create_vector_tables.sql` :

```

CREATE TABLE IF NOT EXISTS analytics.product_embeddings (
 product_id INTEGER PRIMARY KEY,
 embedding vector(1536), -- For OpenAI text-embedding-3-small
 model_name VARCHAR(100) DEFAULT 'text-embedding-3-small',
 updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_product_embeddings_hnsw
ON analytics.product_embeddings
USING hnsw (embedding vector_cosine_ops);

```

## Step 5: Common Utilities

`src/common/config.py` :

```
import os

class Config:
 ENVIRONMENT = os.getenv("ENVIRONMENT", "dev")
 DATABASE_URL = os.getenv(
 "DATABASE_URL",
 "postgresql://postgres:postgres@localhost:5432/ecommerce",
)
 S3_ENDPOINT = os.getenv("S3_ENDPOINT", "http://localhost:9000")
 S3_BUCKET = os.getenv("S3_BUCKET", "ecommerce-data-lake")
 AWS_ACCESS_KEY_ID = os.getenv("AWS_ACCESS_KEY_ID", "minioadmin")
 AWS_SECRET_ACCESS_KEY = os.getenv("AWS_SECRET_ACCESS_KEY", "minioadmin")
 KAFKA_BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")
 OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
```

`src/common/db.py`:

```
import psycopg2
from contextlib import contextmanager
from src.common.config import Config

@contextmanager
def get_db_connection():
 conn = psycopg2.connect(Config.DATABASE_URL)
 try:
 yield conn
 conn.commit()
 except Exception:
 conn.rollback()
 raise
 finally:
 conn.close()

@contextmanager
def get_cursor():
 with get_db_connection() as conn:
 cur = conn.cursor()
 try:
 yield cur
 finally:
 cur.close()
```

## Step 6: Seed Data Generator

Save this as `data/generate_seed_data.py` and run it to create realistic test data:

```

"""Generate realistic seed data for the capstone project."""
import json
import csv
import random
from datetime import datetime, timedelta

random.seed(42)

— Products (500 total: 5 categories × 5 subcategories × 20 products) —
CATEGORIES = {
 "Electronics": ["Headphones", "Speakers", "Chargers", "Cables", "Cases"],
 "Clothing": ["T-Shirts", "Jeans", "Jackets", "Shoes", "Hats"],
 "Home & Kitchen": ["Cookware", "Utensils", "Storage", "Decor", "Lighting"],
 "Sports": ["Running", "Yoga", "Cycling", "Swimming", "Training"],
 "Books": ["Fiction", "Non-Fiction", "Technical", "Self-Help", "Biography"],
}

BRANDS = [
 "TechMax", "StyleCo", "HomePlus", "ActiveGear", "ReadMore",
 "ProElite", "UrbanEdge", "NatureFit", "SmartLife", "CoreBasics",
]

products = []
product_id = 1
for category, subcategories in CATEGORIES.items():
 for subcategory in subcategories:
 for i in range(20):
 brand = random.choice(BRANDS)
 price = round(random.uniform(9.99, 299.99), 2)
 suffix = random.choice(["Pro", "Lite", "Max", "Ultra", "Essential"])
 products.append({
 "product_id": product_id,
 "product_name": f"{brand} {subcategory} {suffix} {random.randint(1,
100)}",
 "description": (
 f"High-quality {subcategory.lower()} from {brand}. "
 f"Perfect for {category.lower()} enthusiasts. "
 f"Features premium materials and modern design."
),
 "category": category,
 "subcategory": subcategory,
 "price": price,
 "brand": brand,
 })
 product_id += 1

with open("data/seed/products.json", "w") as f:
 json.dump(products, f, indent=2)

— Customers (1,000) —
CITIES = [
 ("New York", "NY", "US"), ("Los Angeles", "CA", "US"),
 ("Chicago", "IL", "US"), ("Houston", "TX", "US"),

```

```

 ("Phoenix", "AZ", "US"), ("Austin", "TX", "US"),
 ("Seattle", "WA", "US"), ("Denver", "CO", "US"),
 ("Atlanta", "GA", "US"), ("Miami", "FL", "US"),
]
 FIRST_NAMES = [
 "James", "Emma", "Liam", "Olivia", "Noah",
 "Ava", "William", "Sophia", "Benjamin", "Isabella",
]
 LAST_NAMES = [
 "Smith", "Johnson", "Williams", "Brown", "Jones",
 "Garcia", "Miller", "Davis", "Rodriguez", "Martinez",
]

 customers = []
 for i in range(1, 1001):
 city, state, country = random.choice(CITIES)
 first = random.choice(FIRST_NAMES)
 last = random.choice(LAST_NAMES)
 customers.append({
 "customer_id": i,
 "email": f"{first.lower()}.{last.lower()}@example.com",
 "first_name": first,
 "last_name": last,
 "signup_date": (
 datetime(2024, 1, 1) + timedelta(days=random.randint(0, 730))
).strftime("%Y-%m-%d"),
 "city": city,
 "state": state,
 "country": country,
 })

 with open("data/seed/customers.csv", "w", newline="") as f:
 writer = csv.DictWriter(f, fieldnames=customers[0].keys())
 writer.writeheader()
 writer.writerows(customers)

— Inventory (1,500 records: 500 products × 3 warehouses) —————
inventory = []
for p in products:
 for warehouse_id in range(1, 4):
 inventory.append({
 "product_id": p["product_id"],
 "warehouse_id": warehouse_id,
 "quantity_on_hand": random.randint(0, 500),
 "reorder_point": random.randint(10, 50),
 "last_restocked": (
 datetime.now() - timedelta(days=random.randint(0, 30))
).strftime("%Y-%m-%d %H:%M:%S"),
 })

with open("data/seed/inventory.csv", "w", newline="") as f:
 writer = csv.DictWriter(f, fieldnames=inventory[0].keys())
 writer.writeheader()
 writer.writerows(inventory)

```

```
— Sample Events (10,000 for Kafka simulation) —————
EVENT_TYPES = ["page_view", "add_to_cart", "remove_from_cart", "purchase", "search"]
events = []
for i in range(10000):
 event_type = random.choices(EVENT_TYPES, weights=[50, 20, 5, 15, 10])[0]
 customer_id = random.randint(1, 1000)
 pid = random.randint(1, 500)

 event = {
 "event_id": f"evt_{i:06d}",
 "event_type": event_type,
 "customer_id": customer_id,
 "product_id": pid if event_type != "search" else None,
 "session_id": f"sess_{customer_id}_{random.randint(1, 10):03d}",
 "event_timestamp": (
 datetime.now() - timedelta(hours=random.randint(0, 168))
).isoformat(),
 "event_properties": {
 "page": f"/products/{pid}" if event_type == "page_view" else None,
 "search_query": (
 f"best {random.choice(['headphones', 'shoes', 'books', 'jacket'])}"
 if event_type == "search" else None
),
 "quantity": (
 random.randint(1, 3)
 if event_type in ("add_to_cart", "purchase") else None
),
 },
 }
 events.append(event)

with open("data/sample_events.json", "w") as f:
 json.dump(events, f, indent=2)

print(
 f"Generated: {len(products)} products, {len(customers)} customers, "
 f"{len(inventory)} inventory records, {len(events)} events"
)
```

**Expected output:**

```
Generated: 500 products, 1000 customers, 1500 inventory records, 10000 events
```

## Verify Phase 1

```
Generate seed data
mkdir -p data/seed
python data/generate_seed_data.py

Start the stack
docker-compose up -d

Wait 1-2 minutes for everything to initialize, then verify:

Database schemas exist
docker-compose exec warehouse-db psql -U postgres -d ecommerce -c "\dt raw.*"
docker-compose exec warehouse-db psql -U postgres -d ecommerce -c "\dt analytics.*"

MinIO is running – open http://localhost:9001 (minioadmin/minioadmin)
Airflow is running – open http://localhost:8080 (admin/admin)
```

☐ **Phase 1 Checkpoint:** All services running, schemas created, seed data generated.

## Phase 2: Data Ingestion

**TL;DR:** Build three ingestion pipelines: (1) REST API client that fetches products and uploads to S3 as Parquet, (2) Kafka producer/consumer for real-time user events, (3) CSV file loader for customer and inventory data. Each is wrapped in an Airflow DAG.

### API Ingestion (Products)

`src/ingestion/api_client.py` :

```

"""Product catalog API client with pagination and error handling."""
import json
import time
import boto3
import pyarrow as pa
import pyarrow.parquet as pq
from io import BytesIO
from datetime import datetime
from src.common.config import Config

class ProductAPIClient:
 """Simulates pulling from a REST API by reading seed data.
 In production, replace fetch_products() with actual HTTP requests.
 """

 def __init__(self, seed_file: str = "data/seed/products.json"):
 self.seed_file = seed_file
 self.s3 = boto3.client(
 "s3",
 endpoint_url=Config.S3_ENDPOINT,
 aws_access_key_id=Config.AWS_ACCESS_KEY_ID,
 aws_secret_access_key=Config.AWS_SECRET_ACCESS_KEY,
)

 def fetch_products(self, page: int = 1, page_size: int = 100) -> list[dict]:
 """Fetch a page of products from the API."""
 with open(self.seed_file) as f:
 all_products = json.load(f)
 start = (page - 1) * page_size
 end = start + page_size
 return all_products[start:end]

 def fetch_all_products(self) -> list[dict]:
 """Fetch all products with pagination."""
 all_products = []
 page = 1
 while True:
 batch = self.fetch_products(page=page)
 if not batch:
 break
 all_products.extend(batch)
 page += 1
 time.sleep(0.1) # Simulate rate limiting
 return all_products

 def upload_to_raw(self, products: list[dict]) -> str:
 """Upload products to S3 raw layer as Parquet."""
 table = pa.Table.from_pylist(products)
 buffer = BytesIO()
 pq.write_table(table, buffer)
 buffer.seek(0)

```



```

date_str = datetime.now().strftime("%Y-%m-%d")
key = f"raw/products/date={date_str}/products.parquet"

self.s3.put_object(
 Bucket=Config.S3_BUCKET,
 Key=key,
 Body=buffer.getvalue(),
)
print(f"Uploaded {len(products)} products to s3://{Config.S3_BUCKET}/{key}")
return key

def run(self) -> str:
 """Full ingestion: fetch all products and upload to S3."""
 products = self.fetch_all_products()
 return self.upload_to_raw(products)

```

## Kafka Streaming (Events)

`src/ingestion/kafka_producer.py` :

```

"""Simulate real-time e-commerce events into Kafka."""
import json
import time
import random
from datetime import datetime
from confluent_kafka import Producer
from src.common.config import Config

class EventProducer:
 def __init__(self):
 self.producer = Producer({
 "bootstrap.servers": Config.KAFKA_BOOTSTRAP_SERVERS,
 })
 self.topic = "ecommerce.events"

 def produce_event(self, event: dict) -> None:
 self.producer.produce(
 self.topic,
 key=str(event["customer_id"]).encode(),
 value=json.dumps(event).encode(),
)

 def simulate_traffic(
 self, events_per_second: int = 10, duration_seconds: int = 60
):
 """Generate simulated event traffic."""
 with open("data/sample_events.json") as f:
 sample_events = json.load(f)

 total_sent = 0
 start_time = time.time()

 while time.time() - start_time < duration_seconds:
 event = random.choice(sample_events).copy()
 event["event_timestamp"] = datetime.now().isoformat()
 event["event_id"] = (
 f"evt_{int(time.time() * 1000)}_{random.randint(0, 999)}"
)
 self.produce_event(event)
 total_sent += 1

 if total_sent % events_per_second == 0:
 self.producer.flush()
 time.sleep(1)

 self.producer.flush()
 print(f"Produced {total_sent} events to {self.topic}")

```

`src/ingestion/kafka_consumer.py` :

```

"""Consume events from Kafka and load to raw layer."""
import json
from confluent_kafka import Consumer, KafkaError
from src.common.db import get_db_connection
from src.common.config import Config

class EventConsumer:
 def __init__(self):
 self.consumer = Consumer({
 "bootstrap.servers": Config.KAFKA_BOOTSTRAP_SERVERS,
 "group.id": "ecommerce-ingestion",
 "auto.offset.reset": "earliest",
 })
 self.consumer.subscribe(["ecommerce.events"])

 def consume_and_load(
 self, max_messages: int = 1000, timeout: float = 10.0
) -> int:
 """Consume messages and insert into raw.events."""
 messages = []

 while len(messages) < max_messages:
 msg = self.consumer.poll(timeout=timeout)
 if msg is None:
 break
 if msg.error():
 if msg.error().code() == KafkaError._PARTITION_EOF:
 break
 print(f"Error: {msg.error()}")
 continue
 event = json.loads(msg.value().decode())
 messages.append(event)

 if messages:
 self._insert_to_raw(messages)

 self.consumer.close()
 return len(messages)

 def _insert_to_raw(self, events: list[dict]):
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 for event in events:
 cur.execute("""
 INSERT INTO raw.events
 (event_id, event_type, customer_id, product_id,
 session_id, event_timestamp, event_properties)
 VALUES (%s, %s, %s, %s, %s, %s, %s)
 ON CONFLICT DO NOTHING
 """, (
 event["event_id"],
 event["event_type"],

```

```
 event["customer_id"],
 event.get("product_id"),
 event["session_id"],
 event["event_timestamp"],
 json.dumps(event.get("event_properties", {})),
))
print(f"Loaded {len(events)} events to raw.events")
```

## CSV Ingestion (Inventory + Customers)

`src/ingestion/file_loader.py` :

```

"""Load CSV files into raw tables."""
import csv
from src.common.db import get_db_connection

class FileLoader:
 def load_customers(self, filepath: str = "data/seed/customers.csv"):
 rows = self._read_csv(filepath)
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 for row in rows:
 cur.execute("""
 INSERT INTO raw.customers
 (customer_id, email, first_name, last_name,
 signup_date, city, state, country)
 VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
 """, (
 row["customer_id"], row["email"],
 row["first_name"], row["last_name"],
 row["signup_date"], row["city"],
 row["state"], row["country"],
))
 print(f"Loaded {len(rows)} customers to raw.customers")

 def load_inventory(self, filepath: str = "data/seed/inventory.csv"):
 rows = self._read_csv(filepath)
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 for row in rows:
 cur.execute("""
 INSERT INTO raw.inventory
 (product_id, warehouse_id, quantity_on_hand,
 reorder_point, last_restocked)
 VALUES (%s, %s, %s, %s, %s)
 """, (
 row["product_id"], row["warehouse_id"],
 row["quantity_on_hand"], row["reorder_point"],
 row["last_restocked"],
))
 print(f"Loaded {len(rows)} inventory records to raw.inventory")

 @staticmethod
 def _read_csv(filepath: str) -> list[dict]:
 with open(filepath) as f:
 return list(csv.DictReader(f))

```

## Airflow DAG for Product Ingestion

`dags/ingest_products.py` :

```

from airflow.decorators import dag, task
from datetime import datetime

@dag(
 schedule="@daily",
 start_date=datetime(2026, 1, 1),
 catchup=False,
 tags=["ingestion", "products"],
)
def ingest_products():

 @task()
 def extract_products():
 from src.ingestion.api_client import ProductAPIClient
 client = ProductAPIClient()
 products = client.fetch_all_products()
 return len(products)

 @task()
 def load_products_to_raw():
 from src.ingestion.api_client import ProductAPIClient
 client = ProductAPIClient()
 products = client.fetch_all_products()
 key = client.upload_to_raw(products)

 # Also load to Postgres raw table
 from src.common.db import get_db_connection
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 for p in products:
 cur.execute("""
 INSERT INTO raw.products
 (product_id, product_name, description, category,
 subcategory, price, brand, source)
 VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
 """, (
 p["product_id"], p["product_name"], p["description"],
 p["category"], p["subcategory"], p["price"],
 p["brand"], "api",
))
 return key

 extract_products() >> load_products_to_raw()

ingest_products()

```

#### ❏Pro Tip

Create similar DAGs for events ( `ingest_events.py` ) and inventory ( `ingest_inventory.py` ). The events DAG should trigger the Kafka consumer; the inventory DAG should use the FileLoader.

❏**Phase 2 Checkpoint:** All three ingestion pipelines load data into raw tables. Products also land in S3/MinIO as Parquet.

## Phase 3: Transformation (Star Schema Population)

**TL;DR:** Transform raw data into a star schema with SCD Type 2 for customers, SCD Type 1 for products, and fact tables populated via dimension key lookups. Orders are derived from purchase events.

`src/transformation/star_schema.py` :

```

"""Transform raw data into star schema analytics tables."""
from src.common.db import get_db_connection

class StarSchemaBuilder:

 def populate_dim_products(self):
 """Load product dimension (SCD Type 1 – overwrite)."""
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 cur.execute("""
 INSERT INTO analytics.dim_products
 (product_id, product_name, description, category,
 subcategory, price, brand)
 SELECT DISTINCT ON (product_id)
 product_id, product_name, description, category,
 subcategory, price, brand
 FROM raw.products
 ORDER BY product_id, loaded_at DESC
 ON CONFLICT DO NOTHING
 """)
 print("▢ dim_products populated")

 def populate_dim_customers(self):
 """SCD Type 2 load for customer dimension."""
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 # Compute customer segments and insert new/changed records
 cur.execute("""
 WITH customer_segments AS (
 SELECT
 c.customer_id,
 c.email,
 c.first_name,
 c.last_name,
 c.signup_date,
 c.city,
 c.state,
 c.country,
 CASE
 WHEN c.signup_date > CURRENT_DATE - INTERVAL '30 days'
 THEN 'new'
 WHEN EXISTS (
 SELECT 1 FROM raw.events e
 WHERE e.customer_id = c.customer_id
 AND e.event_type = 'purchase'
 AND e.event_timestamp > NOW() - INTERVAL '90 days'
) THEN 'active'
 ELSE 'lapsed'
 END AS customer_segment
 FROM raw.customers c
)
 -- Insert new records (customers not yet in dimension)
 """)

```



```

 INSERT INTO analytics.dim_customers
 (customer_id, email, first_name, last_name, signup_date,
 city, state, country, customer_segment)
 SELECT
 cs.customer_id, cs.email, cs.first_name, cs.last_name,
 cs.signup_date::date, cs.city, cs.state, cs.country,
 cs.customer_segment
 FROM customer_segments cs
 LEFT JOIN analytics.dim_customers dc
 ON cs.customer_id = dc.customer_id AND dc.is_current = TRUE
 WHERE dc.customer_id IS NULL
 """
 print("▣ dim_customers populated")

def populate_fact_events(self):
 """Load event fact table with dimension key lookups."""
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 cur.execute("""
 INSERT INTO analytics.fact_events
 (event_id, customer_key, product_key, time_key,
 event_type, session_id, event_timestamp, event_properties)
 SELECT
 e.event_id,
 dc.customer_key,
 dp.product_key,
 TO_CHAR(e.event_timestamp::date, 'YYYYMMDD')::INTEGER AS
time_key,

 e.event_type,
 e.session_id,
 e.event_timestamp,
 e.event_properties
 FROM raw.events e
 LEFT JOIN analytics.dim_customers dc
 ON e.customer_id = dc.customer_id AND dc.is_current = TRUE
 LEFT JOIN analytics.dim_products dp
 ON e.product_id = dp.product_id AND dp.is_current = TRUE
 WHERE NOT EXISTS (
 SELECT 1 FROM analytics.fact_events fe
 WHERE fe.event_id = e.event_id
)
 """)
 print("▣ fact_events populated")

def populate_fact_orders(self):
 """Derive orders from purchase events."""
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 cur.execute("""
 INSERT INTO analytics.fact_orders
 (customer_key, time_key, product_key, quantity,
 unit_price, total_amount, order_status,
 order_timestamp, session_id)
 SELECT
 """)

```

```

 dc.customer_key,
 TO_CHAR(e.event_timestamp::date, 'YYYYMMDD')::INTEGER,
 dp.product_key,
 COALESCE((e.event_properties->>'quantity')::INTEGER, 1),
 dp.price,
 dp.price * COALESCE(
 (e.event_properties->>'quantity')::INTEGER, 1
),
 'completed',
 e.event_timestamp,
 e.session_id
 FROM raw.events e
 JOIN analytics.dim_customers dc
 ON e.customer_id = dc.customer_id
 AND dc.is_current = TRUE
 JOIN analytics.dim_products dp
 ON e.product_id = dp.product_id
 AND dp.is_current = TRUE
 WHERE e.event_type = 'purchase'
 AND NOT EXISTS (
 SELECT 1 FROM analytics.fact_orders fo
 WHERE fo.session_id = e.session_id
 AND fo.product_key = dp.product_key
 AND fo.order_timestamp = e.event_timestamp
)
 """
 print("▣ fact_orders populated")

def run_all(self):
 """Execute all transformations in dependency order."""
 print("Starting star schema transformation...")
 self.populate_dim_products()
 self.populate_dim_customers()
 self.populate_fact_events()
 self.populate_fact_orders()
 print("▣ Star schema transformation complete!")

```

## Airflow DAG for Transformation

`dags/transform_star_schema.py` :

```

from airflow.decorators import dag, task
from datetime import datetime

@dag(
 schedule="@daily",
 start_date=datetime(2026, 1, 1),
 catchup=False,
 tags=["transformation"],
)
def transform_star_schema():

 @task()
 def populate_dimensions():
 from src.transformation.star_schema import StarSchemaBuilder
 builder = StarSchemaBuilder()
 builder.populate_dim_products()
 builder.populate_dim_customers()

 @task()
 def populate_facts():
 from src.transformation.star_schema import StarSchemaBuilder
 builder = StarSchemaBuilder()
 builder.populate_fact_events()
 builder.populate_fact_orders()

 populate_dimensions() >> populate_facts()

transform_star_schema()

```

□ **Phase 3 Checkpoint:** Star schema populated. Dimensions have correct SCD behavior. Fact tables have proper foreign key lookups.

## Phase 4: Data Quality & AI Component

**TL;DR:** Add Great Expectations checks on raw event data (null checks, uniqueness, valid event types, row count). Build a product embeddings pipeline that generates vectors and stores them in pgvector. Serve similarity search and semantic product search via FastAPI.

### Great Expectations Integration

`src/quality/expectations.py` :

```

"""Data quality checks using Great Expectations."""
import great_expectations as gx
from src.common.config import Config

def run_raw_events_quality():
 """Run quality checks on raw event data."""
 context = gx.get_context()

 datasource = context.sources.add_or_update_postgres(
 name="warehouse",
 connection_string=Config.DATABASE_URL,
)

 asset = datasource.add_table_asset(
 name="raw_events", table_name="events", schema_name="raw"
)
 batch_request = asset.build_batch_request()

 suite = context.add_or_update_expectation_suite("raw_events_suite")
 validator = context.get_validator(
 batch_request=batch_request,
 expectation_suite_name="raw_events_suite",
)

 # Schema checks
 validator.expect_column_values_to_not_be_null("event_id")
 validator.expect_column_values_to_not_be_null("event_type")
 validator.expect_column_values_to_not_be_null("customer_id")
 validator.expect_column_values_to_be_unique("event_id")

 # Value checks
 validator.expect_column_values_to_be_in_set(
 "event_type",
 ["page_view", "add_to_cart", "remove_from_cart", "purchase", "search"],
)

 # Volume check
 validator.expect_table_row_count_to_be_between(min_value=1)

 validator.save_expectation_suite(discard_failed_expectations=False)

 # Run checkpoint
 checkpoint = context.add_or_update_checkpoint(
 name="raw_events_checkpoint",
 validations=[{
 "batch_request": batch_request,
 "expectation_suite_name": "raw_events_suite",
 }],
)

 result = checkpoint.run()

```

```
if not result.success:
 failures = []
 for vr in result.run_results.values():
 for er in vr["validation_result"]["results"]:
 if not er["success"]:
 failures.append(
 er["expectation_config"]["expectation_type"]
)
 raise Exception(f"Quality checks failed: {failures}")

print("▣ All raw events quality checks passed!")
return True
```

## AI Component — Product Embeddings

`src/ai/embeddings.py` :

```

"""Generate product embeddings and store in pgvector."""
import openai
from src.common.config import Config
from src.common.db import get_db_connection

class ProductEmbedder:
 def __init__(self):
 if Config.OPENAI_API_KEY:
 self.client = openai.OpenAI(api_key=Config.OPENAI_API_KEY)
 self.model = "text-embedding-3-small"
 else:
 # Fallback: use sentence-transformers (free, local)
 from sentence_transformers import SentenceTransformer
 self.local_model = SentenceTransformer("all-MiniLM-L6-v2")
 self.client = None
 self.model = "all-MiniLM-L6-v2"

 def generate_embedding(self, text: str) -> list[float]:
 if self.client:
 response = self.client.embeddings.create(
 model=self.model, input=[text]
)
 return response.data[0].embedding
 else:
 return self.local_model.encode([text])[0].tolist()

 def embed_all_products(self) -> int:
 """Generate embeddings for products that don't have them yet."""
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 cur.execute("""
 SELECT p.product_id, p.product_name, p.description,
 p.category, p.subcategory
 FROM analytics.dim_products p
 LEFT JOIN analytics.product_embeddings pe
 ON p.product_id = pe.product_id
 WHERE pe.product_id IS NULL AND p.is_current = TRUE
 """)
 products = cur.fetchall()

 if not products:
 print("No products need embedding.")
 return 0

 print(f"Generating embeddings for {len(products)} products...")

 for product_id, name, desc, category, subcategory in products:
 text = (
 f"{name}. {desc}. "
 f"Category: {category}/{subcategory}"
)
 embedding = self.generate_embedding(text)

```

```
cur.execute("""
 INSERT INTO analytics.product_embeddings
 (product_id, embedding, model_name)
 VALUES (%s, %s, %s)
 ON CONFLICT (product_id) DO UPDATE SET
 embedding = EXCLUDED.embedding,
 updated_at = NOW()
""", (product_id, str(embedding), self.model))

print(f"▣ Embedded {len(products)} products.")
return len(products)
```

## Similarity Search API

`src/ai/api.py` :

```

"""FastAPI endpoints for product similarity search."""
from fastapi import FastAPI, Query
from pydantic import BaseModel
from src.common.db import get_db_connection

app = FastAPI(title="E-Commerce Product Similarity API")

class SimilarProduct(BaseModel):
 product_id: int
 product_name: str
 category: str
 price: float
 similarity: float

@app.get("/similar/{product_id}", response_model=list[SimilarProduct])
async def get_similar_products(
 product_id: int,
 top_k: int = Query(5, ge=1, le=20),
):
 """Find products similar to the given product."""
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 cur.execute("""
 WITH source_embedding AS (
 SELECT embedding
 FROM analytics.product_embeddings
 WHERE product_id = %s
)
 SELECT
 p.product_id,
 p.product_name,
 p.category,
 p.price,
 1 - (pe.embedding <=> se.embedding) AS similarity
 FROM analytics.product_embeddings pe
 JOIN analytics.dim_products p
 ON pe.product_id = p.product_id AND p.is_current = TRUE
 CROSS JOIN source_embedding se
 WHERE pe.product_id != %s
 ORDER BY pe.embedding <=> se.embedding
 LIMIT %s
 """, (product_id, product_id, top_k))

 return [
 SimilarProduct(
 product_id=row[0],
 product_name=row[1],
 category=row[2],
 price=float(row[3]),
 similarity=round(float(row[4]), 4),
)
]

```



```

 for row in cur.fetchall()
]

@app.get("/search", response_model=list[SimilarProduct])
async def search_products(
 q: str = Query(..., description="Search query"),
 top_k: int = Query(5, ge=1, le=20),
):
 """Semantic search for products by natural language query."""
 from src.ai.embeddings import ProductEmbedder
 embedder = ProductEmbedder()
 query_embedding = embedder.generate_embedding(q)

 with get_db_connection() as conn:
 with conn.cursor() as cur:
 cur.execute("""
 SELECT
 p.product_id,
 p.product_name,
 p.category,
 p.price,
 1 - (pe.embedding <=> %s::vector) AS similarity
 FROM analytics.product_embeddings pe
 JOIN analytics.dim_products p
 ON pe.product_id = p.product_id AND p.is_current = TRUE
 ORDER BY pe.embedding <=> %s::vector
 LIMIT %s
 """, (str(query_embedding), str(query_embedding), top_k))

 return [
 SimilarProduct(
 product_id=row[0],
 product_name=row[1],
 category=row[2],
 price=float(row[3]),
 similarity=round(float(row[4]), 4),
)
 for row in cur.fetchall()
]

@app.get("/health")
async def health():
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 cur.execute(
 "SELECT COUNT(*) FROM analytics.product_embeddings"
)
 count = cur.fetchone()[0]
 return {"status": "healthy", "embedded_products": count}

```

Try the API:

```
Similar products
curl http://localhost:8000/similar/42?top_k=3

Semantic search
curl "http://localhost:8000/search?q=noise+cancelling+headphones&top_k=5"

Health check
curl http://localhost:8000/health
```

□ **Phase 4 Checkpoint:** Quality checks run and pass. Product embeddings stored in pgvector. Similarity API returns results.

## Phase 5: Polish, Testing, and Documentation

**TL;DR:** Write unit tests, set up GitHub Actions CI, create a comprehensive README, and draw an architecture diagram. This is what turns a project into a portfolio piece.

### Tests

`tests/test_ingestion.py`:

```
import pytest
from src.ingestion.api_client import ProductAPIClient

def test_fetch_products_returns_data():
 client = ProductAPIClient()
 products = client.fetch_products(page=1, page_size=10)
 assert len(products) == 10
 assert "product_id" in products[0]
 assert "product_name" in products[0]

def test_fetch_products_pagination():
 client = ProductAPIClient()
 page1 = client.fetch_products(page=1, page_size=5)
 page2 = client.fetch_products(page=2, page_size=5)
 assert page1[0]["product_id"] != page2[0]["product_id"]
```

`tests/test_transformation.py`:

```
import pytest

def test_dim_time_populated():
 """Verify dim_time has expected date range."""
 from src.common.db import get_db_connection
 with get_db_connection() as conn:
 with conn.cursor() as cur:
 cur.execute("SELECT COUNT(*) FROM analytics.dim_time")
 count = cur.fetchone()[0]
 assert count > 1000 # At least 3 years of dates
```

## GitHub Actions CI

`.github/workflows/ci.yml`:

```

name: CI

on:
 push:
 branches: [main]
 pull_request:
 branches: [main]

jobs:
 test:
 runs-on: ubuntu-latest

 services:
 postgres:
 image: pgvector/pgvector:pg16
 env:
 POSTGRES_USER: test
 POSTGRES_PASSWORD: test
 POSTGRES_DB: test_ecommerce
 ports:
 - 5432:5432
 options: >-
 --health-cmd "pg_isready -U test"
 --health-interval 10s
 --health-timeout 5s
 --health-retries 5

 steps:
 - uses: actions/checkout@v4

 - uses: actions/setup-python@v5
 with:
 python-version: "3.11"

 - name: Install dependencies
 run: pip install -e ".[dev]"

 - name: Lint
 run: ruff check src/ tests/

 - name: Run tests
 env:
 DATABASE_URL: postgresql://test:test@localhost:5432/test_ecommerce
 run: pytest tests/ -v

```

**README.md**

```

▢ Real-Time E-Commerce Analytics Platform

A production-grade data platform built as the capstone project for
The Data Engineering Fast Track course.

Architecture

[Include your architecture diagram here]

Data Sources: REST API (products), Kafka stream (user events), CSV (inventory)
Storage: S3/MinIO data lake + Postgres analytics warehouse
Processing: Python + SQL transformations, Airflow orchestration
Data Model: Star schema (fact_orders, fact_events, dim_products, dim_customers,
dim_time)
Data Quality: Great Expectations with automated alerting
AI Component: Product embeddings via pgvector for similarity search

Quick Start

```bash
# Clone and setup
git clone https://github.com/YOUR_USERNAME/ecommerce-data-platform.git
cd ecommerce-data-platform
cp .env.example .env

# Generate seed data
python data/generate_seed_data.py

# Start all services
docker-compose up -d

# Wait 1-2 minutes, then:
# Airflow UI: http://localhost:8080 (admin/admin)
# MinIO UI: http://localhost:9001 (minioadmin/minioadmin)
# Similarity API: http://localhost:8000/docs
```

Tech Stack

Component	Technology
Orchestration	Apache Airflow 2.8
Warehouse	PostgreSQL 16 + pgvector
Data Lake	MinIO (S3-compatible)
Streaming	Apache Kafka
Data Quality	Great Expectations
AI/Embeddings	OpenAI API / sentence-transformers
API	FastAPI
Infrastructure	Docker Compose, Terraform
CI/CD	GitHub Actions
Language	Python 3.11

Data Model

```

```

Star Schema

Fact Tables:
- `fact_orders` – completed purchases (grain: one row per order line)
- `fact_events` – all user events (grain: one row per event)

Dimension Tables:
- `dim_products` – product catalog (SCD Type 1)
- `dim_customers` – customer profiles (SCD Type 2)
- `dim_time` – date dimension (2024-2027)

Running Tests

```bash
pytest tests/ -v
```

License

MIT

```

## Architecture Diagram Guide

For your draw.io / Excalidraw diagram, include these components:

### Left column (Data Sources):

- REST API icon → "Product Catalog API"
- Kafka icon → "User Event Stream"
- File icon → "CSV Inventory Upload"

### Center-top (Ingestion):

- Airflow icon → three sub-boxes: "API Ingester", "Kafka Consumer", "File Loader"

### Center (Storage):

- S3 bucket → "MinIO / S3 Data Lake" with layers: raw/ staging/ prod/
- Database → "PostgreSQL + pgvector" with: Raw Schema, Analytics Star Schema, Vector Embeddings

### Center-bottom (Processing):

- "SQL Transformations" (CTEs, Window Functions, SCD Type 2)
- "Great Expectations" (Schema, Completeness, Freshness, Volume)
- "Embedding Pipeline" (OpenAI / sentence-transformers)

### Right column (Serving):

- "Analytics Queries"
- "FastAPI" with /similar/{id} and /search endpoints
- "Quality Alerts"

### Bottom:

- Docker Compose, Terraform, GitHub Actions icons

---

## Completion Checklist

### Core Requirements (Must Have)

- [ ] **Repository structure** — clean, organized, follows the template
- [ ] **Docker Compose** — full stack starts with `docker-compose up -d`
- [ ] **Data ingestion** — at least 2 of 3 sources working (API, Kafka, CSV)
- [ ] **Star schema** — fact and dimension tables populated with correct relationships
- [ ] **Airflow DAGs** — at least 2 DAGs (ingestion + transformation)
- [ ] **Data quality** — at least 5 Great Expectations checks that run and pass
- [ ] **AI component** — product embeddings stored in pgvector
- [ ] **Similarity API** — FastAPI endpoint returns similar products
- [ ] **Tests** — at least 5 passing tests
- [ ] **README** — includes architecture, setup instructions, tech stack

### Above & Beyond (Bonus)

- [ ] All 3 data sources working
- [ ] SCD Type 2 implemented in customer dimension
- [ ] Volume anomaly detection in quality checks
- [ ] Slack/webhook alerting on quality failures
- [ ] Terraform configs for AWS deployment
- [ ] GitHub Actions CI pipeline
- [ ] Data dictionary / column-level documentation
- [ ] Architecture diagram (draw.io / Excalidraw)
- [ ] Semantic search endpoint (query text → similar products)
- [ ] 5-minute video walkthrough



## Scoring Guide

| Score       | Level              | Description                                         |
|-------------|--------------------|-----------------------------------------------------|
| 10/10 Core  | <b>Complete</b>    | All core requirements met                           |
| 8-9/10 Core | <b>Strong</b>      | Minor gaps but demonstrates full understanding      |
| 6-7/10 Core | <b>Acceptable</b>  | Key components work but some are incomplete         |
| < 6/10 Core | <b>Needs Work</b>  | Major components missing — revisit relevant modules |
| 5+ Bonus    | <b>Outstanding</b> | Portfolio-ready project, go get that job!           |

## Deliverables Summary

When you're done, you should have:

1. **GitHub repository** — public, with clean commit history
2. **Working Docker Compose stack** — one command to run everything
3. **Architecture diagram** — visual overview of the system
4. **README.md** — comprehensive documentation
5. **(Optional) 5-minute video walkthrough** — record yourself explaining the project
6. **(Optional) Blog post** — write up your design decisions for LinkedIn

**This project IS your portfolio piece.** When an interviewer asks "tell me about a data engineering project you've built," this is what you walk them through. Make it clean, make it work, make it yours.

You've got this. Go build something awesome. ☐

---

*End of Module 10 — End of The Data Engineering Fast Track*